

Dobývání znalostí

Internet a klasifikační metody

Učební text k státní závěrečné zkoušce

SZZ Umělá inteligence, MFF UK

červenec 2026

Mapování okruhů na kapitoly

Dokument pokrývá **dva** zkouškové okruhy. Část I odpovídá okruhu „Dobývání znalostí“ (kurz NDBI023, I. Mrázová; kapitola [10](#) navíc vychází ze slidů kurzu *Social Networks and their Analysis*, 2024, téhož vyučujícího). Část II pokrývá okruh předmětu „Internet a klasifikační metody“ (M. Holeňa).

Část I — oficiální znění okruhu „Dobývání znalostí“:

Položka oficiálního okruhu	Kapitola v dokumentu
Základní paradigmata dobývání znalostí	1 Základní paradigmata dobývání znalostí
Příprava dat	2 Příprava dat
Výběr atributů a metody pro analýzu jejich relevance	3 Výběr atributů a analýza relevance
Metody pro dobývání znalostí; asociační pravidla	4 Asociační pravidla a frekventované vzory
Přístupy založené na principu učení s učitelem	5 Učení s učitelem: klasifikace a predikce 6 Vyhodnocování modelů
Klastrová analýza	7 Klastrová analýza
Metody pro extrakci charakteristických diskriminačních pravidel a měření jejich zajímavosti	8 Charakteristická a diskriminační pravidla
Reprezentace, vyhodnocování a vizualizace získaných znalostí	9 Reprezentace a vizualizace znalostí (+ kap. 6)
Modely pro analýzu sociálních sítí; míry centrality, detekce komunit	10 Analýza sociálních sítí (reprezentace a koheze sítí, vlastnosti reálných sítí, modelování vlivu, homofilie a afiliace, strukturní balance, HITS a PageRank, detekce komunit, predikce vazeb)
Praktické využití technik	11 Praktické využití technik

Část II — osnova okruhu „Internet a klasifikační metody“:

Téma osnovy	Kapitola v dokumentu
1. Tři důležité internetové aplikace klasifikačních metod (filtrace spamu, doporučovací systémy, systémy pro odhalení hrozeb v síti)	12 Tři důležité internetové aplikace klasifikačních metod
2. Základní koncepty týkající se klasifikace	13 Základní koncepty týkající se klasifikace (+ kap. 6)
3. Hlavní typy klasifikačních metod	14 Hlavní typy klasifikačních metod (+ kap. 5)
4. Kdy dělá klasifikátor nejméně chyb na nových vstupech?	15 Kdy dělá klasifikátor nejméně chyb na nových vstupech?
5. Kdy je klasifikace srozumitelná uživateli?	16 Kdy je klasifikace srozumitelná uživateli? (+ kap. 8, 9)
6. Tým zvládne více než jedinec	17 Tým zvládne více než jedinec (+ kap. 5.9)

Poznámka k terminologii. Text používá české termíny; anglický ekvivalent je uveden kurzívou v závorce při prvním výskytu.

Zdroje. Část I: kurz NDBI023 *Dobývání znalostí* a kurz *Social Networks and their Analysis* (I. Mrázová, MFF UK), doplněné o standardní literaturu — Han, Kamber, Pei: *Data Mining — Concepts and Techniques*; Tan, Steinbach, Kumar: *Introduction to Data Mining*; Aggarwal: *Data Mining — The Textbook*; pro síť Easley, Kleinberg: *Networks, Crowds and Markets* a Newman: *Networks*. Část II: osnova předmětu *Internet a klasifikační metody* (M. Holeňa) zpracovaná ze standardní literatury — Bishop: *Pattern Recognition and Machine Learning*; Hastie, Tibshirani, Friedman: *The Elements of Statistical Learning*; Kuncheva: *Combining Pattern Classifiers*; Hájek, Havránek: *Mechanizing Hypothesis Formation*; Ricci et al.: *Recommender Systems Handbook*.

Obsah

I	Dobývání znalostí	3
1	Základní paradigmata dobývání znalostí	3
1.1	Motivace a vymezení oboru	3
1.2	Proces KDD	4
1.3	Proces dobývání znalostí očima manažera	4
1.4	Typy úloh dobývání znalostí	4
1.5	Typy dat	5
1.6	Metodologie dobývání znalostí	6
1.7	Shrnutí k zkoušce	7
2	Příprava dat	8
2.1	Proč je příprava dat klíčová	8
2.2	Čištění dat	8
2.2.1	Chybějící hodnoty	8
2.2.2	Šum a odlehlé hodnoty	8
2.3	Integrace dat	9
2.4	Transformace a normalizace	9
2.4.1	Proč normalizovat	9
2.4.2	Převod typů atributů	10
2.5	Diskretizace numerických atributů	10
2.5.1	Nesupervizované metody	11

2.5.2	Supervizované metody	11
2.5.3	Seskupování hodnot kategoriálních atributů (algoritmus KEX)	12
2.6	Redukce dat	12
2.6.1	Příliš mnoho objektů	12
2.6.2	Příliš mnoho atributů	12
2.7	Shrnutí k zkoušce	13
3	Výběr atributů a analýza relevance	14
3.1	Motivace a základní dělení	14
3.2	Kontingenční tabulky jako základ filtračních kritérií	14
3.2.1	Test χ^2	15
3.2.2	Fisherův exaktní test	15
3.3	Kritéria založená na entropii	16
3.4	Korelace numerických atributů	17
3.5	Postupy prohledávání prostoru podmnožin	17
3.6	Shrnutí k zkoušce	18
4	Asociační pravidla a frekventované vzory	19
4.1	Analýza nákupního košíku	19
4.2	Základní pojmy a míry	19
4.3	Hledání frekventovaných množin: Apriori	20
4.4	Hlavní kroky MBA a prořezávání	22
4.5	FP-Growth a FP-strom	24
4.6	Uzavřené a maximální frekventované množiny	25
4.7	Vícenásobný minimální support (MS-Apriori)	25
4.8	Třídní asociační pravidla (CAR)	28
4.9	Dolování sekvenčních vzorů	28
4.10	Rozšíření: vícúrovňová a kvantitativní pravidla	29
4.11	Korelační analýza a míry zajímavosti	30
4.12	Shrnutí k zkoušce	30
5	Učení s učitelem: klasifikace a predikce	32
5.1	Formulace úlohy	32
5.2	Rozhodovací stromy	32
5.2.1	Princip	32
5.2.2	Obecný algoritmus TDIDT	33
5.2.3	Kritéria dělení	33
5.2.4	Převod stromu na pravidla	35
5.2.5	Faktory důležité pro indukci stromu	36
5.2.6	Algoritmus ID3	36
5.2.7	Algoritmus C4.5 a C5.0	37
5.2.8	Algoritmus CART	38
5.2.9	Algoritmus CHAID	38
5.3	Bayesovská klasifikace	39
5.3.1	Bayesova věta a MAP hypotéza	39
5.3.2	Naivní bayesovský klasifikátor	40
5.3.3	Bayesovské sítě	41
5.4	Klasifikace založená na pravidlech	42
5.5	Líné učení: metoda k nejbližších sousedů	43
5.6	Support Vector Machines	43
5.6.1	Základní formulace	43
5.6.2	Duální formulace a support vektory	44
5.6.3	Měkký okraj	44

5.6.4	Jádrový trik	45
5.7	Neuronové sítě pro dobývání znalostí	46
5.7.1	Perceptron a vícevrstvá síť	46
5.7.2	Extreme Learning Machines	46
5.8	Regresní a diskriminační modely	47
5.8.1	Lineární regrese	47
5.8.2	Logistická regrese	48
5.8.3	Diskriminační analýza	49
5.9	Ensemble metody	49
5.9.1	Bagging	49
5.9.2	Náhodné lesy	50
5.9.3	Boosting a AdaBoost	50
5.10	Srovnání klasifikačních metod	52
5.11	Shrnutí k zkoušce	52
6	Vyhodnocování modelů	54
6.1	Kritéria hodnocení klasifikačních metod	54
6.2	Metody odhadu chyby	54
6.3	Matice záměn a odvozené míry	55
6.4	Skórování, řazení a lift analýza	56
6.5	ROC křivka a AUC	57
6.6	Přeučení a srovnávání klasifikátorů	58
6.7	Shrnutí k zkoušce	58
7	Klastrová analýza	60
7.1	Formulace úlohy a míry vzdálenosti	60
7.2	Rozděľující metody: k -means	61
7.3	k -medoids, PAM, CLARA, CLARANS	63
7.4	Hierarchické shlukování	64
7.5	Shlukování založené na hustotě: DBSCAN	65
7.6	Detekce odlehlých hodnot	66
7.7	Další přístupy	67
7.8	Validace shlukování a volba počtu shluků	68
7.9	Srovnání metod shlukování	69
7.10	Shrnutí k zkoušce	69
8	Charakteristická a diskriminační pravidla	71
8.1	Motivace: charakterizace a diskriminace	71
8.2	Koncepční hierarchie	71
8.3	Indukce orientovaná na atributy	72
8.4	Míry t -weight a d -weight	73
8.5	Měření zajímavosti pravidel	74
8.6	Shrnutí k zkoušce	75
9	Reprezentace a vizualizace znalostí	76
9.1	Formy reprezentace znalostí	76
9.2	Vizualizační techniky	77
9.3	Vyhodnocení znalostí z pohledu uživatele	77
10	Analýza sociálních sítí	79
10.1	Vymezení oboru a motivace	79
10.2	Reprezentace sítí	80
10.3	Síla vazeb	80
10.4	Klíčoví hráči: míry centrality	82

10.5	Koheze: míry na úrovni sítě	84
10.6	Vlastnosti reálných komplexních sítí a jejich modely	85
10.7	Modelování vlivu	87
10.7.1	Vliv versus korelace	87
10.8	Homofilie, výběr a sociální vliv	89
10.9	Afiliční sítě a uzávěrové procesy	90
10.10	Schellingův model segregace	92
10.11	Pozitivní a negativní vztahy: strukturní balance	92
10.12	Analýza vazeb: huby, autority a algoritmus HITS	94
10.13	PageRank	96
10.14	Tematické komunity a jejich objevování	100
10.15	Detekce komunit	101
10.15.1	Minimální řez, poměrový a normalizovaný řez	101
10.15.2	Girvan–Newman	102
10.15.3	Louvain	103
10.15.4	Taxonomie metod detekce komunit	103
10.16	Predikce vazeb	106
10.17	Shrnutí k zkoušce	109
11	Praktické využití technik	111
11.1	Typické aplikační oblasti	111
11.2	Analýza nákupního košíku	111
11.3	Kreditní skórování	111
11.4	Detekce podvodů	112
11.5	Segmentace zákazníků a churn	112
11.6	Doporučovací systémy	112
11.7	Webové a textové dolování	112
11.8	Praktické využití analýzy sociálních sítí	113
11.9	Další oblasti	113
11.10	Provozní a etické aspekty nasazení	114
11.11	Shrnutí k zkoušce	114
II	Internet a klasifikační metody	115
12	Tři důležité internetové aplikace klasifikačních metod	115
12.1	Filtrace spamu	115
12.1.1	Klasifikační úlohy vyskytující se při filtraci spamu	115
12.1.2	Klasifikace na základě obsahu zpráv a na základě metainformací	116
12.1.3	Začlenění klasifikace do celkového procesu filtrace	116
12.1.4	Příklady existujících spamových filtrů	117
12.2	Doporučovací systémy	118
12.2.1	Klasifikační úlohy vyskytující se v doporučovacích systémech	118
12.2.2	Klasifikace při obsahovém a při kolaborativním filtrování	118
12.2.3	Příklady existujících doporučovacích systémů	119
12.3	Systémy pro odhalení hrozeb v síti	120
12.3.1	Klasifikační úlohy vyskytující se v systémech pro odhalení hrozeb	120
12.3.2	Příznaky a příklady existujících systémů	120
12.4	Shrnutí k zkoušce	121
13	Základní koncepty týkající se klasifikace	122
13.1	Klasifikace, klasifikátory, počet tříd	122
13.2	Koncepty specifické pro binární klasifikaci	122

13.3	Charakteristiky kvality binární klasifikace	123
13.4	Tvar hranice mezi třídami	124
13.5	Konstrukce klasifikátorů z dat: učení	125
13.6	Klasifikace, regrese a shlukování	125
13.7	Shrnutí k zkoušce	126
14	Hlavní typy klasifikačních metod	127
14.1	Rozdělení metod podle toho, zda hledají hranice mezi třídami	127
14.2	Klasifikátory založené na podobnosti: k -NN	127
14.3	Klasifikátory založené na bodových odhadech pravděpodobnosti	129
14.3.1	Bayesovské klasifikátory	129
14.3.2	Fisherova diskriminační analýza	132
14.4	Klasifikátory hledající hranice pomocí umělých neuronových sítí	133
14.5	Shrnutí k zkoušce	134
15	Kdy dělá klasifikátor nejméně chyb na nových vstupech?	135
15.1	Generalizační schopnost	135
15.2	Hledání klasifikátoru s nejširším pásem jako optimalizační úloha	135
15.3	SVM pro lineárně neseparabilní třídy	136
15.4	Použití SVM v internetových aplikacích	137
15.5	Aktivní učení	138
15.6	Shrnutí k zkoušce	139
16	Kdy je klasifikace srozumitelná uživateli?	141
16.1	Vyjádření klasifikace jazykem formální logiky	141
16.2	Získávání klasifikačních pravidel evolučními algoritmy	142
16.3	Získávání pravidel pomocí observačního kalkulu	144
16.4	Klasifikační stromy	147
16.5	Shrnutí k zkoušce	147
17	Tým zvládne více než jedinec	149
17.1	Spojování více klasifikátorů do týmu	149
17.2	Metody vytváření týmů klasifikátorů	150
17.3	Náhodné lesy	152
17.4	Shrnutí k zkoušce	153

Dobývání znalostí

1 Základní paradigmata dobývání znalostí

1.1 Motivace a vymezení oboru

Organizace shromažďují data v objemech, které daleko přesahují kapacitu člověka je prohlédnout. Klasické dotazování (SQL, OLAP) umí odpovědět na otázku, kterou už umíme položit; *dobývání znalostí* odpovídá na otázku, které jsme položili neuměli — hledá v datech struktury, o jejichž existenci jsme předem nevěděli.

Ilustrativní příklad z přednášky: systém prošel 20 milionů výkonů účtovaných zdravotní pojišťovně, 214 tisíc z nich označil za podivné a 14 tisíc nejpodzřelejších předal k ruční kontrole. Revizní lékaři, kteří jsou schopni prohlédnout řádově stovky případů, z předložených případů okamžitě označili 8 tisíc za podvodné. Stroj tu nenahradil experta — *zaostřil* jeho pozornost.

Dobývání znalostí z databází

Dobývání znalostí z databází (*Knowledge Discovery in Databases*, KDD) je netriviální proces získávání implicitních, dosud neznámých a potenciálně užitečných informací z dat.

Klíčová jsou všechna čtyři slova:

- **netriviální** — nejde o prostý dotaz ani agregaci;
- **implicitní** — znalost v datech není explicitně zapsána;
- **dosud neznámá** — výsledek, který každý zná, je bezcenný;
- **potenciálně užitečná** — musí vést k akci nebo rozhodnutí.

Termín *dobývání dat* (*data mining*, DM) se v úzkém pojetí vztahuje pouze k jednomu kroku procesu KDD (vlastní aplikaci analytických algoritmů), v běžné praxi se však oba pojmy používají zaměnitelně. V komerčním kontextu se obor prolíná s pojmy *business intelligence* (BI) a *big data*. Obor se konstituoval v 90. letech 20. století na průsečíku tří disciplín:

- **umělá inteligence** — zejména metody strojového učení (indukce pravidel, stromy, neuronové sítě);
- **databázové systémy** — ukládání a efektivní přístup k velkým datovým souborům, datové sklady, vyhledávání informací;
- **statistika** — modelování a analýza závislostí nalezených v datech, testování hypotéz.

Čtvrtou — a v praxi rozhodující — složkou je nutnost využít zpracovaná data k podpoře (strategického) rozhodování v podniku. Bez této složky se dobývání znalostí redukuje na akademické cvičení.

Vztah k datovým skladům a OLAP. Typickým zdrojem dat pro KDD je *datový sklad* (*data warehouse*) — integrovaná, subjektivě orientovaná, časově variantní a stálá databáze konsolidovaná z provozních systémů. Data se v něm organizují do *datové kostky*: *dimenze* (čas, produkt, region, ...) × *míry* (tržby, počty kusů); v relační implementaci jde o *hvězdicové schéma* (tabulka faktů obklopená tabulkami dimenzí), fyzicky MOLAP (multidimenzionální uložení) nebo ROLAP (relační). **OLAP** (*On-Line Analytical Processing*) nad kostkou umožňuje agregační operace *roll-up* / *drill-down* a řezy *slice* & *dice* — odpovídá ale jen na otázky, které si analytik umí předem položit. Dobývání znalostí jde o krok dál: hledá vzory, o nichž předem nevíme, že v datech jsou (souvislost OLAP generalizace s AOI viz kap. 8).

1.2 Proces KDD

Dobývání znalostí je **interaktivní a iterativní** proces: výsledky jednoho kroku běžně vedou k návratu ke krokům předchozím. Klasické schéma (Fayyad et al.) má pět fází:

Fáze	Vstup → výstup	Náplň
Výběr (<i>selection</i>)	původní data → vybraná data	výběr relevantních záznamů a atributů z datového skladu
Předzpracování (<i>pre-processing</i>)	vybraná data → předzpracovaná data	čištění, chybějící hodnoty, šum, odlehlé hodnoty, integrace zdrojů
Transformace (<i>transformation</i>)	předzpracovaná data → transformovaná data	normalizace, diskretizace, odvozené atributy, redukce dimenze
Vlastní dobývání (<i>data mining</i>)	transformovaná data → vzory	aplikace algoritmů (stromy, asociace, klastry, ...)
Interpretace (<i>interpretation/evaluation</i>)	vzory → znalost	vyhodnocení z pohledu koncového uživatele, vizualizace, nasazení

Cílem prvních tří fází (souhrnně *příprava dat*) je z dat uložených ve složité struktuře (např. datovém skladu) vybudovat **jednu tabulku** obsahující relevantní údaje o zkoumaných objektech (klientech banky, zákaznících, ...). Tento krok typicky spotřebuje 60–80 % celkového času projektu.

1.3 Proces dobývání znalostí očima manažera

Přednáška uvádí sedmikrokový pohled, který zdůrazňuje organizační stránku (impulsem je „aktuální problém“, výstupem „znalost k jeho řešení“):

1. **Sestavit tým odborníků** — experti na zkoumanou oblast, na data a na metody KDD. Žádná z těchto rolí není zastupitelná.
2. **Specifikovat problém** — převést obchodní zadání do kontextu dobývání znalostí („zvýšit zisk“ → „predikovat pravděpodobnost odchodu zákazníka“).
3. **Získat všechna dostupná data** — může vést k přeformulování problému; hraje roli kvalita databází (data archivovaná v různých systémech) i externí data popisující prostředí analyzovaných procesů (roční období, reklamní kampaně, politická situace, počasí).
4. **Vybrat metodu / metody** — často je nutné kombinovat několik metod (klasifikace, explorativní analýza, asociační pravidla, rozhodovací stromy, genetické algoritmy, bayesovské sítě, neuronové sítě) plus metody vizualizační.
5. **Předzpracovat data** — převod do formy vyžadované zvolenými metodami (odstranění odlehlých hodnot, doplnění chybějících hodnot); výpočetně nákladné.
6. **Dolovat** — opakovaná aplikace metod; výsledky předchozích běhů ovlivňují parametry běhů následujících, typy metod se kombinují podle dílčích výsledků.
7. **Interpretovat výsledky** — část výsledků bývá nezajímavá nebo triviální, část je nutné prezentovat srozumitelněji; vhodné je shrnutí do analytické zprávy, výstupem může být i provedení konkrétní akce.

1.4 Typy úloh dobývání znalostí

Přednáška rozlišuje tři typy úloh podle vztahu nalezené znalosti k cílovému konceptu:

Typ úlohy	Cíl	Charakter výsledku
Klasifikace / predikce	najít znalost umožňující zařadit nové případy, resp. odhadnout budoucí vývoj entity	preferujeme <i>přesnost pokrytí</i> před jednoduchostí; výsledkem je hodně znalosti, obtížně interpretovatelné
Popis (charakterizace, profil)	najít dominantní strukturu či vztahy obsažené v datech	požadujeme <i>srozumitelnost</i> při plném pokrytí konceptu; výsledkem je menší množství méně přesné znalosti
Hledání „nugetů“	najít zajímavou (novou, překvapivou) znalost	znalost <i>nemusí</i> pokrývat celý koncept; hodnotí se překvapivost

Příklady predikce: předpověď počasí, predikce ceny akcií, predikce spotřeby elektřiny. Příklady popisu: profil bonitního klienta. Příklady nugetů: neobvyklá kombinace zboží v košíku, podezřelý vzorec transakcí.

Souběžně se používá dělení podle Han–Kamber:

Deskriptivní vs. prediktivní úlohy

Deskriptivní (*descriptive*) úlohy charakterizují obecné vlastnosti dat: charakterizace a diskriminace, asociační a korelační analýza, shlukování, detekce odlehlých hodnot, analýza vývoje.

Prediktivní (*predictive*) úlohy provádějí inferenci nad daty za účelem predikce: klasifikace (diskrétní cílový atribut), regrese (spojitý cílový atribut), predikce časových řad.

Ortogonální dělení podle dostupnosti cílové proměnné: *učení s učitelem* (*supervised*) — klasifikace, regrese; *učení bez učitele* (*unsupervised*) — shlukování, asociační pravidla, detekce odlehlých hodnot; *poloučené* (*semi-supervised*) — malá označovaná a velká neoznačovaná část.

1.5 Typy dat

Podle typu atributu (určuje, jaké operace jsou smysluplné a jaké metody lze použít):

Škála	Přípustné operace	Příklad	Statistika
Nominální (<i>nominal</i>)	$=, \neq$	barva, druh ovoce	modus, četnosti
Ordinální (<i>ordinal</i>)	$=, <$	známka, {nízký, střední, vysoký}	medián, kvantily
Intervalová (<i>interval-scaled</i>)	$+, -$	teplota ve °C, datum	průměr, směr. odchylka
Poměrová (<i>ratio-scaled</i>)	$\times, /$	příjem, hmotnost, četnost	geometrický průměr

U intervalové škály je klíčové, že *intervaly mají v celém rozsahu stejnou důležitost*: rozdíl věku 10 a 20 let je týž jako mezi 40 a 50 lety. U poměrové škály navíc existuje absolutní nula, takže má smysl poměr; poměrové atributy s exponenciálním chováním (např. počet mikroorganismů Ae^{Bt}) se před dalším zpracováním logaritmují a pak se s nimi zachází jako s intervalovými.

Podle struktury datového souboru:

- **Tabulková / relační data** — objekty $\times$ atributy; standardní vstup většiny metod.
- **Transakční data** — proměnná délka záznamu (nákupní košík); vstup asociačních pravidel.
- **Temporální data** (*temporal*) — např. časové řady kurzů akcií. Typickou úlohou je predikce budoucích hodnot; řada se transformuje do tabulky posuvným oknem: vstupy $y(t_0), y(t_1), y(t_2), y(t_3) \rightarrow$ výstup $y(t_4)$; $y(t_1), \dots, y(t_4) \rightarrow y(t_5)$ atd.
- **Prostorová data** (*spatial*) — geografické informační systémy; implicitní relace sousedství (hodnoty atributů sousedních objektů se od sebe příliš neliší), užitečné zejména pro interpretaci.

- **Strukturní data** (*structural*) — např. chemické sloučeniny zapsané řetězci SMILES nebo soubory faktů v Prologu; úlohou je třeba klasifikace látek podle struktury.
- **Textová a webová data, multimédia, grafy a sítě** — vyžadují specifickou reprezentaci (vektorový model, grafové modely — viz kap. 10).

1.6 Metodologie dobývání znalostí

Cílem metodologie je poskytnout uživatelům jednotný rámec a vést jejich aplikace bez ohledu na obor podnikání — umožnit sdílení a přenos zkušeností a osvědčených postupů z úspěšných projektů. Metodologie vznikají buď u velkých výrobců BI nástrojů (5A od SPSS, SEMMA od SAS Institute), nebo jako neutrální, softwarově nezávislé standardy vytvořené ve spolupráci výzkumných a komerčních institucí (CRISP-DM).

SEMMA (SAS Enterprise Miner)

Použitelná napříč obory (databázový marketing, segmentace trhu, spokojenost a udržení zákazníků, analýza portfolia, analýza rizik, detekce podvodů, predikce bankrotu). Pět fází:

1. **Sample** — výběr dat pro modelování; zahrnuje vzorkování, imputaci a rozdělení na trénovací, testovací a validační množinu.
2. **Explore** — vizuální průzkum dat a jejich redukce (detekce odlehlých hodnot, vícerozměrné histogramy, shlukování, Kohonenovy samoorganizující se mapy).
3. **Modify** — příprava (výběr, tvorba a transformace) objektů, hodnot a proměnných pro modelování.
4. **Model** — aplikace metod (rozhodovací stromy, regresní modely, neuronové sítě).
5. **Assess** — vyhodnocení výsledků z hlediska spolehlivosti a užitečnosti. *Uživatel jim musí rozumět!*

CRISP-DM

CRoss-Industry Standard Process for Data Mining; vývoj financovala iniciativa ESPRIT, konsorcium tvořily ISL (později SPSS, dnes IBM), Teradata, NCR, Daimler AG a pojišťovna OHRA. Cílem bylo navrhnout otevřený, oborově, nástrojově a aplikačně neutrální model procesu — robustní obecný model plus „průvodce“ možnými problémy.

Proces má **6 fází**, jejichž **pořadí není striktní**; výsledky jedné fáze určují volbu dalších kroků a některé kroky je nutné opakovat (proces je typicky zobrazen jako cyklus s vnějším cyklem nasazení a monitoringu).

1. **Porozumění problému** (*Business Understanding*) — určit obchodní cíle (kontext projektu, kritéria úspěchu); posoudit současnou situaci (inventura zdrojů: lidé, informace, software, výpočetní kapacita; předpoklady a omezení; rizika a příležitosti; náklady a přínosy); určit cíle dobývání znalostí a kritéria úspěchu; vytvořit plán projektu.
2. **Porozumění datům** (*Data Understanding*) — sběr počátečních dat; popis dat (množství, vlastnosti, formát); průzkum a vizualizace (rozdělení klíčových atributů, vztahy dvojic atributů, atributy důležitých subpopulací, jednoduché agregace a statistiky); ověření kvality dat (je informace správná, úplná, pokrývá všechny potřebné případy? jsou chybějící hodnoty?).
3. **Příprava dat** (*Data Preparation*) — vytvořit datovou sadu připravenou pro analýzu a modelování; data musí obsahovat informaci relevantní k úloze a ve formě vyžadované algoritmy. Zahrnuje výběr, čištění, konstrukci (odvozené atributy, generované záznamy), integraci (spojení dat), agregaci, formátování. Jednotlivé akce se provádějí iterativně a v různém pořadí.
4. **Modelování** (*Modeling*) — volba techniky (s ohledem na její předpoklady a nastavení parametrů, může vynutit další úpravy dat), návrh testu, sestavení modelu (iterativně, s různými parametry), posouzení modelu.
5. **Vyhodnocení** (*Evaluation*) — vyhodnotit výsledky z pohledu manažera vůči obchodním kritériím úspěchu; schválit modely (s ohledem na rozpočet a čas); revidovat proces; určit další kroky

(seznam možných akcí s odůvodněním).

6. **Nasazení** (*Deployment*) — výsledky se prezentují ve formě snadno interpretovatelné a nasaditelné uživatelem; plán nasazení a jeho monitoring a údržba; závěrečná zpráva a prezentace; revize projektu (dokumentace obtíží a slepých uliček, doporučení pro volbu technik).

ASUM-DM

Hlavní výtkou vůči CRISP-DM je malá podpora projektového řízení. IBM proto v roce 2015 vydala *Analytics Solutions Unified Method for Data Mining / Predictive Analytics* (ASUM-DM), která CRISP-DM zpřesňuje a rozšiřuje. Fáze: **Analyze** (definovat potřeby — funkční i nefunkční atributy jako výkon a použitelnost, získat souhlas), **Design** (definovat komponenty a jejich závislosti, identifikovat zdroje, instalovat vývojové prostředí, upřesnit požadavky), **Configure & Build** (iterativní a inkrementální budování a integrace komponent, plán testování a validace ve více prostředích), **Deploy** (plán provozu a údržby v produkci, komunikace s byznysem), **Operate & Optimize** (údržba a kontrolní body po nasazení). Průřezově přibývá **Project Management**.

Přínosy: minimalizace rizika (kombinace reálné zkušenosti a osvědčených praktik), škálovatelnost a připravenost pro velké organizace, konzistentní projektové řízení a komunikace ve všech fázích, produktově specifické implementační roadmapy.

	SEMMA	CRISP-DM	ASUM-DM
Původ	SAS Institute	ESPRIT / konsorcium	IBM (2015)
Neutralita	vázáno na SAS EM	nástrojově neutrální	vázáno na IBM
Byznys fáze	chybí	ano (fáze 1 a 5)	ano
Projektové řízení	ne	slabé	explicitní, průřezové
Počet fází	5	6	5 + PM

1.7 Shrnutí k zkoušce

- **KDD** = netriviální proces získávání implicitních, dosud neznámých a potenciálně užitečných informací z dat; interaktivní a iterativní.
- **Pět fází KDD**: výběr → předzpracování → transformace → dobývání → interpretace. Příprava dat spotřebuje většinu času.
- **Kořeny oboru**: umělá inteligence (strojové učení), databáze, statistika + potřeba podpory rozhodování.
- **Tři typy úloh** podle přednášky: klasifikace/predikce (přesnost > jednoduchost), popis (srozumitelnost, plné pokrytí), hledání nugetů (překvapivost, nemusí pokrývat).
- **Deskriptivní vs. prediktivní úlohy; s učitelem vs. bez učitele**.
- **Škály atributů**: nominální, ordinální, intervalová, poměrová — určují přípustné operace i metody.
- **Metodologie**: SEMMA (Sample, Explore, Modify, Model, Assess), CRISP-DM (6 fází, pořadí není striktní, cyklické), ASUM-DM (CRISP-DM + projektové řízení).
- Nejčastější doplňující otázka: *proč pořadí fází CRISP-DM není striktní?* — protože výsledky fáze určují další kroky; např. zjištěná kvalita dat vede zpět k přeformulování cíle.

2 Příprava dat

2.1 Proč je příprava dat klíčová

Předzpracování má *klíčový význam pro úspěch aplikace*, je obtížné a časově náročné a výhodou je spolupráce s doménovým expertem. Cílem je:

- vybrat (nebo vytvořit) z dostupných dat ta, která jsou relevantní pro danou úlohu,
- reprezentovat je ve formě vhodné pro zvolený algoritmus,
- výsledným stavem je (jedna) datová tabulka obsahující hodnoty atributů objektů.

Kvalita dat se posuzuje podle rozměrů: *přesnost* (accuracy), *úplnost* (completeness), *konzistence*, *aktuálnost* (timeliness), *důvěryhodnost*, *interpretovatelnost*. Zásada „garbage in, garbage out“ platí bez výjimky: sebelepší algoritmus na špatných datech vytvoří jen sebejistý nesmysl.

2.2 Čištění dat

2.2.1 Chybějící hodnoty

Podle přednášky existuje pět základních strategií:

1. **Ignorovat objekty** s libovolnou chybějící hodnotou (*listwise deletion*). Jednoduché, ale při mnoha attributech ztratíme většinu dat; navíc zavádí zkreslení, pokud chybějící hodnoty nejsou náhodné.
2. **Nahradit novou hodnotou** „nevím“ (*I don't know*). Chybějící hodnota se stane plnohodnotnou kategorií — vhodné, když samotná absence nese informaci (nevyplněný příjem u žádosti o úvěr).
3. **Nahradit existující hodnotou atributu**: nejčastější hodnotou (modus, u numerických průměr či medián), úměrně poměru všech hodnot (náhodně podle empirického rozdělení), nebo libovolnou hodnotou.
4. **Doplnit hodnotu pomocí modelu** — naučit prediktivní model (regrese, rozhodovací strom, k -NN, EM) předpovídající chybějící atribut z ostatních. Nejpresnější, výpočetně nejnáročnější, riskuje zavlčení umělé závislosti mezi atributy.
5. Doplnění **globální konstantou** nebo průměrem **v rámci třídy** (u úloh s učitelem) jako kompromis mezi (3) a (4).

Terminologicky se rozlišuje MCAR (*missing completely at random*), MAR (*missing at random*, závisí na pozorovaných attributech) a MNAR (*missing not at random*, závisí na nepozorované hodnotě samotné) — jen u MCAR je prosté vypuštění nezakreslené.

2.2.2 Šum a odlehlé hodnoty

Šum (*noise*) je náhodná chyba nebo rozptyl v měřené veličině. Techniky vyhlazování:

- **binning** — hodnoty se rozdělí do košů a nahradí průměrem/mediánem/hranicí koše;
- **regrese** — hodnoty se nahradí predikcí regresní funkce;
- **shlukování** — hodnoty mimo shluky se označí za odlehlé;
- **kombinace stroje a člověka** — podezřelé hodnoty se předloží ke kontrole.

Rozpětí, kvartily, mezikvartilové rozpětí, odlehlá hodnota

Rozpětí (*range*) množiny dat je rozdíl nejvyšší a nejnižší hodnoty.

Kvartily Q_1, Q_2, Q_3 dělí data na čtyři části: 25 % pozorování má hodnotu menší než dolní kvartil Q_1 , 50 % pod mediánem Q_2 a 75 % pod horním kvantilem Q_3 .

Mezikvartilové rozpětí (*interquartile range*): $IQR = Q_3 - Q_1$.

Odlehlá hodnota (*outlier*) je extrémní hodnota ležící mimo celkový vzorec dat. Běžné pravidlo: odlehlá je každá hodnota

$$x > Q_3 + 1,5 \cdot IQR \quad \text{nebo} \quad x < Q_1 - 1,5 \cdot IQR.$$

Výpočet kvartilů pro n vzestupně seřazených hodnot (diskrétní data): pro Q_1 dělíme n čtyřmi — je-li $n/4$ celé číslo, bereme střed mezi odpovídajícím a následujícím členem; není-li celé, zaokrouhlíme nahoru a vezmeme odpovídající člen. Pro Q_3 analogicky s $3n/4$. U spojitých dat se místo zaokrouhlení interpoluje.

Příklad — krabicový graf a odlehlé hodnoty

Hladina glukózy v krvi u 30 žen (mmol/l), stonkový diagram (klíč: 2 | 1 znamená 2,1):

2 | 2 2 3 3 5 7 (6)

3 | 1 2 6 7 7 7 8 8 8 8 9 9 9 (13)

4 | 0 0 0 0 4 5 6 7 8 (9)

5 | 1 5 (2)

$n = 30$, $30/4 = 7,5 \rightarrow$ zaokrouhlíme na 8, osmé číslo je 3,2, tedy $Q_1 = 3,2$. Medián $Q_2 = 3,8$.

Pro Q_3 : $3 \cdot 30/4 = 22,5 \rightarrow 23$, 23. číslo je 4,0, tedy $Q_3 = 4,0$.

$IQR = 4,0 - 3,2 = 0,8$; dolní mez $= 3,2 - 1,5 \cdot 0,8 = 2,0$; horní mez $= 4,0 + 1,5 \cdot 0,8 = 5,2$.

Odehlá hodnota je 5,5. Nejvyšší hodnota, která odehlá není, je 5,1 — v krabicovém grafu (*box plot*) k ní vede vous, odehlá hodnota se vyznačí křížkem.

Poznámka: násobek 1,5 je konvence; při použití násobku 1 (jak ukazuje druhý příklad z přednášky se želvami) dostaneme přísnější kritérium. Pro želví samce s $Q_1 = 400$ g, $Q_3 = 580$ g je $IQR = 180$, meze $400 - 180 = 220$ a $580 + 180 = 760$ — žádná z hodnot 400, 260, 550, 640 g není odehlá a největší hmotnost bez označení za outlier je 760 g. Pro samice s $Q_1 = 260$, $Q_3 = 340$, $IQR = 80$ jsou meze 180 a 420, takže odehlé jsou hodnoty 170 g a 440 g.

2.3 Integrace dat

Při *integraci* (*data integration*) spojujeme více zdrojů do jedné tabulky. Hlavní problémy:

- **problém identity entity** (*entity identification problem*) — atribut `cust_id` v jedné databázi odpovídá atributu `customer_number` v druhé; nutná metadata a párování záznamů (*record linkage*);
- **redundance** — atribut odvoditelný z jiných (roční příjem = 12×měsíční); detekuje se korelační analýzou u numerických atributů a χ^2 testem u kategoriálních (viz kap. 3);
- **konflikty hodnot** — různé jednotky (Kč vs. EUR), různé škály hodnocení, různé kódování;
- **duplicity** — tentýž objekt vícekrát, s drobně odlišnými hodnotami.

2.4 Transformace a normalizace

2.4.1 Proč normalizovat

V euklidovském prostoru je standardizace atributů doporučená, aby všechny atributy mohly mít *stejný vliv* na výpočet vzdáleností. Příklad z přednášky: pro body $x_i = (0,1; 20)$ a $x_j = (0,9; 720)$ je

$$\text{dist}(x_i, x_j) = \sqrt{(0,9 - 0,1)^2 + (720 - 20)^2} = 700,000457,$$

tedy vzdálenost je zcela ovládnuta rozdílem $720 - 20 = 700$ ve druhém atributu; první atribut nemá prakticky žádný vliv. Normalizace nutí atributy do společného rozsahu hodnot.

Základní standardizační transformace

Nechť f je atribut a x_{if} jeho hodnota u i -tého objektu.

Desítkové škálování (*decimal scaling*) do $[-1, 1]$: dělíme hodnoty nejmenší mocninou 10, která udrží všechny transformované hodnoty v $[-1, 1]$.

Min-max normalizace (*range standardization*) do $[0, 1]$:

$$\text{range}(x_{if}) = \frac{x_{if} - \min(f)}{\max(f) - \min(f)};$$

do $[-1, 1]$ pak $\text{range}_{[-1,1]}(x_{if}) = 2 \cdot \text{range}(x_{if}) - 1$. Obecně do $[\text{new_min}, \text{new_max}]$: $x' = \frac{x - \min(f)}{\max(f) - \min(f)}(\text{new_max} - \text{new_min}) + \text{new_min}$.

Z-skóre s průměrnou absolutní odchylkou: transformuje hodnoty tak, aby měly nulový průměr a jednotkovou střední absolutní odchylku:

$$m_f = \frac{1}{n}(x_{1f} + \dots + x_{nf}), \quad s_f = \frac{1}{n}(|x_{1f} - m_f| + \dots + |x_{nf} - m_f|), \quad z(x_{if}) = \frac{x_{if} - m_f}{s_f}.$$

Standardizace podle směrodatné odchylky (*std-score*): nulový průměr a jednotková (korigovaná) výběrová směrodatná odchylka:

$$\text{cors}_f = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_{if} - m_f)^2}, \quad \text{std}(x_{if}) = \frac{x_{if} - m_f}{\text{cors}_f}.$$

Použití průměrné absolutní odchylky s_f místo směrodatné odchylky je robustnější vůči odlehlým hodnotám: u s_f se odchylky nepovyšují na druhou, takže odlehlé hodnoty s_f nenafouknou tolik, a jejich z -skóre zůstane výrazně velké (tedy detekovatelné).

Min-max normalizace zachovává tvar rozdělení, ale je citlivá na odlehlé hodnoty (jediný extrém stlačí zbytek dat do úzkého pásma) a nová data mimo původní rozsah vypadnou z $[0, 1]$. Z-skóre je robustnější vůči novým datům, ale nezaručuje ohraničený rozsah.

2.4.2 Převod typů atributů

Nominální na numerické: nechť nominální atribut má v hodnot; vytvoříme v binárních atributů (*one-hot kódování*). Pokud instance nabývá dané hodnoty, příslušný binární atribut je 1, ostatní 0. Výsledné binární atributy lze používat jako numerické s hodnotami 0 a 1.

Příklad: nominální atribut *ovoce* s hodnotami {jablko, pomeranč, hruška} → tři binární atributy *jablko*, *pomeranč*, *hruška*; instance s hodnotou „jablko“ má *jablko*=1, *pomeranč*=0, *hruška*=0.

Ordinální atributy: běžný přístup je zacházet s nimi jako s intervalovými (přiřadit hodnotám pořadová čísla, případně je normalizovat do $[0, 1]$ jako $(r_i - 1)/(M - 1)$). Ztrácíme tím informaci o nerovnoměrnosti rozestupů mezi kategoriemi.

Poměrové atributy: nejprve logaritmická transformace $\log(x_{if})$, pak zacházení jako s intervalovými.

2.5 Diskretizace numerických atributů

Diskretizace = rozdělení oboru hodnot numerického atributu na intervaly. Je nutná pro metody, které pracují jen s kategoriálními atributy (ID3, naivní Bayes v základní variantě, asociační pravidla). Přednáška rozlišuje metody podle:

- **strategie tvorby intervalů** — shora dolů (postupné dělení intervalů) vs. zdola nahoru (postupné slučování);
- **počtu získaných intervalů**;

- **typu** intervalů (ostré vs. fuzzy);
- **kritéria kvality** — minimální klasifikační chyba, entropie, χ^2 test.

2.5.1 Nesupervizované metody

Diskretizace na stejně široké intervaly (*equal-width binning*): rozsah $[\min, \max]$ se rozdělí na k intervalů délky $(\max - \min)/k$. Jednoduché, ale citlivé na odlehlé hodnoty a při šikmém rozdělení vzniknou téměř prázdné koše.

Diskretizace na stejně četné intervaly (*equal-frequency / equal-depth binning*): každý interval obsahuje přibližně n/k objektů. Robustní vůči šikmosti, ale může rozdělit shluk stejných hodnot.

Diskretizace shlukováním: 1-D k -means na hodnotách atributu; hranice intervalů jsou mezi sousedními centroidy.

2.5.2 Supervizované metody

Entropická diskretizace (Fayyad–Irani, používá ji C4.5 pro spojité atributy): rekurzivně shora dolů. Pro každý kandidátní dělicí bod T (typicky střed mezi dvěma sousedními různými hodnotami s různou třídou) spočteme váženou entropii

$$H(S, T) = \frac{|S_1|}{|S|} H(S_1) + \frac{|S_2|}{|S|} H(S_2)$$

a vybereme T minimalizující $H(S, T)$ (ekvivalentně maximalizující informační zisk). Rekursi zastavíme podle kritéria MDL (*minimum description length*): dělení přijmeme jen tehdy, když zisk převáží cenu popisu dělicího bodu.

ChiMerge (Kerber): zdola nahoru. Každá hodnota tvoří vlastní interval; opakovaně spočteme χ^2 pro každou dvojici *sousedních* intervalů (kontingenční tabulka $2 \times m$, kde m je počet tříd) a sloučíme dvojici s *nejnižší* hodnotou χ^2 (tedy tu, jejíž rozdělení tříd se nejméně liší — intervaly jsou „nezávislé“ na třídě, takže hranice mezi nimi je zbytečná). Slučujeme, dokud je minimální χ^2 pod kritickou hodnotou na zvolené hladině významnosti, nebo dokud nedosáhneme cílového počtu intervalů.

Fuzzy diskretizace (procedura *Interval*)

Hranice fuzzy intervalů jsou „rozmazané“. Přednáška uvádí proceduru *Interval*, která hledá vhodné intervaly zřetězením posloupností vstupních hodnot se stejnou (fuzzy) třídou:

3.1. Má-li posloupnost hodnot stejnou třídu, vytvoř z nich interval s charakteristickou funkcí rovnou 1 na celém intervalu.

3.2. Pro každý interval Int_i : patří-li Int_i třídě „? $?$ “, pak

- patří-li sousední intervaly Int_{i-1} a Int_{i+1} téže třídě, spoj $Int_{i-1} \cup Int_i \cup Int_{i+1}$ (charakteristická funkce 1 na celém intervalu);
- jinak vytvoř dva fuzzy intervaly $Int_{i-1} \cup Int_i$ a $Int_i \cup Int_{i+1}$ s charakteristickými funkcemi

$$\mu_1(x) = \frac{Llim_{i+1} - x}{Llim_{i+1} - Ulim_{i-1}}, \quad \mu_2(x) = \frac{x - Ulim_{i-1}}{Llim_{i+1} - Ulim_{i-1}}$$

pro $x \in \langle Ulim_{i-1}, Llim_{i+1} \rangle$ (a rovnou 1 na jádrech obou intervalů).

3.3. Zajisti spojité pokrytí oboru hodnot (mezery mezi intervaly se chápou jako intervaly třídy „? $?$ “).

Jedné numerické hodnotě tak mohou odpovídat *dvě* diskretizované hodnoty, přičemž součet příslušných charakteristických funkcí je roven 1. Vznikají „fuzzy objekty“ s vahou < 1 , přičemž váha objektu je součin hodnot charakteristických funkcí všech atributů: $w(obj) = \prod_i \mu(x_i)$. Rizika: možná ztráta informace a *spory* — objekty se stejnými hodnotami diskretizovaných atributů patří do různých tříd.

2.5.3 Seskupování hodnot kategoriálních atributů (algoritmus KEX)

Má-li kategoriální atribut příliš mnoho hodnot, může být užitečné je slučovat (na základě charakteristik spočtených na trénovacích datech).

Cíl: vytvořit skupiny $A(\text{Grp})$ takové, že $P(\text{Class} \mid A(\text{Grp}))$ se *významně liší* od $P(\text{Class})$, a vytvořit tolik skupin, kolik je tříd plus jedna navíc („?“).

Hlavní cyklus:

1. vytvoř uspořádaný seznam uvažovaných hodnot atributu;
2. pro každou hodnotu: (2.1) spočti četnosti výskytu objektů s touto hodnotou pro jednotlivé třídy; (2.2) přiřaď hodnotě kód třídy procedurou *Assign*;
3. vytvoř intervaly hodnot procedurou *Group*.

Procedura *Assign*: patří-li všechny objekty s danou hodnotou téže třídy, přiřaď hodnotě kód této třídy; jinak — liší-li se rozdělení tříd pro danou hodnotu významně (podle χ^2 testu) od apriorního rozdělení tříd — přiřaď kód nejčetnější třídy, jinak přiřaď kód „?“.

Procedura *Group*: vytvoř skupinu z hodnot, jimž byl přiřazen stejný kód třídy.

2.6 Redukce dat

2.6.1 Příliš mnoho objektů

Dávkové zpracování je pak problematické. Přednáška nabízí tři cesty:

- použít jen **vzorek** objektů vybraný z dat;
- ukládat data způsobem, který umožní přístup ke všem objektům **bez nutnosti držet je v operační paměti**;
- postavit **více modelů** nad podmnožinami objektů a modely následně zkombinovat (viz ensemble metody, kap. 5.9).

Klíčová je **reprezentativnost** vybraných dat: vybrané objekty by měly co nejlépe zachytit povahu dat (např. shodu rozdělení hodnot atributů na vzorku a na celých datech). Typy vzorkování: prostý náhodný výběr bez/s vrácením, *stratifikovaný* (zachová poměr tříd), *klastrový*.

Nevyvážené třídy v původních datech vedou k tendenci preferovat majoritní třídu. Řešení:

- různé **váhy** pro různé typy chybné klasifikace (cenově citlivé učení);
- výběr příkladů z různých tříd s **různou pravděpodobností** (převzorkování minoritní / podvzorkování majoritní třídy, SMOTE);
- vhodné metriky (viz kap. 6, kde přesnost selhává: při 1 % průniků dosáhneme 99 % přesnosti tím, že neuděláme nic).

Souvisejícím nástrojem je **křížová validace**: z dat vybereme část pro trénink a jinou (odlišnou) část pro testování. Pomáhá i efektivnější ukládání a paralelní zpracování.

2.6.2 Příliš mnoho atributů

Dvě cesty:

- **Redukce expertem** — doménový expert vyřadí irelevantní atributy.
- **Automatická redukce**:
 - **transformací** — z existujících atributů vytvoříme méně nových (PCA); nové atributy vznikají jako lineární kombinace původních. Nevýhody: je nutné dodat hodnoty *všech* původních atributů a nové atributy nemusí mít jasnou interpretaci.
 - **výběrem** (*feature selection*) — z existujících atributů vybereme jen ty nejdůležitější (viz kap. 3).

Analýza hlavních komponent (PCA)

Principal Component Analysis hledá ortogonální báze prostoru tak, aby první osa zachycovala maximum rozptylu dat, druhá maximum zbývajících rozptylu při kolmosti na první atd.

Postup: (1) centruj data (odečti průměr, obvykle i standardizuj); (2) spočti kovarianční matici $\Sigma = \frac{1}{n-1} X^T X$; (3) najdi její vlastní čísla $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m$ a vlastní vektory $u_1, \dots, u_m$; (4) prvních p vlastních vektorů tvoří novou bázi, projekce $Z = XU_p$ je redukovaná reprezentace.

Podíl vysvětleného rozptylu prvních p komponent je $\sum_{i \leq p} \lambda_i / \sum_i \lambda_i$; p se volí např. tak, aby tento podíl dosáhl 90–95 %, nebo podle „lokty“ v grafu vlastních čísel (*scree plot*).

Vlastnosti: PCA je citlivá na škálování (proto nutná standardizace), zachycuje jen *lineární* závislosti, je neřízená (nepoužívá cílový atribut — na rozdíl od LDA), komponenty jsou nekorelované.

Další techniky redukce dimenze: SVD, faktorová analýza, náhodné projekce (lemma Johnsona a Lindenstrausse), nelineární metody (t-SNE, UMAP — vhodné spíše pro vizualizaci než pro modelování), autoenkodéry.

Redukce počtu hodnot (*numerosity reduction*): parametrické (regresní model místo dat), neparametrické (histogramy, shlukování, vzorkování). **Komprese dat**: waveletová transformace, PCA jako ztrátová komprese.

2.7 Shrnutí k zkoušce

- **Cíl předzpracování:** vybrat relevantní data a převést je do jedné tabulky ve formě vhodné pro algoritmus. Časově nejnáročnější fáze.
- **Chybějící hodnoty:** ignorovat objekt / hodnota „nevím“ / nejčastější hodnota / úměrně poměru / doplnit modelem.
- **Odlehle hodnoty:** $x > Q_3 + 1,5 \text{ IQR}$ nebo $x < Q_1 - 1,5 \text{ IQR}$; umět spočítat kvartily a nakreslit box plot.
- **Normalizace:** min-max do $[0, 1]$; z -skóre s průměrnou absolutní odchylkou s_f ; std-skóre s korigovanou směrodatnou odchylkou $\text{cor } s_f$; desítkové škálování. *Proč?* Aby všechny atributy měly stejný vliv na vzdálenost — příklad $(0, 1; 20)$ vs. $(0, 9; 720)$.
- **Typy atributů:** nominální $\rightarrow$ v binárních atributů; ordinální $\rightarrow$ jako intervalové; poměrové $\rightarrow$ nejdřív logaritmus.
- **Diskretizace:** stejně široké / stejně četné intervaly (bez učitele), entropická (Fayyad–Irani, shora dolů) a ChiMerge (zdola nahoru, slučuje dvojice s *nejnižším* χ^2) s učitelem; fuzzy diskretizace (procedura *Interval*, váhy objektů jako součin charakteristických funkcí); KEX pro seskupování kategoriálních hodnot.
- **Redukce:** vzorkování (reprezentativnost, stratifikace), PCA (lineární kombinace, vlastní vektory kovarianční matice, ztráta interpretovatelnosti), výběr atributů.
- **Nevyvážené třídy:** různé váhy chyb, různé pravděpodobnosti výběru, vhodné metriky.

3 Výběr atributů a analýza relevance

3.1 Motivace a základní dělení

Hledáme ty atributy, které nejlépe přispívají ke správné klasifikaci objektů. Důvody: menší modely jsou srozumitelnější, rychleji se učí i vyhodnocují, méně trpí přeučením a *prokletím dimenzionality* (*curse of dimensionality*) — v mnoha dimenzích jsou si všechny body přibližně stejně vzdálené, takže metody založené na vzdálenosti přestávají fungovat.

Filter, wrapper, embedded

Filtrační metody (*filter*) — ke každému atributu spočtou charakteristiku odrážející jeho přínos ke klasifikaci; výběr probíhá *nezávisle* na použitém klasifikátoru. Rychlé, škálovatelné, ale ignorují interakce s modelem.

Obalové metody (*wrapper*) — pomocí algoritmu strojového učení se postaví model nad podmnožinou atributů a model se vyhodnotí; použije se nejlepší z postavených modelů (s vybranými atributy). Dělí se na:

- „zdola nahoru“ (*bottom-up, forward selection*) — začneme od modelů postavených nad jednotlivými atributy a další atributy postupně přidáváme;
- „shora dolů“ (*top-down, backward elimination*) — začneme s modelem nad úplnou množinou atributů a nepotřebné postupně odstraňujeme.

Přesnější (zohlední interakce i vlastnosti klasifikátoru), ale výpočetně drahé a hrozí přeučení výběru.

Vestavěné metody (*embedded*) — výběr je součástí učení modelu: L1 regularizace (LASSO), volba dělicích atributů v rozhodovacím stromu, důležitost atributů v náhodném lese.

3.2 Kontingenční tabulky jako základ filtračních kritérií

Pro vstupní atribut A s hodnotami $v_1, \dots, v_R$ a cílový atribut C s hodnotami $v_1, \dots, v_S$ sestavíme kontingenční tabulku četností a_{kl} kombinace $(A(v_k) \wedge C(v_l))$:

	$C(v_1)$	$C(v_2)$	$\dots$	$C(v_S)$	Σ
$A(v_1)$	a_{11}	a_{12}	$\dots$	a_{1S}	r_1
$A(v_2)$	a_{21}	a_{22}	$\dots$	a_{2S}	r_2
$\vdots$					$\vdots$
$A(v_R)$	a_{R1}	a_{R2}	$\dots$	a_{RS}	r_R
Σ	s_1	s_2	$\dots$	s_S	n

kde $r_k = \sum_l a_{kl}$, $s_l = \sum_k a_{kl}$, $n = \sum_k \sum_l a_{kl}$ jsou řádkové a sloupcové součty (*marginální hodnoty*) a

$$e_{kl} = \frac{r_k \cdot s_l}{n}$$

je *očekávaná* četnost kombinace při předpokladu nezávislosti A a C . Odhady pravděpodobností: $P(A(v_k) \wedge C(v_l)) = a_{kl}/n$, $P(A(v_k)) = r_k/n$, $P(C(v_l)) = s_l/n$.

3.2.1 Test χ^2

χ^2 statistika

$$\chi^2 = \sum_{k=1}^R \sum_{l=1}^S \frac{(a_{kl} - e_{kl})^2}{e_{kl}} = n \sum_{k=1}^R \sum_{l=1}^S \frac{(a_{kl} - \frac{r_k s_l}{n})^2}{r_k s_l} \quad \text{alternativně} \quad \chi^2 = \sum_{k=1}^R \sum_{l=1}^S \frac{a_{kl}^2}{e_{kl}} - n.$$

Za platnosti nulové hypotézy o nezávislosti

$$H_0 : P(A = v_k \wedge C = v_l) = P(A = v_k)P(C = v_l) \quad \forall k, l$$

má statistika χ^2 rozdělení s $(R-1)(S-1)$ stupni volnosti. Je-li hodnota statistiky $\geq$ kritické hodnotě $\chi^2_{(R-1)(S-1)}(\alpha)$ na zvolené hladině významnosti α , nulovou hypotézu zamítáme a přijímáme alternativní hypotézu o závislosti.

Jako kritérium výběru atributů platí: **čím vyšší hodnota, tím lepší atribut.**

Příklad — χ^2 test na čtyřpolní tabulce

		Poskytnutí úvěru		
		ANO	NE	Σ
Výše příjmu	vysoký	50	0	50
	nízký	30	40	70
Σ		80	40	120

Očekávané četnosti: $e_{11} = \frac{50 \cdot 80}{120} = 33,3$; $e_{12} = \frac{50 \cdot 40}{120} = 16,7$; $e_{21} = \frac{70 \cdot 80}{120} = 46,7$; $e_{22} = \frac{70 \cdot 40}{120} = 23,3$. Rozdíly jsou po řadě $+16,7, -16,7, -16,7, +16,7$.

$$\chi^2 = \frac{16,7^2}{33,3} + \frac{16,7^2}{16,7} + \frac{16,7^2}{46,7} + \frac{16,7^2}{23,3} = 42,857.$$

Pro $(2-1)(2-1) = 1$ stupeň volnosti a $\alpha = 0,05$ je kritická hodnota $\chi^2_{(1)}(0,05) = 3,84$. Protože $42,857 > 3,84$, **mezi výší příjmu a poskytnutím úvěru existuje závislost.**

3.2.2 Fisherův exaktní test

χ^2 test lze použít jen pro dostatečně velké četnosti — podmínkou je $(r_k \cdot s_l)/n \geq 5$ pro všechna k, l . Pro čtyřpolní tabulky s nízkými četnostmi je možné použít *Fisherův exaktní test*. Pravděpodobnost, že čtyřpolní tabulka má právě četnosti a_{kl} při daných marginálech, je

$$p = \frac{r_1! r_2! s_1! s_2!}{n! a_{11}! a_{12}! a_{21}! a_{22}!} = \frac{\binom{r_1}{a_{11}} \binom{r_2}{a_{21}}}{\binom{n}{s_1}}.$$

Pravděpodobnosti se sečtou přes všechny „alespoň tak extrémní“ tabulky. Zvolíme-li za sledovanou buňku tu s *nejnižší* četností (označme ji a ; marginály jsou pevné, takže tabulku určuje jediné číslo), sčítáme

$$P = \sum_{i=0}^a \frac{r_1! r_2! s_1! s_2!}{n! (a-i)! (a_{12}+i)! (a_{21}+i)! (a_{22}-i)!} = \sum_{j=0}^a P(j),$$

kde $P(j)$ je pravděpodobnost tabulky, v níž sledovaná buňka nabývá hodnoty j . Je-li $P \leq \alpha$, nulovou hypotézu na hladině α zamítáme.

Příklad z přednášky: tabulka D_A : (5, 13), D_B : (1, 11) s marginály $r = (18, 12)$, $s = (6, 24)$, $n = 30$. Sledovanou buňkou je D_B /první sloupec s **pozorovanou hodnotou 1**; ta může nabýt hodnot $0, \dots, 6$ a příslušné pravděpodobnosti jsou

$$P(0) = \frac{12! 18! 6! 24!}{30! 0! 12! 6! 12!} = 0,031, \quad P(1) = 0,173, \quad P(2) = 0,340, \quad P(3) = 0,302,$$

$$P(4) = 0,128, \quad P(5) = 0,024, \quad P(6) = 0,002 \quad (\text{kontrola: součet} = 1,000).$$

Extrémnější než pozorovaný stav je jen $j = 0$, takže testová hodnota je $P = P(1) + P(0) = 0,173 + 0,031 = 0,204 > 0,05$, tedy **přijímáme nulovou hypotézu o nezávislosti**. (Ekvivalentně: $P(j)$ je hypergeometrické rozdělení $\binom{12}{j} \binom{18}{6-j} / \binom{30}{6}$.)

3.3 Kritéria založená na entropii

Entropie a podmíněná entropie atributu

Entropie (entropy) je míra neuspořádanosti („překvapení“) v systému:

$$H = - \sum_{j=1}^m p_j \log_2 p_j,$$

kde p_j je pravděpodobnost (relativní četnost) výskytu třídy j a m počet tříd. Konvence $0 \log_2 0 = 0$. Převod báze: $\log_2 x = \log_{10} x / 0,301$.

Entropie pro jednu hodnotu v_k atributu A (nad objekty pokrytými kategorií $A(v_k)$):

$$H(A(v_k)) = - \sum_{l=1}^S \frac{a_{kl}}{r_k} \log_2 \frac{a_{kl}}{r_k}.$$

Vážená průměrná entropie atributu A :

$$H(A) = \sum_{k=1}^R \frac{r_k}{n} H(A(v_k)).$$

Jako kritérium výběru: **čím nižší hodnota, tím lepší atribut**.

Informační zisk, poměrný informační zisk, Gini index

Informační zisk (information gain) — očekávané snížení entropie po rozdělení dat podle atributu A :

$$\text{GAIN}(S, A) = \text{ENTROPY}(S) - \sum_{v \in \text{VALUES}(A)} \frac{|S_v|}{|S|} \text{ENTROPY}(S_v).$$

Nevýhoda: preferuje atributy s mnoha hodnotami (v extrému rodné číslo, které rozdělí data na jednoprvkové podmnožiny s nulovou entropií a maximálním ziskem, ale je bezcenné).

Poměrný informační zisk (gain ratio, C4.5) tuto tendenci koriguje dělením tzv. vnitřní informací (*split information*):

$$\text{GAIN_RATIO}(S, A) = \frac{\text{GAIN}(S, A)}{- \sum_{v \in \text{VALUES}(A)} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}}.$$

Gini index (CART) — pro atribut s r hodnotami $v_1, \dots, v_r$, n objekty a m třídami:

$$G = \sum_{i=1}^r \frac{n_i G(v_i)}{n}, \quad G(v_i) = 1 - \sum_{j=1}^m p_j^2, \quad \sum_{i=1}^r n_i = n,$$

kde n_i je počet objektů s hodnotou v_i a p_j pravděpodobnost třídy j mezi těmito n_i objekty. **Nižší hodnoty G znamenají lepší diskriminaci.**

Gini index a entropie se chovají velmi podobně (pro dvě třídy nabývají obě maxima při $p = 0,5$); Gini je výpočetně levnější (nepotřebuje logaritmus), entropie o něco silněji penalizuje velmi nevyvážená rozdělení.

Vzájemná informace a informační míra závislosti

Vzájemná informace (*mutual information*):

$$MI(A, C) = \sum_{k=1}^R \sum_{l=1}^S P(A(v_k) \wedge C(v_l)) \log_2 \frac{P(A(v_k) \wedge C(v_l))}{P(A(v_k)) P(C(v_l))} = \frac{1}{n} \sum_{k=1}^R \sum_{l=1}^S a_{kl} \log_2 \frac{n a_{kl}}{r_k s_l}.$$

Informační míra závislosti (*information measure of dependence*) je normalizovaná vzájemná informace:

$$ID(A, C) = \frac{MI(A, C)}{H(C)} = \frac{MI(A, C)}{-\sum_{l=1}^S \frac{s_l}{n} \log_2 \frac{s_l}{n}} \in [0, 1].$$

Čím vyšší hodnota, tím lepší atribut. Platí $MI(A, C) = H(C) - H(C | A)$, takže vzájemná informace je totéž co informační zisk.

3.4 Korelace numerických atributů

Pearsonův korelační koeficient

Posouzení „množství“ lineární závislosti mezi numerickými veličinami:

$$\rho(x, y) = \frac{S_{xy}}{\sqrt{S_x^2 S_y^2}}, \quad \bar{x} = \frac{\sum_i x_i}{n}, \quad \bar{y} = \frac{\sum_i y_i}{n},$$

kde korigovaná výběrová kovariance a rozptyly jsou

$$S_{xy} = \frac{1}{n-1} \sum_i (x_i - \bar{x})(y_i - \bar{y}), \quad S_x^2 = \frac{1}{n-1} \sum_i (x_i - \bar{x})^2, \quad S_y^2 = \frac{1}{n-1} \sum_i (y_i - \bar{y})^2.$$

Platí $\rho \in [-1, 1]$; $|\rho| = 1$ znamená dokonalou lineární závislost, $\rho = 0$ *nekorelovanost* (nikoli nezávislost — $y = x^2$ na symetrickém intervalu má $\rho = 0$).

Korelace se používá dvojím způsobem: (a) jako míra relevance atributu vůči numerickému cíli, (b) jako detektor *redundance* mezi dvojicemi vstupních atributů — ze silně korelované dvojice stačí ponechat jeden. Pro ordinální data se používá Spearmanův korelační koeficient (Pearson nad pořadími).

3.5 Postupy prohledávání prostoru podmnožin

Atributy lze uspořádat podle hodnot uvažovaných kritérií a vybrat určitý počet nejlepších (*ranking*). Přednáška ale upozorňuje na zásadní omezení: **každý atribut je vyhodnocován samostatně**, takže vzájemný vliv několika atributů na přesnost klasifikace se obtížně zachytí. Řešením je počítat hodnotu kritéria pro *množinu* atributů (pro všechny jejich kombinace) — prostor 2^m podmnožin je však nutné prohledávat heuristicky:

Strategie	Popis
Dopředný výběr (<i>forward selection</i> , bottom-up)	Start s prázdnou množinou. V každém kroku přidej atribut, který nejvíce zlepší kritérium. Konec, když už žádný nepomůže. Levné, ale nenajde dvojice atributů užitečné jen společně (XOR).
Zpětná eliminace (<i>backward elimination</i> , top-down)	Start s úplnou množinou. V každém kroku odeber atribut, jehož odebrání nejméně uškodí. Dražší (modely nad mnoha atributy), ale lépe zachytí interakce.
Obousměrný výběr (<i>stepwise</i>)	Kombinace obou: po každém přidání se zkusí odebrat dříve přidaný atribut.
Rekurzivní eliminace atributů (RFE)	Opakovaně natrénuj model, odeber k atributů s nejnižší důležitostí (např. podle vah SVM).
Heuristické / stochastické prohledávání	Genetické algoritmy, simulované žíhání — pro velmi velké m .

Typický kompromis v praxi: nejprve filtrem odstranit zjevně irelevantní a redundantní atributy (levné), pak nad zbytkem spustit wrapper (drahé, ale přesné) — a výsledek validovat na *oddělené* testovací množině, protože samotný výběr atributů je zdrojem přeučení (musí probíhat uvnitř křížové validace, nikoli před ní).

3.6 Shrnutí k zkoušce

- **Filter** (nezávisle na modelu, rychlé), **wrapper** (model jako černá skříňka, přesné, drahé; bottom-up / top-down), **embedded** (LASSO, stromy, random forest).
- **Kontingenční tabulka**: a_{kl} , marginály r_k, s_l , $e_{kl} = r_k s_l / n$.
- χ^2 : $\sum \frac{(a_{kl} - e_{kl})^2}{e_{kl}}$, $(R - 1)(S - 1)$ stupňů volnosti, čím větší tím lepší; podmínka $r_k s_l / n \geq 5$, jinak Fisherův exaktní test.
- **Entropie** $H(A) = \sum_k \frac{r_k}{n} H(A(v_k))$ — čím menší tím lepší; **information gain** $= H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$; **gain ratio** (dělení split-info, koriguje preferenci mnohohodnotových atributů); **Gini** $G(v) = 1 - \sum p_j^2$ (čím menší tím lepší).
- $MI(A, C) = \text{information gain}$; $ID(A, C) = MI(A, C) / H(C)$ normalizuje do $[0, 1]$.
- **Korelace** pro numerické atributy — relevance i redundance; $\rho = 0$ neznámá nezávislost.
- **Klíčové omezení rankingů**: hodnotí atributy izolovaně; interakce vyžadují prohledávání podmnožin (forward / backward / stepwise).
- Umět spočítat χ^2 na čtyřpolní tabulce a rozhodnout o závislosti (příklad příjem/úvěr $\rightarrow 42,857$ vs. $3,84$).

4 Asociační pravidla a frekventované vzory

4.1 Analýza nákupního košíku

Analýza nákupního košíku (*market basket analysis*, MBA) odpovídá na otázku: **jaké položky se vyskytují společně v košíku?** Výsledky se vyjadřují ve formě pravidel a lze je bezprostředně aplikovat: plánování a uspořádání prodejny, nabídka kuponů a časově omezených slev, „balíčkování“ produktů.

Dobré asociační pravidlo má být:

- **srozumitelné** — jakmile je vztah nalezen, lze jej snadno ověřit;
- **aplikovatelné** — obsahuje užitečnou informaci vedoucí k zásahu.

Naopak nemá být:

- **triviální** — výsledek už každý zná;
- **nevysvětlitelné** — neexistuje pro něj vysvětlení a nevede k akci.

Virtuální položky (*virtual items*) neodpovídají žádnému výrobku ani službě, ale označují např. typ prodejny, kde transakce proběhla. Umožňují srovnání nových a existujících prodejen: (1) poskytneme transakční data z nových prodejen za určitou dobu, (2) přibližně stejné množství dat z existujících prodejen, (3) MBA najde asociační pravidla v obou množinách, (4) zaměříme se zejména na pravidla s virtuálními položkami.

4.2 Základní pojmy a míry

Položka, transakce, frekvenční tabulka

Položka (item) — nabízený produkt nebo služba. *Transakce* obsahuje jednu nebo více položek. *Frekvenční tabulka* udává počet společných výskytů libovolných dvou položek v provedených transakcích (kolikrát byly tyto dva produkty koupeny současně); hodnoty na diagonále odpovídají počtu transakcí obsahujících danou položku.

Příklad (potraviny):

Zákazník	Položky		chléb	máslo	mléko	káva
1	chléb, máslo	chléb	4	3	1	2
2	mléko, chléb, máslo	máslo	3	4	1	2
3	chléb, káva	mléko	1	1	1	0
4	chléb, máslo, káva	káva	2	2	0	3
5	káva, máslo					

Z tabulky je vidět povaha prodeje: *chléb a máslo se kupují spolu; mléko se nikdy nekupuje s kávou.*

Support, confidence, lift

Pravidlo má tvar IF Podmínka THEN Závěr, resp. $i \Rightarrow j$.

Support (podpora) — jak často lze pravidlo použít:

$$\text{support}(r) = \frac{\text{počet transakcí obsahujících } i \text{ a } j}{\text{počet všech transakcí}} \cdot 100 \%$$

Confidence (spolehlivost) — jak spolehlivé jsou výsledky pravidla:

$$\text{confidence}(r) = \frac{\text{počet transakcí obsahujících } i \text{ a } j}{\text{počet transakcí obsahujících } i} \cdot 100 \% = \frac{\text{support}(i \cup j)}{\text{support}(i)}$$

Lift (zdvih) — kolikrát je lepší použít pravidlo pro predikci než jen předpokládat jeho výsledek:

$$\text{lift}(r) = \frac{p(i \wedge j)}{p(i) \cdot p(j)} = \frac{\text{confidence}(i \Rightarrow j)}{p(j)}$$

Platí-li $\text{lift} < 1$: pravidlo je pro predikci *horší než náhodná volba* a **negace závěru** může dát lepší pravidlo (IF Podmínka THEN NOT Závěr). $\text{lift} = 1$ znamená nezávislost, $\text{lift} > 1$ pozitivní korelaci.

Příklad — výpočet support, confidence a lift

Nad transakční databází výše:

Pravidlo 1: *Kupuje-li zákazník chléb, kupuje i máslo.*

$\text{support} = 3/5 \cdot 100 \% = 60 \%$ (transakce 1, 2, 4), $\text{confidence} = 3/4 \cdot 100 \% = 75 \%$ (chléb je ve 4 transakcích).

Pravidlo 2: *Kupuje-li zákazník kávu, kupuje i máslo.*

$\text{support} = 2/5 \cdot 100 \% = 40 \%$, $\text{confidence} = 2/3 \cdot 100 \% = 66 \%$.

Pravidlo 3: *Kupuje-li zákazník mléko, kupuje i máslo.*

$\text{support} = 1/5 \cdot 100 \% = 20 \%$, $\text{confidence} = 1/1 \cdot 100 \% = 100 \%$,

$$\text{lift} = \frac{1/5}{(1/5) \cdot (4/5)} = \frac{0,2}{0,16} = \frac{5}{4} = 1,25.$$

Pravidlo 3 má nejvyšší confidence, ale nejnižší support — opírá se o jedinou transakci. To je typická past: **vysoká confidence sama o sobě nestačí**, je nutné sledovat i support a lift.

Kontrapříklad k confidence. Mějme 10 000 transakcí, z toho 6000 obsahuje kávu, 7500 čaj a 4000 obojí. Pravidlo $\text{čaj} \Rightarrow \text{káva}$ má $\text{confidence} = 4000/7500 = 53 \%$, což vypadá dobře — ale $P(\text{káva}) = 60 \%$, takže $\text{lift} = 0,53/0,60 = 0,89 < 1$: nákup čaje pravděpodobnost nákupu kávy ve skutečnosti *snižuje*.

4.3 Hledání frekventovaných množin: Apriori

Frekventovaná množina položek

Frekventovaná množina položek (*frequent itemset*) je kombinace (konjunkce) kategorií, která dosahuje předem zadané četnosti v datech — podpory **minsup**.

Vlastnost uzavřenosti dolů (*downward closure property*), též *apriori vlastnost*: libovolná podmnožina frekventované množiny je také frekventovaná. Ekvivalentně (kontrapozice): je-li množina neřekventovaná, jsou neřekventované i všechny její nadmnožiny. To je klíčová idea prořezávání.

Algoritmus Apriori (R. Agrawal) hledá kombinace délky k pomocí známých kombinací délky $k - 1$ — **prohledávání do šířky**. Aby byla kombinace délky k vygenerována, musí všechny její podkombinace délky $k - 1$ splňovat požadavek na četnost.

Algoritmus APRIORI

Krok 1: Přiřaď do L_1 všechny kategorie, které dosahují alespoň požadované četnosti.

Krok 2: Polož $k = 2$.

Krok 3: Dokud $L_{k-1} \neq \emptyset$:

3.1 Na základě L_{k-1} a pomocí funkce APRIORI-GEN vygeneruj množinu kandidátů C_k .

3.2 Přiřaď do L_k všechny kombinace z C_k , které dosáhly alespoň požadované četnosti.

3.3 Zvyš k .

Funkce APRIORI-GEN(L_{k-1})

Krok 1: (*spojení*) Pro všechny dvojice kombinací $\text{Comb}_p, \text{Comb}_q$ z L_{k-1} : shodují-li se Comb_p a Comb_q v $k - 2$ kategoriích, přidej $\text{Comb}_p \cup \text{Comb}_q$ do C_k .

Krok 2: (*prořezání*) Pro každou kombinaci Comb z C_k : není-li některá z jejích podkombinací délky $k - 1$ obsažena v L_{k-1} , odstraň Comb z C_k .

Příklad — průchod algoritmem Apriori

Transakční databáze ($n = 5$), $\text{minsup} = 2$ transakce (40 %):

TID	položky
T1	A, C, D
T2	B, C, E
T3	A, B, C, E
T4	B, E
T5	A, B, C, E

Krok $k = 1$. Četnosti: $A:3, B:4, C:4, D:1, E:4$. Položka D nesplňuje $\text{minsup} \rightarrow$ vypadává.

$$L_1 = \{\{A\} : 3, \{B\} : 4, \{C\} : 4, \{E\} : 4\}.$$

Krok $k = 2$. APRIORI-GEN vytvoří $C_2 = \{AB, AC, AE, BC, BE, CE\}$ (dvojice se shodují v $k - 2 = 0$ kategoriích, tedy všechny dvojice). Průchod daty:

$$AB:2, AC:3, AE:2, BC:3, BE:4, CE:3.$$

Všechny splňují minsup , tedy $L_2 = \{AB, AC, AE, BC, BE, CE\}$.

Krok $k = 3$. Spojení dvojic shodujících se v $k - 2 = 1$ položce: $AB+AC \rightarrow ABC$, $AB+AE \rightarrow ABE$, $AC+AE \rightarrow ACE$, $BC+BE \rightarrow BCE$. Prořezání: všechny 2-podmnožiny každého kandidáta musí být v L_2 — pro ABC jsou to AB, AC, BC (všechny v L_2 , kandidát zůstává); totéž platí pro ostatní. Průchod daty:

$$ABC:2 \text{ (T3, T5)}, ABE:2, ACE:2, BCE:3.$$

$$L_3 = \{ABC, ABE, ACE, BCE\}.$$

Krok $k = 4$. Spojení: $ABC+ABE \rightarrow ABCE$ (shoda v AB). Prořezání: 3-podmnožiny ABC, ABE, ACE, BCE — všechny jsou v L_3 , kandidát projde. Průchod daty: $ABCE:2$ (T3, T5) $\rightarrow L_4 = \{ABCE\}$.

Krok $k = 5$. $C_5 = \emptyset$, konec.

Generování pravidel z množiny $\{B, C, E\}$ se supportem $3/5 = 60\%$: každou kombinaci rozdělíme na všechny možné dvojice podkombinací Ant a Suc = Comb – Ant (Ant $\cap$ Suc = $\emptyset$, Ant $\cup$ Suc = Comb). Support pravidla odpovídá četnosti kombinace Comb, confidence poměru četností Comb a Ant:

Pravidlo	support	confidence	lift
$BC \Rightarrow E$	3/5	3/3 = 100 %	1,00/0,8 = 1,25
$BE \Rightarrow C$	3/5	3/4 = 75 %	0,75/0,8 = 0,94
$CE \Rightarrow B$	3/5	3/3 = 100 %	1,00/0,8 = 1,25
$B \Rightarrow CE$	3/5	3/4 = 75 %	0,75/0,6 = 1,25
$C \Rightarrow BE$	3/5	3/4 = 75 %	0,75/0,8 = 0,94
$E \Rightarrow BC$	3/5	3/4 = 75 %	0,75/0,6 = 1,25

Při $\text{minconf} = 80\%$ projdou jen $BC \Rightarrow E$ a $CE \Rightarrow B$. Všimněte si, že support je pro všechna pravidla z téže množiny stejný — proto je fáze hledání frekventovaných množin oddělena od fáze generování pravidel.

Generování pravidel využívá další monotónní vlastnost: má-li pravidlo $\text{Ant} \Rightarrow \text{Suc}$ nízkou confidence, mají ji i všechna pravidla se *zúženým* antecedentem (např. z $AB \Rightarrow C$ plyne, že $A \Rightarrow BC$ nemá vyšší confidence). Kandidáty proto generujeme po hladinách podle velikosti konsekventu.

4.4 Hlavní kroky MBA a prořezávání

Hlavní kroky analýzy nákupního košíku:

- zvolit vhodné položky na adekvátní úrovni;
- generovat pravidla na základě frekvenční tabulky — spočítat (podmíněné) pravděpodobnosti výskytu položek a jejich kombinací v transakcích a omezit hledání prahem na support;
- najít nejlepší pravidla analýzou pravděpodobností — překonat limity dané počtem položek a jejich kombinací v „zajímavých“ transakcích.

Volba vhodných položek. Transakční data mívají horší kvalitu a vyžadují rozsáhlejší předzpracování; relevance položek se navíc mění s časem. Adekvátní úroveň zpracování je kompromis: rostoucí počet kombinací položek $\times$ okamžitá aplikovatelnost výsledků (konkrétní položky) $\times$ dostatečně vysoký support.

Taxonomie (hierarchie kategorií) řídí složitost generovaných pravidel: zpočátku použijeme obecnější položky, později generujeme pravidla pro konkrétní položky jen na základě transakcí, které tyto položky obsahují. Aby byly výsledky použitelné, měly by se uvažované položky vyskytovat zhruba ve stejném počtu transakcí:

- *vzácné položky přesuň výše* v taxonomii (tam se objevují relativně častěji);
- *běžnější položky ponech níže* (aby nejčastější položky neovládly pravidla).

Virtuální položky překračují rámec tradiční taxonomie: stírají hranice mezi typy produktů (např. značky — Calvin Klein) a mohou nést informaci o transakci samotné — anonymní (den v týdnu, čas) nebo podepsanou (informace o zákazníkovi a jeho chování). Mohou ale způsobit redundance: jediná virtuální položka odpovídá položkám z taxonomie („Je-li produkt_Škoda, pak Škoda“), nebo se v pravidle objeví virtuální i obecná položka současně („Je-li produkt_Škoda a malé auto, pak stan“ místo „Je-li Fabia, pak stan“).

Prořezávání podle minimálního supportu eliminuje méně časté položky — prováděné akce se musí týkat dostatečného počtu transakcí. Dvě možnosti: (a) eliminovat vzácné položky a odpovídající pravidla, (b) použít taxonomii k vytvoření obecných položek splňujících práh. Proměnlivý minimální support má *kaskádový efekt*.

Disociační pravidla (*dissociation rules*) mají tvar IF A AND NOT B THEN C. Zavedeme nové položky komplementární k původním — neobsahuje-li transakce původní položku, obsahuje její negaci. Nevýhody: dvojnásobný počet položek, zvětšená velikost transakcí, negované položky se vyskytují častěji než původní (a pravidla se všemi položkami negovanými se obtížně využívají).

Analýza časových řad pomocí MBA (příčina–následek): potřebujeme informaci o čase nebo pořadí událostí a často identifikaci zákazníka. Převod na MBA: do transakcí, které nastaly *před* uvažovanou událostí, zavedeme nové položky (pro příčiny) a analogicky *po* události (pro následky); duplicitní položky se z transakcí odstraní. *Okno* je „snímek“ všech dat z určitého časového období (např. všechny transakce z posledního měsíce) — umožňuje sledovat trendy vzácných položek.

Výhody MBA	Nevýhody MBA
Jasně a srozumitelné výsledky — IF-THEN pravidla, okamžitě aplikovatelná	Výpočetní náklady: prostor asociačních pravidel je exponenciální, $O(2^m)$, kde m je počet položek
Dobývání bez cílových výstupních hodnot (bez učitele) — důležité při zpracování velkých dat bez apriorní znalosti	Nutnost taxonomií a virtuálních položek
Umožňuje zpracovat data proměnné délky	Obtížné určení vhodného počtu položek; položky by měly mít zhruba stejnou četnost
Jednoduchý a srozumitelný výpočet	MBA typicky produkuje <i>nadměrné</i> množství pravidel — tisíce, desetitisíce, miliony

4.5 FP-Growth a FP-strom

Apriori používá přístup „generuj a testuj“: generuje kandidátní množiny a testuje, zda jsou frekventované. To je drahé — generování kandidátů (v čase i v prostoru) i počítání supportu (kontrola podmnožin je výpočetně náročná a je nutných více průchodů databází, tedy I/O).

FP-Growth umožňuje objevit frekventované množiny *bez generování kandidátů*. Dvoustupňový přístup:

1. postavit kompaktní datovou strukturu *FP-strom* (*FP-tree*) — dvěma průchody daty;
2. extrahovat frekventované množiny přímo z *FP-stromu* jeho procházením.

FP-strom

Uzly odpovídají položkám a mají čítač. *FP-Growth* čte transakce po jedné a mapuje je na cestu ve stromu. Používá se *pevné pořadí* položek, takže se cesty mohou překrývat, když transakce sdílejí položky (mají stejný prefix); v takovém případě se inkrementují čítače. Mezi uzly obsahující stejnou položku se udržují ukazatele tvořící jednosměrné spojové seznamy. Čím více cest se překrývá, tím vyšší je komprese — *FP-strom* se pak vejde do paměti.

Konstrukce (2 průchody):

- *Průchod 1*: projdi data a najdi support každé položky; vyřaď nefrekventované položky; seřaď frekventované sestupně podle supportu (např. a, b, c, d, e). Toto pořadí se použije při stavbě stromu, aby bylo možné sdílet společné prefixy.
- *Průchod 2*: postav *FP-strom*. Např. transakce $\{a, b\}$: vytvoř uzly a a b a cestu $\text{null} \rightarrow a \rightarrow b$, čítače nastav na 1. Transakce $\{b, c, d\}$: vytvoř uzly b, c, d a cestu $\text{null} \rightarrow b \rightarrow c \rightarrow d$ — ačkoli transakce 1 a 2 sdílejí b , cesty jsou disjunktní, protože nesdílejí společný *prefix*; přidej odkaz mezi oběma uzly b . Transakce $\{a, c, d, e\}$ sdílí prefix a s transakcí 1, takže se cesty překryjí a čítač uzlu a se zvýší o 1; přidej odkazy mezi uzly c a mezi uzly d . Pokračuj, dokud nejsou všechny transakce namapovány na cestu.

Velikost FP-stromu. Obvykle menší než nekomprimovaná data, protože mnoho transakcí sdílí položky (a tedy prefixy). Nejlepší případ: všechny transakce obsahují tutéž množinu položek $\rightarrow$ jediná cesta. Nejhorší případ: každá transakce má unikátní množinu položek (žádné společné položky) $\rightarrow$ strom je alespoň tak velký jako původní data a paměťové nároky jsou vyšší (je nutné ukládat ukazatele a čítače). Velikost závisí na pořadí položek — řazení podle klesajícího supportu je heuristika, která nevede vždy k nejmenšímu stromu.

Extrakce frekventovaných množin je algoritmus *zdola nahoru* od listů ke kořeni, typu rozděl a panuj: nejprve hledáme frekventované množiny končící na e , pak na de atd., pak na d , cd atd. Nejprve se extrahují *prefixové podstromy* končící danou položkou (s využitím spojových seznamů).

Podmíněný FP-strom

Podmíněný FP-strom (*conditional FP-tree*) pro množinu X je *FP-strom*, který bychom postavili, kdybychom uvažovali jen transakce obsahující X (a X pak ze všech transakcí odstranili).

Postup získání podmíněného *FP-stromu* pro e z prefixového podstromu končícího v e :

1. aktualizuj čítače podél prefixových cest (od e) tak, aby odrážely počet transakcí obsahujících e ;
2. odstraň uzly obsahující e (informace o nich už není potřeba díky předchozímu kroku);
3. odstraň z prefixových cest nefrekventované položky.

Příklad ($\text{minsup} = 2$), extrakce všech frekventovaných množin obsahujících e :

1. Získej prefixový podstrom pro e .
2. Ověř, zda je e frekventovaná položka — sečti čítače podél spojového seznamu. Ano, čítač = 3, takže $\{e\}$ se extrahuje jako frekventovaná množina.

3. Protože je e frekventovaná, hledej frekventované množiny končící na e , tj. de , ce , be , ae — problém dekomponuj rekurzivně. K tomu je nejprve nutné získat podmíněný FP-strom pro e : čítače b a c se nastaví na 1 a a na 2, uzly e se odstraní; položka b má support 1 (což znamená, že be má support 1), takže be je nefrekventovaná a b se odstraní.
4. Podmíněný FP-strom pro e použij k nalezení frekventovaných množin končících na de , ce , ae (nikoli be , protože b v podmíněném stromu není). Pro každou z nich najdi prefixové cesty v podmíněném stromu, extrahuj frekventované množiny, vygeneruj podmíněný FP-strom atd. (rekurze). Například $e \rightarrow de \rightarrow ade$: nalezeny jsou $\{d, e\}$ a $\{a, d, e\}$; $e \rightarrow ce$: nalezena je $\{c, e\}$.
5. Totéž zopakuj pro d , c , b , a ; dílčí řešení se sloučí.

Výhody FP-Growth	Nevýhody FP-Growth
Pouze 2 průchody daty	FP-strom se nemusí vejít do paměti
„Komprimuje“ datovou sadu	FP-strom je drahé postavit (i když pak se množiny čtou snadno)
Žádné generování kandidátů, mnohem rychlejší než Apriori	Prořezávat lze jen jednotlivé položky; support lze spočítat až po vložení všech dat do stromu

4.6 Uzavřené a maximální frekventované množiny

Počet frekventovaných množin roste exponenciálně; kompaktní reprezentace:

Uzavřené a maximální frekventované množiny

Frekventovaná množina X je **uzavřená** (*closed frequent itemset*), pokud neexistuje žádná vlastní nadmnožina $Y \supset X$ se stejným supportem.

Frekventovaná množina X je **maximální** (*maximal frequent itemset*), pokud žádná její vlastní nadmnožina není frekventovaná.

Platí: maximální $\subseteq$ uzavřené $\subseteq$ všechny frekventované.

Z uzavřených množin lze zrekonstruovat *všechny* frekventované množiny *včetně jejich supportů* (bezeztrátová reprezentace), z maximálních jen jejich seznam *bez* supportů (ztrátová, ale nejkompaktější reprezentace).

Příklad. Data $\{a, b, c, d, e\}$ v 2 transakcích a $\{a, b, c\}$ ve 3 dalších, $\text{minsup}=2$. Množina $\{a, b\}$ má support 5, ale i $\{a, b, c\}$ má support 5 $\rightarrow \{a, b\}$ *není* uzavřená. $\{a, b, c\}$ (support 5) uzavřená je, protože každá nadmnožina má support 2. $\{a, b, c, d, e\}$ je uzavřená i maximální. Maximální jsou tedy $\{a, b, c, d, e\}$; uzavřené $\{a, b, c\}$ a $\{a, b, c, d, e\}$; algoritmy: A-Close, CHARM (uzavřené), MaxMiner, MAFIA (maximální).

4.7 Vícenásobný minimální support (MS-Apriori)

Problém jediného minsup: předpokládá, že všechny položky v datech mají stejnou povahu a/nebo podobné četnosti. V mnoha aplikacích se však některé položky vyskytují velmi často a jiné velmi vzácně — v supermarketu se kuchyňský robot a pánev kupují mnohem méně často než chléb a mléko.

Problém vzácných položek (*rare item problem*): je-li minsup nastaven příliš vysoko, pravidla obsahující vzácné položky nebudou nalezena; aby byla nalezena pravidla obsahující zároveň časté i vzácné položky, musel by být minsup velmi nízký, což způsobí *kombinatorickou explozi* (časté položky se pospojují všemi možnými způsoby).

Model vícenásobných minsup (MIS)

Minimální support pravidla se vyjádří pomocí *minimálních supportů položek* (*minimum item support*, MIS) položek, které se v pravidle vyskytují. Každá položka může mít vlastní MIS; zadáním různých hodnot MIS uživatel vyjádří různé požadavky na support pro různá pravidla.

Minsup pravidla $R : a_1, a_2, \dots, a_k \rightarrow a_{k+1}, \dots, a_r$ je *nejnižší* MIS hodnota položek v pravidle; pravidlo splňuje svůj minimální support, je-li jeho skutečný support $\geq \min(\text{MIS}(a_1), \dots, \text{MIS}(a_r))$.

Aby se velmi časté a velmi vzácné položky neobjevovaly v téže množině, zavádí se *omezení rozdílu supportů*:

$$\max_{i \in s} \{\text{sup}(i)\} - \min_{i \in s} \{\text{sup}(i)\} \leq \varphi,$$

kde $0 \leq \varphi \leq 1$ je uživatelem zadaný maximální rozdíl supportů (stejný pro všechny množiny).

Příklad. $\text{MIS}(\text{chléb}) = 2\%$, $\text{MIS}(\text{boty}) = 0,1\%$, $\text{MIS}(\text{oblečení}) = 0,2\%$.

- R_1 : oblečení $\rightarrow$ chléb [$\text{sup} = 0,15\%$, $\text{conf} = 70\%$] — minsup pravidla je $\min\{0,2; 2\} = 0,2\%$, a protože $0,15 < 0,2$, pravidlo *nesplňuje* svůj minsup.
- R_2 : oblečení $\rightarrow$ boty [$\text{sup} = 0,15\%$, $\text{conf} = 70\%$] — minsup pravidla je $\min\{0,2; 0,1\} = 0,1\%$, a protože $0,15 \geq 0,1$, pravidlo *splňuje*.

V novém modelu neplatí vlastnost uzavřenosti dolů! Minimální supporty položek totiž dovolují vyšší minimální supporty pro pravidla obsahující jen časté položky a nižší pro pravidla s méně častými položkami. Příklad: $\text{MIS}(1) = 10\%$, $\text{MIS}(2) = 20\%$, $\text{MIS}(3) = 5\%$, $\text{MIS}(4) = 6\%$. Množina $\{1, 2\}$ se supportem 9% je nefrekventovaná, ale $\{1, 2, 3\}$ a $\{1, 2, 4\}$ frekventované být mohou — proto $\{1, 2\}$ *nesmíme* zahodit.

Řešení: seřaď všechny položky z I podle jejich MIS hodnot vzestupně. Toto pořadí se zafixuje a použije ve všech následných operacích v každé množině. Každá množina w má pak tvar $\{w[1], \dots, w[k]\}$, kde platí

$$\text{MIS}(w[1]) \leq \text{MIS}(w[2]) \leq \dots \leq \text{MIS}(w[k]).$$

Algoritmus MS-Apriori

```

1: Vstup: transakce  $T$ , hodnoty  $MS$  všech MIS, parametr  $\varphi$ 
2:  $M \leftarrow \text{sort}(I, MS)$  ▷ seřad' položky podle MIS( $i$ )
3:  $L \leftarrow \text{INIT-PASS}(M, T)$  ▷ první průchod daty;  $L$  je množina semen
4:  $F_1 \leftarrow \{\{i\} \mid i \in L, i.\text{count}/n \geq \text{MIS}(i)\}$ 
5: for  $k = 2$ ;  $F_{k-1} \neq \emptyset$ ;  $k++$  do
6:   if  $k = 2$  then
7:      $C_k \leftarrow \text{LEVEL2-CANDIDATE-GEN}(L, \varphi)$ 
8:   else
9:      $C_k \leftarrow \text{MSCANDIDATE-GEN}(F_{k-1}, \varphi)$ 
10:  end if
11:  for každou transakci  $t \in T$  do
12:    for každého kandidáta  $c \in C_k$  do
13:      if  $c \subseteq t$  then
14:         $c.\text{count}++$ 
15:      end if
16:      if  $c - \{c[1]\} \subseteq t$  then
17:         $c.\text{tailCount}++$  ▷ pro generování pravidel
18:      end if
19:    end for
20:  end for
21:   $F_k \leftarrow \{c \in C_k \mid c.\text{count}/n \geq \text{MIS}(c[1])\}$ 
22: end for
23: return  $F \leftarrow \bigcup_k F_k$ 

```

První průchod daty (INIT-PASS). Algoritmus zaznamená support každé položky, pak podle seřazeného pořadí najde první položku i z M , která splňuje $\text{MIS}(i)$; i se vloží do množiny semen L . Pro každou následující položku j v M za i platí: je-li $j.\text{count}/n \geq \text{MIS}(i)$, vloží se j také do L (splňuje totiž požadavek na minimální support pro pravidlo s jedinou položkou).

Příklad. Položky 1–4 s $\text{MIS}(1) = 10\%$, $\text{MIS}(2) = 20\%$, $\text{MIS}(3) = 5\%$, $\text{MIS}(4) = 6\%$; $n = 100$ transakcí, četnosti $\{3\}:6$, $\{4\}:3$, $\{1\}:9$, $\{2\}:25$. Pak $L = \{3, 1, 2\}$ a $F_1 = \{\{3\}, \{2\}\}$. Položka 4 není v L , protože $4.\text{count}/n < \text{MIS}(3) = 5\%$; $\{1\}$ není v F_1 , protože $1.\text{count}/n < \text{MIS}(1) = 10\%$.

Funkce $\text{LEVEL2-CANDIDATE-GEN}(L, \varphi)$ vrací nadmnožinu všech frekventovaných 2-množin: pro každou položku l v L (v témž pořadí) s $l.\text{count}/n \geq \text{MIS}(l)$ a pro každou položku h v L za l platí: je-li $h.\text{count}/n \geq \text{MIS}(l)$ a $|\text{sup}(h) - \text{sup}(l)| \leq \varphi$, vlož $\{l, h\}$ do C_2 . (Musíme použít L , nikoli F_1 — F_1 totiž neobsahuje položky, které splňují MIS dřívější položky, ale ne svůj vlastní MIS, jako položka 1 v příkladu výše.)

Funkce $\text{MSCANDIDATE-GEN}(F_{k-1}, \varphi)$ má krok spojení (jako v Apriori: dvojice lišící se jen v poslední položce, navíc s podmínkou $|\text{sup}(i_{k-1}) - \text{sup}(i'_{k-1})| \leq \varphi$) a krok prořezání: pro každou $(k-1)$ -podmnožinu s kandidáta c , není-li s v F_{k-1} , lze c smazat — s výjimkou případu, kdy s neobsahuje $c[1]$ (taková s je právě jedna), tj. první položku c s nejnižší MIS hodnotou. Pak c smazat nesmíme (nevíme, zda s nesplňuje $\text{MIS}(c[1])$, víme jen, že nesplňuje $\text{MIS}(c[2])$) — ledaže $\text{MIS}(c[2]) = \text{MIS}(c[1])$.

Volba hodnot MIS: (a) přiřadit MIS každé položce podle jejího skutečného supportu v datech, např. $\text{MIS}(i) = \lambda \cdot \text{sup}(i)$, kde $\lambda \in [0, 1]$ je stejné pro všechny položky; (b) seskupit položky do shluků (bloků) s podobnými četnostmi a přiřadit stejné MIS všem položkám z téhož shluku.

Generování pravidel v MS-Apriori vyžaduje $c.\text{tailCount}$: u Apriori platí, že je-li f frekventovaná a $f_{\text{sub}} \subseteq f$, je i f_{sub} frekventovaná — v MS-Apriori nestačí zaznamenat support každé frekventované množiny, abychom určili confidence pravidel. Problém by nastal jen tehdy, kdyby položka s nejnižší MIS hodnotou byla v *konsekventu* pravidla.

Model vícenásobných minsup zahrnuje model s jediným supportem jako speciální případ, je reálnější pro praktické aplikace a umožňuje najít pravidla o vzácných položkách bez nadprodukce bezcenných pravidel s obecnými častými položkami. Nastavením MIS položky na 100 % (nebo více) efektivně algoritmu zakážeme generovat pravidla obsahující pouze tyto položky.

4.8 Třídní asociační pravidla (CAR)

Běžné dobývání asociačních pravidel nepracuje s cílovým atributem — najde všechna pravidla, která v datech existují, přičemž libovolná položka může být podmínkou i závěrem. V některých aplikacích uživatele zajímají spíše cíle (atributy/třídy): má např. množinu textových dokumentů publikovaných na známá témata a chce zjistit, která slova jsou asociovaná s jednotlivými tématy.

Třídní asociační pravidlo

Nechť T je transakční datová sada s n transakcemi, každá transakce je navíc označována třídou y . Nechť I je množina všech položek v T , Y množina všech tříd a $I \cap Y = \emptyset$.

Třídní asociační pravidlo (*class association rule*, CAR) je implikace tvaru $X \rightarrow y$, kde $X \subseteq I$ a $y \in Y$. Definice supportu a confidence jsou stejné jako pro běžná asociační pravidla.

Příklad — textové dokumenty:

dok. 1: Student, Teach, School	: Education
dok. 2: Student, School	: Education
dok. 3: Teach, School, City, Game	: Education
dok. 4: Baseball, Basketball	: Sport
dok. 5: Basketball, Player, Spectator	: Sport
dok. 6: Baseball, Coach, Game, Team	: Sport
dok. 7: Basketball, Team, City, Game	: Sport

Při minsup = 20 % a minconf = 60 % jsou příklady CAR:

Student, School $\rightarrow$ Education [sup = 2/7, conf = 2/2], Game $\rightarrow$ Sport [sup = 2/7, conf = 2/3].

Na rozdíl od běžných asociačních pravidel lze CAR dolovat *přímo v jednom kroku*. Klíčovou operací je najít všechny *rule-items* se supportem nad minsup. Rule-item má tvar (condset, y), kde condset $\subseteq I$ a $y \in Y$; každý rule-item reprezentuje pravidlo condset $\rightarrow y$. Algoritmus Apriori lze pro generování CAR modifikovat.

Myšlenku vícenásobného minimálního supportu lze použít i zde — *vícenásobné minimální supporty tříd*: uživatel může zadat různé minimální supporty různým třídám (což efektivně přiřadí různý minimální support pravidlům pro každou třídu). Např. u dat se třídami Yes a No můžeme chtít, aby pravidla třídy Yes měla minimální support 5 % a pravidla třídy No 10 %. Nastavením minimálních supportů tříd na 101 % (nebo více) zakážeme generovat pravidla pro tyto třídy — užitečný trik v aplikacích.

Na CAR je založen klasifikátor **CBA** (*Classification Based on Associations*): pravidla se seřadí podle confidence (při shodě podle supportu), vybere se minimální podmnožina pokrývající trénovací data a klasifikuje se prvním pravidlem, které nový objekt pokrývá.

4.9 Dolování sekvenčních vzorů

Dobývání asociačních pravidel neuvažuje *pořadí* transakcí; v mnoha aplikacích je však pořadí významné: v analýze nákupního košíku je zajímavé vědět, zda lidé kupují položky v sekvenci (nejprve postel a o nějaký čas později povlečení); ve *web usage mining* je užitečné najít navigační vzory uživatelů na webu na základě sekvencí navštívených stránek.

Sekvence, velikost, délka, podsekvence

Nechť $I = \{i_1, \dots, i_m\}$ je množina položek. *Sekvence* je uspořádaný seznam množin položek. *Prvek* (*element*, *itemset*) sekvence je neprázdná množina $X \subseteq I$; sekvenci značíme $s = \langle a_1 a_2 \dots a_r \rangle$, kde a_i je prvek. Položky v prvku předpokládáme v lexikografickém pořadí.

Velikost (*size*) sekvence je počet jejích prvků; *délka* (*length*) je počet položek v sekvenci. Sekvence délky k se nazývá k -sekvence.

Sekvence $s_1 = \langle a_1 \dots a_r \rangle$ je *podsekvencí* sekvence $s_2 = \langle b_1 \dots b_v \rangle$ (a s_2 je *nadsekvencí* s_1), existují-li celá čísla $1 \leq j_1 < j_2 < \dots < j_r \leq v$ taková, že $a_1 \subseteq b_{j_1}$, $a_2 \subseteq b_{j_2}$, $\dots$, $a_r \subseteq b_{j_r}$. Říkáme též, že s_2 obsahuje s_1 .

Příklad. $I = \{1, \dots, 9\}$. Sekvence $\langle \{3\}\{4, 5\}\{8\} \rangle$ je obsažena v $\langle \{6\}\{3, 7\}\{9\}\{4, 5, 8\}\{3, 8\} \rangle$, protože $\{3\} \subseteq \{3, 7\}$, $\{4, 5\} \subseteq \{4, 5, 8\}$ a $\{8\} \subseteq \{3, 8\}$. Naproti tomu $\langle \{3\}\{8\} \rangle$ není obsažena v $\langle \{3, 8\} \rangle$ ani naopak. Velikost sekvence $\langle \{3\}\{4, 5\}\{8\} \rangle$ je 3, délka 4.

Úloha: pro danou množinu S vstupních datových sekvencí (sekvenční databázi) najít všechny sekvence, které mají uživatelem zadaný minimální support. Každá taková sekvence se nazývá *frekvencovaná sekvence* nebo *sekvenční vzor*. Support sekvence je podíl datových sekvencí v S , které tuto sekvenci obsahují.

Algoritmus GSP (*Generalized Sequential Pattern*) generuje sekvenční vzory podobně jako Apriori: po hladinách podle délky, s generováním kandidátů spojením a jejich prořezáním na základě apriori vlastnosti (každá podsekvence frekvencované sekvence je frekvencovaná). Rozdíl oproti Apriori je ve spojovacím kroku: dvě k -sekvence s_1 , s_2 se spojí, je-li sekvence vzniklá odstraněním první položky s_1 shodná se sekvencí vzniklou odstraněním poslední položky s_2 ; poslední položka s_2 se pak přidá k s_1 buď jako součást posledního prvku (byla-li v s_2 součástí prvku), nebo jako nový prvek. GSP navíc umí pracovat s časovými omezeními (*min-gap*, *max-gap*, *sliding window*) a s taxonomií položek. Alternativní algoritmy: SPADE (vertikální formát), PrefixSpan (růst vzorů, obdoba FP-Growth).

4.10 Rozšíření: vícúrovňová a kvantitativní pravidla

Vícúrovňová asociační pravidla (*multilevel association rules*) využívají koncepční hierarchii položek (mléko $\rightarrow$ polotučné mléko $\rightarrow$ Tatra polotučné 1,5 l). Strategie prahů:

- *Jednotný minsup* — stejný práh na všech úrovních; jednoduché, ale na nízkých úrovních nenajde nic (položky jsou příliš vzácné).
- *Klesající minsup* (*reduced support*) — nižší práh na nižších úrovních hierarchie.
- *Minsup podle skupiny* (*group-based*) — expert nastaví práh pro každou skupinu položek.

Problémem jsou *redundantní pravidla*: pravidlo „mléko $\Rightarrow$ chléb [8 %, 70 %]“ a jeho potomek „polotučné mléko $\Rightarrow$ chléb [2 %, 72 %]“ — potomek je redundantní, pokud jeho support i confidence odpovídají očekávání odvozenému z předka.

Kvantitativní asociační pravidla (*quantitative association rules*) pracují s numerickými atributy, které je nutné diskretizovat:

- *statická diskretizace* předem (equal-width, equal-depth) — riskuje nevhodné hranice;
- *dynamická diskretizace* podle rozdělení dat (např. ARCS — *Association Rule Clustering System*: pravidla se hledají v 2-D mřížce a sousední buňky se slučují do obdélníků);
- *diskretizace podle vzdálenosti* — intervaly odrážejí hustotu dat.

Příklad: věk(30..39) $\wedge$ příjem(42K..48K) $\Rightarrow$ koupí(TV).

4.11 Korelační analýza a míry zajímavosti

Support a confidence samy o sobě generují mnoho *zavádějících* pravidel. Proto se používá analýza korelace — pravidlo se rozšíří na

$$A \Rightarrow B \text{ [support, confidence, correlation].}$$

Objektivní míry zajímavosti

Lift: $\text{lift}(A, B) = \frac{P(A \cup B)}{P(A)P(B)}$; = 1 nezávislost, > 1 pozitivní, < 1 negativní korelace.

χ^2 : nad kontingenční tabulkou 2×2 pro $A, \neg A$ a $B, \neg B$; testuje statistickou významnost závislosti.

All-confidence: $\text{all_conf}(A, B) = \frac{\sup(A \cup B)}{\max\{\sup(A), \sup(B)\}} = \min\{P(A | B), P(B | A)\}$.

Max-confidence: $\max\{P(A | B), P(B | A)\}$.

Kulczynski: $\text{Kulc}(A, B) = \frac{1}{2}(P(A | B) + P(B | A))$.

Cosine: $\cos(A, B) = \frac{P(A \cup B)}{\sqrt{P(A)P(B)}} = \sqrt{P(A | B)P(B | A)}$.

Koeficient jistoty (conviction): $\text{conv}(A \Rightarrow B) = \frac{P(A)P(\neg B)}{P(A \wedge \neg B)}$.

Leverage: $P(A \cup B) - P(A)P(B)$.

Nulová invariance

Míra je *nulově invariantní* (*null-invariant*), pokud její hodnota nezávisí na počtu transakcí, které neobsahují ani A , ani B (tzv. nulových transakcí).

Nulově invariantní jsou: all-confidence, max-confidence, Kulczynski, cosine.

Nulově invariantní *nejsou*: lift, χ^2 , leverage.

Proč to vadí: v transakční databázi supermarketu s milionem transakcí obsahuje mléko a kaviár současně 100 transakcí, kaviár sám 100 a mléko 100 000. Nulových transakcí (bez obou položek) je 899 900. Lift i χ^2 nafouknou hodnotu do závratné výše jen proto, že je nulových transakcí hodně, ačkoli reálný vztah je nejasný. Kulczynski dá $\frac{1}{2}(100/100 + 100/100\,000) = 0,5005$ a doplněný *koeficient nevyváženosti* (*imbalance ratio*) $\text{IR}(A, B) = \frac{|\sup(A) - \sup(B)|}{\sup(A) + \sup(B) - \sup(A \cup B)}$ ukáže, že vztah je silně nevyvážený (kaviár se prakticky vždy kupuje s mlékem, ale mléko jen výjimečně s kaviárem).

Objektivní vs. subjektivní míry zajímavosti

Objektivní míry vycházejí pouze ze struktury dat: support, confidence, lift, χ^2 , Kulczynski, cosine, koeficient jistoty, informační zisk. Jsou automaticky vyčíslitelné, ale nedokáží odlišit triviální pravidlo od objevu.

Subjektivní míry vycházejí ze znalostí a očekávání uživatele. Pravidlo je zajímavé, pokud je:

- *neočekávané* (*unexpected*) — překvapí uživatele, odporuje jeho modelu světa;
- *akceschopné* (*actionable*) — uživatel na jeho základě může něco udělat.

Realizace: uživatel zadá své očekávání („soft belief“ systémy), systém ukáže pravidla, která mu odporují; nebo se použije šablonový jazyk (*rule templates*) pro filtrování zajímavých a nezajímavých tvarů pravidel.

Prakticky se doporučuje kombinace: nejprve prah na support (aby pravidlo bylo statisticky opřené), pak prah na confidence, pak nulově invariantní míra (Kulczynski / cosine) doplněná koeficientem nevyváženosti, a nakonec subjektivní filtr doménového experta.

4.12 Shrnutí k zkoušce

- **Support** = podíl transakcí obsahujících $i \cup j$; **confidence** = $\sup(i \cup j) / \sup(i)$; **lift** = $\frac{P(i \wedge j)}{P(i)P(j)}$; lift < 1 → negace závěru dá lepší pravidlo.

- **Apriori vlastnost** (uzavřenost dolů): každá podmnožina frekventované množiny je frekventovaná. APRIORI-GEN = spojení (shoda v $k-2$ položkách) + prořezání (všechny $(k-1)$ -podmnožiny musí být v L_{k-1}).
- Umět projít Apriori na malé databázi a vygenerovat pravidla se support/confidence/lift.
- **FP-Growth**: 2 průchody, FP-strom (pořadí podle klesajícího supportu, sdílené prefixy, spojové seznamy), prefixové podstromy, *podmíněný FP-strom* (aktualizuj čítače, odstraň uzly položky, odstraň nefrekventované položky), rekurze zdola nahoru. Bez generování kandidátů; problém je paměť.
- **Uzavřené** (žádná nadmnožina se stejným supportem) vs. **maximální** (žádná nadmnožina není frekventovaná); $\text{maximální} \subseteq \text{uzavřené} \subseteq \text{všechny}$.
- **Rare item problem** a **MS-Apriori**: MIS pro každou položku, $\text{minsup pravidla} = \min \text{MIS položek}$, omezení φ na rozdíl supportů, *neplatí uzavřenost dolů* $\rightarrow$ řazení podle MIS, speciální LEVEL2-CANDIDATE-GEN a výjimka v prořezávání pro podmnožiny neobsahující $c[1]$.
- **CAR**: $X \rightarrow y$, $X \subseteq I$, $y \in Y$; dolování v jednom kroku přes rule-items; vícenásobné minimální supporty tříd; klasifikátor CBA.
- **Sekvenční vzory**: velikost = počet prvků, délka = počet položek; podsekvence přes vnoření prvků; algoritmus GSP (obdoba Apriori), PrefixSpan.
- **Míry zajímavosti**: objektivní (lift, χ^2 , all-conf, max-conf, Kulczynski, cosine, conviction) vs. subjektivní (neočekávanost, akceschopnost); **nulová invariance** — lift a χ^2 ji nemají, Kulczynski, cosine, all-conf a max-conf ano.
- **Složitost**: prostor pravidel je $O(2^m)$; MBA produkuje nadměrné množství pravidel $\rightarrow$ taxonomie, virtuální položky, prořezávání, míry zajímavosti.

5 Učení s učitelem: klasifikace a predikce

5.1 Formulace úlohy

Klasifikace a predikce

Je dána trénovací množina $D = \{(\vec{x}_1, y_1), \dots, (\vec{x}_p, y_p)\}$, kde $\vec{x}_i = (x_{i1}, \dots, x_{in})$ je popis objektu pomocí atributů $A_1, \dots, A_n$ a y_i je hodnota cílového atributu.

Klasifikace (classification): cílový atribut je diskrétní, $y_i \in C = \{C_1, \dots, C_m\}$ (třídy). Hledáme funkci $h: \mathcal{X} \rightarrow C$, která co nejlépe zobecňuje na nová, dosud neviděná data.

Predikce / regrese (prediction, regression): cílový atribut je spojitý, $y_i \in \mathbb{R}$.

Řešení má dvě fáze: (1) *indukce* (učení) modelu z trénovacích dat; (2) *aplikace* modelu na nové objekty. Kvalita se měří na *oddělených* testovacích datech (kap. 6).

5.2 Rozhodovací stromy

5.2.1 Princip

Rozhodovací strom modeluje klasifikační proces a dělí prostor příznaků na *obdélníkové oblasti*; objekt se klasifikuje podle oblasti, do níž patří.

Rozhodovací strom

Nechť D je databáze $D = \{\vec{t}_1, \dots, \vec{t}_p\}$, $\vec{t}_i = (t_{i1}, \dots, t_{in})$, s atributy $\{A_1, \dots, A_n\}$ a množinou tříd $C = \{C_1, \dots, C_m\}$.

Rozhodovací strom (decision tree) pro D je strom, kde:

- každý vnitřní uzel je označen atributem A_a , $1 \leq a \leq n$ (*rozhodovací atribut*);
- každá hrana je označena predikátem aplikovatelným na atribut rodičovského uzlu (*rozhodovací predikát*);
- každý list nese označení třídy C_j .

Výhody	Nevýhody
Snadná implementace a interpretace	Obtížné zpracování spojitých dat (kategorizace atributu = dělení prostoru na obdélníky; nelze vždy použít — např. bankovní úvěry, kde je hranice šikmá)
Efektivita: indukce $O(np \log p)$, klasifikace q objektů stromem hloubky $O(\log p)$ je $O(q \log p)$	Obtížné zpracování chybějících dat
Extrakce jednoduchých pravidel	Přeučení $\rightarrow$ nutné <i>prořezávání</i>
Použitelné i na velké datové sady	Nezohledňují vzájemné korelace mezi atributy

5.2.2 Obecný algoritmus TDIDT

TDIDT — Top Down Induction of Decision Trees

Indukce stromů metodou shora dolů (*rozděl a panuj*):

1. Zvol jeden atribut jako kořen (pod)stromu.
2. Rozděl data přicházející do tohoto uzlu na podmnožiny podle hodnot zvoleného atributu a přidej uzel pro každou podmnožinu.
3. Existuje-li uzel takový, že data do něj přicházející nepatří všechna do téže třídy, opakuj proces pro tento uzel od kroku 1; jinak skonči.

Obecný (generický) tvar:

- 1: **Vstup:** D (trénovací data); **Výstup:** T (rozhodovací strom)
- 2: $T = \{\}$; urči nejlepší kritérium pro dělení
- 3: $T =$ vytvoř kořenový uzel a označ jej rozhodovacím atributem
- 4: $T =$ přidej hranu pro každý rozhodovací predikát a označ ji
- 5: **for** každou hranu **do**
- 6: $D' =$ databáze vzniklá aplikací rozhodovacího predikátu na D
- 7: **if** je pro danou cestu splněno zastavovací kritérium **then**
- 8: $T' =$ vytvoř list a označ jej odpovídající třídou
- 9: **else**
- 10: $T' =$ generuj rozhodovací strom(D')
- 11: **end if**
- 12: $T =$ připoj T' k hraně
- 13: **end for**

5.2.3 Kritéria dělení

Cílem je zvolit atribut, který nejlépe oddělí objekty z různých tříd.

$$H = - \sum_{j=1}^m p_j \log_2 p_j, \quad H(A(v)) = - \sum_{j=1}^m \frac{n_j(A(v))}{n(A(v))} \log_2 \frac{n_j(A(v))}{n(A(v))},$$

$$H(A) = \sum_{v \in \text{Val}(A)} \frac{n(A(v))}{n} H(A(v)).$$

Vybíráme atribut s *minimální* váženou entropií $H(A)$, ekvivalentně s *maximálním* informačním ziskem $\text{GAIN}(S, A) = H(S) - H(A)$. Alternativy: gain ratio (C4.5), Gini index (CART), χ^2 (CHAID) — viz kap. 3.

Příklad — indukce stromu, žádost o úvěr (1/2): volba kořene

Klient	Příjem	Konto	Pohlaví	Nezaměstnaný	Úvěr schválen
C1	vysoký	vysoké	žena	ne	ano
C2	vysoký	vysoké	muž	ne	ano
C3	nízký	nízké	muž	ne	ne
C4	nízký	vysoké	žena	ano	ano
C5	nízký	vysoké	muž	ano	ano
C6	nízký	nízké	žena	ano	ne
C7	vysoký	nízké	muž	ne	ano
C8	vysoký	nízké	žena	ano	ano
C9	nízký	střední	muž	ano	ne
C10	vysoký	střední	žena	ne	ano
C11	nízký	střední	žena	ano	ne
C12	nízký	střední	muž	ne	ano

Entropie kořene: 8 z 12 objektů má „ano“, 4 mají „ne“:

$$H(S) = -\frac{8}{12} \log_2 \frac{8}{12} - \frac{4}{12} \log_2 \frac{4}{12} = 0,9183.$$

Čtyřpolní kontingenční tabulka pro příjem a schválení úvěru:

	Úvěr ano	Úvěr ne
Příjem vysoký	5	0
Příjem nízký	3	4

PŘÍJEM:

$$H(\text{PŘÍJEM}(\text{vysoký})) = -\frac{5}{5} \log_2 \frac{5}{5} - \frac{0}{5} \log_2 \frac{0}{5} = 0,$$

$$H(\text{PŘÍJEM}(\text{nízký})) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0,9852,$$

$$H(\text{PŘÍJEM}) = \frac{5}{12} \cdot 0 + \frac{7}{12} \cdot 0,9852 = \mathbf{0,5747}.$$

KONTO: vysoké (4 obj., všechny ano) $\rightarrow H = 0$; střední (4 obj., 2:2) $\rightarrow H = 1$; nízké (4 obj., 2:2) $\rightarrow H = 1$:

$$H(\text{KONTO}) = \frac{1}{3} \cdot 0 + \frac{1}{3} \cdot 1 + \frac{1}{3} \cdot 1 = 0,6667.$$

POHLAVÍ: muž (6 obj., 4:2) $\rightarrow H = 0,9183$; žena (6 obj., 4:2) $\rightarrow H = 0,9183$:

$$H(\text{POHLAVÍ}) = \frac{1}{2} \cdot 0,9183 + \frac{1}{2} \cdot 0,9183 = 0,9183.$$

NEZAMĚSTNANÝ: ano (6 obj., 3:3) $\rightarrow H = 1$; ne (6 obj., 5:1) $\rightarrow H = 0,65$:

$$H(\text{NEZAMĚSTNANÝ}) = \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 0,65 = 0,825.$$

Informační zisky ($= H(S) - H(A)$): PŘÍJEM 0,344; KONTO 0,252; POHLAVÍ 0,000; NEZAMĚSTNANÝ 0,093. **Kořenem se stává PŘÍJEM** (nejnižší $H(A)$ = nejvyšší zisk). Povšimněte si, že POHLAVÍ má zisk přesně nulový — rozdělení tříd je v obou jeho větvích totožné s rozdělením v celých datech.

Příklad — indukce stromu, žádost o úvěr (2/2): další větvení

První větvení podle PŘÍJEM. Větev *vysoký*: všech 5 objektů má úvěr schválen $\rightarrow$ list „ano“. Větev *nízký*: 7 klientů, dále větvíme:

$$H(\text{KONTO}) = \frac{2}{7} \cdot 0 + \frac{3}{7} \cdot 0,9183 + \frac{2}{7} \cdot 0 = \mathbf{0,3935},$$

$$H(\text{POHLAVÍ}) = 0,965, \quad H(\text{NEZAMĚSTNANÝ}) = 0,9792.$$

Druhé větvení podle KONTO (3 hodnoty). Větev *vysoké*: všechny „ano“. Větev *nízké*: všechny „ne“. Větev *střední*: 3 klienti (C9, C11, C12), dále větvíme:

$$H(\text{POHLAVÍ}) = \frac{2}{3} \cdot 1 + \frac{1}{3} \cdot 0 = 0,6667, \quad H(\text{NEZAMĚSTNANÝ}) = 0.$$

Třetí větvení podle NEZAMĚSTNANÝ. Výsledný strom:

PŘÍJEM = vysoký $\rightarrow$ **ano**

PŘÍJEM = nízký $\rightarrow$ KONTO

KONTO = vysoké $\rightarrow$ **ano**

KONTO = nízké $\rightarrow$ **ne**

KONTO = střední $\rightarrow$ NEZAMĚSTNANÝ

NEZAMĚSTNANÝ = ano $\rightarrow$ **ne**

NEZAMĚSTNANÝ = ne $\rightarrow$ **ano**

Příklad — totéž kritériem Gini (CART)

Na týchž datech spočteme Gini index $G(v) = 1 - \sum_j p_j^2$ a jeho vážený průměr $G(A) = \sum_i \frac{n_i}{n} G(v_i)$. Kořen: $G(S) = 1 - (\frac{8}{12})^2 - (\frac{4}{12})^2 = 0,4444$; pro větev *nízký příjem* (3 ano, 4 ne) je $G = 1 - (\frac{3}{7})^2 - (\frac{4}{7})^2 = 0,4898$ a analogicky pro ostatní hodnoty.

Atribut	Gini jednotlivých hodnot	$G(A)$
PŘÍJEM	vysoký 5:0 $\rightarrow$ 0; nízký 3:4 $\rightarrow$ 0,4898	$\frac{5}{12} \cdot 0 + \frac{7}{12} \cdot 0,4898 = \mathbf{0,2857}$
KONTO	vysoké 4:0 $\rightarrow$ 0; střední 2:2 $\rightarrow$ 0,5; nízké 2:2 $\rightarrow$ 0,5	$\frac{1}{3}(0 + 0,5 + 0,5) = 0,3333$
NEZAMĚSTNANÝ	ano 3:3 $\rightarrow$ 0,5; ne 5:1 $\rightarrow$ 0,2778	$\frac{1}{2}(0,5 + 0,2778) = 0,3889$
POHLAVÍ	muž 4:2 $\rightarrow$ 0,4444; žena 4:2 $\rightarrow$ 0,4444	0,4444

Nejnižší $G(A)$ má opět PŘÍJEM (pokles nečistoty $0,4444 - 0,2857 = 0,1587$) a **pořadí atributů je totožné jako u entropie** ($0,5747 < 0,6667 < 0,825 < 0,9183$) — typická situace: obě míry se chovají velmi podobně.

CART navíc dělí jen binárně, takže u tříhodnotového atributu KONTO prověří všechna tři rozdělení na dvě skupiny:

$$\{\text{vysoké}\} \mid \{\text{střední, nízké}\} : \frac{4}{12} \cdot 0 + \frac{8}{12} \cdot 0,5 = \mathbf{0,3333},$$

$$\{\text{nízké}\} \mid \{\text{vysoké, střední}\} : \frac{4}{12} \cdot 0,5 + \frac{8}{12} \cdot 0,375 = 0,4167 \quad (\text{a symetricky pro } \{\text{střední}\}).$$

Nejlepší binární dělení podle KONTO tedy odděluje *vysoké* konto od zbytku; kořenem zůstává PŘÍJEM s $G = 0,2857$.

5.2.4 Převod stromu na pravidla

Každá cesta mezi kořenem a listem odpovídá jednomu pravidlu: nelistové uzly (atributy příznaků) se objeví spolu s hodnotou příslušné hrany na levé straně (v těle) pravidla, listový uzel (cílový atribut třídy) tvoří pravou stranu (hlavu) pravidla.

Pro strom z příkladu:


```

IF PŘÍJEM(vysoký) THEN ÚVĚR(ano)
IF PŘÍJEM(nízký) ∧ KONTO(vysoké) THEN ÚVĚR(ano)
IF PŘÍJEM(nízký) ∧ KONTO(nízké) THEN ÚVĚR(ne)
IF PŘÍJEM(nízký) ∧ KONTO(střední) ∧ NEZAMĚSTNANÝ(ano) THEN ÚVĚR(ne)
IF PŘÍJEM(nízký) ∧ KONTO(střední) ∧ NEZAMĚSTNANÝ(ne) THEN ÚVĚR(ano)

```

Takto získaná sada pravidel je *vzájemně výlučná a vyčerpávající* (každý objekt padne právě do jednoho pravidla).

5.2.5 Faktory důležité pro indukci stromu

- **Volba rozhodovacích atributů** — ovlivňuje efektivitu vybudovaného stromu; přínosem je „informovaný“ vstup experta pro danou oblast.
- **Pořadí rozhodovacích atributů** — eliminuje redundantní porovnání.
- **Větvení** — počet nutných dělení; náročnější pro spojité hodnoty nebo mnoho hodnot.
- **Tvar stromu** — vhodnější jsou vyvážené stromy s co nejméně úrovněmi (za cenu složitějších porovnání); některé algoritmy vytvářejí pouze binární stromy (často dost hluboké).
- **Zastavovací kritéria** — správná klasifikace trénovacích dat; předčasné zastavení brání budování velkých stromů a přeučení (kompromis přesnost $\times$ efektivita); *Occamova břitva* (William Occam, 1320): preference nejjednodušší hypotézy pro D .
- **Trénovací data a jejich množství** — příliš málo $\rightarrow$ strom nemusí být dostatečně spolehlivý pro obecnější data; příliš mnoho $\rightarrow$ nebezpečí přeučení.
- **Prořezávání** — vyšší přesnost a efektivita na testovacích datech; odstranění redundantních porovnání, celých podstromů a irelevantních atributů; při přeučení odstranění celých podstromů na nižších úrovních. Lze provádět už během indukce (brání vzniku nadměrně velkých stromů) nebo až na hotovém stromu.
- **Časová a prostorová složitost** — závisí na počtu trénovacích dat p , počtu atributů n a tvaru stromu.

5.2.6 Algoritmus ID3

Iterative Dichotomiser (Ross Quinlan, 1986): učení booleovských funkcí, hladová metoda, buduje strom shora dolů; v každém uzlu volí atribut, který nejlépe klasifikuje lokální trénovací data — nejlepší atribut má *největší informační zisk*. Proces pokračuje, dokud nejsou všechna trénovací data správně klasifikována nebo dokud nejsou využity všechny atributy.

Algoritmus ID3

ID3(EXAMPLES, TARGET_ATTRIBUTE, ATTRIBUTES)

- Vytvoř kořen ROOT stromu.
- Jsou-li všechny EXAMPLES pozitivní, vrať strom ROOT s jediným uzlem označeným „+“.
- Jsou-li všechny EXAMPLES negativní, vrať strom ROOT s jediným uzlem označeným „−“.
- Je-li ATTRIBUTES = {}, vrať strom ROOT s jediným uzlem označeným nejčastější hodnotou TARGET_ATTRIBUTE v EXAMPLES.
- Jinak:
 - $A \leftarrow$ atribut z ATTRIBUTES, který nejlépe klasifikuje EXAMPLES;
 - rozhodovací atribut pro ROOT $\leftarrow A$;
 - pro všechny možné hodnoty v_i atributu A :
 - * připoj k ROOT novou hranu odpovídající predikátu $A = v_i$;
 - * nechť EXAMPLES_{v_i} je podmnožina EXAMPLES s hodnotou v_i pro A ;
 - * je-li $\text{EXAMPLES}_{v_i} = \{\}$, připoj k hraně list označený nejčastější hodnotou TARGET_ATTRIBUTE v EXAMPLES;
 - * jinak připoj k hraně podstrom ID3(EXAMPLES_{v_i} , TARGET_ATTRIBUTE, ATTRIBUTES − { A }).
- Vrať ROOT.

Omezení ID3: pracuje jen s kategoriálními atributy, neumí chybějící hodnoty, neprořezává, preferuje atributy s mnoha hodnotami.

5.2.7 Algoritmus C4.5 a C5.0

C4.5 (Quinlan, 1993) je modifikace ID3, která odstraňuje jeho hlavní omezení.

Chybějící data. Během konstrukce stromu se ignorují — při vyhodnocování kritéria pro dělení dat se uvažují pouze známé hodnoty daného atributu (zisk se váží podílem známých hodnot). Během klasifikace se chybějící hodnoty odhadují na základě hodnot známých pro ostatní objekty (objekt se „rozdělí“ do všech větví s vahami odpovídajícími četnostem).

Spojité data. Kategorizují se podle hodnot daného atributu na objektech z trénovací množiny (binární dělení $A \leq t$ vs. $A > t$; kandidátní prahy t jsou středy mezi sousedními hodnotami, vybere se práh s maximálním ziskem).

Kritérium dělení: poměrný informační zisk.

$$\text{GAIN_RATIO}(S, A) = \frac{\text{GAIN}(S, A)}{-\sum_{v \in \text{VALUES}(A)} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}}$$

Pro dělení se použije atribut s *největším* poměrným informačním ziskem; kritérium znevýhodňuje nadměrné větvení.

Příklad na datech o úvěru: pro PŘÍJEM je vnitřní informace $-\frac{5}{12} \log_2 \frac{5}{12} - \frac{7}{12} \log_2 \frac{7}{12} = 0,980$, tedy $\text{GAIN_RATIO} = 0,344/0,980 = 0,351$. Pro KONTO je vnitřní informace $\log_2 3 = 1,585$, tedy $\text{GAIN_RATIO} = 0,252/1,585 = 0,159$. PŘÍJEM zůstává nejlepší, ale rozdíl se zvýraznil — KONTO bylo zvyhodněno vyšším počtem hodnot.

Prořezávání.

- *Validace:* rozděl trénovací data na trénovací (2/3) a validační (1/3) množinu.
- *Nahrazení podstromů listy (reduced-error pruning):* podstrom se nahradí listem označeným nejčastější třídou v příslušných trénovacích objektech. Nový strom *nesmí* být na validační množině horší než původní — jinak k nahrazení nedojde.
- *Vyzdvižení podstromu (subtree raising):* podstrom se nahradí svým nejčastěji používaným podstromem, který se vyzdvihne na vyšší úroveň; testuje se klasifikační přesnost.

Post-prořezávání pravidel (rule post-pruning):

1. Indukuj rozhodovací strom na trénovací množině — přeučení je povoleno.
2. Převeď strom na ekvivalentní sadu pravidel (pro každou cestu kořen–list jedno pravidlo).
3. Prořez (zobecní) pravidla vypuštěním všech předpokladů, jejichž odstranění nezhorší odhadovanou klasifikační přesnost.
4. Seřaď prořezaná pravidla podle jejich očekávané přesnosti a v tomto pořadí je používej ke klasifikaci.

Výsledkem je snazší a radikálnější prořezávání než u celých stromů a transparentní, srozumitelná pravidla.

Odhad přesnosti pravidel a stromů. Standardně validační množinou; C4.5 ale používá odhad přímo z trénovacích dat: přesnost $\hat{p} = p_1/p$ (poměr objektů správně klasifikovaných pravidlem ku všem objektům pravidlem pokrytým). Předpokládá se binomické rozdělení, takže směrodatná odchylka přesnosti je $\sqrt{\hat{p}(1-\hat{p})/p}$, a jako charakteristika pravidla se bere *dolní mez* zvoleného konfidenčního intervalu. Například pro 95% hladinu spolehlivosti:

$$\text{dolní mez} = \hat{p} - 1,96 \sqrt{\frac{\hat{p}(1-\hat{p})}{p}}.$$

Tento pesimistický odhad automaticky penalizuje pravidla opřená o málo objektů.

C5.0 je komerční verze C4.5 pro velké databáze: podobná indukce stromu, efektivnější generování pravidel, vyšší přesnost pomocí *boostingu* (kombinace více klasifikátorů) — z původních trénovacích dat se vytvoří různé trénovací podmnožiny, každému trénovacímu objektu se přiřadí váha odpovídající jeho vlivu při klasifikaci, pro každou kombinaci vah se vytvoří samostatný klasifikátor a při následné klasifikaci má každý klasifikátor svůj hlas, přičemž vítězí většinová třída.

5.2.8 Algoritmus CART

Classification And Regression Trees generuje **binární** stromy: při každém dělení dat vznikají právě dvě hrany. Volba nejlepšího rozhodovacího atributu se řídí entropií nebo *Gini indexem* (nižší hodnoty G znamenají lepší diskriminaci). Dělení se provádí podle „nejlepšího“ nalezeného bodu pro daný uzel t ; prověřují se všechny možné hodnoty v rozhodovacího atributu A , $v \in \text{VALUES}(A)$.

Označme L , R levý, resp. pravý podstrom a P_L , P_R pravděpodobnosti zpracování příslušným podstromem:

$$P_L = \frac{\text{počet objektů v levém podstromu } L}{\text{počet objektů v celé trénovací množině}}.$$

(V případě rovnosti se použije pravý podstrom.) Dále $P(C_j | t_L)$, $P(C_j | t_R)$ je pravděpodobnost, že objekt patří do třídy C_j a je v levém, resp. pravém podstromu uzlu t :

$$P(C_j | t) = \frac{\text{počet objektů třídy } j \text{ v podstromu}}{\text{počet objektů v uvažovaném uzlu}}.$$

Kritérium pro volbu dělení (*twoing criterion*):

$$\Phi(v | t) = 2P_L P_R \sum_{j=1}^m |P(C_j | t_L) - P(C_j | t_R)|,$$

jehož maximální hodnoty nastávají, jsou-li všechny objekty z dané třídy seskupeny v témže podstromu. (Přednáška uvádí tento zjednodušený tvar; Breimanova původní definice *twoing* má sumu umocněnou na druhou a dělenou čtyřmi, $\frac{P_L P_R}{4} [\sum_j |\cdot|]^2$ — pořadí kandidátních dělení tím ale zůstává zachováno.)

Charakteristické vlastnosti CART: pořadí atributů odpovídá jejich vlivu při klasifikaci; chybějící data se ignorují a nezapočítávají se do vyhodnocení rozhodovacích kritérií (v praxi se používají *náhradní dělení*, *surrogate splits*); zastavovací kritérium — žádné další dělení by nezlepšilo klasifikační přesnost; vysoká přesnost na trénovací množině nemusí odrážet přesnost na testovacích datech. CART umí i *regresní stromy*: v listech jsou průměry cílové hodnoty a kritériem dělení je snížení součtu čtverců.

Prořezávání v CART se provádí metodou *cost-complexity pruning*: minimalizuje se $R_\alpha(T) = R(T) + \alpha |\text{listy}(T)|$, kde $R(T)$ je chyba a α penalizace za složitost; optimální α se volí křížovou validací.

5.2.9 Algoritmus CHAID

Chi-square Automatic Interaction Detection. Kritériem pro větvení je χ^2 . Algoritmus automaticky seskupuje hodnoty kategoriálních atributů: hodnoty se postupně slučují od původního počtu až po dvě skupiny, načež se vybere atribut a jeho kategorizace, které jsou pro dělení dat v daném kroku nejlepší. Při větvení tak nevznikne tolik větví, kolik má atribut hodnot.

Seskupování hodnot atributu:

1. Opakuj, dokud nezbydou jen dvě skupiny hodnot:
 - (a) zvol dvojici kategorií atributu, které jsou si z hlediska χ^2 nejpodobnější a lze je sloučit;
 - (b) novou kategorii považuj za možné seskupení k provedení v daném kroku.

2. Pro každé z možných seskupení hodnot otestuj χ^2 testem jeho relevanci vůči predikované proměnné.
3. Zvol seskupení, které dává „nejlepší“ seskupení hodnot atributu s nejvýznamnějším dělením.
4. Urči, zda toto seskupení statisticky významně přispívá k rozlišení objektů z různých tříd.

	ID3	C4.5	CART	CHAID
Kritérium	information gain	gain ratio	Gini / twoing	χ^2
Větvení	vícecestné	vícecestné	binární	vícecestné se slučováním
Spojité atributy	ne	ano (prahy)	ano	ano (po diskretizaci)
Chybějící hodnoty	ne	ano	ano (surrogate)	ano
Prořezávání	ne	reduced-error, rule post-pruning	cost-complexity	statistický test zastaví růst
Regrese	ne	ne	ano	ne

5.3 Bayesovská klasifikace

5.3.1 Bayesova věta a MAP hypotéza

Bayesova věta

$$P(H | E) = \frac{P(E | H) P(H)}{P(E)}$$

— pravděpodobnost, že hypotéza H je pravdivá, pozorujeme-li E . Zde

- $P(H)$ — *apriorní* pravděpodobnost hypotézy H ,
- $P(H | E)$ — *aposteriorní* (podmíněná) pravděpodobnost,
- $P(E)$ — pravděpodobnost jevu E ,
- $P(E | H)$ — věrohodnost (*likelihood*).

MAP hypotéza

Nejpravděpodobnější hypotéza H_{MAP} (*maximum a posteriori*) je ta s největší aposteriorní pravděpodobností:

$$H_{MAP} = H_J; \quad P(H_J | E) = \max_t \frac{P(E | H_t)P(H_t)}{P(E)}.$$

Jmenovatel $P(E) = \sum_t P(E | H_t)P(H_t)$ je pro všechny hypotézy stejný, lze jej tedy zanedbat:

$$H_{MAP} = H_J \iff P(E | H_J)P(H_J) = \max_t P(E | H_t)P(H_t).$$

Příklad. Máme poskytnout úvěr klientovi s vysokým příjmem? Dáno:

$$P(\text{PŮJČIT}) = 0,667, \quad P(\text{ZAMÍTNOUT}) = 0,333,$$

$$P(\text{VYSOKÝ} | \text{PŮJČIT}) = 0,91, \quad P(\text{VYSOKÝ} | \text{ZAMÍTNOUT}) = 0,12,$$

$$P(\text{NÍZKÝ} | \text{PŮJČIT}) = 0,09, \quad P(\text{NÍZKÝ} | \text{ZAMÍTNOUT}) = 0,88.$$

Pak

$$P(\text{VYSOKÝ} | \text{PŮJČIT}) P(\text{PŮJČIT}) = 0,607,$$

$$P(\text{VYSOKÝ} | \text{ZAMÍTNOUT}) P(\text{ZAMÍTNOUT}) = 0,040,$$

tedy $H_{MAP} = \text{PŮJČIT}$.

5.3.2 Naivní bayesovský klasifikátor

Naivní bayesovský klasifikátor

Předpoklad: jevy $E_1, \dots, E_K$ (hodnoty jednotlivých atributů) jsou *podmíněně nezávislé* za platnosti hypotézy H . V reálných úlohách je tento předpoklad splněn zřídka — proto „naivní“.

$$P(H | E_1, \dots, E_K) = \frac{P(E_1, \dots, E_K | H)P(H)}{P(E_1, \dots, E_K)} = \frac{P(H)}{P(E_1, \dots, E_K)} \prod_{k=1}^K P(E_k | H).$$

Klasifikujeme hypotézou s největší aposteriorní pravděpodobností:

$$H_{MAP} = H_J \iff P(H_J) \prod_{k=1}^K P(E_k | H_J) = \max_t P(H_t) \prod_{k=1}^K P(E_k | H_t).$$

Odhady pravděpodobností z trénovacích dat:

$$P(H_t) = P(\text{třída}_t) = \frac{\text{počet objektů ze třídy } t}{\text{počet všech objektů}},$$

$$P(E_k | H_t) = \frac{\text{počet objektů ze třídy } t \text{ splňujících } E_k}{\text{počet objektů ze třídy } t}.$$

Výhoda: možnost klasifikovat i neúplně popsane objekty (chybějící atribut se jednoduše vynechá ze součinu).

Nevýhoda: nulová podmíněná pravděpodobnost $P(E_k | H_t)$ (jde o *věrohodnost* atributu při dané třídě, nikoli o aposteriorní pravděpodobnost) v případě, že v trénovací množině není žádný objekt s E_k (jediný nulový činitel vynuluje celý součin), nebo podhodnocené členy $P(E_k | H_t)$ při nízkém společném výskytu E_k a H_t .

Laplaceova korekce

Řešením nulové pravděpodobnosti je *Laplaceova korekce* (*Laplace smoothing*, aditivní vyhlazení): ke každému čítací se přičte 1 (obecně α) a ke jmenovateli počet možných hodnot atributu v :

$$P(E_k | H_t) = \frac{n_{k,t} + 1}{n_t + v}.$$

Odhad zůstane konzistentní (pro velká n_t se blíží původnímu), ale nikdy není nulový.

Příklad: třída má 1000 objektů, atribut *příjem* má 3 hodnoty s četnostmi 0, 990, 10. Bez korekce $P(\text{nízký} | C) = 0$; s korekcí $\frac{1}{1003} = 0,001$, $\frac{991}{1003} = 0,988$, $\frac{11}{1003} = 0,011$.

Příklad — naivní Bayes na datech o úvěru

Použijeme tutéž tabulku 12 klientů jako u rozhodovacích stromů (cílový atribut ÚVĚR). Apriorní pravděpodobnosti:

$$P(\text{Úvěr} = \text{ano}) = 8/12 = 0,667, \quad P(\text{Úvěr} = \text{ne}) = 4/12 = 0,333.$$

Podmíněné pravděpodobnosti:

$$P(\text{Konto} = \text{střední} \mid \text{ano}) = 2/8 = 0,25, \quad P(\text{Konto} = \text{střední} \mid \text{ne}) = 2/4 = 0,5,$$

$$P(\text{Nezam.} = \text{ne} \mid \text{ano}) = 5/8 = 0,625, \quad P(\text{Nezam.} = \text{ne} \mid \text{ne}) = 1/4 = 0,25.$$

Pro klienta, který žádá o úvěr, má *střední* zůstatek na kontě a *není* nezaměstnaný:

$$P(\text{ano}) \cdot P(\text{střední} \mid \text{ano}) \cdot P(\text{ne} \mid \text{ano}) = 0,667 \cdot 0,25 \cdot 0,625 = \mathbf{0,1042},$$

$$P(\text{ne}) \cdot P(\text{střední} \mid \text{ne}) \cdot P(\text{ne} \mid \text{ne}) = 0,333 \cdot 0,5 \cdot 0,25 = \mathbf{0,0416}.$$

Klient bude zařazen do třídy **ÚVĚR(ano)**. (Normalizace: $0,1042/(0,1042 + 0,0416) = 71,5\%$.)

Spojitě atributy se řeší buď diskretizací, nebo předpokladem normálního rozdělení uvnitř třídy:

$$P(x_k \mid H_t) = \frac{1}{\sqrt{2\pi}\sigma_{k,t}} \exp\left(-\frac{(x_k - \mu_{k,t})^2}{2\sigma_{k,t}^2}\right),$$

kde $\mu_{k,t}$ a $\sigma_{k,t}$ se odhadnou z trénovacích dat příslušné třídy. Numericky se místo součinu pravděpodobností sčítají logaritmy (prevence podtečení).

Proč naivní Bayes funguje i při porušení předpokladu? Pro *rozhodnutí* stačí správné pořadí aposteriorních pravděpodobností, nikoli jejich správné hodnoty. Korelované atributy sice zkreslí absolutní hodnoty, ale často zachovají pořadí. Naivní Bayes je proto silný baseline zejména pro klasifikaci textu.

5.3.3 Bayesovské sítě

Bayesovská síť

Bayesovská síť (*Bayesian belief network*) je acyklický orientovaný graf, jehož uzly jsou náhodné veličiny a hrany vyjadřují přímou pravděpodobnostní závislost. Ke každému uzlu X_i je přiřazena tabulka podmíněných pravděpodobností $P(X_i \mid \text{Parents}(X_i))$.

Sdružené rozdělení se pak faktorizuje:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i)).$$

Bayesovská síť zobecňuje naivní Bayes: naivní Bayes je bayesovská síť, v níž je třída jediným rodičem všech atributů a mezi atributy nevede žádná hrana. Bayesovská síť umožňuje zachytit skutečné závislosti mezi atributy za cenu složitějšího učení (struktura se hledá skórovacím kritériem, např. BIC, nebo testy podmíněné nezávislosti; parametry maximální věrohodností či EM při skrytých proměnných) a složitější inference (exaktní eliminací proměnných, přibližné vzorkováním).

5.4 Klasifikace založená na pravidlech

Sada rozhodovacích pravidel

Model je množina pravidel tvaru IF podmínka THEN třída. Rozlišujeme:

- *uspořádanou* sadu (*decision list*) — pravidla se zkouší v pořadí, aplikuje se první, které objekt pokrývá;
- *neuspořádanou* sadu — všechna pokrývající pravidla hlasují (podle své přesnosti nebo pokrytí).

Vlastnosti sady: *vzájemná vylučnost* (*mutually exclusive*) — každý objekt pokrývá nejvýše jedno pravidlo; *vyčerpávající* (*exhaustive*) — každý objekt pokrývá alespoň jedno pravidlo. Není-li sada vyčerpávající, doplní se *implicitní pravidlo* (*default rule*) s majoritní třídou.

Sekvenční pokrývání

Sekvenční pokrývání (*sequential covering*) učí pravidla po jednom:

- 1: $R \leftarrow \emptyset$
- 2: **for** každou třídu c (v pořadí od nejméně četné) **do**
- 3: **while** v D zbývají objekty třídy c **do**
- 4: $r \leftarrow \text{LEARN-ONE-RULE}(D, c)$ ▷ hladově přidávej podmínky do těla pravidla
- 5: odstraň z D objekty pokryté pravidlem r
- 6: $R \leftarrow R \cup \{r\}$
- 7: **end while**
- 8: **end for**
- 9: přidej implicitní pravidlo s majoritní třídou

LEARN-ONE-RULE začíná s prázdným tělem a hladově přidává podmínky maximalizující kritérium: přesnost, *FOIL information gain* $p_1 \left(\log_2 \frac{p_1}{p_1+n_1} - \log_2 \frac{p_0}{p_0+n_0} \right)$, nebo Laplaceův odhad. Aby se nezasekl v lokálním optimu, používá se *paprskové prohledávání* (*beam search*) — udržuje se k nejlepších kandidátů.

RIPPER (*Repeated Incremental Pruning to Produce Error Reduction*, Cohen 1995) je nejpoužívanější algoritmus tohoto typu:

1. data se rozdělí na *growing set* (2/3) a *pruning set* (1/3);
2. pravidlo se hladově vypěstuje na growing set (kritérium FOIL gain) až do 100% přesnosti;
3. pravidlo se okamžitě prořeže na pruning set — postupně se odstraňuje poslední podmínka, dokud roste metrika $\frac{p-n}{p+n}$;
4. pravidla se přidávají, dokud kritérium MDL nezačne růst;
5. následuje *optimalizační fáze*: pro každé pravidlo se zkusí jeho náhrada nově vypěstovaným pravidlem nebo jeho revizí a ponechá se varianta minimalizující MDL.

RIPPER škáluje téměř lineárně s počtem objektů a dobře zvládá nevyvážené třídy.

Sada pravidel může vzniknout i **extrakcí z rozhodovacího stromu** (viz C4.5 rule post-pruning) — takto vzniklá pravidla jsou zpočátku vzájemně vylučná a vyčerpávající, po prořezání tuto vlastnost ztrácejí a je nutné je uspořádat.

5.5 Líné učení: metoda k nejbližších sousedů

k -NN

Líné učení (*lazy learning*, *instance-based learning*) nebuduje explicitní model; trénink spočívá v uložení dat, veškerá práce se odkládá na dobu klasifikace.

Klasifikace objektu $\vec{z}$ metodou k nejbližších sousedů (*k-nearest neighbours*):

1. spočti vzdálenost $\vec{z}$ ke všem trénovacím objektům;
2. vyber k nejbližších;
3. přiřaď třídu, která mezi nimi převažuje (u regrese průměr jejich hodnot).

Vážená varianta dává sousedům váhu $1/d(\vec{z}, \vec{x}_i)^2$.

Vlastnosti: žádná fáze učení, ale drahá klasifikace ($O(pn)$ na dotaz, urychlení pomocí k -d stromů, ball-tree nebo LSH); nutná normalizace atributů (jinak dominují atributy s velkým rozsahem — viz kap. 2); silná citlivost na irelevantní atributy a prokletí dimenzionality; hranice mezi třídami je libovolně složitá (neparametrický model). Volba k : malé $k \rightarrow$ vysoký rozptyl (citlivost na šum), velké $k \rightarrow$ vysoké zkreslení; k se volí křížovou validací, obvykle liché (kvůli remízám).

5.6 Support Vector Machines

SVM vynalezl V. Vapnik se spolupracovníky v 70. letech v Rusku; na Západě se metoda stala známou v roce 1992. SVM jsou lineární klasifikátory, které hledají nadrovinu oddělující dvě třídy dat, pozitivní a negativní; pro nelineární separaci se používají *jádrové funkce*. SVM mají nejen důkladný teoretický základ, ale v aplikacích klasifikují přesněji než většina jiných metod, zejména pro vysokorozměrná data — patří k nejlepším klasifikátorům pro klasifikaci textu.

5.6.1 Základní formulace

Trénovací množina $D = \{(\vec{x}_1, y_1), \dots, (\vec{x}_r, y_r)\}$, $\vec{x}_i \in X \subseteq \mathbb{R}^n$, $y_i \in \{1, -1\}$ (1 = pozitivní, -1 = negativní třída). SVM hledá lineární funkci $f(\vec{x}) = \langle \vec{w} \cdot \vec{x} \rangle + b$ tak, že

$$y_i = \begin{cases} 1 & \text{je-li } \langle \vec{w} \cdot \vec{x}_i \rangle + b \geq 0, \\ -1 & \text{je-li } \langle \vec{w} \cdot \vec{x}_i \rangle + b < 0. \end{cases}$$

Nadrovina $\langle \vec{w} \cdot \vec{x} \rangle + b = 0$ se nazývá *rozhodovací hranice* (*decision boundary*). Takových nadrovin je nekonečně mnoho — kterou zvolit?

Nadrovina s maximálním okrajem

SVM hledá oddělující nadrovinu s *největším okrajem* (*margin*). Teorie strojového učení říká, že tato nadrovina minimalizuje mez chyby (souvisí s VC dimenzí a strukturální minimalizací rizika). Uvažujme nejbližší pozitivní bod $\vec{x}^+$ a nejbližší negativní bod $\vec{x}^-$ a definujme dvě rovnoběžné nadroviny H_+ a H_- jimi procházející. Vzdálenost bodu $\vec{x}_i$ od nadroviny je

$$\frac{|\langle \vec{w} \cdot \vec{x}_i \rangle + b|}{\|\vec{w}\|}, \quad \|\vec{w}\| = \sqrt{\langle \vec{w} \cdot \vec{w} \rangle} = \sqrt{w_1^2 + \dots + w_n^2}.$$

Po normalizaci škály tak, že $H_+ : \langle \vec{w} \cdot \vec{x} \rangle + b = 1$ a $H_- : \langle \vec{w} \cdot \vec{x} \rangle + b = -1$, dostaneme $d_+ = d_- = \frac{1}{\|\vec{w}\|}$ a

$$\text{margin} = d_+ + d_- = \frac{2}{\|\vec{w}\|}.$$

Maximalizace okraje je tedy ekvivalentní minimalizaci $\|\vec{w}\|$:

Lineární SVM: separabilní případ (primární úloha)

$$\text{minimalizuj } \frac{\langle \vec{w} \cdot \vec{w} \rangle}{2} \quad \text{za podmínky } y_i(\langle \vec{w} \cdot \vec{x}_i \rangle + b) \geq 1, \quad i = 1, \dots, r.$$

Podmínka $y_i(\langle \vec{w} \cdot \vec{x}_i \rangle + b) \geq 1$ shrnuje obě nerovnosti: $\langle \vec{w} \cdot \vec{x}_i \rangle + b \geq 1$ pro $y_i = 1$ a $\langle \vec{w} \cdot \vec{x}_i \rangle + b \leq -1$ pro $y_i = -1$.

5.6.2 Duální formulace a support vektory

Úloha se řeší standardní Lagrangeovou metodou:

$$L_P = \frac{1}{2} \langle \vec{w} \cdot \vec{w} \rangle - \sum_{i=1}^r \alpha_i [y_i(\langle \vec{w} \cdot \vec{x}_i \rangle + b) - 1], \quad \alpha_i \geq 0.$$

Optimální řešení musí splňovat *Kuhnovy–Tuckerovy podmínky*, které jsou obecně nutné, nikoli však postačující. Pro naši úlohu s *konvexní* účelovou funkcí a lineárními omezeními jsou však KKT podmínky nutné i postačující.

Support vektory

Podmínka komplementarity $\alpha_i [y_i(\langle \vec{w} \cdot \vec{x}_i \rangle + b) - 1] = 0$ ukazuje, že $\alpha_i > 0$ mohou mít pouze body ležící na okrajových nadrovinách H_+ a H_- (pro ně platí $y_i(\langle \vec{w} \cdot \vec{x}_i \rangle + b) - 1 = 0$). Tyto body se nazývají *support vektory* (*support vectors*); pro všechny ostatní body je $\alpha_i = 0$.

Důsledek: model je určen *jen* support vektory — ostatní trénovací data lze zahodit, aniž se řešení změní. To je zdroj robustnosti i řidkosti modelu.

Nulováním parciálních derivací L_P podle primárních proměnných $\vec{w}$ a b a dosazením zpět do Lagranžíanu (eliminace primárních proměnných) dostaneme *Wolfeho duální úlohu*:

$$L_D = \sum_{i=1}^r \alpha_i - \frac{1}{2} \sum_{i,j=1}^r y_i y_j \alpha_i \alpha_j \langle \vec{x}_i \cdot \vec{x}_j \rangle \rightarrow \max, \quad \sum_{i=1}^r \alpha_i y_i = 0, \quad \alpha_i \geq 0.$$

Pro konvexní účelovou funkci a lineární omezení primární úlohy platí, že maximum L_D nastává při týchž hodnotách $\vec{w}$, b , α_i jako minimum L_P . Řešení vyžaduje numerické techniky (např. SMO — *Sequential Minimal Optimization*).

Rozhodovací hranice a klasifikace testovacího objektu $\vec{z}$:

$$\langle \vec{w} \cdot \vec{z} \rangle + b = \sum_{i \in sv} \alpha_i y_i \langle \vec{x}_i \cdot \vec{z} \rangle + b = 0, \quad \text{sign} \left(\sum_{i \in sv} \alpha_i y_i \langle \vec{x}_i \cdot \vec{z} \rangle + b \right).$$

Vrátí-li výraz 1, je $\vec{z}$ klasifikován jako pozitivní, jinak jako negativní.

5.6.3 Měkký okraj

Nejsou-li data lineárně separabilní (šum, překryv tříd), zavedeme *pomocné proměnné* $\xi_i \geq 0$ (*slack variables*):

$$\text{minimalizuj } \frac{\langle \vec{w} \cdot \vec{w} \rangle}{2} + C \sum_{i=1}^r \xi_i \quad \text{za podmínky } y_i(\langle \vec{w} \cdot \vec{x}_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Parametr $C > 0$ řídí kompromis mezi šířkou okraje a počtem chyb: velké C penalizuje chyby (úzký okraj, riziko přeučení), malé C preferuje široký okraj (více chyb, lepší zobecnění). V duální úloze se změní jediná věc: $0 \leq \alpha_i \leq C$ (*box constraint*).

5.6.4 Jádrový trik

Formulace SVM vyžaduje lineární separaci; reálná data mohou vyžadovat separaci nelineární. Používá se stejná formulace i techniky — pouze data transformujeme do jiného prostoru (obvykle mnohem vyšší dimenze), kde už lineární rozhodovací hranice pozitivní a negativní příklady oddělí. Transformovaný prostor se nazývá *prostor příznaků* (*feature space*), původní prostor *vstupní prostor* (*input space*):

$$\phi : X \rightarrow F, \quad \vec{x} \mapsto \phi(\vec{x}).$$

Příklad transformace. Pro dvourozměrný vstupní prostor a zobrazení $(x_1, x_2) \mapsto (x_1^2, x_2^2, \sqrt{2}x_1x_2)$ se trénovací objekt $((2, 3), -1)$ zobrazí na $((4, 9, 8, 5), -1)$.

Problém explicitní transformace: může trpět prokletím dimenzionality — počet dimenzí prostoru příznaků může být pro užitečné transformace obrovský i při rozumném počtu atributů ve vstupním prostoru, což činí výpočet neproveditelným. Naštěstí explicitní transformace není potřeba.

Jádrová funkce a jádrový trik

V duální formulaci vyžaduje jak konstrukce optimální nadroviny, tak vyhodnocení rozhodovací funkce *pouze skalární součiny* $\langle \phi(\vec{x}) \cdot \phi(\vec{z}) \rangle$, nikdy zobrazený vektor $\phi(\vec{x})$ v explicitní podobě. To je klíčové: máme-li způsob, jak spočítat $\langle \phi(\vec{x}) \cdot \phi(\vec{z}) \rangle$ přímo ze vstupních vektorů $\vec{x}$ a $\vec{z}$, nepotřebujeme znát $\phi(\vec{x})$ ani samotné ϕ . Toho se dosáhne *jádrovou funkcí*

$$K(\vec{x}, \vec{z}) = \langle \phi(\vec{x}) \cdot \phi(\vec{z}) \rangle.$$

Nahrazení všech skalárních součinů jádrovou funkcí se nazývá *jádrový trik* (*kernel trick*).

Ověření pro polynomiální jádro $K(\vec{x}, \vec{z}) = \langle \vec{x} \cdot \vec{z} \rangle^d$ se stupněm $d = 2$ ve dvourozměrném prostoru, $\vec{x} = (x_1, x_2)$, $\vec{z} = (z_1, z_2)$:

$$\begin{aligned} \langle \vec{x} \cdot \vec{z} \rangle^2 &= (x_1z_1 + x_2z_2)^2 = x_1^2z_1^2 + 2x_1z_1x_2z_2 + x_2^2z_2^2 \\ &= \langle (x_1^2, x_2^2, \sqrt{2}x_1x_2) \cdot (z_1^2, z_2^2, \sqrt{2}z_1z_2) \rangle = \langle \phi(\vec{x}) \cdot \phi(\vec{z}) \rangle. \end{aligned}$$

Jádro $\langle \vec{x} \cdot \vec{z} \rangle^2$ je tedy skalárním součinem v transformovaném prostoru příznaků.

Jak poznat, že je funkce jádrem (tj. že skutečně odpovídá skalárnímu součinu v nějakém prostoru příznaků), aniž bychom prováděli výše uvedené odvození? Odpovídá *Mercerova věta*: symetrická spojitá funkce K je jádrem právě tehdy, když je pro libovolnou konečnou množinu bodů odpovídající Gramova matice $[K(\vec{x}_i, \vec{x}_j)]$ pozitivně semidefinitní.

Jádro	Předpis	Poznámka
lineární	$K(\vec{x}, \vec{z}) = \langle \vec{x} \cdot \vec{z} \rangle$	identita jako ϕ ; vysokorozměrná řádká data (text)
polynomiální	$K(\vec{x}, \vec{z}) = (\langle \vec{x} \cdot \vec{z} \rangle + \theta)^d$	interakce atributů do stupně d
gaussovské (RBF)	$K(\vec{x}, \vec{z}) = e^{-\ \vec{x} - \vec{z}\ ^2 / (2\sigma^2)}$	nekonečně rozměrný prostor příznaků; nejuniverzálnější
sigmoidální	$K(\vec{x}, \vec{z}) = \tanh(\kappa \langle \vec{x} \cdot \vec{z} \rangle - \delta)$	odpovídá dvouvrstvé neuronové síti

Další otázky u SVM: SVM pracují pouze v reálném prostoru — kategoriální atributy je nutné převést na numerické (one-hot). SVM provádějí pouze dvoutřídní klasifikaci; pro více tříd se používají strategie *jeden proti zbytku* (*one-against-rest*), *jeden proti jednomu* nebo kódy opravující chyby (*error-correcting output coding*). Nadrovina vytvořená SVM je pro člověka obtížně srozumitelná a jádra situaci ještě zhoršují — SVM se proto běžně používají v aplikacích, které lidské porozumění nevyžadují.

5.7 Neuronové sítě pro dobývání znalostí

5.7.1 Perceptron a vícevrstvá síť

Perceptron počítá $y = g(\sum_j w_j x_j + b)$, kde g je aktivační funkce (skoková, sigmoida $s(u) = \frac{1}{1+e^{-u}}$, ReLU). Jediný perceptron realizuje pouze lineárně separabilní funkce (nikoli XOR). Vícevrstvá dopředná síť (multilayer perceptron) s jednou skrytou vrstvou a nelineární aktivací je univerzální aproximátor.

Zpětné šíření chyby (backpropagation) je gradientní minimalizace chybové funkce $E = \frac{1}{2} \sum_i \|t_i - y_i\|^2$ (nebo křížové entropie): dopředným průchodem se spočítají výstupy, zpětným průchodem se pomocí řetízového pravidla propagují parciální derivace δ od výstupní vrstvy ke vstupní a váhy se upraví $\Delta w_{jk} = -\eta \frac{\partial E}{\partial w_{jk}}$. Problémy: lokální minima, volba počáteční inicializace, mizející gradient, přeučení, pomalá konvergence, nutnost hladké přenosové funkce.

Pro dobývání znalostí jsou neuronové sítě přesné, ale netransparentní („černá skříňka“); používají se metody extrakce pravidel ze sítí a analýzy citlivosti.

5.7.2 Extreme Learning Machines

Síť ELM

Dopředná síť s n vstupními neurony, jednou skrytou vrstvou o L neuronech a výstupní vrstvou. Výstup skrytých neuronů:

$$\vec{h}(\vec{x}) = [G(\vec{a}_1, b_1, \vec{x}), \dots, G(\vec{a}_L, b_L, \vec{x})],$$

kde $\vec{a}_i$ je váhový vektor i -tého neuronu a b_i jeho práh. Přenosová funkce může být

$$\text{sigmoidální: } G(\vec{a}_i, b_i, \vec{x}) = g(\vec{a}_i \vec{x} + b_i), \quad \text{RBF: } G(\vec{a}_i, b_i, \vec{x}) = g(b_i \|\vec{x} - \vec{a}_i\|).$$

Výstup sítě:

$$f_L(\vec{x}) = \sum_{i=1}^L \beta_i G(\vec{a}_i, b_i, \vec{x}),$$

kde β_i je váha mezi i -tým skrytým a výstupním neuronem.

Princip ELM — učení bez iterativních úprav

Nechť g je po částech spojitá funkce. Lze-li spojitou funkci $f(\vec{x})$ aproximovat dopřednou neuronovou sítí se skrytou vrstvou nastavitelných neuronů počítajících funkci g , **není nutné parametry skrytých neuronů takové sítě adaptovat**. Parametry všech skrytých neuronů lze vygenerovat *náhodně*, dokonce i bez znalosti trénovacích dat.

Pro každou spojitou funkci f a náhodně vygenerovanou posloupnost $\{(\vec{a}_i, b_i)\}_{i=1}^L$ platí s pravděpodobností jedna:

$$\lim_{L \rightarrow \infty} \|f(\vec{x}) - f_L(\vec{x})\| = \lim_{L \rightarrow \infty} \left\| f(\vec{x}) - \sum_{i=1}^L \beta_i G(\vec{a}_i, b_i, \vec{x}) \right\| = 0$$

za předpokladu, že hodnoty β_i jsou nastaveny tak, aby minimalizovaly $\|f(\vec{x}) - f_L(\vec{x})\|$.

Matematický model. Pro N různých náhodně zvolených trénovacích vzorů $(\vec{x}_i, \vec{t}_i) \in \mathbb{R}^n \times \mathbb{R}^m$ požadujeme

$$\sum_{i=1}^L \beta_i G(\vec{a}_i, b_i, \vec{x}_j) = \vec{t}_j; \quad j = 1, \dots, N,$$

což je ekvivalentní maticové rovnici $H\beta = T$, kde

$$H = \begin{pmatrix} \vec{h}(\vec{x}_1) \\ \vdots \\ \vec{h}(\vec{x}_N) \end{pmatrix} = \begin{pmatrix} G(\vec{a}_1, b_1, \vec{x}_1) & \cdots & G(\vec{a}_L, b_L, \vec{x}_1) \\ \vdots & \ddots & \vdots \\ G(\vec{a}_1, b_1, \vec{x}_N) & \cdots & G(\vec{a}_L, b_L, \vec{x}_N) \end{pmatrix}_{N \times L},$$

$$\beta = \begin{pmatrix} \beta_1^T \\ \vdots \\ \beta_L^T \end{pmatrix}_{L \times m}, \quad T = \begin{pmatrix} t_1^T \\ \vdots \\ t_N^T \end{pmatrix}_{N \times m}.$$

H je matice výstupů skryté vrstvy; její i -tý sloupec je výstup i -tého skrytého neuronu pro vstupy $\vec{x}_1, \dots, \vec{x}_N$.

Trénovací algoritmus ELM

Vstup: trénovací množina $\aleph = \{(\vec{x}_j, \vec{t}_j) \mid \vec{x}_j \in \mathbb{R}^n, \vec{t}_j \in \mathbb{R}^m, j = 1, \dots, N\}$, přenosová funkce skrytých neuronů $G(\vec{a}, b, \vec{x})$ a počet skrytých neuronů L .

1. Zvol náhodně parametry skrytých neuronů $(\vec{a}_i, b_i)$, $1 \leq i \leq L$.

2. Spočti matici výstupů skryté vrstvy H .

3. Spočti váhy výstupních neuronů: $\beta = H^+ T$,
kde H^+ je Mooreova–Penroseova pseudoinverze matice H .

Regularizovaná varianta (stabilnější, lepší zobecnění): k prvkům na diagonále $H^T H$ se přičte kladná hodnota $1/C$, takže

$$f(\vec{x}) = \vec{h}(\vec{x})\beta = \vec{h}(\vec{x})(H^T H)^{-1} H^T T \rightarrow \vec{h}(\vec{x})(I/C + H^T H)^{-1} H^T T.$$

Výhody ELM:

- jednoduchá neiterativní procedura o třech krocích (mnoho tradičních trénovacích algoritmů je podstatně složitějších);
- velmi rychlý a přímočarý trénink — není nutné řešit lokální minima, vhodnou inicializaci parametrů, přeučení apod.;
- parametry skrytých neuronů $\vec{a}_i$ a b_i jsou nezávislé na trénovacích datech i navzájem;
- ELM může vygenerovat parametry skrytých neuronů ještě *před* předložením trénovacích dat (konvenční techniky musí data „vidět“ dříve, než parametry nastaví);
- lze použít libovolnou omezenou, po částech spojitou nekonstantní přenosovou funkci (gradientní algoritmy vyžadují hladké přenosové funkce).

Nevýhody: obvykle je potřeba více skrytých neuronů než u sítě trénované gradientně; výsledek závisí na náhodné inicializaci; pseudoinverze velké matice je paměťově náročná.

5.8 Regresní a diskriminační modely

5.8.1 Lineární regrese

Regresní analýza zjišťuje vztah veličiny Y k jedné či více veličinám $X_1, \dots, X_n$. Důvody pro její použití: (1) měření výstupních hodnot je nákladné $\Rightarrow$ predikuj výstup ze snadno dostupných vstupů; (2) vstupní hodnoty jsou známy dříve než výstup $\Rightarrow$ odhadni výstup; (3) řízené vstupní hodnoty mohou pomoci nalézt správné chování výstupu; (4) mezi vstupem a výstupem může být kauzální vztah $\Rightarrow$ najdi jej.

Korelační analýza odpovídá na otázku, *zda* mezi dvěma numerickými veličinami existuje lineární vztah; *lineární regrese* určuje *parametry* tohoto vztahu — aproximaci pozorovaných hodnot $[x_i, y_i]$ pomocí $y = \beta x + \alpha + \varepsilon$.

Metoda nejmenších čtverců minimalizuje součet čtverců odchylek skutečných a očekávaných hodnot:

$$\text{SSE} = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - y'_i)^2 = \sum_{i=1}^n (y_i - \alpha - \beta x_i)^2 \rightarrow \min.$$

Derivace podle α a β položíme rovny nule:

$$\frac{\partial(\text{SSE})}{\partial \alpha} = -2 \sum_i (y_i - \alpha - \beta x_i) = 0, \quad \frac{\partial(\text{SSE})}{\partial \beta} = -2 \sum_i ((y_i - \alpha - \beta x_i)x_i) = 0,$$

odkud *normální rovnice*

$$n\alpha + \beta \sum_i x_i = \sum_i y_i, \quad \alpha \sum_i x_i + \beta \sum_i x_i^2 = \sum_i x_i y_i.$$

Z první plyne $\alpha + \beta \bar{x} = \bar{y}$, tedy $\alpha = \bar{y} - \beta \bar{x}$; dosazením do druhé

$$\beta = \frac{\sum_i x_i (y_i - \bar{y})}{\sum_i x_i (x_i - \bar{x})} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad \alpha = \bar{y} - \beta \bar{x},$$

tedy β je podíl výběrové kovariance a výběrového rozptylu x . Predikce: $y = \alpha + \beta x$.

Vícerozměrná lineární regrese. Pro i -té pozorování $y_i = \alpha + \beta_1 x_{i1} + \dots + \beta_m x_{im} + \varepsilon_i$; maticově $\vec{y} = \beta X$, kde

$$X = \begin{pmatrix} 1 & x_{11} & \dots & x_{1m} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \dots & x_{nm} \end{pmatrix}, \quad \beta = (\alpha, \beta_1, \dots, \beta_m)^T.$$

Minimalizací $\text{SSE} = (Y - \beta X)^T (Y - \beta X)$ dostaneme $X^T X \beta = X^T Y$, tedy

$$\boxed{\beta = (X^T X)^{-1} X^T Y}$$

Řešení složitých praktických úloh může být výpočetně nákladné $\Rightarrow$ aproximativní řešení (gradientní sestup, iterativní metody).

5.8.2 Logistická regrese

Příklad nelineární regrese pro *kategoriální* (např. dvouhodnotovou) závislou proměnnou. Modeluje se pravděpodobnost, že y nabude určité hodnoty, v závislosti na kombinaci hodnot nezávislých veličin $\vec{x}$; pracuje se s „podmíněnou šancí“ $P(y | \vec{x}) / (1 - P(y | \vec{x}))$. Pro binární y :

$$\ln \frac{P(y = 1 | x_1, \dots, x_m)}{1 - P(y = 1 | x_1, \dots, x_m)} = \alpha + \beta_1 x_1 + \dots + \beta_m x_m,$$

resp.

$$P(y = 1 | x_1, \dots, x_m) = \frac{e^{\alpha + \sum_j \beta_j x_j}}{1 + e^{\alpha + \sum_j \beta_j x_j}} = \frac{1}{1 + e^{-\alpha - \sum_j \beta_j x_j}} = s(\text{sum}), \quad \text{sum} = \alpha + \sum_j \beta_j x_j.$$

Parametry se odhadují *metodou maximální věrohodnosti* (maximalizace L):

$$L = \prod_{i=1}^n P(y_i | x_{i,1}, \dots, x_{i,m}) = \prod_{i=1}^n \left(\frac{1}{1 + e^{-\alpha - \sum_j \beta_j x_{i,j}}} \right)^{y_i} \left(\frac{1}{1 + e^{\alpha + \sum_j \beta_j x_{i,j}}} \right)^{1-y_i}.$$

Místo maximalizace L se minimalizuje *křížová entropie* $E = -\log L$:

$$E(\beta | X) = - \sum_i \left[y_i \log s(\text{sum}_i) + (1 - y_i) \log (1 - s(\text{sum}_i)) \right].$$

Protože $s'(u) = s(u)(1 - s(u))$, adaptační pravidla mají elegantní tvar

$$\Delta \beta_j = -\eta \frac{\partial E}{\partial \beta_j} = \eta \sum_i (y_i - s(\text{sum}_i)) x_{ij}, \quad \Delta \alpha = \eta \sum_i (y_i - s(\text{sum}_i)),$$

kde η je rychlost učení. Logistická regrese je tedy totéž co jednovrstvá neuronová síť se sigmoidou trénovaná gradientním sestupem.

5.8.3 Diskriminační analýza

Klasifikace objektů do známých tříd — hledá se závislost nominální veličiny (určující příslušnost do třídy) na m ostatních numerických veličinách. Předpokládáme, že ke každé třídě c_t , $t = 1, \dots, T$, existuje *diskriminační funkce* f_t a platí

$$f_t(\vec{x}) = \max_k f_k(\vec{x}), \quad k = 1, \dots, T \iff \vec{x} = (x_1, \dots, x_m) \text{ patří do } c_t.$$

Lineární diskriminační analýza: $f_t = q_{0t} + q_{1t}x_1 + \dots + q_{mt}x_m$. Při diskriminaci do dvou tříd lze místo f_1, f_2 hledat jedinou funkci $f(\vec{x}) = f_1(\vec{x}) - f_2(\vec{x})$ a klasifikovat podle jejího *znaménka* („+“ třída 1, „-“ třída 2).

Optimální klasifikace ve smyslu minimální chyby $\Rightarrow$ diskriminační funkce $\equiv$ podmíněné (aposteriorní) pravděpodobnosti zařazení pozorování $\vec{x}$ do třídy c_t :

$$f_t(\vec{x}) = P(c_t | \vec{x}) = \frac{P(\vec{x} | c_t)P(c_t)}{\sum_k P(\vec{x} | c_k)P(c_k)}; \quad f(\vec{x}) = P(\vec{x} | c_1)P(c_1) - P(\vec{x} | c_2)P(c_2).$$

Normální rozdělení se středními hodnotami $\vec{\mu}_1, \vec{\mu}_2$ a kovariančními maticemi S_1, S_2 dává *kvadratickou* diskriminační funkci:

$$f(\vec{x}) = \frac{1}{2}\vec{x}^T(S_1^{-1} - S_2^{-1})\vec{x} + (\vec{\mu}_1^T S_1^{-1} - \vec{\mu}_2^T S_2^{-1})\vec{x} + \frac{1}{2} \ln \frac{|S_2|}{|S_1|} - \frac{1}{2}(\vec{\mu}_1^T S_1^{-1} \vec{\mu}_1 - \vec{\mu}_2^T S_2^{-1} \vec{\mu}_2) - \ln \frac{P(c_1)}{P(c_2)}.$$

Při *shodných* kovariančních maticích ($S_1 = S_2 = S$) se kvadratický člen vyruší a dostaneme *lineární* diskriminační funkci:

$$f(\vec{x}) = (\vec{\mu}_1^T - \vec{\mu}_2^T)S^{-1}\vec{x} - \frac{1}{2}(\vec{\mu}_1 - \vec{\mu}_2)^T S^{-1}(\vec{\mu}_1 - \vec{\mu}_2) - \ln \frac{P(c_1)}{P(c_2)}.$$

Jsou-li navíc kovarianční matice jednotkové a obě třídy stejně pravděpodobné:

$$f(\vec{x}) = (\vec{\mu}_1^T - \vec{\mu}_2^T)\vec{x} - \frac{1}{2}(\vec{\mu}_1 - \vec{\mu}_2)^T(\vec{\mu}_1 - \vec{\mu}_2),$$

což je klasifikace podle nejbližší střední hodnoty. Pro normální rozdělení se tedy hledání diskriminačních funkcí redukuje na odhad středních hodnot $\vec{\mu}_i$ (výběrovými průměry) a kovariančních matic S_i (z korigovaných výběrových rozptylů).

5.9 Ensemble metody

5.9.1 Bagging

Bagging (*bootstrapped aggregating*) se snaží snížit **rozptyl** klasifikátoru: je-li rozptyl klasifikátoru σ^2 , rozptyl průměru k nezávislých a stejně rozdělených (i.i.d.) klasifikátorů klesne na σ^2/k . Bagging funguje nejlépe, když jednotlivé klasifikátory splňují vlastnost i.i.d.

Rozhodovací stromy jsou pro bagging ideální volbou, protože mají *nízké zkreslení a vysoký rozptyl* (jsou-li dostatečně hluboké). Naopak bagging *nesnižuje zkreslení* modelu a v případech, kdy je hlavním zdrojem chyby zkreslení, může přesnost dokonce mírně zhoršit (kvůli korelacím mezi komponentami ensemble a nesplněnímu předpokladu i.i.d.).

Bootstrap

Data se vzorkují *rovnoměrně s vrácením* z původních dat:

- vzorek má přibližně tutéž velikost jako původní data;
- může obsahovat duplikáty a obsahuje zhruba $1 - (1 - 1/n)^n \approx 1 - 1/e = 63,2\%$ různých objektů z původních dat;
- pravděpodobnost, že objekt *není* zahrnut, je $(1 - 1/n)^n$;
- nezávisle se nakreslí celkem k různých bootstrapových vzorků, každý velikosti n , a na každém se natrénuje klasifikátor;
- pro daný testovací objekt vrací ensemble třídu jako *většinový hlas* různých klasifikátorů.

Zbýlých 36,8 % objektů (*out-of-bag*) lze použít k nestrannému odhadu chyby.

5.9.2 Náhodné lesy

Mají-li k různých klasifikátorů, každý s rozptylem σ^2 , kladnou párovou korelaci ρ , pak rozptyl zprůměrované predikce je

$$\rho\sigma^2 + \frac{(1 - \rho)\sigma^2}{k}.$$

Člen $\rho\sigma^2$ *nezávisí* na počtu komponent k — právě on omezuje zisk z baggingu. Bootstrapované stromy přitom bývají kladně korelované.

Náhodný les

Náhodný les (*random forest*) je ensemble rozhodovacích stromů, do jejichž procesu budování byla explicitně vložena *náhodnost*, aby byly komponenty méně korelované. Lze na něj pohlížet jako na zobecnění baggingu aplikovaného na rozhodovací stromy.

Hlavní myšlenka: je-li jeden strom dobrý, mnoho stromů (les) by mělo být lepší — *je-li mezi stromy dostatečná různorodost*. Náhodnost se zavádí dvěma způsoby:

- **bagging** — stromy se trénují na mírně odlišných datech (bootstrapové vzorky);
- **omezený počet atributů** — v každém uzlu dostane strom náhodnou podmnožinu m atributů a může vybírat pouze z ní, nikoli z celé množiny. To zároveň urychluje trénink (v každém kroku se prohledává méně atributů) a stromy není nutné prořezávat.

Snížení rozptylu přitom *negativně neovlivní zkreslení*. Výstup lesa se pro klasifikaci určí většinovým hlasováním.

Algoritmus: pro každý z N stromů: (1) vytvoř nový bootstrapový vzorek trénovací množiny; (2) natrénuj na něm rozhodovací strom; (3) v každém uzlu náhodně vyber m atributů, spočítej informační zisk pouze na této množině a vyber optimální atribut; (4) opakuj, dokud není strom hotov.

Parametry: počet atributů m — běžnou volbou je $\sqrt{\text{počet atributů}}$ (náhodné lesy nejsou na tento parametr příliš citlivé); počet stromů — stromy lze přidávat, dokud chyba klesá.

Náhodný les navíc poskytuje *důležitost atributů* (*feature importance*): buď průměrné snížení Gini indexu, nebo pokles přesnosti po náhodné permutaci hodnot atributu v out-of-bag vzorku.

5.9.3 Boosting a AdaBoost

Hlavní princip: každému trénovacímu objektu je přiřazena *váha* a různé klasifikátory se trénují s použitím těchto vah; váhy objektů se iterativně upravují podle výkonu klasifikátoru. Budoucí modely tedy závisí na modelech předchozích a každý klasifikátor je konstruován tímž algoritmem na jinak váženém trénovacím souboru.

Myšlenka: v dalších iteracích se soustředí na chybně klasifikované objekty tím, že zvýšíš jejich relativní váhu. Iterativním použitím tohoto přístupu a vytvořením vážené kombinace klasifikátorů lze vytvořit klasifikátor s nižším celkovým **zkreslením**.

Algoritmus AdaBoost (Freund & Schapire, 1996)

```

AdaBoost(Data:  $D$ ; bázový klasifikátor:  $A$ ; max. počet kol:  $T$ )
1:  $t = 0$ ; pro každý objekt  $i$  inicializuj váhu  $W_1(i) = 1/n$ 
2: repeat
3:    $t = t + 1$ 
4:   urči váženou chybovost  $\epsilon_t$  na  $D$ , když se bázový algoritmus  $A$  aplikuje na vážená data
     s vahami  $W_t(\cdot)$ 
5:    $\alpha_t = \frac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t}$   $\triangleright e^{\alpha_t} = \sqrt{(1 - \epsilon_t)/\epsilon_t}$ 
6:   for každý chybně klasifikovaný  $X_i \in D$  do
7:      $W_{t+1}(i) = W_t(i)e^{\alpha_t} = W_t(i)\sqrt{(1 - \epsilon_t)/\epsilon_t}$ 
8:   end for
9:   for každý správně klasifikovaný  $X_i \in D$  do
10:     $W_{t+1}(i) = W_t(i)e^{-\alpha_t} = W_t(i)\sqrt{\epsilon_t/(1 - \epsilon_t)}$ 
11:  end for
12:  normalizuj:  $W_{t+1}(i) = W_{t+1}(i) / \sum_j W_{t+1}(j)$ ,  $1 \leq j \leq n$ 
13: until  $t \geq T$  nebo  $\epsilon_t = 0$  nebo  $\epsilon_t \geq 0,5$ 
14: klasifikuj testovací objekt komponentami s váhami  $\alpha_t$ :  $H(\vec{x}) = \text{sign}(\sum_t \alpha_t h_t(\vec{x}))$ 

Zde  $\epsilon_t$  je podíl chybně predikovaných trénovacích objektů (spočtený po vážení vahami  $W_t(i)$ )
modelem v  $t$ -té iteraci.

```

Shrnutí AdaBoost. *Trénink:* vytváří posloupnost klasifikátorů (se stejným **bázovým učícím algoritmem**); každý klasifikátor závisí na předchozím a soustřeďuje se na jeho chyby; objekty chybně predikované v předchozích klasifikátorech dostávají vyšší váhy. *Testování:* pro testovací případ se výsledky série klasifikátorů zkombinují a určí se výsledná třída. Skutečný výkon závisí na datech a **bázovém učícím algoritmu**. AdaBoost je *citlivý na šum* — je-li počet odlehlých hodnot velmi vysoký, důraz kladený na obtížné příklady může výkon zhoršit.

Náhodné lesy	Boosting
Rychlé — mohou běžet paralelně a v každém kroku prohledávají jen malou množinu atributů	Vyčerpávající — v každém kroku prohledává celou množinu atributů a každý krok závisí na předchozím
Stromy jsou levné na trénink, lze jich ve stejném výpočetním čase vypěstovat více i na velkých a složitých datech	Musí běžet sekvenčně a může být drahý
Přes malé části prostoru problému viděné každým stromem dávají překvapivě dobré výsledky	Pro stejný počet stromů překonává boosting náhodné lesy (ty prohledávají v každém kroku jen malou podmnožinu dat)
Snižuje <i>rozptyl</i>	Snižuje <i>zkreslení</i>

Moderní varianty boostingu: *gradient boosting* (obecná ztrátová funkce, stromy fitují reziduální gradient), implementace XGBoost, LightGBM, CatBoost — v praxi nejúspěšnější metody pro tabulková data. Třetí typ ensemble je *stacking*: predikce **bázových modelů** se stanou vstupy meta-modelu.

5.10 Srovnání klasifikačních metod

Metoda	Interp.	Trénink	Klasif.	Škál.	Typ hranice
Rozhodovací strom	vysoká	rychlý	velmi rychlá	není nutné	osově rovnoběžná
Sada pravidel (RIPPER)	vysoká	střední	rychlá	není nutné	osově rovnoběžná
Naivní Bayes	střední	velmi rychlý	velmi rychlá	není nutné	kvadratická nebo lineární
k -NN	nízká	žádný	pomalá	nutné	libovolná
Lineární / log. regrese	vysoká	rychlý	velmi rychlá	vhodné	lineární
SVM (lineární)	nízká	střední	rychlá	nutné	lineární, max. margin
SVM (RBF)	velmi nízká	pomalý	střední	nutné	libovolná
Neuronová síť	velmi nízká	pomalý	rychlá	nutné	libovolná
ELM	velmi nízká	rychlý	rychlá	nutné	libovolná
Náhodný les	nízká	střední	střední	není nutné	osově rovnoběžná (agreg.)
AdaBoost / GBM	nízká	pomalý	střední	není nutné	osově rovnoběžná (agreg.)

	Vysoké zkreslení (<i>bias</i>)	Vysoký rozptyl (<i>variance</i>)
Typický model	mělký strom, lineární model, naivní Bayes	hluboký strom, k -NN s $k = 1$, velká NN
Symptom	podučení, špatná i trénovací chyba	přeučení, velký rozdíl trénovací a testovací chyby
Lék	složitější model, více atributů, boosting	více dat, regularizace, prořezávání, bagging

5.11 Shrnutí k zkoušce

- **TDIDT / ID3**: hladově shora dolů, kritérium *maximální informační zisk* = minimální vážená entropie. Umět spočítat $H(S)$, $H(A(v))$, $H(A)$ a GAIN na malé tabulce.
- **C4.5**: gain ratio (dělení split-info), spojitě atributy (prahy), chybějící hodnoty (váženě), prořezávání (reduced-error, subtree raising, rule post-pruning), pesimistický odhad přesnosti z binomického rozdělení ($\hat{p} - 1,96\sqrt{\hat{p}(1-\hat{p})/p}$). **C5.0**: komerční, boosting.
- **CART**: binární stromy, Gini / twoing kritérium $\Phi(v | t) = 2P_L P_R \sum_j |P(C_j | t_L) - P(C_j | t_R)|$, regresní stromy, cost-complexity pruning. **CHAID**: χ^2 , slučování hodnot atributů. Umět spočítat i **Gini** na téže tabulce (PŘÍJEM 0,2857 < KONTO 0,3333 < NEZAM. 0,3889 < POHLAVÍ 0,4444 — stejné pořadí jako u entropie) a vědět, že CART u tříhodnotového atributu prověří všechna binární rozdělení.
- **Extrakce pravidel ze stromu**: cesta kořen–list = jedno pravidlo; podmínky v těle, třída v hlavě.
- **Bayes**: $P(H | E) = \frac{P(E|H)P(H)}{P(E)}$; MAP hypotéza (jmenovatel lze zanedbat). **Naivní Bayes**: podmíněná nezávislost atributů, $H_{MAP} = \arg \max_t P(H_t) \prod_k P(E_k | H_t)$; výhoda — umí neúplně popsané objekty; nevýhoda — nulové pravděpodobnosti $\rightarrow$ **Laplaceova korekce**. Umět spočítat příklad s úvěrem (0,1042 vs. 0,0416).
- **Bayesovské sítě**: acyklický orientovaný graf a tabulky podmíněných pravděpodobností; faktORIZACE $\prod_i P(X_i | \text{Parents}(X_i))$; naivní Bayes je speciální případ.
- **Pravidlové klasifikátory**: sekvenční pokrývání, LEARN-ONE-RULE (FOIL gain, beam search),

RIPPER (grow + prune + optimalizace, MDL).

- **k-NN**: líné učení, nutná normalizace, volba k křížovou validací, drahá klasifikace, citlivost na irrelevantní atributy.
- **SVM**: maximální margin $= 2/\|\vec{w}\|$; primární úloha $\min \frac{1}{2}\langle \vec{w} \cdot \vec{w} \rangle$ za $y_i(\langle \vec{w} \cdot \vec{x}_i \rangle + b) \geq 1$; KKT podmínky (pro konvexní úlohu nutné i postačující); *support vektory* = body s $\alpha_i > 0$ ležící na okrajových nadrovinách; duální úloha L_D obsahuje jen skalární součiny $\Rightarrow$ **jádrový trik** $K(\vec{x}, \vec{z}) = \langle \phi(\vec{x}) \cdot \phi(\vec{z}) \rangle$; měkký okraj s C a ξ_i ; jádra: lineární, polynomiální, RBF, sigmoidální; Mercerova věta; jen 2 třídy $\rightarrow$ one-against-rest.
- **ELM**: náhodné parametry skryté vrstvy, výstupní váhy $\beta = H^+T$ (Mooreova–Penroseova pseudoinverze), regularizace $(I/C + H^T H)^{-1} H^T T$; extrémně rychlé, bez lokálních minim, funguje s libovolnou omezenou po částech spojitou přenosovou funkcí.
- **Regrese**: MNČ, $\beta = (X^T X)^{-1} X^T Y$; logistická regrese — sigmoida, maximální věrohodnost, křížová entropie, $\Delta\beta_j = \eta \sum_i (y_i - s(\text{sum}_i)) x_{ij}$.
- **Diskriminační analýza**: $f_t(\vec{x})$ maximální $\Rightarrow$ třída c_t ; při shodných kovariančních maticích lineární, jinak kvadratická diskriminační funkce.
- **Ensemble**: *bagging* snižuje rozptyl (bootstrap, 63,2 % různých objektů, většinové hlasování); *náhodný les* navíc omezí atributy v uzlu ($\sqrt{n}$), aby snížil korelaci ρ ($\rho\sigma^2$ nezávisí na $k!$); *boosting* snižuje zkreslení (váhy objektů, $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$, sekvenční, citlivý na šum).

6 Vyhodnocování modelů

6.1 Kritéria hodnocení klasifikačních metod

- **Prediktivní přesnost** (*predictive accuracy*):

$$\text{Accuracy} = \frac{\text{počet správných klasifikací}}{\text{celkový počet testovaných případů}}, \quad \text{Error} = 1 - \text{Accuracy}.$$

- **Efektivita**: čas potřebný na konstrukci modelu a čas na jeho použití.
- **Robustnost**: zvládání šumu a chybějících hodnot.
- **Škálovatelnost**: efektivita nad databázemi uloženými na disku.
- **Interpreovatelnost**: srozumitelnost a vhled poskytnutý modelem.
- **Kompaktnost modelu**: velikost stromu, počet pravidel apod.

6.2 Metody odhadu chyby

Holdout, křížová validace, leave-one-out, validační množina

Holdout — dostupná data D se rozdělí na dvě *disjunktní* podmnožiny: trénovací množinu D_{train} (učení modelu) a testovací množinu D_{test} (testování modelu). *Důležité*: trénovací množina se nesmí použít při testování a testovací množina při učení — neviděná testovací množina poskytuje *nestranný* odhad přesnosti. Používá se hlavně pro velké D (typicky 2/3 : 1/3).

k -násobná křížová validace (*k-fold cross-validation*) — dostupná data se rovnoměrně rozdělí na k stejně velkých disjunktních podmnožin. Každá podmnožina se použije jako testovací množina a zbývajících $k - 1$ podmnožin se spojí do trénovací množiny. Procedura běží k -krát a poskytne k přesností Accuracy_i ; výsledný odhad je jejich průměr:

$$\overline{\text{Accuracy}} = \frac{1}{k} \sum_{i=1}^k \text{Accuracy}_i.$$

Běžné je 10-násobná a 5-násobná křížová validace; metoda je vhodná spíše pro menší data. *Stratifikovaná* varianta zachovává v každém foldu poměr tříd.

Leave-one-out — speciální případ křížové validace pro velmi malá data: každý fold má jediný testovací příklad a všechna ostatní data se použijí k učení. Má-li původní množina m příkladů, jde o m -násobnou křížovou validaci.

Validační množina — data se rozdělí na tři podmnožiny: trénovací, validační a testovací. Validační množina slouží k odhadu parametrů (hyperparametrů) učicích algoritmů: použijí se hodnoty, které dávají nejlepší přesnost na validační množině. Ke stejnému účelu lze použít i křížovou validaci (vnořenou).

Konfidenční interval pro křížovou validaci

Přibližný N % konfidenční interval:

$$\overline{\text{Accuracy}} \mp t_{N,k-1} s_{\overline{\text{Accuracy}}}, \quad s_{\overline{\text{Accuracy}}} = \sqrt{\frac{1}{k(k-1)} \sum_{i=1}^k (\text{Accuracy}_i - \overline{\text{Accuracy}})^2}.$$

Hodnoty $t_{N,\nu}$ pro oboustranné konfidenční intervaly:

	hladina spolehlivosti N		
	90 %	95 %	99 %
$\nu = 2$	2,92	4,30	9,92
$\nu = 10$	1,81	2,23	3,17
$\nu = 30$	1,70	2,04	2,75
$\nu = \infty$	1,64	1,96	2,58

Často se $t_{N,\nu}$ klade rovno 1,96 (pro $\nu = \infty$ a $N = 95$ %).

Bootstrap — alternativa pro velmi malá data: opakovaně se losuje n objektů *s vrácením* (trénovací množina), zbylých přibližně 36,8 % (*out-of-bag*) tvoří testovací množinu. Odhad 0,632 *bootstrap* kombinuje optimistickou trénovací a pesimistickou *out-of-bag* přesnost:

$$\text{Acc} = 0,632 \cdot \text{Acc}_{\text{oob}} + 0,368 \cdot \text{Acc}_{\text{train}}.$$

6.3 Matice záměn a odvozené míry

Přesnost je jen jedna míra a v některých aplikacích není vhodná. V textovém dolování nás mohou zajímat pouze dokumenty určitého tématu, které tvoří jen malou část velké kolekce. U klasifikace se *šikmými* nebo *silně nevyváženými* daty (detekce průniků do sítě, finanční podvody) nás zajímá jen minoritní třída: vysoká přesnost neznamená, že byl detekován jakýkoli průnik — například při 1 % průniků dosáhneme 99 % přesnosti tím, že neuděláme nic. Zajímavá třída se nazývá *pozitivní*, ostatní *negativní*.

Matice záměn

		Predikovaná třída	
		pozitivní	negativní
Skutečná třída	pozitivní	TP	FN
	negativní	FP	TN

- TP (*true positive*) — počet správných klasifikací pozitivních příkladů;
- FN (*false negative*) — počet nesprávných klasifikací pozitivních příkladů;
- FP (*false positive*) — počet nesprávných klasifikací negativních příkladů;
- TN (*true negative*) — počet správných klasifikací negativních příkladů.

$$p = \text{precision} = \frac{TP}{TP + FP}, \quad r = \text{recall} = \frac{TP}{TP + FN},$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Přesnost p (precision) je počet správně klasifikovaných pozitivních příkladů dělený celkovým počtem příkladů klasifikovaných jako pozitivní. *Úplnost r (recall)* je počet správně klasifikovaných pozitivních příkladů dělený celkovým počtem skutečně pozitivních příkladů v testovací množině.

Poznámka: precision i recall měří klasifikaci *pouze na pozitivní třídě*.

Příklad z přednášky: matice záměn s $TP = 1$, $FN = 99$, $FP = 0$, $TN = 1000$ dává $p = 100$ % a $r = 1$ %, protože jsme správně klasifikovali jen jediný pozitivní příklad a žádný negativní příklad

špatně. Samotná precision tedy o kvalitě modelu nic neříká.

F_1 míra

Porovnávat dva klasifikátory pomocí dvou měr je obtížné; F_1 skóre spojuje precision a recall do jediné míry — jde o *harmonický průměr*:

$$F_1 = \frac{2pr}{p+r} = \frac{2}{\frac{1}{p} + \frac{1}{r}}.$$

Harmonický průměr dvou čísel táhne k menšímu z nich; aby byla hodnota F_1 velká, musí být *obě* hodnoty p a r velké.

Obecněji $F_\beta = \frac{(1+\beta^2)pr}{\beta^2 p + r}$: $\beta > 1$ zvýhodní recall, $\beta < 1$ precision.

Pro vícetřídní úlohy se počítá *makro-průměr* (průměr měr přes třídy, každá třída má stejnou váhu) nebo *mikro-průměr* (sečtou se TP, FP, FN přes všechny třídy a teprve pak se počítá míra — převáží četné třídy).

6.4 Skórování, řazení a lift analýza

Skórování (*scoring*) souvisí s klasifikací: zajímá nás jediná třída (pozitivní), např. třída kupujících v marketingové databázi. Místo přiřazení definitivní třídy přiřadí skórování každému testovacímu objektu *odhad pravděpodobnosti* (*probability estimate*, PE), který udává, jak pravděpodobně objekt do pozitivní třídy patří.

Po přiřazení PE lze objekty seřadit a rozdělit do n (např. 10) košů; podle počtu pozitivních příkladů v jednotlivých koších se vykreslí *lift křivka*. Klasifikační systémy lze pro skórování použít, pokud produkují odhad pravděpodobnosti (v rozhodovacím stromu např. hodnota confidence v listu).

Příklad — lift analýza

Chceme rozeslat propagační materiály potenciálním zákazníkům, abychom prodali hodinky. Každý balíček stojí \$0,50 (materiál a poštovné); prodej hodinek přinese \$5 zisku. Kolik balíčků poslat a komu?

Testovací množina má 10 000 objektů, z toho 500 pozitivních. Po sestavení klasifikátoru objekty oskórujeme, seřadíme a rozdělíme do 10 košů po 1000:

Koš	1	2	3	4	5	6	7	8	9	10
Poz. případů	210	120	60	40	22	18	12	7	6	5
Podíl	42 %	24 %	12 %	8 %	4,4 %	3,6 %	2,4 %	1,4 %	1,2 %	1 %
Kumulativně	42 %	66 %	78 %	86 %	90,4 %	94 %	96,4 %	97,8 %	99 %	100 %

Lift křivka vynáší kumulativní procento pozitivních případů proti procentu testovaných případů; úhlopříčka odpovídá náhodnému výběru. Rozešleme-li balíčky prvním třem košům (3000 balíčků, náklad \$1500), zasáhneme 390 pozitivních zákazníků (zisk $390 \cdot \$5 = \1950 , čistý zisk \$450). Poslední koše se nevyplatí: 1000 balíčků za \$500 přinese jen 5 prodejů, tedy \$25.

6.5 ROC křivka a AUC

ROC křivka

Receiver Operating Characteristic curve se běžně používá k vyhodnocení klasifikačních výsledků na pozitivní třídě u dvoutřídních úloh. Je to graf *míry správné pozitivní klasifikace* (TPR) proti *míře falešné pozitivní klasifikace* (FPR):

$$\text{TPR} = \frac{TP}{TP + FN} \quad (\text{podíl správně klasifikovaných pozitivních; totéž co } recall, \text{ též } senzitivita),$$

$$\text{FPR} = \frac{FP}{TN + FP} \quad (\text{podíl skutečně negativních klasifikovaných jako pozitivní}),$$

$$\text{TNR} = \frac{TN}{TN + FP} \quad (\text{specifická, tj. recall negativní třídy}); \quad \text{FPR} = 1 - \text{specifická}.$$

Každá ROC křivka začíná v bodě (0, 0) — každý testovaný případ klasifikován jako negativní — a končí v (1, 1) — každý případ klasifikován jako pozitivní. Hlavní úhlopříčka odpovídá náhodnému hádání (predikce pozitivní třídy s pevnou pravděpodobností), kdy $\text{TPR} = \text{FPR}$.

Konstrukce ROC křivky (je-li klasifikace dána jako uspořádání testovaných případů):

1. Získej uspořádání seřazením testovaných případů podle klesající výstupní hodnoty klasifikátoru (např. aposteriorní pravděpodobnosti).
2. Rozděľ seznam na dvě podmnožiny v každém testovaném případě; každý případ v horní části (s vyšší výstupní hodnotou) považuj za pozitivní a každý případ v dolní části za negativní.
3. Pro každé takové rozdělení spočti dvojici hodnot TPR a FPR. Je-li horní část prázdná, dostaneme bod (0, 0); je-li prázdná dolní část, dostaneme bod (1, 1).
4. Nakonec spoj sousední body.

Příklad — výpočet bodů ROC křivky

Máme 10 testovaných případů seřazených klasifikátorem (pozice 1 je nejvyšší rank):

Rank	1	2	3	4	5	6	7	8	9	10
Skutečná třída	+	+	-	+	-	+	-	-	+	-

Celkem $P = 5$ pozitivních a $N = 5$ negativních. Pro každé rozdělení po k -tém případě:

k	0	1	2	3	4	5	6	7	8	9	10
TP	0	1	2	2	3	3	4	4	4	5	5
FP	0	0	0	1	1	2	2	3	4	4	5
TPR	0,0	0,2	0,4	0,4	0,6	0,6	0,8	0,8	0,8	1,0	1,0
FPR	0,0	0,0	0,0	0,2	0,2	0,4	0,4	0,6	0,8	0,8	1,0

Plocha pod křivkou (lichoběžníkovým pravidlem po úsecích šířky 0,2):

$$\text{AUC} = 0,2 \cdot 0,4 + 0,2 \cdot 0,6 + 0,2 \cdot 0,8 + 0,2 \cdot 0,8 + 0,2 \cdot 1,0 = 0,72.$$

Kontrola Mannovým–Whitneyho vztahem: $\text{AUC} =$ podíl dvojic (pozitivní, negativní), v nichž je pozitivní případ ohodnocen výše. Pozitivní jsou na pozicích 1, 2, 4, 6, 9 a předbíhají po řadě 5, 5, 4, 3 a 1 negativních případů, tedy $\text{AUC} = (5 + 5 + 4 + 3 + 1)/(5 \cdot 5) = 18/25 = 0,72$.

AUC a srovnávání klasifikátorů

Uvažujme dva klasifikátory C_1 , C_2 na týchž datech. Jejich ROC křivky se mohou křížit: může platit, že „při FPR menším než 0,43 je lepší C_1 , při FPR větším než 0,43 je lepší C_2 “. Pak nelze jednoznačně říci, který je lepší — záleží na provozním bodě.

Souhrnně se proto používá **plocha pod ROC křivkou** (*Area Under the Curve*, AUC):

- je-li AUC klasifikátoru C_i větší než AUC klasifikátoru C_j , říkáme, že C_i je lepší než C_j ;
- je-li klasifikátor dokonalý, jeho AUC je 1;
- hádá-li klasifikátor zcela náhodně, jeho AUC je 0,5.

Interpretace: AUC je pravděpodobnost, že náhodně zvolený pozitivní případ dostane vyšší skóre než náhodně zvolený negativní případ. AUC je *necitlivá na nevyváženost tříd* — proto se u nevyvážených úloh doporučuje spíše *PR křivka* (precision proti recall) a AUPRC.

6.6 Přeučení a srovnávání klasifikátorů

Přeučení

Přeučení (*overfitting*) nastává, když model zachytí i šum a specifika trénovací množiny, takže jeho chyba na trénovacích datech klesá, ale chyba na nezávislých datech roste. *Podučení* (*underfitting*) je opačný jev — model je příliš jednoduchý na to, aby zachytil skutečnou strukturu.

Očekávaná chyba se rozkládá na

$$\mathbb{E}[\text{chyba}] = \underbrace{\text{zkreslení}^2}_{\text{bias}} + \underbrace{\text{rozptyl}}_{\text{variance}} + \underbrace{\text{neredukovatelný šum}}_{\sigma^2}.$$

Složitější model snižuje zkreslení a zvyšuje rozptyl — odtud *kompromis mezi zkreslením a rozptylem*.

Obrana: prořezávání, regularizace (L_1 , L_2), předčasné zastavení, více dat, ensemble metody, a především *poctivé* oddělení testovací množiny.

Časté metodické chyby: použití testovací množiny při výběru atributů či ladění hyperparametrů (*data leakage* — výběr atributů musí být *uvnitř* křížové validace); normalizace spočítaná na celých datech včetně testovacích; u časových řad náhodné rozdělení místo rozdělení podle času; opakované testování mnoha modelů na téže testovací množině („přeučení testovací množiny“).

Statistické srovnávání klasifikátorů:

- *Párový t-test* nad výsledky k -násobné křížové validace: testuje se, zda je průměrný rozdíl přesností významně různý od nuly. Pozor: folds nejsou nezávislé, test je proto mírně optimistický (doporučuje se opakovaná 5×2 křížová validace).
- *McNemarův test* nad jedinou testovací množinou: porovnává počty případů, kde se klasifikátory liší (n_{01} , n_{10}); statistika $\frac{(|n_{01}-n_{10}|-1)^2}{n_{01}+n_{10}}$ má rozdělení χ^2 s 1 stupněm volnosti.
- *Friedmanův test* + Nemenyiho post-hoc pro srovnání více klasifikátorů na více datových sadách.

Cenově citlivé hodnocení: má-li chyba různý dopad (FN u detekce nádoru vs. FP), definuje se matice nákladů $c(i, j)$ a minimalizují se očekávané náklady $\sum_{i,j} c(i, j) \cdot n_{ij}$ místo chybovosti. Odpovídající prahová hodnota skóre je $\frac{c(FP)}{c(FP)+c(FN)}$.

6.7 Shrnutí k zkoušce

- **Kritéria:** přesnost, efektivita, robustnost, škálovatelnost, interpretovatelnost, kompaktnost.
- **Odhad chyby:** holdout (velká data), k -fold CV (malá data, průměr k přesností, konfidenční interval $\overline{Acc} \pm t_{N,k-1}s$), leave-one-out (velmi malá data), bootstrap (0,632), validační množina pro hyperparametry.
- **Matice záměn:** TP, FN, FP, TN; $p = \frac{TP}{TP+FP}$, $r = \frac{TP}{TP+FN}$, $F_1 = \frac{2pr}{p+r}$ (harmonický průměr — musí být velké obojí).

- **Proč nestačí accuracy:** nevyvážené třídy — při 1 % pozitivních dosáhneme 99 % přesnosti nicneděláním.
- **ROC:** TPR (senzitivita, recall) proti FPR ($= 1 - \text{specificita}$); od (0,0) do (1,1); úhlopříčka = náhodné hádání. **AUC:** 1 dokonalý, 0,5 náhodný; interpretace jako pravděpodobnost správného pořadí náhodné dvojice. Umět zkonstruovat body ROC z uspořádaného seznamu.
- **Lift analýza:** skórování $\rightarrow$ řazení $\rightarrow$ koše $\rightarrow$ kumulativní podíl pozitivních; ekonomické rozhodnutí (kolik balíčků poslat).
- **Přeučení:** rozklad chyby na zkreslení + rozptyl + šum; obrana prořezáváním, regularizací, více daty, ensemble.
- **Srovnávání:** párový t -test nad CV, McNemarův test, cenově citlivé hodnocení.

7 Klastrová analýza

7.1 Formulace úlohy a míry vzdálenosti

Shlukování

Shlukování (clustering) rozděluje pozorované objekty do skupin (*shluků, clusters*) vzájemně podobných objektů. Předpoklad: umíme měřit *vzdálenost* mezi objekty. Každý objekt je charakterizován m numerickými veličinami.

Jde o *učení bez učitele*: neexistuje správná odpověď, kvalita se posuzuje interními (kompaktnost, oddělenost) nebo externími (shoda se známým rozdělením) kritérii.

Cílem je *vysoká vnitroshluková podobnost a nízká mezishluková podobnost*.

Míry vzdálenosti

Pro $\vec{x}_1 = (x_{11}, \dots, x_{1m})$ a $\vec{x}_2 = (x_{21}, \dots, x_{2m})$:

1. **Hammingova** (Manhattanská, městských bloků): $d_H(\vec{x}_1, \vec{x}_2) = \sum_{j=1}^m |x_{1j} - x_{2j}|$
2. **Euklidovská**: $d_E(\vec{x}_1, \vec{x}_2) = \sqrt{\sum_{j=1}^m (x_{1j} - x_{2j})^2}$
3. **Čebyševova**: $d_C(\vec{x}_1, \vec{x}_2) = \max_j |x_{1j} - x_{2j}|$
4. **Minkowského metrika** (1.–3. jsou jejími speciálními případy):

$$L^{(z)}(\vec{x}_1, \vec{x}_2) = \sqrt[z]{\sum_{j=1}^m |x_{1j} - x_{2j}|^z}, \quad z \geq 1$$

Platí $d_H = L^{(1)}$, $d_E = L^{(2)}$, $d_C = \lim_{z \rightarrow \infty} L^{(z)}$. *Absolutní hodnota je podstatná*: bez ní by $L^{(1)}$ dalo pouhý algebraický součet rozdílů (a mohlo být záporné) a pro liché z by výraz pod odmocninou nebyl nezáporný.

Volba metriky závisí na zúčastněných veličinách a jejich rozsahu $\Rightarrow$ veličiny je nutné *normalizovat* (dělit průměrem, směrodatnou odchylkou, rozpětím $\max - \min$, ...). Předpokládáme přitom stejný rozptyl všech veličin; mají-li veličiny *různý* rozptyl (a jsou-li korelované), použije se **Mahalanobisova vzdálenost**

$$d_M^2(\vec{x}_1, \vec{x}_2) = (\vec{x}_1 - \vec{x}_2)^T S^{-1} (\vec{x}_1 - \vec{x}_2),$$

kde S je kovarianční matice. Pro binární a kategoriální data se používá Jaccardův koeficient, kosinová podobnost (text) nebo Gowerova vzdálenost (smíšené typy).

Vzdálenost mezi shluky (metody spojování)

Pro shluky U a V :

- **Metoda nejbližšího souseda** (*single linkage*) — minimum ze vzdáleností jejich prvků:

$$D(U, V) = \min_{k,l} d(\vec{x}_k, \vec{x}_l); \quad \vec{x}_k \in U, \vec{x}_l \in V$$

- **Metoda nejvzdálenějšího souseda** (*complete linkage*) — maximum ze vzdáleností:

$$D(U, V) = \max_{k,l} d(\vec{x}_k, \vec{x}_l)$$

- **Metoda průměrné vzdálenosti** (*average linkage*) — průměr vzdáleností mezi objekty (n_U, n_V jsou počty objektů ve shlucích):

$$D(U, V) = \frac{1}{n_U n_V} \sum_{k=1}^{n_U} \sum_{l=1}^{n_V} d(\vec{x}_k, \vec{x}_l)$$

- **Centroidní metoda** — vzdálenost středů shluků ($\vec{u}, \vec{v}$ jsou centroidy U, V):

$$D(U, V) = d(\vec{u}, \vec{v})$$

- **Wardova metoda** — slučuje dvojici, jejíž sloučení nejméně zvýší celkový vnitroshlukový součet čtverců.

Chování metod: *single linkage* umí najít protáhlé shluky libovolného tvaru, ale trpí *řetězovým efektem* (*chaining*) — dva shluky spojí jediný můstek bodů. *Complete linkage* vytváří kompaktní kulovité shluky a je citlivá na odlehlé hodnoty. *Average* a *Ward* jsou kompromisem, Ward tíhne k shlukům podobné velikosti.

Centroid

Centroid je střed shluku — „prototyp“, který daný shluk reprezentuje. Tentýž shluk může být reprezentován jedním i více centroidy současně, podle tvaru shluku a zvolené metriky pro vyhodnocení vzdáleností.

7.2 Rozdělovací metody: k -means

Algoritmus k -means (Lloyd, 1982)

1. Rozděli objekty náhodně do K neprázdných shluků $U_1, \dots, U_K$, které jsou *párově disjunkt* a pokrývají celá data: $U_k \neq \emptyset$, $U_k \cap U_l = \emptyset$ pro $k \neq l$, $\bigcup_{k=1}^K U_k = D$.
2. Urči centroidy všech shluků v aktuálním rozdělení, např. podle

$$\vec{c}_k = \frac{1}{|U_k|} \sum_{\vec{x}_i \in U_k} \vec{x}_i.$$

3. Pro každý objekt $\vec{x}$:
 - (a) urči vzdálenosti $d(\vec{x}, \vec{c}_k)$, $1 \leq k \leq K$ ($\vec{c}_k$ je centroid k -tého shluku);
 - (b) nechť $d(\vec{x}, \vec{c}_l) = \min_k d(\vec{x}, \vec{c}_k)$;
 - (c) nepatří-li $\vec{x}$ do shluku l (s nejbližší vzdáleností k centroidu $\vec{c}_l$), přesuň $\vec{x}$ do shluku l .
4. Došlo-li k přesunu, jdi na krok 2, jinak KONEC.

Následně jsou shluky reprezentovány svými centroidy.

Varianty: při počátečním rozdělení prohlásit prvních k objektů za centroidy (eliminuje krok 2); upravovat centroidy po každém přesunu (v cyklu kroku 3).

k -means minimalizuje součet čtverců vzdáleností objektů od jejich centroidů (*within-cluster sum of*

squares, SSE):

$$\text{SSE} = \sum_{k=1}^K \sum_{\vec{x} \in U_k} \|\vec{x} - \vec{c}_k\|^2.$$

Jde o *alternující minimalizaci* (přiřazení $\rightarrow$ centroidy $\rightarrow$ přiřazení), která konverguje k *lokálnímu* minimu.

Příklad — průchod algoritmem k -means

Body: $P_1 = (1, 1)$, $P_2 = (2, 1)$, $P_3 = (4, 3)$, $P_4 = (5, 4)$; $k = 2$; počáteční centroidy $\vec{c}_1 = (1, 1)$, $\vec{c}_2 = (2, 1)$ (euklidovská vzdálenost).

Iterace 1 — přiřazení:

Bod	d k $\vec{c}_1 = (1, 1)$	d k $\vec{c}_2 = (2, 1)$	shluk
$P_1(1, 1)$	0,000	1,000	1
$P_2(2, 1)$	1,000	0,000	2
$P_3(4, 3)$	$\sqrt{9+4} = 3,606$	$\sqrt{4+4} = 2,828$	2
$P_4(5, 4)$	$\sqrt{16+9} = 5,000$	$\sqrt{9+9} = 4,243$	2

Nové centroidy: $\vec{c}_1 = (1; 1)$, $\vec{c}_2 = (\frac{2+4+5}{3}, \frac{1+3+4}{3}) = (3,667; 2,667)$.

Iterace 2 — přiřazení:

Bod	d k $(1; 1)$	d k $(3,667; 2,667)$	shluk
P_1	0,000	3,145	1
P_2	1,000	2,357	1 (přesun)
P_3	3,606	0,471	2
P_4	5,000	1,886	2

Nové centroidy: $\vec{c}_1 = (1,5; 1)$, $\vec{c}_2 = (4,5; 3,5)$.

Iterace 3: P_1 : 0,5 vs. 4,30; P_2 : 0,5 vs. 3,54; P_3 : 3,20 vs. 0,707; P_4 : 4,61 vs. 0,707. **Žádný přesun $\Rightarrow$ konec.**

Výsledek: $\{P_1, P_2\}$ s centroidem $(1,5; 1)$ a $\{P_3, P_4\}$ s centroidem $(4,5; 3,5)$;

$$\text{SSE} = (0,25 + 0,25) + (0,5 + 0,5) = 1,5.$$

Rozšíření Lloydova algoritmu

Kroky alternující minimalizace neposkytují záruku aproximace, Lloydův algoritmus však nikdy nezvýší cenu počátečního shlukování $\Rightarrow$ je vhodné poskytnout algoritmu prokazatelně dobrou inicializaci.

k -means++ seeding (Arthur, Vassilvitskii, 2007): první centroid zvol rovnoměrně náhodně; každý další vzorkuj s pravděpodobností úměrnou *kvadratické vzdálenosti* k „nejbližšímu“ již zvolenému centroidu. Označíme-li $D(\vec{x}) = \min_{\vec{c} \in C} d(\vec{x}, \vec{c})$ vzdálenost bodu k nejbližšímu dosud zvolenému centroidu, je pravděpodobnost výběru bodu $\vec{x}$ rovna

$$\frac{D^2(\vec{x})}{\sum_{q=1}^P D^2(\vec{x}_q)}.$$

Dává $O(\log k)$ aproximaci optimálního řešení v očekávání.

LocalSearch++ (Lattanzi, Sohler, 2019): nejprve k -means++ pro volbu k počátečních centroidů, poté se nové objekty vzorkují s analogickou pravděpodobností a nahrazují existující centroidy, pokud sníží celkovou kvadratickou vzdálenost objektů k nejbližším centroidům.

Vlastnosti k -means: časová složitost $O(tkn d)$ (t iterací); jednoduchý a rychlý; *nevýhody* — nutnost zadat k předem, citlivost na inicializaci (lokální minima), citlivost na odlehlé hodnoty (centroid se táhne za outlierem), předpoklad kulovitých shluků podobné velikosti a hustoty, nepracuje s kategoriálními atributy (varianta k -modes používá modus a Hammingovu vzdálenost).

7.3 k -medoids, PAM, CLARA, CLARANS

Algoritmus k -medoids

Uvažujeme datovou sadu D o n d -rozměrných objektech $\bar{X}_1, \dots, \bar{X}_n$. Cílem je určit k reprezentantů $\bar{Y}_1, \dots, \bar{Y}_k$ vybraných z původní databáze D (k zadá uživatel) tak, aby minimalizovaly účelovou funkci danou součtem vzdáleností mezi objekty a jejich nejbližšími reprezentanty:

$$O = \sum_{i=1}^n \min_j \text{Dist}(\bar{X}_i, \bar{Y}_j).$$

Časová složitost jedné iterace je $O(k \cdot n \cdot d)$; obvykle stačí malý konstantní počet iterací.

Proč vybírat reprezentanty z D ?

1. Shluky k -means mohou být zkresleny odlehlými hodnotami $\Rightarrow$ reprezentanti shluků se mohou ocitnout v prázdných oblastech nereprezentativních pro většinu objektů v daném shluku $\Rightarrow$ takoví reprezentanti mohou vést k částečnému splynutí různých shluků. (Tento problém lze alespoň částečně vyřešit pečlivým ošetřením odlehlých hodnot.)
2. U složitých dat (např. množiny časových řad různé délky) může být obtížné spočítat optimálního centrálního reprezentanta $\Rightarrow$ výběr reprezentantů z původní datové sady je velmi užitečný. k -medoids lze definovat prakticky pro libovolný typ dat, pokud je k dispozici vhodná funkce podobnosti či vzdálenosti.

Algoritmus používá *horolezectví* (*hill-climbing*) s množinou reprezentantů S inicializovanou body z D ; S se iterativně zlepšuje výměnou bodu z S za objekt z D . Možné strategie výměny:

1. vyzkoušet všech $|S| \cdot |D|$ možností nahrazení reprezentanta v S objektem z D a vybrat nejlepší $\Rightarrow$ extrémně drahé;
2. použít náhodně vybranou množinu r dvojic $(\bar{X}_i, \bar{Y}_j)$ pro možnou výměnu ($\bar{X}_i$ z D , $\bar{Y}_j$ z S); nejlepší z r dvojic se použije pro výměnu. Čas úměrný $r \cdot |D|$; algoritmus konvergoval, když se účelová funkce nezlepší, resp. je-li průměrné zlepšení pod uživatelem daným prahem.

k -medoids je pomalejší než k -means, ale použitelný pro různé typy dat; jeho škálovatelnou verzí jsou CLARA/CLARANS.

CLARA a CLARANS

Motivace: v mnoha aplikacích jsou data velmi rozsáhlá, nevejdou se do hlavní paměti a musí zůstat na disku — to klade omezení na algoritmičtý návrh. CLARA a CLARANS jsou škálovatelné implementace k -medoids; dědí výhody i nevýhody bazového algoritmu (CLARANS je snadno zobecnitelný na různé typy dat, ale dědí relativně vysokou výpočetní složitost k -medoids).

CLARA (*Clustering LARge Applications*) vychází z varianty k -medoids zvané PAM (*Partitioning Around Medoids*): testují se všechny možné dvojice $k \cdot (n - k)$ medoidů a nemedoidů pro výměnu a vymění se dvojice s nejlepším zlepšením účelové funkce; výměny se provádějí, dokud algoritmus nekonverguje k lokálně optimální hodnotě. Každá iterace vyžaduje $O(k \cdot n^2)$ výpočtů vzdáleností, tj. $O(k \cdot n^2 \cdot d)$ času pro d -rozměrná data — *poměrně drahé*.

Složitost lze snížit aplikací algoritmu na menší vzorek velikosti $f \cdot n$ (kde $f \ll 1$ je vzorkovací podíl) pro nalezení medoidů; zbývající (nevzorkované) objekty se přiřadí k nalezeným medoidům. Postup se opakovaně aplikuje na nezávisle zvolené vzorky téže velikosti a nejlepší shlukování přes všechny vzorky se vybere jako výsledné řešení. Pro malá f může být CLARA řádově rychlejší (složitost iterace $O(k \cdot f^2 \cdot n^2 \cdot d + k \cdot (n - k))$). *Hlavní problém CLARA* nastane, když předvybrané vzorky neobsahují dobrou volbu medoidů.

CLARANS (*Clustering Large Applications based on RANdomized Search*) pracuje s úplnými daty, aby se vyhnul problému s nesprávně předvybranými vzorky. Iterativně testuje výměny mezi náhodnými medoidy a náhodnými nemedoidy: po každém pokusu se ověří, zda se kvalita zlepšila; pokud ano, výměna se stane definitivní, jinak se počítají neúspěšné pokusy. Lokálně optimální řešení je nalezeno, když se dosáhne uživatelem zadaného počtu neúspěšných pokusů **MaxAttempt**. Celý proces hledání lokálního optima se opakuje **MaxLocal**-krát a vybere se nejlepší z těchto lokálních optim. **CLARANS prozkoumá větší různorodost prostoru řešení než CLARA.**

7.4 Hierarchické shlukování

Aglomerativní hierarchické shlukování

Přístup „zdola nahoru“ (*bottom-up*, aglomerativní):

Inicializace:

1. Urči vzájemné vzdálenosti mezi všemi objekty.
2. Přiřaď každý objekt do vlastního shluku.

Hlavní cyklus:

1. Dokud je shluků více než jeden:
 - (a) najdi dva vzájemně nejbližší shluky a slouč je;
 - (b) pro nový shluk spočti jeho vzdálenost od ostatních shluků.

Opačný přístup „shora dolů“ (*divisive*) začíná s jediným shlukem a rekurzivně jej dělí (např. algoritmus DIANA nebo bisekční k -means); je výpočetně náročnější, ale dokáže lépe zachytit globální strukturu.

Dendrogram

Dendrogram zobrazuje (zleva doprava, resp. zdola nahoru) postupné slučování shluků. Výška spojení odpovídá vzdálenosti, při níž ke sloučení došlo.

Optimální počet shluků není znám předem — určí se „řezem“ dendrogramu ve zvolené výšce (např. tam, kde je mezi dvěma sousedními sloučenými největší skok výšky).

Příklad — aglomerativní shlukování, single vs. complete linkage

Objekty na přímce: $a = 1, b = 2, c = 5, d = 9, e = 10$. Matice vzdáleností:

	a	b	c	d	e
a	0	1	4	8	9
b		0	3	7	8
c			0	4	5
d				0	1
e					0

Single linkage: nejmenší vzdálenost je 1 — slouč $\{a, b\}$ (výška 1) a $\{d, e\}$ (výška 1). Nové vzdálenosti: $D(\{a, b\}, c) = \min(4, 3) = 3$; $D(c, \{d, e\}) = \min(4, 5) = 4$; $D(\{a, b\}, \{d, e\}) = \min(8, 9, 7, 8) = 7$. Slouč $\{a, b\}$ s c ve výšce 3 $\rightarrow \{a, b, c\}$. Nakonec $D(\{a, b, c\}, \{d, e\}) = \min(7, 4) = 4 \rightarrow$ sloučení ve výšce 4.

Řez pod výškou 4 dává shluky $\{a, b, c\}$ a $\{d, e\}$.

Complete linkage: nejprve rovněž $\{a, b\}$ a $\{d, e\}$ ve výšce 1. Nové vzdálenosti: $D(\{a, b\}, c) = \max(4, 3) = 4$; $D(c, \{d, e\}) = \max(4, 5) = 5$; $D(\{a, b\}, \{d, e\}) = \max(8, 9, 7, 8) = 9$. Slouč $\{a, b\}$ s c ve výšce 4. Nakonec $D(\{a, b, c\}, \{d, e\}) = \max(8, 9, 7, 8, 4, 5) = 9 \rightarrow$ sloučení ve výšce 9.

Struktura je stejná, ale complete linkage „roztáhne“ dendrogram (výška poslední fúze 9 místo 4) — rozdíl mezi shluky je zdůrazněn. Na datech s můstkem mezi shluky by se struktury lišily i topologicky.

Složitost naivního aglomerativního shlukování je $O(n^3)$ (resp. $O(n^2 \log n)$ s haldou) a paměťová $O(n^2)$ — proto se pro velká data používají škálovatelné varianty (BIRCH, CURE).

CURE

Clustering Using REpresentatives — aglomerativní hierarchický algoritmus; velmi rychlý a schopný objevit shluky *libovolného tvaru* (na rozdíl např. od CLARANS). Každý shluk je reprezentován několika rozptýlenými body, které jsou staženy směrem k centroidu.

Postup:

1. Navzorkuj s objektů z databáze D velikosti n .
2. Rozděť navzorkovaných s objektů do p částí velikosti s/p .
3. Shlukuj každou část nezávisle hierarchickým slučováním do k' shluků; celkový počet shluků $k' \cdot p$ přes všechny části je stále větší než uživatelem požadované k .
4. Proveď hierarchické shlukování nad $k' \cdot p$ shluky ze všech částí až na požadované k .
5. Přiřaď každý z $(n - s)$ nevzorkovaných objektů do shluku obsahujícího nejbližšího reprezentanta.

7.5 Shlukování založené na hustotě: DBSCAN

DBSCAN

Density-Based Spatial Clustering of Applications with Noise (Ester et al., 1996) definuje shluk jako maximální souvislou oblast vysoké hustoty. Parametry: poloměr ε a minimální počet objektů MinPts.

- ε -okolí bodu p : $N_\varepsilon(p) = \{q \in D \mid d(p, q) \leq \varepsilon\}$.
- p je **jádrový bod** (*core point*), je-li $|N_\varepsilon(p)| \geq \text{MinPts}$.
- q je **přímo hustotně dosažitelný** z p , je-li p jádrový a $q \in N_\varepsilon(p)$.
- q je **hustotně dosažitelný** z p , existuje-li řetěz $p = p_1, \dots, p_l = q$, kde každý p_{i+1} je přímo hustotně dosažitelný z p_i .
- p a q jsou **hustotně spojené**, existuje-li bod o , z něhož jsou oba hustotně dosažitelné.
- **Hraniční bod** (*border point*) leží v okolí jádrového bodu, ale sám jádrový není; **šum** (*noise*) je bod, který není ani jádrový, ani hraniční.

Shluk je maximální množina hustotně spojených bodů.

Algoritmus DBSCAN

```
1: for každý dosud nenavštívený bod  $p \in D$  do
2:   označ  $p$  jako navštívený;  $N \leftarrow N_\varepsilon(p)$ 
3:   if  $|N| < \text{MinPts}$  then
4:     označ  $p$  jako ŠUM ▷ může být později přeznačen na hraniční bod
5:   else
6:     vytvoř nový shluk  $C$ ; přidej  $p$  do  $C$ 
7:     for každý  $q \in N$  do
8:       if  $q$  nebyl navštíven then
9:         označ  $q$  jako navštívený;  $N' \leftarrow N_\varepsilon(q)$ 
10:        if  $|N'| \geq \text{MinPts}$  then
11:           $N \leftarrow N \cup N'$ 
12:        end if
13:      end if
14:      if  $q$  dosud nepatří do žádného shluku then
15:        přidej  $q$  do  $C$ 
16:      end if
17:    end for
18:  end if
19: end for
```

Složitost $O(n \log n)$ s prostorovým indexem (R-strom, k-d strom), jinak $O(n^2)$.

Výhody: nevyžaduje zadat počet shluků, najde shluky libovolného tvaru, explicitně identifikuje šum a odlehlé hodnoty. **Nevýhody:** citlivost na volbu ε a MinPts ; selhává při *proměnlivé hustotě* shluků; ve vysokých dimenzích ztrácí vzdálenost rozlišovací schopnost. Heuristika pro ε : graf vzdálenosti ke k -tému nejbližšímu sousedu seřazený sestupně — „loket“ udává vhodnou hodnotu. $\text{MinPts} \approx 2d$.

Rozšíření:

- **OPTICS** — uspořádá body podle dosažitelnosti, díky čemuž zvládne i proměnlivou hustotu shluků;
- **DENCLUE** — odhad hustoty jádrovými funkcemi;
- **HDBSCAN** — hierarchická varianta.

7.6 Detekce odlehlých hodnot

Detekce odlehlých hodnot (*outlier / anomaly detection*) je samostatná deskriptivní úloha KDD (kap. 1); univariátní přístup přes IQR a box plot byl v kap. 2, zde jde o vícerozměrné metody. Rozlišujeme *globální* odlehlost (objekt je daleko od všech ostatních dat) a *lokální* (objekt je odlehlý jen vzhledem k hustotě svého okolí — např. bod na okraji řídkého shluku globálně nevyčnívá).

Přístupy k detekci odlehlých hodnot

- **Statistické:** objekt je odlehlý, je-li málo pravděpodobný podle předpokládaného modelu dat — vzdálenost přes 3σ od průměru, Mahalanobisova vzdálenost (zohlední kovarianci), malá hustota podle GMM.
- **Vzdálenostní:** skóre objektu = vzdálenost k jeho k -tému nejbližšímu sousedovi; velká hodnota $\Rightarrow$ globální outlier. Jednoduché, ale selhává při shlucích různé hustoty.
- **Hustotní — LOF** (*Local Outlier Factor*): porovnává lokální hustotu objektu s hustotou jeho sousedů,

$$\text{LOF}_k(\vec{x}) = \frac{1}{|N_k(\vec{x})|} \sum_{\vec{y} \in N_k(\vec{x})} \frac{\text{lrd}_k(\vec{y})}{\text{lrd}_k(\vec{x})},$$

kde lrd_k je *lokální dosažitelná hustota* (převrácená hodnota průměrné dosažitelné vzdálenosti k k nejbližším sousedům). Interpretace: $\text{LOF} \approx 1$ — hustota jako u sousedů (normální bod), $\text{LOF} \gg 1$ — lokální outlier. Najde outliery i při proměnlivé hustotě shluků.

- **Modelové:** *jednotřídní SVM* (naučí se hranici kolem normálních dat, vše vně je anomálie), *izolační les* (*isolation forest*) — anomálie se v náhodně stavěných stromech oddělí krátkou cestou od kořene, skóre = průměrná hloubka izolace; *autoenkodér* — anomálie mají velkou rekonstrukční chybu.

DBSCAN označí šum jako vedlejší produkt shlukování (binárně); specializované metody výše dávají každému objektu spojitě *skóre odlehlosti*, takže lze řídit počet hlášených anomálií. Typické aplikace: detekce podvodů, průniků do sítě (kap. 11), chyb měření.

7.7 Další přístupy

Mřížkové metody (*grid-based*): prostor se rozdělí na konečný počet buněk mřížky a shlukuje se nad buňkami, nikoli nad objekty $\Rightarrow$ doba běhu nezávisí na počtu objektů, jen na počtu buněk. *STING* (hierarchická mřížka se statistikami v buňkách), *CLIQUE* (kombinace mřížky a hustoty, hledá shluky v podprostorech pomocí apriori vlastnosti — vhodné pro vysoké dimenze), *WaveCluster* (waveletová transformace hustoty).

Modelové metody (*model-based*): předpokládá se, že data jsou generována směsí rozdělání, např. *gaussovskou směsí* (*Gaussian Mixture Model*); parametry se odhadují **algoritmem EM** (*Expectation-Maximization*): krok E spočítá aposteriorní pravděpodobnosti příslušnosti objektů ke komponentám, krok M přepočítá parametry komponent. Výsledkem je *měkké* shlukování (objekt patří do shluku s určitou pravděpodobností); k -means je limitním případem GMM se sférickými kovariancemi a tvrdým přiřazením.

Spektrální shlukování: pracuje s grafem podobnosti, používá vlastní vektory Laplaceanu (viz kap. 10); zvládne shluky nekonvexního tvaru.

Vektorová kvantizace: algoritmus LVQ

Learning Vector Quantization je metoda pro *shlukování s učitelem* — třídy jsou označeny C .

Krok 1: Inicializuj všechny váhové vektory $\vec{w}_j(0)$ a rychlosti učení $\mu(0)$; polož $k = 0$.

Krok 2: Otestuj zastavovací podmínku: je-li NEPRAVDA $\Rightarrow$ POKRAČUJ, je-li PRAVDA $\Rightarrow$ KONEC.

Krok 3: Pro každý trénovací objekt $\vec{x}_i$ proved' kroky 4 a 5.

Krok 4: Urči index váhového vektora ($j = q$) tak, že $\min_j \|\vec{x}_i - \vec{w}_j(k)\|^2$ (euklidovská vzdálenost); $\vec{w}_q(k)$ je váhový vektor s minimální vzdáleností.

Krok 5: Uprav příslušný váhový vektor $\vec{w}_q(k)$:

$$\text{je-li } C_{w_q} = C_{\vec{x}_i} : \quad \vec{w}_q(k+1) = \vec{w}_q(k) + \mu(k)(\vec{x}_i - \vec{w}_q(k)),$$

$$\text{je-li } C_{w_q} \neq C_{\vec{x}_i} : \quad \vec{w}_q(k+1) = \vec{w}_q(k) - \mu(k)(\vec{x}_i - \vec{w}_q(k)).$$

Krok 6: Polož $k \leftarrow k + 1$; sniž rychlosti učení, např. podle $\mu(k) = \mu(k-1)/(k+1)$ pro $k > 0$; jdi na krok 2.

Váhové vektory se typicky inicializují prvními m vektory z trénovací množiny (musí obsahovat objekty ze všech tříd) — tím se neurony automaticky kalibrují s třídami.

Samoorganizující se mapa (SOM, Kohonen)

Protějšek LVQ pro *učení bez učitele*. Neurony s váhovými vektory $\vec{w}_j$ jsou uspořádány do pevné *mřížky* (typicky dvourozměrné). Pro každý trénovací objekt $\vec{x}_i$ se najde *vítězný neuron* $q = \arg \min_j \|\vec{x}_i - \vec{w}_j\|$ a adaptuje se vítěz **i jeho okolí v mřížce**:

$$\vec{w}_j(k+1) = \vec{w}_j(k) + \mu(k) h_{qj}(k) (\vec{x}_i - \vec{w}_j(k)),$$

kde h_{qj} je *funkce sousedství* (např. gaussovská ve vzdálenosti neuronů q a j na mřížce), jejíž šířka i rychlost učení μ v čase klesají. Výsledkem je *topologii zachovávající* zobrazení: blízké vstupy se mapují na blízké neurony mřížky. SOM tak slouží současně ke shlukování a k **vizualizaci vícerozměrných dat ve 2D** — *U-matice* (zobrazení vzdáleností mezi váhami sousedních neuronů) ukazuje hranice shluků jako „hřebeny“, *component planes* rozložení hodnot jednotlivých atributů po mapě. Odtud zařazení SOM ve fázi Explore metodologie SEMMA (kap. 1); jako vizualizační technika doplňuje přehled v kap. 9.

7.8 Validace shlukování a volba počtu shluků

Silueta

Pro objekt i nechť $a(i)$ je průměrná vzdálenost k ostatním objektům *jeho* shluku (koheze) a $b(i)$ minimální průměrná vzdálenost k objektům jiného shluku (separace). *Siluetový koeficient*:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1].$$

Interpretace: $s(i) \approx 1$ — objekt je dobře zařazen; $s(i) \approx 0$ — leží na hranici dvou shluků; $s(i) < 0$ — je pravděpodobně zařazen špatně. *Průměrná silueta* přes všechna data měří kvalitu celého shlukování a její maximalizace přes k je běžný způsob volby počtu shluků.

Příklad. Pro shlukování z k -means příkladu: $a(P_1) = d(P_1, P_2) = 1$; $b(P_1) = \frac{d(P_1, P_3) + d(P_1, P_4)}{2} = \frac{3,606 + 5,000}{2} = 4,303$; tedy $s(P_1) = \frac{4,303 - 1}{4,303} = 0,768$ — velmi dobré zařazení.

Další kritéria kvality:

Kritérium	Typ	Popis
SSE / „loket“ (<i>elbow</i>)	interní	vynes SSE proti k ; vyber k v místě zlomu, kde další shluk už SSE výrazně nesnižuje
Silueta	interní	viz výše; maximalizuj průměr
Dunnův index	interní	$\frac{\min \text{mezishluková vzdálenost}}{\max \text{průměr shluku}}$; maximalizuj
Daviesův–Bouldinův index	interní	průměr nejhorších poměrů rozptylu a separace; minimalizuj
Gap statistika	interní	porovná SSE s očekávanou SSE na referenčních náhodných datech
Randův index, ARI	externí	shoda s referenčním rozdělením (podíl souhlasných dvojic)
Normalizovaná vzájemná informace	externí	informační shoda s referenčním rozdělením
Čistota (<i>purity</i>)	externí	podíl objektů majoritní třídy v každém shluku

7.9 Srovnání metod shlukování

Metoda	Typ	Nutné k	Tvar shluků	Odlehlé h.	Složitost
k -means	rozdělující	ano	kulovité	citlivý	$O(tknd)$
k -medoids, PAM	rozdělující	ano	kulovité	robustní	$O(kn^2d)/\text{iter.}$
CLARA	rozdělující	ano	kulovité	robustní	$O(kf^2n^2d)$
CLARANS	rozdělující	ano	kulovité	robustní	mezi PAM a CLARA
Aglomerativní	hierarch.	ne (řez)	dle linkage	dle linkage	$O(n^2 \log n)$
CURE	hierarch.	ano	libovolný	robustní	$O(s^2 \log s)$
DBSCAN	hustotní	ne	libovolný	detekuje	$O(n \log n)$
OPTICS	hustotní	ne	libovolný	detekuje	$O(n \log n)$
STING / CLIQUE	mřížková	ne	obdélníkový	podle prahu	$O(\text{buněk})$
EM / GMM	modelová	ano	elipsoidní	citlivý	$O(tknd^2)$
Spektrální	grafová	ano	libovolný	střední	$O(n^3)$
LVQ	s učitelem	ano	kulovité	střední	$O(tnmd)$

Volba algoritmu závisí na konkrétní aplikaci! Praktická vodítka: kulovité shluky podobné velikosti a velká data $\rightarrow k$ -means (s k -means++); neznámý počet shluků a nepravidelné tvary $\rightarrow$ DBSCAN; potřeba hierarchie nebo malá data $\rightarrow$ aglomerativní shlukování s dendrogramem; nenumernická data nebo silné odlehlé hodnoty $\rightarrow k$ -medoids; pravděpodobnostní model a měkké přiřazení $\rightarrow$ EM/GMM.

7.10 Shrnutí k zkoušce

- **Metriky:** Hammingova ($L^{(1)}$), euklidovská ($L^{(2)}$), Čebyševova ($L^{(\infty)}$), Minkowského $L^{(z)}$, Mahalanobisova (různé rozptyly, kovarianční matice). Vždy normalizovat!
- **Linkage:** nejbližší soused (single, řetězový efekt, libovolné tvary), nejvzdálenější soused (complete, kompaktní shluky), průměrná vzdálenost, centroidní, Wardova.
- **k -means:** 4 kroky (náhodné rozdělení $\rightarrow$ centroidy $\rightarrow$ přiřazení k nejbližšímu centroidu $\rightarrow$ opakuji při přesunu). Minimalizuje SSE, konverguje k lokálnímu minimu. **k -means++** vzorkuje počáteční centroidy s pravděpodobností úměrnou d^2 . Umět projít příklad na 4 bodech.
- **k -medoids:** reprezentanti z dat, účelová funkce $O = \sum_i \min_j \text{Dist}(\bar{X}_i, \bar{Y}_j)$; robustní k odlehlým hodnotám a použitelný pro libovolný typ dat; hill-climbing s výměnami. **PAM** testuje všech $k(n-k)$ výměn; **CLARA** pracuje na vzorcích (problém: špatně zvolený vzorek); **CLARANS** náhodné výměny nad plnými daty (**MaxAttempt**, **MaxLocal**), prozkoumá větší diverzitu.
- **Hierarchické:** aglomerativní (zdola nahoru) vs. divizivní; **dendrogram** a řez určuje počet shluků. **CURE** — vzorkování, dělení na části, dvoufázové hierarchické slučování, libovolné tvary.

- **DBSCAN**: ε , MinPts; jádrový / hraniční / šumový bod; hustotní dosažitelnost a spojenost; nevyžaduje k , najde libovolné tvary, detekuje šum; selhává při proměnlivé hustotě.
- **Validace**: silueta $s(i) = \frac{b(i)-a(i)}{\max\{a(i),b(i)\}}$, loket na SSE, Dunn, Davies–Bouldin, gap; externí: Rand/ARI, NMI, purity.
- **Detekce odlehlých hodnot**: globální vs. lokální; statistické (3σ , Mahalanobis) / vzdálenostní (k -NN vzdálenost) / hustotní (**LOF** $\approx$ poměr hustoty sousedů k vlastní, $\gg 1$ = outlier) / modelové (jednotřídní SVM, izolační les, autoenkodér).
- **LVQ**: shlukování s učitelem; při shodě třídy přitáhne váhový vektor, při neshodě odtlačí; klesající rychlost učení. **SOM**: bez učitele, neurony na mřížce, adaptuje se vítěz i okolí (h_{qj}); topologii zachovávající zobrazení, vizualizace U-maticí.

8 Charakteristická a diskriminační pravidla

8.1 Motivace: charakterizace a diskriminace

Deskriptivní dobývání znalostí odpovídá na dvě příbuzné, ale odlišné otázky:

Charakterizace vs. diskriminace

Charakterizace dat (*data characterization*) — shrnutí obecných vlastností *cílové třídy* (*target class*). Odpověď na otázku: „Jak vypadá typický klient s hypotékou?“ Výsledkem jsou *charakteristická pravidla* (*characteristic rules*).

Diskriminace dat (*data discrimination*) — porovnání obecných vlastností cílové třídy s vlastnostmi jedné či více *kontrastních tříd* (*contrasting classes*). Odpověď na otázku: „Čím se klient s hypotékou liší od klienta bez hypotéky?“ Výsledkem jsou *diskriminační pravidla* (*discriminant rules*).

Rozdíl je zásadní: vlastnost může být pro cílovou třídu *typická* (vysoký podíl uvnitř třídy), a přesto nemá žádnou *rozlišovací sílu*, protože je stejně typická i pro kontrastní třídu.

Obě úlohy řeší metoda *indukce orientované na atributy* (*attribute-oriented induction*, AOI), navržená Hanem, Cai a Cercone (1991). Alternativou je datově-kostkový (*data cube*) přístup známý z OLAP: operace *roll-up* odpovídá generalizaci, *drill-down* specializaci. AOI je flexibilnější (zvládne i atributy bez předem materializované kostky), OLAP je rychlejší (předpočítané agregace).

8.2 Koncepční hierarchie

Koncepční hierarchie

Koncepční hierarchie (*concept hierarchy*) atributu *A* je částečné uspořádání jeho hodnot od nejkonkrétnějších (listy) po nejobecnější (kořen ANY). Typy:

- *schémátová* (*schema hierarchy*) — daná strukturou dat: ulice $\prec$ město $\prec$ kraj $\prec$ stát;
- *seskupovací* (*set-grouping*) — explicitní seskupení hodnot: {20–29} $\prec$ mladý;
- *operační* (*operation-derived*) — odvozená výpočtem: e-mailová adresa $\rightarrow$ doména;
- *pravidlová* (*rule-based*) — definovaná pravidly nad více atributy.

Hierarchie zadává expert, nebo se generuje automaticky (diskretizací u numerických atributů, podle počtu různých hodnot u kategoriálních — atribut s více různými hodnotami je obvykle na nižší úrovni hierarchie).

8.3 Indukce orientovaná na atributy

Algoritmus AOI

Vstup: databáze, dotaz vymezující cílovou třídu, seznam atributů, prahy generalizace T_i pro jednotlivé atributy a celkový práh T pro velikost výsledné relace.

- 1: **sběr dat:** vyber pomocí dotazu množinu objektů cílové třídy (*initial working relation*)
- 2: ke každému objektu přidej atribut `count` = 1
- 3: **for** každý atribut A_i **do**
- 4: **if** A_i má mnoho různých hodnot a *neexistuje* pro něj koncepční hierarchie **then**
- 5: **odstranění atributu** (*attribute removal*)
- 6: **else if** A_i má mnoho různých hodnot, ale hierarchie existuje **then**
- 7: **generalizace atributu** (*attribute generalization*): nahraď hodnoty jejich předky
- 8: o úroveň výš, dokud počet různých hodnot $\leq T_i$
- 9: **end if**
- 10: **end for**
- 11: **agregace:** slouč identické zobecněné objekty a sečti jejich `count`
- 12: **while** počet řádků výsledné relace $> T$ **do**
- 13: zobecni dále atribut, jehož generalizace nejvíce sníží počet řádků
- 14: **end while**
- 15: **prezentace:** zobraz *primární relaci* (*prime relation*) jako tabulku, křížovou tabulku,
- 16: graf nebo ji převed' na kvantitativní pravidla

Dvě základní generalizační operace:

- **Odstranění atributu** — je-li atributů příliš mnoho různých hodnot a (a) neexistuje pro něj generalizační operátor, nebo (b) jeho vyšší úrovně jsou vyjádřeny jinými atributy, atribut se z relace odstraní. Příklad: *jméno, telefon, rodné číslo*.
- **Generalizace atributu** — existuje-li generalizační operátor (koncepční hierarchie), nahradí se hodnoty hodnotami na vyšší úrovni. Příklad: *datum narození* $\rightarrow$ *věk* $\rightarrow$ *věková kategorie*; *město* $\rightarrow$ *kraj* $\rightarrow$ *region*.

Řízení generalizace probíhá dvěma prahy:

- *práh generalizace atributu* T_i — maximální povolený počet různých hodnot atributu (typicky 2–8);
- *práh velikosti relace* T — maximální počet řádků výsledné primární relace (typicky 10–30).

Příliš vysoká generalizace vede k triviálním výsledkům („všichni klienti jsou lidé“), příliš nízká k nepřehledné tabulce. Vhodná úroveň se hledá interaktivně (drill-down / roll-up).

Složitost: $O(na)$ pro n objektů a a atributů (jeden průchod daty a agregace), pokud se generalizace realizuje přes hašovací tabulku zobecněných záznamů — AOI je tedy *lineární* v počtu objektů, což je pro velké databáze zásadní.

8.4 Míry t-weight a d-weight

t-weight a d-weight

Nechť primární relace obsahuje zobecněné záznamy (*generalized tuples*) $q_1, \dots, q_N$.

Typičnost (*typicality*, t-weight) záznamu q_a v cílové třídě:

$$t\text{-weight}(q_a) = \frac{\text{count}(q_a)}{\sum_{i=1}^N \text{count}(q_i)} \in [0, 1],$$

tj. podíl objektů cílové třídy, které záznam q_a pokrývá. Součet t-weight přes všechny záznamy primární relace je 1 (100 %). Slouží pro *charakteristická* pravidla.

Diskriminační váha (d-weight) záznamu q_a vůči cílové třídě C_{target} a kontrastním třídám $C_1, \dots, C_m$:

$$d\text{-weight}(q_a) = \frac{\text{count}(q_a \in C_{target})}{\sum_{j=1}^m \text{count}(q_a \in C_j) + \text{count}(q_a \in C_{target})} \in [0, 1],$$

tj. podíl objektů pokrytých záznamem q_a , které patří do cílové třídy. Slouží pro *diskriminační* pravidla. Vysoká d-weight ($\rightarrow 1$) znamená, že záznam cílovou třídu silně odlišuje; hodnota blízká podílu tříd znamená nulovou rozlišovací schopnost.

Kvantitativní charakteristické, diskriminační a popisné pravidlo

Kvantitativní charakteristické pravidlo (nutná podmínka — implikace z třídy):

$$\forall X \text{ target_class}(X) \Rightarrow \text{cond}_1(X) [t: w_1] \vee \dots \vee \text{cond}_N(X) [t: w_N]$$

Čte se: patří-li X do cílové třídy, pak splňuje cond_1 s typičností w_1 , nebo cond_2 s typičností w_2 , ...

Kvantitativní diskriminační pravidlo (postačující podmínka — implikace do třídy):

$$\forall X \text{ target_class}(X) \Leftarrow \text{cond}(X) [d: w]$$

Čte se: splňuje-li X podmínku cond , patří s pravděpodobností w do cílové třídy.

Kvantitativní popisné pravidlo (*quantitative description rule*) kombinuje obojí (nutná i postačující podmínka):

$$\forall X \text{ target_class}(X) \Leftrightarrow \text{cond}_1(X) [t: w_1, d: w'_1] \vee \dots \vee \text{cond}_N(X) [t: w_N, d: w'_N]$$

Příklad — charakterizace a diskriminace klientů banky

Cílová třída: klienti s hypotékou (200 osob). **Kontrastní třída:** klienti bez hypotéky (800 osob).

Po AOI (odstraněno *jméno, rodné číslo, telefon*; generalizováno *datum narození* → *věková kategorie*, *plat* → *příjmová kategorie*, *obec* → *region*) vznikne primární relace:

Věk	Příjem	Region	počet		<i>t</i> -weight	<i>d</i> -weight
			cíl	kontrast	(cíl)	
30–40	vysoký	Praha	90	10	45 %	90 %
30–40	střední	Praha	50	150	25 %	25 %
40–50	vysoký	mimo Prahu	40	160	20 %	20 %
20–30	nízký	mimo Prahu	20	480	10 %	4 %
Celkem			200	800	100 %	

Výpočet. *t*-weight prvního záznamu = $90/200 = 45\%$; *d*-weight = $90/(90 + 10) = 90\%$. Pro druhý záznam $t = 50/200 = 25\%$, $d = 50/(50 + 150) = 25\%$. Pro čtvrtý $t = 20/200 = 10\%$, $d = 20/(20 + 480) = 4\%$.

Kvantitativní charakteristické pravidlo:

$\forall X$ hypotéka(X) $\Rightarrow$ ($\text{věk}(X)=30-40 \wedge \text{příjem}(X)=\text{vysoký} \wedge \text{region}(X)=\text{Praha}$) [$t: 45\%$]
 $\vee$ ($\text{věk}(X)=30-40 \wedge \text{příjem}(X)=\text{střední} \wedge \text{region}(X)=\text{Praha}$) [$t: 25\%$] $\vee \dots$

Kvantitativní diskriminační pravidlo:

$\forall X$ hypotéka(X) $\Leftarrow$ ($\text{věk}(X)=30-40 \wedge \text{příjem}(X)=\text{vysoký} \wedge \text{region}(X)=\text{Praha}$) [$d: 90\%$]

Interpretace rozdílů. Poslední řádek ukazuje, proč jsou obě míry potřeba: profil „20–30, nízký příjem, mimo Prahu“ pokrývá 10 % klientů s hypotékou (nezanedbatelná typičnost), ale jeho *d*-weight je pouhá 4 % — výrazně *méně* než apriorní podíl cílové třídy $200/1000 = 20\%$. Tento profil tedy hypotéku spíše *vyklučuje*. Naopak druhý a třetí řádek mají *t*-weight i *d*-weight shodně 25 %, resp. 20 %: u druhého řádku je *d*-weight jen nepatrně *nad* apriorním podílem (25 % vs. 20 %), takže rozlišovací síla je zanedbatelná, a u třetího je *d*-weight *přesně rovna* apriornímu podílu (20 %), tedy rozlišovací síla je **nulová** — profil o příslušnosti k cílové třídě neříká vůbec nic, přestože pokrývá pětinu klientů s hypotékou.

8.5 Měření zajímavosti pravidel

Zajímavost odvozených pravidel se posuzuje obdobně jako u asociačních pravidel (kap. 4), s důrazem na následující míry:

Míra	Význam
<i>t</i> -weight	typičnost profilu uvnitř cílové třídy; pravidlo s nízkou <i>t</i> -weight popisuje okrajový jev
<i>d</i> -weight	rozlišovací síla profilu; <i>zajímavá</i> jsou pravidla s <i>d</i> -weight výrazně vyšší (nebo nižší) než apriorní podíl třídy
Lift / zdvih	$\frac{d\text{-weight}}{P(C_{\text{target}})}$ — kolikrát profil zvýší šanci na cílovou třídu
Pokrytí (<i>coverage</i>)	podíl všech objektů pokrytých pravidlem; nízké pokrytí $\Rightarrow$ statisticky nespolehlivé
χ^2	statistická významnost vztahu profilu a třídy
Informační zisk	snížení entropie třídy po znalosti profilu
Jednoduchost	délka pravidla (počet podmínek), počet pravidel; Occamova břitva
Neočekávanost	rozpor s modelem světa uživatele (subjektivní míra)
Akceschopnost	lze na jeho základě jednat (subjektivní míra)

Prakticky: pravidlo je zajímavé, je-li současně (a) *srozumitelné*, (b) *platné* na nových datech s vysokou spolehlivostí, (c) *potenciálně užitečné* a (d) *nové*. První tři jsou objektivně měřitelné, poslední vyžaduje znalost uživatele.

Klasická past: pravidlo s vysokou *t*-weight, ale *d*-weight rovnou apriornímu podílu třídy, není objevem, i když vypadá impozantně („80 % klientů s hypotékou má bankovní účet“ — ale ten má i 80 % klientů bez hypotéky).

8.6 Shrnutí k zkoušce

- **Charakterizace** popisuje cílovou třídu; **diskriminace** ji porovnává s kontrastními třídami.
- **AOI:** sběr dat dotazem $\rightarrow$ přidání **count** $\rightarrow$ *odstranění atributu* (mnoho hodnot, žádná hierarchie) nebo *generalizace atributu* (podle koncepční hierarchie) $\rightarrow$ agregace stejných záznamů $\rightarrow$ další generalizace, dokud relace není menší než práh $T \rightarrow$ prezentace.
- **Koncepční hierarchie:** schémátová, seskupovací, operační, pravidlová.
- **t-weight** $= \frac{\text{count}(q_a)}{\sum_i \text{count}(q_i)}$ — typičnost uvnitř cílové třídy, součet přes všechny záznamy je 100 %; pro *charakteristická* pravidla ($\Rightarrow$, nutná podmínka).
- **d-weight** $= \frac{\text{count}(q_a \in C_{\text{target}})}{\sum_j \text{count}(q_a \in C_j)}$ — rozlišovací síla; pro *diskriminační* pravidla ($\Leftarrow$, postačující podmínka). Popisné pravidlo kombinuje obojí ($\Leftrightarrow$).
- Umět spočítat obě váhy z tabulky a vysvětlit, proč vysoká *t*-weight neznamena vysokou *d*-weight.
- **Zajímavost:** objektivní (*t/d*-weight, lift, pokrytí, χ^2 , informační zisk, jednoduchost) vs. subjektivní (neočekávanost, akceschopnost).

9 Reprezentace a vizualizace znalostí

9.1 Formy reprezentace znalostí

Forma	Typický zdroj	Vlastnosti
IF-THEN pravidla	asociační pravidla, RIPPER, C4.5	nejsrozumitelnější; lokální (každé pravidlo se čte samostatně); riziko konfliktů a nadprodukce
Rozhodovací strom	ID3/C4.5/CART	globální model, snadno čitelný do hloubky 4–5; nad ni nepřehledný
Rozhodovací tabulka	diskretizovaná data	úplný výčet kombinací; přehledná jen pro málo atributů
Matematická funkce	regrese, SVM, NN	kompaktní a přesná; obtížně interpretovatelná (kromě lineárního modelu s normalizovanými vahami)
Prototypy / centroidy	k -means, k -medoids, LVQ	„typický zástupce“ skupiny; velmi intuitivní pro byznys
Pravděpodobnostní model	naivní Bayes, bayesovská síť, GMM	explicitní nejistota; graf sítě je čitelný
Grafy a sítě	analýza sociálních sítí	vztahy mezi entitami; vizuálně silné
Instance (paměť případů)	k -NN, CBR	„model“ je samotná data; vysvětlení odkazem na podobné případy

Interpreovatelnost a vysvětlitelnost

Interpreovatelnost (interpretability) — schopnost člověka porozumět tomu, jak model dospěl k výstupu. *Vysvětlitelnost (explainability)* — schopnost podat srozumitelné odůvodnění konkrétního rozhodnutí.

Rozlišujeme:

- *modely inherentně interpreovatelné* — lineární modely, rozhodovací stromy, sady pravidel, prototypy;
- *post-hoc vysvětlení černých skříněk* — důležitost atributů (permutační, SHAP), lokální surrogátní modely (LIME), grafy částečné závislosti (PDP), extrakce pravidel z neuronové sítě, analýza citlivosti, kontrafaktuální vysvětlení („kdyby byl váš příjem o 5000 vyšší, úvěr by byl schválen“).

Interpreovatelnost není jen pohodlí: v regulovaných doménách (úvěry, pojištění, medicína) je právním požadavkem (GDPR, čl. 22 — právo na vysvětlení automatizovaného rozhodnutí; AI Act pro vysoce rizikové systémy).

9.2 Vizualizační techniky

Technika	K čemu	Omezení / poznámka
Histogram	rozdělení jedné numerické veličiny	citlivý na volbu šířky košů
Krabicový graf (<i>box plot</i>)	medián, kvartily, odlehle hodnoty; srovnání skupin	neukáže bimodalitu (řeší <i>violin plot</i>)
Bodový graf (<i>scatter plot</i>)	vztah dvou numerických veličin	při mnoha bodech překryv → průhlednost, hexbin
Matice bodových grafů (SPLOM)	všechny dvojice atributů najednou	použitelné do zhruba 10 atributů
Paralelní souřadnice (<i>parallel coordinates</i>)	vysokorozměrná data — objekt je lomená čára přes svislé osy atributů	silně závisí na pořadí os; nutné škálování; křížení čar = negativní korelace
Teplotní mapa (<i>heatmap</i>)	korelační matice, matice změn, expresní data	volba barevné škály (divergentní pro korelace)
Dendrogram	výsledek hierarchického shlukování	pořadí listů není jednoznačné
Projekce (PCA, MDS, t-SNE, UMAP)	2-D pohled na vysokorozměrná data	t-SNE/UMAP zkreslují globální vzdálenosti — <i>velikosti</i> shluků a mezery mezi nimi nelze interpretovat
ROC / lift křivka	kvalita klasifikátoru	viz kap. 6
Mozaikový graf	kontingenční tabulka kategoriálních atributů	alternativa k χ^2 pro vizuální posouzení
Vizualizace sítě	sociální sítě, grafy vztahů	„chlupatá koule“ u velkých grafů → agregace komunit
Chernoffovy tváře, hvězdíkové grafy	malý počet objektů, mnoho atributů	vnímání je nelineární; dnes spíše kuriozita

Vizualizace v procesu KDD má tři role:

1. *explorativní analýza* (fáze porozumění datům): odhalí rozdělení, odlehle hodnoty, nečekané vztahy — Anscombeho kvartet ukazuje čtyři datové sady s identickými průměry, rozptyly a korelacemi, ale zcela odlišnou strukturou, kterou odhalí jedině graf;
2. *vizualizace modelu*: strom, dendrogram, síť, hranice rozhodování;
3. *prezentace výsledků* manažerovi: agregace, jednoduchost, jeden graf — jedno sdělení.

Zásady dobré vizualizace: vysoký poměr datové a inkoustové složky (Tufta); nezkreslovat (nulová osa u sloupcových grafů, lineární osy nebo zřetelně označené logaritmické); barvu volit podle typu dat (kategorie → kvalitativní paleta, pořadí → sekvenční, odchylka od středu → divergentní) a s ohledem na barvosleposti; popisky a jednotky vždy; vyhnout se 3-D efektům a koláčovým grafům s mnoha výsečemi.

9.3 Vyhodnocení znalostí z pohledu uživatele

Nalezená znalost se hodnotí z pohledu koncového uživatele — část výsledků může být nezajímavá nebo z jeho hlediska zřejmá, část lze použít přímo a část je nutné prezentovat srozumitelněji. Bývá užitečné shrnout výsledky do *analytické zprávy*; výstupem může být i provedení rozumné akce.

Kritéria zajímavosti znalosti (souhrnně):

- *srozumitelnost* — lze ji vysvětlit člověku;
- *platnost* — drží na nových datech s přijatelnou spolehlivostí;
- *užitečnost* — vede k akci s měřitelným přínosem;

- *novost* — nebyla dosud známa (subjektivní vůči konkrétnímu uživateli).

10 Analýza sociálních sítí

10.1 Vymezení oboru a motivace

Analýza sociálních sítí

Analýza sociálních sítí (*Social Network Analysis*, SNA) je interdisciplinární obor s původem ve společenských vědách, síťové analýze a teorii grafů:

- *společenské vědy* — otázky a problémy vysvětlující sociální chování analýzou společnosti;
- *síťová analýza* — formulace a řešení problémů, které mají síťovou strukturu;
- *teorie grafů* — matematický aparát abstraktních pojmů a metod pro reprezentaci a analýzu grafů.

Klíčové je, že SNA se soustředí na **vztahy** mezi sociálními entitami (jednotlivci, skupinami, institucemi), nikoli na entity samotné a jejich atributy. Metody SNA zahrnují vizualizační i analytické nástroje.

Studium společnosti ze síťové perspektivy chápe jednotlivce jako *zasazené* do sítě vztahů a hledá vysvětlení sociálního chování ve struktuře těchto sítí, nikoli pouze v jednotlivcích. Historicky nejstarším příkladem síťové analýzy je Eulerovo řešení problému sedmi mostů města Königsberg.

Motivace 1: analýza vazeb (*link analysis*). Cíle: nalézt vztahy mezi daty a vizualizovat vazby a vztahy. Aplikační oblasti: telekomunikace; kriminalistika a právo (skupiny zločinců jsou vzájemně propojené, analýza vztahů může pomoci je odhalit); marketing (vztahy mezi zákazníky).

Šest stupňů odloučení (Milgram, 1967) — experiment o fenoménu malého světa: náhodní lidé z Nebrasky měli poslat dopis (přes prostředníky, vždy jen někomu, koho znali křestním jménem) burzovnímu makléři v Bostonu. Mezi dopisy, které cíl našly, byl *medián* počtu vazeb roven šesti. Poznámky: mnoho dopisů nedorazilo; experiment předznamenal pozdější objevy (huby, tendence postupovat podle geografie a profese).

Motivace 2: bezškálové sítě (*scale-free networks*). Některé uzly mají extrémně mnoho vazeb — *huby*; většina uzlů má jen několik vazeb. Odtud odolnost vůči náhodným poruchám a zranitelnost vůči koordinovaným útokům. Konkrétně: může selhat 80 % náhodně zvolených uzlů, aniž by se klastr rozpadl, zatímco odstranění 5–15 % všech hubů může způsobit kolaps celého systému.

Oblast	Aplikace bezškálových sítí
Výpočetní technika	sítě s bezškálovou architekturou (WWW): extrémně odolné vůči náhodným výpadkům, velmi zranitelné vůči koordinovaným útokům či sabotáži; vymýcení internetových virů je v podstatě nemožné
Medicína	očkovací kampaně cílené na huby (lidé s mnoha kontakty — jejich identifikace je obtížná); nové léky cílící na klíčové molekuly (huby) dané nemoci; minimalizace vedlejších účinků pomocí nalezených sítí vazeb uvnitř buněk
Byznys	finanční krachy (pochopení vazeb mezi firmami, odvětvím a ekonomikou; monitorování a eliminace kaskádových krachů); marketing (šíření „nákazy“, efektivnější reklama na nové produkty)

Příklady bezškálových sítí: *vědecká spolupráce* (Paul Erdős napsal přes 1500 článků s 507 spoluautory; *Erdősovo číslo* je počet vazeb spojujících vědce s Erdősem přes spoluautorství — mezi matematiky je průměrné Erdősovo číslo 4,65 a maximum 13); Hollywood (herci hrající ve stejném filmu); biologické sítě (buněčný metabolismus, regulační sítě proteinů); socio-technické sítě (internet — routery a spoje; WWW — stránky a URL).

Praktické aplikace SNA: firmy (analýza a zlepšení toku komunikace v organizaci, s partnery či zákazníky; marketingové kampaně); orgány činné v trestním řízení a armáda (identifikace zločineckých a teroristických sítí ze zachycené komunikace, identifikace klíčových hráčů); sociální sítě (doporu-

čení potenciálních přátel na základě přátel přátel); občanská sdružení (odhalování střetů zájmů ve skrytých vazbách mezi státními orgány, lobby a firmami); síťoví operátoři (optimalizace struktury a kapacity komunikačních sítí); medicína (nakažlivé nemoci a jejich prevence; pohlavně přenosné choroby — epidemie syfilis v Rockdale County u Atlanty v roce 1996).

10.2 Reprezentace sítí

Reprezentace grafu

Sociální síť modelujeme grafem $G = (V, E)$: vrcholy (*vertices*, uzly) jsou aktéři, hrany (*edges*, vazby) jsou vztahy. Tři ekvivalentní reprezentace:

- **seznam hran** (*edge list*) — dvojice (vrchol, vrchol), případně s třetím sloupcem *váhy*;
- **matice sousednosti** (*adjacency matrix*) — $A_{ij} = 1$, vede-li hrana z i do j ; u *orientovaných* grafů nemusí být symetrická, u neorientovaných symetrická je; váženou variantu dostaneme dosazením vah místo jedniček;
- **seznam sousedů** (*adjacency list*) — pro řídké sítě paměťově nejúspornější.

Týž seznam hran má *různou interpretaci* podle orientovanosti: orientovaně „kdo koho kontaktuje“, neorientovaně „kdo koho zná“.

Váhy hran mohou vyjadřovat: četnost interakce ve sledovaném období; počet vyměněných položek; individuální vnímání síly vztahu; náklady na komunikaci či výměnu (např. vzdálenost); kombinace uvedeného.

Ego síť a „celé“ síť

Ego síť (*ego network*) je osobní síť jednoho aktéra (*ego*) a jeho sousedů (*alters*) včetně vazeb mezi nimi. Ego samo není pro další analýzu potřeba — všichni alteri jsou k němu z definice připojeni; po jeho odstranění se objeví *izoláty* (*isolates*).

Žádná zkoumaná síť není v praxi „celá“: obvykle jde o částečný obraz reálné sítě — *problém vymezení hranic* (*boundary specification problem*).

10.3 Síla vazeb

Sociální média propojují uživatele velmi snadno — uživatel může mít online tisíce přátel, ale vazby nejsou stejně silné a hrají různou roli při formování komunit a šíření informací. *Silné vazby* jsou blízcí přátelé, *slabé vazby* známí.

Síla slabých vazeb (Granovetter, 1973)

Příležitostná setkání se vzdálenými známými mohou poskytnout důležité informace o nových příležitostech (např. při hledání práce), které v hustě propojeném okruhu blízkých přátel kolují dokola. Slabé vazby jsou proto informačně cennější, než odpovídá jejich „síle“.

Tři propojené pojmy:

- **Homofilie** (*homophily*) — tendence vázat se na lidi s podobnými charakteristikami (status, přesvědčení). Vede ke vzniku homogenních skupin (klastřů), kde se vztahy tvoří snadněji; extrémní homogenizace však může působit proti inovacím a generování myšlenek, takže *heterofilie* je v některých kontextech žádoucí. Homofilní vazby mohou být silné i slabé.
- **Tranzitivita** (*transitivity*) — „je-li vazba mezi A a B a mezi B a C, budou v tranzitivní síti propojeni i A a C“. Silné vazby jsou tranzitivní častěji než slabé, takže tranzitivita naznačuje existenci silných vazeb (není to však nutná ani postačující podmínka). Tranzitivita a homofilie dohromady vedou ke vzniku *klik* (úplně propojených klastřů).
- **Mosty** (*bridges*) — uzly a hrany spojující různé skupiny; obvykle jde o slabé vazby, ale ne každá slabá vazba je most. Usnadňují meziskupinovou komunikaci, zvyšují sociální kohezi a podporují inovace.

Most, zkratkový most, překryv sousedství

Most (*bridge*): hrana je mostem, pokud její odstranění rozpojí její koncové uzly. V reálných sítích jsou mosty vzácné.

Zkratkový most (*shortcut bridge*): uvolněná definice — kontrolujeme, zda se po odstranění hrany zvětší vzdálenost mezi koncovými uzly (na více než 2). Čím větší vzdálenost, tím slabší vazba. Příklad: je-li $d(2, 5) = 4$ po odstranění $e(2, 5)$, ale $d(5, 6) = 2$ po odstranění $e(5, 6)$, je $e(5, 6)$ silnější vazbou než $e(2, 5)$.

Překryv sousedství (*neighborhood overlap*) — čím větší překryv, tím silnější vazba mezi v_i a v_j :

$$\text{overlap}(v_i, v_j) = \frac{\text{počet společných přátel } v_i \text{ a } v_j}{\text{počet přátel sousedících alespoň s } v_i \text{ nebo } v_j} = \frac{|N_i \cap N_j|}{|N_i \cup N_j| - 2},$$

kde -2 ve jmenovateli vylučuje samotné v_i a v_j . Pro graf $G = (\{1, 2\}, \{(1, 2)\})$ definujeme $\text{overlap}(1, 2) =_{df} 0$.

Odhad síly vazeb z profilů a interakcí. Na Twitteru (lze sledovat bez potvrzení) je skutečná síť přátelství určena četností vzájemné komunikace, nikoli vztahem follower–followee; skutečná síť přátelství přitom silněji ovlivňuje používání Twitteru. Na Facebooku lze sílu vazeb předpovědět z příspěvků iniciovaných přítelem, zpráv na zdi, počtu společných přátel apod. Číselnou sílu vazby lze učit metodou maximální věrohodnosti: podobnost profilů určuje sílu vazby, síla vazby zase určuje interakci uživatelů, takže maximalizujeme věrohodnost vzhledem k pozorovaným profilům a interakcím.

Odhad síly vazeb z aktivity uživatelů. Je-li přístupný log aktivit, můžeme se naučit, jak uživatelé ovlivňují své přátele — maximalizací věrohodnosti aktivity podle zvoleného modelu šíření vlivu (LTM či ICM, viz kap. 10.7). Modelování vlivu je důležité pro pochopení difuze informací, šíření nových myšlenek a virálního marketingu.

10.4 Klíčoví hráči: míry centrality

Přehled měr centrality

Stupňová centralita (*degree centrality*) — vstupní a výstupní stupeň uzlu, tj. počet vazeb vedoucích do/z uzlu (u neorientovaných grafů jsou shodné). Měří propojenost, vliv a/nebo popularitu uzlu; pomáhá určit, které uzly jsou centrální z hlediska šíření informací a ovlivňování ostatních ve svém okolí.

$$C_D(v) = \deg(v), \quad C'_D(v) = \frac{\deg(v)}{n-1}.$$

Mezilehlostní centralita (*betweenness centrality*) — počet nejkratších cest procházejících uzlem dělený všemi nejkratšími cestami v síti. Pro daný uzel v spočteme počet nejkratších cest mezi uzly i a j procházejících v , vydělíme všemi nejkratšími cestami mezi i a j a sečteme přes všechny dvojice; někdy se normalizuje tak, aby nejvyšší hodnota byla 1:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}},$$

kde σ_{st} je počet nejkratších cest z s do t a $\sigma_{st}(v)$ počet těch z nich, které procházejí v . *Úmluva:* u neorientovaných grafů se počítá přes **neuspořádané** dvojice $\{s, t\}$ (jinak by každá dvojice byla započtena dvakrát); normalizovaná varianta se dělí počtem dvojic $\binom{n-1}{2} = \frac{(n-1)(n-2)}{2}$. Indikuje uzly na komunikačních cestách mezi jinými uzly a pomáhá určit místa, kde se síť může rozpadnout.

Blízkostní centralita (*closeness centrality*) — míra *dosahu*: rychlost, s jakou se informace z daného uzlu dostane k ostatním. Spočteme průměrnou délku všech nejkratších cest z uzlu ke všem ostatním (kolik „skoků“ je v průměru potřeba) a vezmeme převrácenou hodnotu; vyšší hodnoty jsou lepší:

$$C_C(v) = \frac{n-1}{\sum_{u \neq v} d(v, u)}.$$

Užitečná tam, kde jde především o rychlost šíření.

Vlastní centralita (*eigenvector centrality*) — úměrná součtu vlastních centralit všech přímo připojených uzlů. Vychází z myšlenky, že moc a status aktéra jsou *rekurzivně* definovány mocí a statusem jeho sousedů. Pro matici sousednosti A :

$$x_v = \frac{1}{\lambda} \sum_{t \in M(v)} x_t = \frac{1}{\lambda} \sum_{t \in G} a_{v,t} x_t \quad \Longleftrightarrow \quad \lambda \mathbf{x} = A \mathbf{x}.$$

Existuje mnoho vlastních čísel λ s nenulovým vlastním vektorem; požadavek *nezápornosti* všech složek vede k tomu, že žádoucí míru centrality dává pouze *největší* vlastní číslo. Vlastní vektor je určen až na společný násobek — je nutné jej normalizovat (např. na součet 1 nebo na počet vrcholů n).

Katzova centralita — řeší problém vlastní centrality v orientovaných grafech (uzly bez vstupních hran mají nulové skóre) přidáním konstantního příspěvku:

$$x_v = \alpha \sum_u A_{uv} x_u + \beta, \quad \mathbf{x} = \beta (I - \alpha A^T)^{-1} \mathbf{1}, \quad \alpha < 1/\lambda_{\max}.$$

PageRank — viz kap. 10.13.

Míra	Interpretace v sociálních sítích
Stupňová	Kolik lidí může tato osoba oslovit přímo? <i>Síť hudebních spoluprací</i> : s kolika lidmi spolupracoval/a?
Mezilehlostní	Jak pravděpodobně leží tato osoba na nejpřímější cestě v síti? <i>Síť špiónů</i> : přes koho pravděpodobně poteče většina informací?
Blízkostní	Jak rychle dosáhne tato osoba všech v síti? <i>Síť sexuálních vztahů</i> : jak rychle se pohlavně přenosná choroba rozšíří do zbytku sítě?
Vlastní	Jak dobře je tato osoba propojena s jinými dobře propojenými lidmi? <i>Citační síť</i> : který autor je nejvíce citován jinými dobře citovanými autory?

Algoritmy nejkratších cest (nutné pro blízkostní a mezilehlostní centralitu): jeden zdroj a obecné grafy se zápornými vahami — Bellmanův–Fordův algoritmus, $O(|V| \cdot |E|)$; bez záporných vah — Dijkstrův algoritmus, $O(|V|^2)$. Všechny dvojice: Floydův–Warshallův algoritmus $O(|V|^3)$ (i se zápornými vahami), Johnsonův algoritmus $O(|E||V| + |V|^2 \log_2 |V|)$ pro řídké grafy. Mezilehlostní centralitu pro všechny uzly najednou počítá Brandesův algoritmus v $O(|V||E|)$.

Příklad — centrality na malém grafu

Graf: dva trojúhelníky spojené mostem. Vrcholy $\{A, B, C, D, E, F\}$, hrany $AB, AC, BC, CD, DE, DF, EF$ ($|E| = 7$).

Stupňová centralita: $\deg(A) = \deg(B) = \deg(E) = \deg(F) = 2$, $\deg(C) = \deg(D) = 3$.

Blízkostní centralita. Vzdálenosti z C : $A=1, B=1, D=1, E=2, F=2$, součet 7, tedy $C_C(C) = 5/7 = 0,714$. Vzdálenosti z A : $B=1, C=1, D=2, E=3, F=3$, součet 10, tedy $C_C(A) = 5/10 = 0,500$.

Mezilehlostní centralita vrcholů. Vrcholem C prochází nejkratší cesta pro dvojice $A-D$, $A-E$, $A-F$, $B-D$, $B-E$, $B-F$, tedy $C_B(C) = 6$; symetricky $C_B(D) = 6$. Pro A : žádná nejkratší cesta mezi jinými vrcholy jím nevede ($B-C$ je přímá hrana), tedy $C_B(A) = 0$; stejně $C_B(B) = C_B(E) = C_B(F) = 0$.

Mezilehlost hran. Hranou CD musí projít všech $3 \times 3 = 9$ nejkratších cest mezi $\{A, B, C\}$ a $\{D, E, F\}$, tedy $C_B(CD) = 9$ — nejvyšší hodnota v grafu. Dále $C_B(AC) = C_B(BC) = C_B(DE) = C_B(DF) = 4$ a $C_B(AB) = C_B(EF) = 1$.

Kontrola: součet mezilehlostí všech hran musí být roven součtu délek nejkratších cest přes všechny dvojice: $1 + 4 + 4 + 9 + 4 + 4 + 1 = 27$, což odpovídá.

Míry se často neshodují — vrchol s nízkým stupněm může mít obrovskou mezilehlost, leží-li na jediném mostě mezi dvěma komunitami (přesně případ C a D výše).

Množiny klíčových hráčů a strukturní ekvivalence

Uvažovat *množiny* klíčových hráčů je často užitečnější než hledat jediného. Typický příklad: uzel 10 je nejcentrálnější podle stupňové centrality, ale uzly 3 a 5 *dohromady* dosáhnou na více uzlů; navíc vazba mezi 3 a 5 je kritická — při jejím přerušení se síť rozpadne na dvě izolované podsítě. Za jinak stejných okolností jsou tedy uzly 3 a 5 společně pro síť důležitější než uzel 10 samotný.

Strukturní ekvivalence (*structural equivalence*) vyjadřuje podobnost aktérů: je založena na sousedech, které sdílejí (ekvivalentně na počtu identických vazeb). Dva aktéři jsou strukturně ekvivalentní, lze-li jejich pozice zaměnit, aniž se změní celková struktura sítě. Přibližnou strukturní ekvivalenci lze použít pro hierarchické shlukování sociálních sítí.

10.5 Koheze: míry na úrovni sítě

Reciprocita, hustota, shlukovací koeficient, průměr

Reciprocita (*reciprocity*) — míra vzájemnosti a reciproční výměny, související se sociální kohezí: podíl počtu opěťovaných vztahů (hrana vede oběma směry) ku celkovému počtu vztahů v síti (dva vrcholy jsou ve vztahu, existuje-li mezi nimi alespoň jedna hrana). Má smysl jen v orientovaných grafech. *Příklad:* pro síť se dvěma opěťovanými z pěti vztahů je reciprocita $2/5 = 0,4$.

Hustota (*density*) — podíl počtu hran ku celkovému počtu možných hran; pro neorientovaný graf o n vrcholech je možných hran $\binom{n}{2} = \frac{n(n-1)}{2}$, pro orientovaný $n(n-1)$ (orientované grafy tedy mají poloviční hustotu než jejich neorientované protějšky). Dokonale propojená síť se nazývá *klika* a má hustotu 1. *Příklad:* síť se 4 vrcholy a 5 neorientovanými hranami má hustotu $5/6 = 0,83$; táž síť orientovaně $5/12 = 0,42$.

Shlukovací koeficient (*clustering coefficient*) — indikuje přítomnost (pod)komunit; měří hustotu okolí uzlu. Pro graf $G = (V, E)$ je okolí vrcholu i definováno jako $N(i) = \{j \in V; e(i, j) \in E \vee e(j, i) \in E\}$ a $k(i) = |N(i)|$:

$$C(i) = \frac{2|\{e(j, k) \in E; j, k \in N(i)\}|}{k(i)(k(i) - 1)} \quad (\text{neorientovaný graf}),$$

$$C(i) = \frac{|\{e(j, k) \in E; j, k \in N(i)\}|}{k(i)(k(i) - 1)} \quad (\text{orientovaný graf}).$$

Shlukovací koeficient sítě je průměr koeficientů jejích uzlů: $C = \frac{1}{|V|} \sum_{i=1}^{|V|} C(i)$.

Průměr (*diameter*) — nejdelší nejkratší cesta (vzdálenost) mezi libovolnými dvěma uzly; užitečná míra dosahu sítě udávající, jak dlouho bude nejvýše trvat dosažení libovolného uzlu. Řidší sítě mají obecně větší průměr. **Průměrná vzdálenost** je průměr všech nejkratších cest. **Excentricita** vrcholu je největší geodetická vzdálenost mezi ním a libovolným jiným vrcholem grafu.

Příklad výpočtu shlukovacího koeficientu (síť o 7 uzlech z přednášky): koeficienty jednotlivých uzlů jsou 1, 0,67, 0,33, 0,17, 0, 0, 0; jejich součet je $1 + 0,67 + 0,33 + 0,17 = 2,17$ a po vydělení sedmi vychází shlukovací koeficient sítě $C = 0,31$.

Malý svět

Malý svět (*small world*) je síť, která vypadá téměř náhodně, ale vykazuje **významně vysoký shlukovací koeficient** a **relativně krátkou průměrnou délku cesty** — uzly se lokálně shlukují a přitom jsou dosažitelné v několika krocích.

V sociálních sítích je to velmi časté kvůli: (a) tranzitivitě silných sociálních vazeb a (b) schopnosti slabých vazeb dosáhnout napříč klastry. Sítě typu malý svět mají mnoho klastrů, ale i mnoho mostů mezi nimi, které zkracují průměrnou vzdálenost mezi uzly.

Centralizace a struktura jádro–periferie

Centralizace (*centralization*) měří, do jaké míry je síť centralizovaná či decentralizovaná; vychází z rozdílů ve stupních mezi uzly a obvykle se normalizuje do $[0, 1]$. Síť silně závislá na 1–2 vysoce propojených uzlech (např. v důsledku preferenčního připojování) vykazuje větší rozdíly ve stupňové centralitě. Centralizované struktury mohou být lepší v úlohách vyžadujících koordinaci (týmové řešení problémů), ale jsou náchylnější k selhání, odpojí-li se klíčoví hráči.

Jádro–periferie (*core-periphery*): mnoho velkých skupin má *jádro* hustě propojených uživatelů, kritické pro připojení mnohem větší periferie. Jádra lze identifikovat vizuálně, zkoumáním polohy uzlů s vysokým stupněm a jejich sdružených rozdělení stupňů, nebo *motýlkovou analýzou* (*bow-tie analysis*) používanou pro strukturu webu.

10.6 Vlastnosti reálných komplexních sítí a jejich modely

Šest vlastností reálných komplexních sítí

Reálné komplexní sítě jsou nenáhodné a nepravidelné grafy s charakteristickými rysy (příklady: sociální sítě, informační a znalostní sítě, technologické sítě, biologické sítě):

1. **Efekt malého světa** — Milgram; medián délky úspěšných cest byl 6. Sociální sítě jsou charakterizovány velmi krátkými cestami mezi náhodně zvolenými dvojicemi lidí, což má důsledky pro rychlost šíření informací i nemocí.
2. **Tranzitivita / shlukování** — v reálných, zejména sociálních sítích je vysoká pravděpodobnost nalezení úplných podsítí, kde jsou všechny uzly propojeny („každý zná každého“).
3. **Mocninné rozdělení stupňů** — rozdělení stupňů (histogram podílu uzlů se stupněm k) je u reálných sítí zcela odlišné od náhodných grafů.
4. **Odolnost sítě** (*resilience*) — dopad na souvislost sítě při odstranění jednoho či více uzlů. Většina sítí je robustní vůči náhodnému odstranění vrcholů, ale podstatně méně vůči cílenému odstranění vrcholů s nejvyšším stupněm. Odstranění *vrátných* (*gatekeepers*) silně mění schopnost komunikace mezi dvojicemi uzlů. Odstranění jediného uzlu bývá bezvýznamné — vhodnější je testovat odolnost odstraněním určitého *procenta* uzlů.
5. **Vzorci míšení** (*mixing patterns*) — koexistují-li v síti různé typy uzlů, pozorujeme selektivitu při vytváření vazeb. Selektivní propojování se nazývá *asortativní míšení* (*assortative mixing*, homofilie); klasickým příkladem je míšení podle rasy. Reálné sítě vykazují vyšší sklon k asortativnímu míšení.
6. **Komunitní struktura** — většina reálných sociálních sítí vykazuje komunitní strukturu, která vzniká jako důsledek globální i lokální heterogenity rozložení hran: vysoké koncentrace hran uvnitř určitých oblastí grafu (komunit) a nízké mezi nimi.

Preferenční připojování a bezškálové sítě

Bezškálové sítě (Barabási, Albert, 1999) jsou sítě, jejichž rozdělení stupňů se řídí **mocninným zákonem**:

$$P(k) \sim k^{-\gamma}, \quad \gamma \approx 3 \text{ pro čistý BA model, reálné sítě } \gamma \in \langle 2, 3 \rangle.$$

Mocnná rozdělení vznikají obvykle tehdy, když množství, které něčeho získáte, závisí na množství, které již máte; topologicky se to projevuje jako síť s několika vysoce propojenými uzly (*huby*) a velkým počtem uzlů s nízkým stupněm. Mechanismus se nazýval *rich-get-richer* / *poor-get-poorer*, *Matoušův efekt*, *kumulativní výhoda* (Price) a nověji **preferenční připojování** (Barabási, Albert): nové uzly vstupující do sítě se připojují k dobře propojeným uzlům, které jsou často spojeny s centrálními a prestižními pozicemi.

Dva mechanismy BA modelu: (1) *růst* — v každém kroku přibude nový uzel s m hranami; (2) *preferenční připojování* — nový uzel se připojí k uzlu i s pravděpodobností

$$\Pi(k_i) = \frac{k_i}{\sum_j k_j}.$$

Důvody preferenčního připojování:

- *Popularita* („the rich get richer“) — chceme být spojováni s populárními lidmi, myšlenkami a věcmi, což jejich popularitu dále zvyšuje bez ohledu na objektivní, měřitelné charakteristiky.
- *Kvalita* („the good get better“) — hodnotíme podle objektivních kritérií kvality, takže kvalitnější uzly přirozeně přitáhnou více pozornosti rychleji.
- *Smíšený model* — „nelze předpovědět, kdo se stane hvězdou, i když na kvalitě záleží“: mezi uzly s podobnými atributy se hvězdami stanou ty, které první dosáhnou kritického množství (efekt svatozáře).

Výsledná síť má málo vysoce propojených uzlů a mnoho uzlů s nízkým stupněm — *dlouhoocasé rozdělení stupňů* a strukturu malého světa. Tranzitivita a charakteristiky silných/slabých vazeb jsou běžné a mohou vést ke struktuře malého světa, *nejdou však k jejímu vysvětlení nutné*.

Pro srovnání uvádíme dva klasické referenční modely (nad rámec přednášky, ale standardní v literatuře):

	Erdős–Rényi $G(n, p)$	Watts–Strogatz	Barabási–Albert
Konstrukce	každá hrana nezávisle s pravděpodobností p	kruhová mřížka + přepojení s pravd. β	růst + preferenční připojování
Rozdělení stupňů	Poissonovo (úzké)	úzké	mocnné $k^{-\gamma}$
Průměrná cesta ℓ	$\ln n / \ln \langle k \rangle$ (krátká)	krátká	velmi krátká ($\sim \ln \ln n$)
Shlukovací koef. C	nízký ($\langle k \rangle / n$)	vysoký	nízký
Huby	ne	ne	ano
Co vysvětlí	krátké cesty	krátké cesty + shlukování	krátké cesty + huby + odolnost

Model Watts–Strogatz ukazuje, že již pro malé β (řádu 0,01) klesne průměrná délka cesty téměř na úroveň náhodného grafu, zatímco shlukovací koeficient zůstává vysoký — existuje tedy široké pásmo, kde má síť současně vysoké C i krátké ℓ , což odpovídá reálným sítím.

10.7 Modelování vlivu

Společné vlastnosti metod modelování vlivu

- sociální síť je reprezentována *orientovaným* grafem, každý aktér je jeden uzel;
- každý uzel začíná jako *aktivní* nebo *neaktivní*;
- jakmile je uzel aktivován, aktivuje své sousední uzly;
- jakmile je uzel aktivován, *nemůže* být deaktivován.

Model lineárního prahu (LTM)

Linear Threshold Model — aktér provede akci, pokud počet jeho přátel, kteří akci provedli, překročí určitý práh.

- Každý uzel v volí práh θ_v náhodně z rovnoměrného rozdělení na intervalu $\langle 0, 1 \rangle$.
- V každém diskrétním kroku zůstávají všechny uzly aktivní z předchozího kroku aktivní.
- Aktivovány budou uzly splňující

$$\sum_{w \in N(v), w \text{ je aktivní}} b_{w,v} \geq \theta_v.$$

Model se soustředí na *příjemce* vlivu.

Model nezávislých kaskád (ICM)

Independent Cascade Model — soustředí se na *odesílatele* spíše než na příjemce.

- Uzel w aktivovaný v kroku t má *jednu* příležitost aktivovat každého ze svých sousedů, a to náhodně.
- Pro sousední uzel v se aktivace zdaří s pravděpodobností $p_{w,v}$ (např. $p = 0,5$).
- Zdaří-li se aktivace, stane se v aktivním v kroku $t + 1$.
- V dalších kolech se už w nepokouší aktivovat v .
- Proces difuze začíná počáteční aktivovanou množinou uzlů a pokračuje, dokud není možná žádná další aktivace.

Maximalizace vlivu

Je dána síť a parametr k : které k uzlů má být vybráno do aktivační množiny B , aby byl maximalizován vliv měřený počtem aktivních uzlů na konci? Označíme-li $\sigma(B)$ *očekávaný* počet uzlů, které lze ovlivnit množinou B :

$$\max_{B \subseteq V, |B| \leq k} \sigma(B).$$

Úloha je **NP-těžká**, ale lze dokázat, že *hladový přístup* dává řešení na úrovni **63 %** optima (přesněji $1 - 1/e \approx 0,632$ díky submodularitě a monotonii σ).

Hladový algoritmus:

- začni s $B = \emptyset$;
- vyhodnoť $\sigma(v)$ pro každý uzel a vyber uzel s maximálním $\sigma(v)$ jako první uzel v_1 , čímž vznikne $B = \{v_1\}$;
- vyber uzel, který $\sigma(B)$ nejvíce zvýší, bude-li do B zahrnut, tj. hladově najdi $v \in V \setminus B$ takové, že $\arg \max_{v \in V \setminus B} \sigma(B \cup \{v\})$.

10.7.1 Vliv versus korelace

Atributy a chování uživatelů často korelují s jejich sociálními sítěmi. Předpokládejme binární atribut u každého uzlu (např. zda je kuřák). Koreluje-li atribut se sítí, očekáváme, že aktéři budou sdílet stejné hodnoty atributu pozitivně korelované se sociálními vazbami — kuřáci budou spíše interagovat s kuřáky a nekuřáci s nekuřáky.

Test korelace

Vyskytuje-li se podíl hran spojujících uzly s *různými* hodnotami atributu významně méně často než očekávaná pravděpodobnost, jde o důkaz korelace.

Příklad. Jsou-li vazby nezávislé na kuřáctví a podíl kuřáků je $p = 4/9$, nekuřáků $1 - p = 5/9$, pak jedna hrana spojuje dva kuřáky s pravděpodobností $p \times p$, dva nekuřáky s pravděpodobností $(1 - p) \times (1 - p)$ a kuřáka s nekuřákem s pravděpodobností $2p(1 - p)$. Bez korelace by tedy pravděpodobnost hrany mezi kuřákem a nekuřákem byla

$$2 \times \frac{4}{9} \times \frac{5}{9} = \frac{40}{81} \approx 49\%.$$

Ve zkoumaném grafu je však tento podíl $2/14 = 14\% < 49\%$, takže síť vykazuje jistý stupeň **korelace** vzhledem ke kuřáckému chování.

Tři sociální procesy vysvětlující korelaci

- **Homofilie** — tendence vázat se na ty, kdo s námi sdílejí určitou podobnost („vrána k vráně sedá“). Směr: *atributy* $\rightarrow$ *propojení*.
- **Matoucí proměnná** (*confounding*) — korelace mezi aktéry může vzniknout i vlivem vnějšího prostředí („jednotlivci žijící v téže městě se spíše stanou přáteli než náhodní jednotlivci“).
- **Vliv** (*influence*) — proces způsobující korelaci chování mezi sousedními aktéry („přejde-li většina přátel k jinému mobilnímu operátorovi, může být člověk ovlivněn a přejít také“). Směr: *propojení* $\rightarrow$ *atributy*.

Proč odlišovat vliv od homofilie a matoucích proměnných? *Analýza:* predikovat dynamiku systému — zda se nová norma chování, technologie či myšlenka může šířit jako epidemie. *Návrh:* navrhnout systém vyvolávající určité chování, např. očkovací strategie (náhodné, cílené na demografickou skupinu, cílené na náhodné známé) nebo virální marketingové kampaně.

Logistický model aktivace a věrohodnost

V mnoha studiích je vliv určen *časovými razítky*. Pro hranu (u, v) může uzel u ovlivnit uzel v v diskretních krocích $t = 0, 1, 2, \dots$; v každém t se každý neaktivní uzel stane aktivním s pravděpodobností $p(a)$, kde a je počet aktivních kontaktů. Pravděpodobnost aktivace je *logistickou funkcí* počtu aktivních přátel:

$$p(a) = \frac{e^{\alpha \ln(a+1) + \beta}}{1 + e^{\alpha \ln(a+1) + \beta}} \quad \text{ekvivalentně} \quad \ln \frac{p(a)}{1 - p(a)} = \alpha \ln(a + 1) + \beta,$$

kde a je počet aktivních přátel, α **koefficient sociální korelace**, β konstanta vysvětlující vrozený sklon k aktivaci a $\ln(a + 1)$ vysvětlující proměnná.

Věrohodnost aktivace. Necht se v čase t aktivuje $Y_{a,t}$ uživatelů s a aktivními přáteli a $N_{a,t}$ uživatelů s a aktivními přáteli zůstane neaktivních. Věrohodnost aktivace v čase t je

$$\prod_a p(a)^{Y_{a,t}} (1 - p(a))^{N_{a,t}},$$

a věrohodnost celé posloupnosti $\{Y_{a,t}, N_{a,t}\}_{t=0}^T$ je

$$\prod_{t=0}^T \prod_a p(a)^{Y_{a,t}} (1 - p(a))^{N_{a,t}}.$$

Protože $Y_{a,t}$ i $N_{a,t}$ s rostoucím a rychle klesají, používá se jen $a \leq R$ pro nějakou konstantu R . Z logu aktivit uživatelů spočteme α a β maximalizující $\prod_a p(a)^{Y_a} (1 - p(a))^{N_a}$, kde $Y_a = \sum_t Y_{a,t}$ a $N_a = \sum_t N_{a,t}$; α pak měří korelaci mezi uzly.

Shuffle test — vliv vs. korelace

Klíčová myšlenka: nehraje-li vliv roli, mělo by být načasování aktivace nezávislé na načasování ostatních aktérů. I když tedy *náhodně zpřeházíme* časová razítka aktivit uživatelů, měli bychom dostat podobnou sociální korelaci.

Test vlivu: Zpřeházej časová razítka aktivit uživatelů. Liší-li se nový odhad sociální korelace významně od odhadu založeného na skutečném logu aktivit, jde o důkaz *vlivu*.

Postup. Nechť G je sociální síť, $W = \{w_1, \dots, w_\ell\}$ množina uzlů aktivovaných během celého měření o T krocích, přičemž uzel w_i byl aktivován v čase t_i .

1. Pro všechna $t = 1, \dots, T$ spočti $Y_{a,t}$ a $N_{a,t}$.
2. Spočti α a β maximalizující $\prod_a p(a)^{Y_a} (1 - p(a))^{N_a}$.
3. Vyber náhodnou permutaci π množiny $\{1, \dots, \ell\}$ a nastav čas aktivace uzlu w_i na $t'_i = t_{\pi(i)}$; na základě $t'_1, \dots, t'_\ell$ spočti $Y'_{a,t}$ a $N'_{a,t}$.
4. Znovu spočti α' a β' maximalizující $\prod_a p'(a)^{Y'_a} (1 - p'(a))^{N'_a}$.
5. Model *nevykazuje* sociální vliv, jsou-li hodnoty α a α' blízké.

10.8 Homofilie, výběr a sociální vliv

Homofilie a test homofilie

Homofilie: princip, že máme tendenci být podobní svým přátelům — „vrána k vráně sedá“. Vaši přátelé jsou vám podobnější věkem, rasou, zájmy a názory než náhodný výběr jednotlivců.

Homofilie a triadický uzávěr: sdílí-li dva jednotlivci společného přítele, je i mezi nimi pravděpodobnější vznik přátelství. Homofilie naznačuje, že dva jednotlivci jsou si podobnější *kvůli* společnému příteli, takže vazba může vzniknout, i když ani jeden o společném příteli neví. Obvykle je obtížné přisoudit vznik vazby jedinému faktoru.

Test homofilie: Je-li podíl *heterogenních* (např. mezipohlavních) hran *významně nižší* než $2pq$, jde o důkaz homofilie. Zde p je podíl uzlů první hodnoty a $q = 1 - p$ druhé; kdyby byla každému uzlu náhodně přiřazena hodnota podle skutečného poměru, neměl by se počet heterogenních hran oproti skutečné síti významně změnit.

Příklad. Síť má 9 uzlů, z toho $p = 6/9 = 2/3$ mužů a $q = 3/9 = 1/3$ žen, a 18 hran, z nichž je 5 mezipohlavních. Bez homofilie by mezipohlavních hran mělo být

$$2pq = 2 \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{4}{9}, \quad \text{tj.} \quad \frac{4}{9} \cdot 18 = 8 \text{ z } 18.$$

Skutečných 5 je významně méně než očekávaných 8 $\Rightarrow$ **důkaz homofilie**.

Poznámky k testu. Jak významné je „významně nižší“? Vhodné je posuzovat odchylku pod střední hodnotou. Co když má síť *významně více* než $2pq$ heterogenních hran? Pak jde o **inverzní homofilii** — příkladem jsou partnerské vztahy muž-žena. Test lze použít na libovolnou charakteristiku (rasu, věk, mateřský jazyk, preference); pro charakteristiky s více než dvěma hodnotami se provede analogický výpočet: porovnáme počet heterogenních hran s tím, jak by vypadal náhodně generovaný graf používající skutečná data jako pravděpodobnosti.

Výběr vs. sociální vliv

Výběr (*selection*) — lidé si volí přátele podobné sobě samým. Působí na různých úrovních a s různou mírou záměrnosti: aktivně volíte přátele podobné sobě v malé skupině; populace vaší školy je relativně homogenní vůči celkové populaci, takže vás prostředí nutí volit přátele podobné sobě.

Sociální vliv (*social influence*) — lidé jsou náchylní k sociálnímu vlivu a mohou měnit své chování tak, aby se blížilo chování jejich přátel („Špatná společnost kazí dobré mravy.“).

Sociální vliv je opakem výběru: výběr znamená, že *individuální charakteristiky řídí vznik vazeb*; sociální vliv znamená, že *existující vazby formují měnitelné charakteristiky lidí*.

Měnitelné a neměnitelné charakteristiky (*mutable / immutable*): výběr funguje odlišně podle typu charakteristiky. Neměnitelné se nemění (pohlaví, rasa) nebo se mění konzistentně s populací (věk, generace); měnitelné se mohou v čase měnit (chování, přesvědčení, zájmy, názory).

Longitudinální studie. Z jediného snímku sítě je obtížné rozlišit, zda působí výběr nebo sociální vliv. Longitudinální studie sledující sociální vazby a individuální chování v čase pomáhají odhalit efekt sociálního vlivu: *mění se chování po změnách v síti, nebo se mění síť po změnách v chování?*

Příklad — školní prospěch a užívání drog. Pochopení těchto efektů pomáhá při návrhu intervencí: vykazuje-li užívání drog v síti homofilii, může nejlépe fungovat program cílený na sociální vliv (přimět přátele, aby ovlivnili ostatní přátele k ukončení). Vznikla-li však homofilie *výběrem*, mohou si bývalí uživatelé zvolit nové přátele a chování ostatních tím není silně ovlivněno.

Příklad — obezita. Christakis a Fowler sledovali stav obezity a sociální síť 12 000 lidí po dobu 32 let a našli homofilii podle obezity. Zvažovali tři hypotézy: (1) efekty výběru — volí si lidé přátele s podobným stavem obezity? (2) matoucí efekty — existují další faktory korelující s obezitou? (3) sociální vliv — ovlivnila změna obezity přátel budoucí stav člověka? Nalezen byl **významný důkaz pro hypotézu 3**: obezita je typem „nákazy“, která se může šířit sociálním vlivem.

10.9 Afilační síť a uzávěrové procesy

Afilační a sociálně-afilační síť

Afilační síť (*affiliation network*) vnáší do sítě kontext zobrazením vazeb na aktivity, firmy, organizace, čtvrti apod. Jde o **bipartitní graf**: každá hrana spojuje dva uzly patřící do různých množin — *lidé* a *ohniska* (*foci*, *afiliace*).

Sociálně-afilační síť má **dva typy hran**:

- *člověk–člověk*: přátelství nebo jiný sociální vztah;
- *člověk–ohnisko*: účast v daném ohnisku.

Tři uzávěrové procesy

- **Triadický uzávěr** (*triadic closure*) — dva lidé se společným přítelem se propojí. *Tranzitivita na lidech*.
- **Ohniskový uzávěr** (*focal closure*) — dva lidé sdílející ohnisko se propojí. Uzávěr díky *výběru*.
- **Členský uzávěr** (*membership closure*) — člověk se zapojí do ohniska, do kterého jsou už zapojeni jeho přátelé. Uzávěr díky *sociálnímu vlivu*.

Otázky o uzávěrech. Je triadický uzávěr závislý na *počtu* sdílených přátel? Formálně: jaká je pravděpodobnost, že dva lidé vytvoří vazbu, jako funkce počtu společných přátel? Analogicky pro ohniskový uzávěr (jako funkce počtu společně sdílených ohnisek) a členský uzávěr (jako funkce počtu přátel, kteří už jsou v ohnisku zapojeni).

Metodologie měření triadického uzávěru

1. Pořídí dva snímky sítě v časech t_1 a t_2 .
2. Pro každé k identifikuj všechny dvojice uzlů, které mají v t_1 přesně k společných přátel, ale nejsou přímo spojeny hranou.
3. Definuj $T(k)$ jako podíl těchto dvojic, které do času t_2 vytvoří hranu — to je pravděpodobnost vzniku vazby mezi dvěma lidmi s k společnými přáteli.
4. Vynes $T(k)$ jako funkci k a ilustruj efekt společných přátel na vznik vazby.

Základní (nulový) model: vzniká-li vazba nezávisle s pravděpodobností p pro každého společného přítele, pak

$$T_{base}(k) = 1 - (1 - p)^k \quad (\text{resp. } T_{base}(k) = 1 - (1 - p)^{k-1}).$$

Empirické výsledky. *E-mailová síť* (Kossinets, Watts, 2006): e-mailová komunikace 22 000 studentů velké americké univerzity po dobu jednoho roku; vazba se předpokládá, byl-li mezi dvěma lidmi za posledních 60 dní odeslán e-mail; snímky po jednom dni, $T(k)$ zprůměrováno přes více dvojic snímků. Pozorování: téměř žádné e-maily bez společných přátel; *významný nárůst* při přechodu z 1 na 2 společné přátele; další nárůst je významný, ale na mnohem menší subpopulaci — tedy *klesající výnosy*.

Ohniskový uzávěr: titíž autoři získali rozvrhy 22 000 studentů a vytvořili sociálně-afiliační síť, kde ohnisky jsou přednášky. Sdílení přednášky má *téměř stejný efekt* jako sdílení přítele; opět *klesající výnosy*.

Členský uzávěr: Backstrom et al. (2006) vytvořili sociálně-afiliační síť pro LiveJournal (přátelství podle profilů, ohniska jsou uživatelsky definované komunity); Crandall et al. (2008) totéž pro Wikipedii (uzly jsou editoři udržující diskusní stránku, vazba znamená komunikaci přes diskusní stránku, ohnisky jsou editované články). V obou případech opět významný nárůst z 1 na 2 a *klesající výnosy*.

Výběr vs. vliv na datech Wikipedie. Podobnost dvou editorů se měří jako

$$\frac{\text{počet článků editovaných oběma editory}}{\text{počet článků editovaných alespoň jedním editorem}}$$

(analogie překryvu sousedství pro hrany). Dvojice editorů, kteří spolu komunikovali, jsou si v chování významně podobnější než ti, kteří spolu nikdy nekomunikovali. Vzniká homofilie proto, že editoři mluví s editory téhož článku (*výběr*), nebo proto, že jsou k článkům přivedeni těmi, s nimiž mluví (*sociální vliv*)? Zkoumá se průměr interakčních historií, kde 0 odpovídá okamžiku vzniku vazby a podobnost se měří relativně k němu (jen editoři s alespoň 100 akcemi před i po první interakci; základní linií je podobnost náhodné dvojice editorů).

10.10 Schellingův model segregace

Schellingův model

Jednoduchý model, který zavedl Thomas Schelling v 70. letech; ukazuje, jak mohou **globální vzorce** samovolně zvolené segregace vznikat kvůli **homofilii na lokální úrovni**.

Nastavení: populaci tvoří dva typy jednotlivců (agenti X a O) reprezentující *neměnitelné* charakteristiky (rasa, etnicita, země původu, mateřský jazyk). Agenti jsou náhodně rozmístěni v mřížce. Agent je **spokojen** se svou polohou, je-li alespoň t jeho sousedů agenty téhož typu (t je předem daný práh).

Průběh kola:

1. Identifikuj všechny nespokojené agenty (pro $t = 3$ chce každý agent alespoň 3 sousedy téhož typu).
2. Všichni nespokojení agenti se přesunou na pozici, kde budou spokojeni.
3. Přesuny mohou způsobit, že dříve spokojený agent přestane být spokojen $\Rightarrow$ opakuj.

Někdy agent nenajde novou polohu, která by ho uspokojila — pak zůstane, nebo se přesune na náhodnou pozici.

Výsledek: mřížka je po několika kolech *výrazně segregovanější* než na začátku.

Varianty modelu: agenti se přesouvají v náhodném pořadí nebo systematicky; přesun na nejbližší vyhovující nebo náhodnou pozici; práh jako procento nebo proměnlivý mezi skupinami i uvnitř nich; svět se může „zaobalit“ (levý okraj navazuje na pravý). Výsledek bývá vždy stejný: **samovolně vzniklá segregace**.

Závěr. Schellingův model je příkladem toho, jak se *neměnitelné* charakteristiky mohou stát silně korelovanými s *měnitelnými*: volba místa bydliště (měnitelná) se časem přizpůsobí typu agenta (neměnitelnému), čímž vzniká segregace, tedy homofilie.

10.11 Pozitivní a negativní vztahy: strukturní balance

Dosud měl vztah v síti převážně pozitivní konotaci, existují však sociální sítě, kde lze vyjádřit i *negativní* (nepřátelské) vztahy. Otázka zní: *kdy je síť ve vyváženém stavu?* Uvidíme, jak lokální vlastnosti vynucují vlastnosti globální (srovnej se Schellingovým modelem). Zjednodušíme: uvažujeme síť, kde jsou *všechny* dvojice uzlů spojeny pozitivním nebo negativním vztahem (úplný graf, klika) — to je užitečné pro malé skupiny lidí či pro státy.

Strukturní balance

Pro trojici uzlů existují čtyři možné konfigurace znamének:

Konfigurace	Interpretace	Vyvážená?
+, +, +	A, B a C jsou vzájemní přátelé	ano
+, -, -	A a B jsou přátelé, C je jejich společný nepřítel	ano
+, +, -	B a C jsou přáteli A, ale spolu nevycházejí	ne
-, -, -	A, B a C jsou vzájemní nepřátelé	ne

Vlastnost strukturní balance: pro každý trojúhelník platí, že buď jsou všechny tři hrany pozitivní, nebo je pozitivní právě jedna.

Věta o bilanci

Věta. Je-li označovaný úplný graf vyvážený, pak buď

- jsou všechny dvojice uzlů přátelé, nebo
- lze uzly rozdělit do dvou skupin X a Y tak, že každá dvojice uzlů v X se má ráda, každá dvojice v Y se má ráda, a každý v X je nepřítelem každého v Y .

Důkaz. Nechť N je vyvážená síť. Jsou-li všechny hrany pozitivní, jsme hotovi. Existuje-li alespoň jedna negativní hrana, zvolme uzel A a položme $X :=$ všichni přátelé A (včetně A), $Y :=$ všichni nepřátelé A . Zbývá ověřit tři body:

1. *Každé dva uzly v X jsou přátelé.* Jsou-li $B, C \in X$, pak $AB = +$ a $AC = +$; kdyby $BC = -$, byl by trojúhelník ABC typu $+, +, -$, tedy nevyvážený. Proto $BC = +$.
2. *Každé dva uzly v Y jsou přátelé.* Jsou-li $D, E \in Y$, pak $AD = -$ a $AE = -$; kdyby $DE = -$, byl by trojúhelník typu $-, -, -$, tedy nevyvážený. Proto $DE = +$.
3. *Každý uzel v X je nepřítelem každého uzlu v Y .* Je-li $B \in X$ a $D \in Y$, pak $AB = +$ a $AD = -$; kdyby $BD = +$, byl by trojúhelník typu $+, +, -$, tedy nevyvážený. Proto $BD = -$.

□

Slabá strukturní balance

Vlastnost slabé strukturní balance: neexistuje trojice uzlů, jejíž hrany tvoří přesně dvě pozitivní a jednu negativní hranu.

Motivace: konfigurace $+, +, -$ je *silnější nevyváženost* — B a C se snaží vyřešit svou nevraživost, nebo dvojice vytvoří alianci proti třetímu. Naproti tomu $-, -, -$ (všichni vzájemní nepřátelé) je ve slabém pojetí přípustná.

Věta. Je-li označovaný úplný graf slabě vyvážený, lze jeho uzly rozdělit do skupin vzájemných přátel tak, že libovolné dva uzly patřící do různých skupin jsou nepřátelé.

Důkaz. (1) Nechť A je uzel a X množina přátel A včetně A . (2) Všichni přátelé A jsou přáteli mezi sebou. (3) A i všichni jeho přátelé jsou nepřáteli všech ostatních. (4) Odstraň X ze sítě a pokračuj krokem 1. □

Zobecnění. V praxi (a) nejsou spojeny všechny dvojice a (b) nemusí být vyvážené všechny trojúhelníky. Neúplný graf nazveme vyváženým, lze-li jej „doplnit“ přidáním hran na vyvážený označovaný úplný graf. Je-li vyvážena většina trojúhelníků, lze síť přibližně rozdělit na dvě frakce.

Aplikace strukturní balance. *Vývoj aliancí v Evropě 1872–1907* (Spolek tří císařů 1872–81, Trojspolek 1882, zánik německo-ruské smlouvy 1890, francouzsko-ruská aliance 1891–94, Srdečná dohoda 1904, britsko-ruská aliance 1907): síť postupně sklouzává do vyváženého označování — a do první světové války. *Konflikt o odtržení Bangladéše od Pákistánu (1972)*: překvapivá podpora Pákistánu ze strany USA je méně překvapivá, uvědomíme-li si, že SSSR byl nepřítelem Číny, Čína nepřítelem Indie a Indie měla tradičně špatné vztahy s Pákistánem; protože USA v té době zlepšovaly vztahy s Čínou, podpořily nepřátele nepřátel Číny.

10.12 Analýza vazeb: huby, autority a algoritmus HITS

Huby a autority

Autorita (*authority*) — zhruba stránka s mnoha *vstupními* vazbami. Myšlenka: stránka může mít dobrý či autoritativní obsah k nějakému tématu, a proto jí mnoho lidí důvěřuje a odkazuje na ni.

Hub — stránka s mnoha *výstupními* vazbami. Slouží jako organizátor informací o daném tématu a ukazuje na mnoho dobrých autoritativních stránek.

Klíčová myšlenka HITS: dobrý hub ukazuje na mnoho dobrých autorit a na dobrou autoritu ukazuje mnoho dobrých hubů — autority a huby jsou ve vztahu *vzájemného posilování* (hustě propojený bipartitní podgraf).

Proč oddělovat huby od autorit? Konkurenční firmy na sebe navzájem neodkazují a nelze je považovat za vzájemné doporučení. V PageRanku naopak doporučení přechází přímo z jedné významné stránky na druhou — to je dominantní režim v akademických a vládních stránkách, mezi blogery, osobními stránkami či ve vědecké literatuře.

Algoritmus HITS

Hypertext Induced Topic Search (Kleinberg, 1998) — na rozdíl od PageRanku, který je statický, je HITS **závislý na dotazu**.

Sběr stránek. Pro široký dotaz q : pošli dotaz vyhledávači a sesbírej t nejlépe hodnocených stránek ($t = 200$ v původním článku) — to je *kořenová množina* (*root set*) W . Rozšiř W o každou stránku, na kterou ukazuje stránka z W , i o každou stránku, která na stránku z W ukazuje — vznikne větší *bázová množina* (*base set*) S :

$$S = W \cup \{d \mid s \rightarrow d \vee d \rightarrow s; s \in W\}.$$

V praxi se počet vstupních odkazů na jednu stránku z W omezuje (např. na 50), aby S nenarostla přes únosnou mez.

Skóre. Nechť L je matice sousednosti grafu S : $L_{ij} = 1$, je-li $(i, j) \in E$, jinak 0. Vzájemné posilování autoritního skóre $a(i)$ a hubovského skóre $h(i)$:

$$a(i) = \sum_{(j,i) \in E} h(j) \quad \text{a} \quad h(i) = \sum_{(i,j) \in E} a(j),$$

maticově

$$\mathbf{a} = L^T \mathbf{h}, \quad \mathbf{h} = L \mathbf{a}.$$

Algoritmus HITS-ITERATE(G):

- 1: $\mathbf{a}_0 \leftarrow \mathbf{h}_0 \leftarrow (1, 1, \dots, 1)^T$; $k \leftarrow 1$
- 2: **repeat**
- 3: $\mathbf{a}_k \leftarrow L^T L \mathbf{a}_{k-1}$
- 4: $\mathbf{h}_k \leftarrow L L^T \mathbf{h}_{k-1}$
- 5: $\mathbf{a}_k \leftarrow \mathbf{a}_k / \|\mathbf{a}_k\|_1$; $\mathbf{h}_k \leftarrow \mathbf{h}_k / \|\mathbf{h}_k\|_1$ ▷ normalizace
- 6: $k \leftarrow k + 1$
- 7: **until** $\|\mathbf{a}_k - \mathbf{a}_{k-1}\|_1 < \varepsilon_a$ a $\|\mathbf{h}_k - \mathbf{h}_{k-1}\|_1 < \varepsilon_h$
- 8: **return** $\mathbf{a}_k$ a $\mathbf{h}_k$

Normalizace je nutná, protože hodnoty mají tendenci růst do velkých čísel; dělí se součtem všech autoritních, resp. hubovských skóre.

Poznámka: zápis $\mathbf{a}_k \leftarrow L^T L \mathbf{a}_{k-1}$ je jen zkratkou za dvojici kroků $\mathbf{h} \leftarrow L \mathbf{a}$, $\mathbf{a} \leftarrow L^T \mathbf{h}$; obě verze mají stejnou limitu (hlavní vlastní vektor), liší se však o jeden „půlkrok“, takže po *konečném* počtu iterací dávají různá mezivýsledná čísla. Následující příklad počítá po půlkrocích.

Spektrální analýza HITS

Označme M matici sousednosti ($M_{ij} = 1$ pro vazbu $i \rightarrow j$). Pravidla aktualizace zapíšeme jako násobení matice vektorem: $\mathbf{h} := M\mathbf{a}$, $\mathbf{a} := M^T\mathbf{h}$. Po k krocích tedy

$$\mathbf{a}^{(k)} = (M^T M)^{k-1} M^T \mathbf{h}^{(0)}, \quad \mathbf{h}^{(k)} = (M M^T)^k \mathbf{h}^{(0)}.$$

Velikosti hodnot rostou s každou aktualizací a konvergují teprve po normalizaci: existují konstanty c a d tak, že $\mathbf{h}^{(k)}/c^k$ a $\mathbf{a}^{(k)}/d^k$ konvergují. Vektor $\mathbf{h}^{(k)}$ konverguje k limitě $\mathbf{h}^*$, pokud se po vynásobení maticí MM^T nemění jeho *směr* (velikost se změnit může, o faktor c):

$$MM^T \mathbf{h}^* = c \mathbf{h}^*,$$

tedy $\mathbf{h}^*$ musí být **vlastním vektorem** matice MM^T .

Důkaz konvergence. MM^T je symetrická, má tedy n vlastních vektorů $v_1, \dots, v_n$ s vlastními čísly $|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_n|$. Rozvineme-li $\mathbf{h}^{(0)} = q_1 v_1 + \dots + q_n v_n$, dostaneme

$$\mathbf{h}^{(k)} = (MM^T)^k \mathbf{h}^{(0)} = q_1 \lambda_1^k v_1 + \dots + q_n \lambda_n^k v_n,$$

a při předpokladu $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$

$$\frac{\mathbf{h}^{(k)}}{\lambda_1^k} = q_1 v_1 + q_2 \left(\frac{\lambda_2}{\lambda_1}\right)^k v_2 + \dots + q_n \left(\frac{\lambda_n}{\lambda_1}\right)^k v_n.$$

Pro $k \rightarrow \infty$ jdou všechny členy kromě prvního k nule, takže $\mathbf{h}^{(k)}/\lambda_1^k$ konverguje k $q_1 v_1$. (Zbývalo by dokázat konvergenci nezávisle na počátečním vektoru a uvolnit předpoklad $|\lambda_1| > |\lambda_2|$.)

Pro reálné symetrické matice jsou všechna vlastní čísla reálná a vlastní vektory příslušné různým vlastním číslům jsou ortogonální.

Vztah ke spolucitovanosti a bibliografické vazbě

Spolucitovanost (*co-citation*) stránek i a j — počet dokumentů, které citují *obě*:

$$C_{ij} = \sum_{k=1}^n L_{ki} L_{kj} = (L^T L)_{ij}.$$

Autoritní matice $L^T L$ algoritmu HITS je tedy *maticí spolucitovanosti*.

Bibliografická vazba (*bibliographic coupling*) stránek i a j — počet dokumentů citovaných *oběma*:

$$B_{ij} = \sum_{k=1}^n L_{ik} L_{jk} = (L L^T)_{ij}.$$

Hubovská matice $L L^T$ algoritmu HITS je tedy *maticí bibliografické vazby*.

Příklad — výpočet HITS

Graf o 4 uzlech s hranami $A \rightarrow C$, $A \rightarrow D$, $B \rightarrow C$, $B \rightarrow D$, $C \rightarrow D$.

Start: $\mathbf{a}_0 = \mathbf{h}_0 = (1, 1, 1, 1)^T$ v pořadí (A, B, C, D) .

Iterace 1. $\mathbf{a} = L^T \mathbf{h}_0$, tj. autoritní skóre = součet hubovských skóre uzlů, které na daný uzel ukazují:

$$a_A = 0, \quad a_B = 0, \quad a_C = h_A + h_B = 2, \quad a_D = h_A + h_B + h_C = 3 \Rightarrow \mathbf{a}_1 = (0, 0, 2, 3).$$

$\mathbf{h} = L \mathbf{a}_1$, tj. hubovské skóre = součet autoritních skóre uzlů, na které daný uzel ukazuje:

$$h_A = a_C + a_D = 5, \quad h_B = 5, \quad h_C = a_D = 3, \quad h_D = 0 \Rightarrow \mathbf{h}_1 = (5, 5, 3, 0).$$

Iterace 2.

$$\mathbf{a}_2 = (0, 0, h_A+h_B=10, h_A+h_B+h_C=13), \quad \mathbf{h}_2 = (10+13=23, 23, 13, 0).$$

Normalizace (dělení součtem složek): $\mathbf{a} = (0; 0; 10/23; 13/23) = (0; 0; 0,435; 0,565)$ a $\mathbf{h} = (23/59; 23/59; 13/59; 0) = (0,390; 0,390; 0,220; 0)$.

Interpretace: nejlepší autoritou je D (ukazují na ni všechny tři ostatní uzly, včetně dobrého hubu C), následuje C . Nejlepšími huby jsou A a B (ukazují na obě autority); D má nulové hubovské skóre, protože nikam neukazuje.

Konvergence: uvedené hodnoty jsou stav *po dvou iteracích*. V limitě platí pro poměr $r = a_C/a_D$ vztah $r = \frac{2(r+1)}{2(r+1)+1}$, tj. $2r^2 + r - 2 = 0$, odkud $r = \frac{-1+\sqrt{17}}{4} = 0,7808$ a normovaně $\mathbf{a}^* = (0; 0; 0,4385; 0,5615)$ — dvě iterace tedy stačí na tři platné číslice pořadí.

Silné a slabé stránky HITS. *Síla:* schopnost hodnotit stránky podle tématu dotazu, což může poskytnout relevantnější autority a huby. *Slabiny:* (a) snadno se dá *spamovat* — přidat výstupní vazby na vlastní stránku je velmi snadné; (b) *tematický drift* — mnoho stránek v rozšířené množině nemusí být k tématu; (c) *neefektivita v době dotazu* — sběr kořenové množiny, její rozšíření i výpočet vlastních vektorů jsou nákladné operace.

10.13 PageRank

Prestiž podle hodnoty (rank prestige)

U měř prestiže se uvažuje důležitý faktor — *významnost jednotlivých aktérů, kteří „hlasují“*. Ve skutečném světě je osoba i zvolená důležitou osobou prestižnější než osoba zvolená méně důležitou osobou (hlas generálního ředitele váží víc než hlas řadového pracovníka). Je-li něco okruh vlivu plný prestižních aktérů, je i jeho vlastní prestiž vysoká.

Prestiž $PR(i)$ je definována jako lineární kombinace hodnoty vazeb, které do i ukazují:

$$PR(i) = A_{1i}PR(1) + A_{2i}PR(2) + \dots + A_{ni}PR(n),$$

kde $A_{ji} = 1$, ukazuje-li j na i , a 0 jinak. Maticově s $\mathbf{P} = (PR(1), \dots, PR(n))^T$:

$$\mathbf{P} = A^T \mathbf{P},$$

což je charakteristická rovnice — $\mathbf{P}$ je *vlastním vektorem* matice A^T .

PageRank: základní definice

PageRank spoléhá na demokratickou povahu webu a využívá jeho rozsáhlou odkazovou strukturu jako indikátor hodnoty stránky. Hypertextový odkaz ze stránky j na stránku i interpretuje jako *hlas* stránky j pro stránku i . PageRank ale nesleduje jen počet hlasů — analyzuje i stránku, která hlas odevzdává: hlasy „důležitých“ stránek váží více. To je přesně myšlenka prestiže podle hodnoty.

Web jako orientovaný graf $G = (V, E)$ s n stránkami; O_j je počet výstupních vazeb j :

$$P(i) = \sum_{(j,i) \in E} \frac{P(j)}{O_j}.$$

Maticově s $A_{ij} = 1/O_i$ pro $(i, j) \in E$ a 0 jinak:

$$\mathbf{P} = A^T \mathbf{P}.$$

Intuice. PageRank je jakási „tekutina“: obíhá sítě, přechází od uzlu k uzlu po hranách a hromadí se v nejdůležitějších uzlech. *Celkový PageRank v síti zůstává konstantní* — každá stránka svůj PageRank rozdělí a předá po vazbách, takže se nikdy nevytváří ani neztrácí, jen přesouvá. Na rozdíl od HITS proto *není nutné* PageRank normalizovat, aby hodnoty nerostly.

Problémy základní definice a jejich řešení

Problém 1: „nesprávné“ uzly získají všichni PageRank. Ukazují-li si uzly F a G navzájem místo na A , PageRank, který přiteče z C do F a G , se už nikdy nemůže vrátit — pro velká k dostane každý z F a G hodnotu $\frac{1}{2}$ a všechny ostatní 0. Jde o „pomalý únik“ (*slow leak*): malé množiny uzlů, které jsou ze zbytku grafu dosažitelné, ale nemají cestu zpět. Řešení: posílat malý zlomek PageRanku všem uzlům.

Škálované pravidlo aktualizace. Zvol škálovací faktor $s \in (0, 1)$: (1) aplikuj základní pravidlo; (2) zmenši všechny hodnoty faktorem s (celkový PageRank klesne z 1 na s); (3) rozděl zbylých $1 - s$ jednotek rovnoměrně mezi všechny uzly, tedy $(1 - s)/n$ každému. Jde o „koloběh vody“: v každém kroku se odpaří $1 - s$ jednotek PageRanku a rovnoměrně naprší na všechny uzly. Opakovaná aplikace konverguje k jedinečné rovnováze (hodnoty ovšem závisí na volbě s); v praxi se používá $s \in \langle 0,8; 0,9 \rangle$ — *škálovací faktor* = *tlumicí faktor* (*damping factor*).

Problém 2: viset zůstávající uzly (*dangling nodes*) — stránky bez výstupních vazeb dávají v A nulové řádky, takže $\sum_j A_{ij} = 1$ neplatí a A není stochastická matice. Řešení: (a) tyto uzly během výpočtu odstranit (neovlivňují přímo hodnocení jiných stránek); (b) položit $A_{ii} = 1$; (c) přidat z nich úplnou sadu výstupních vazeb na všechny ostatní stránky, tj. $A_{ij} = 1/n$.

PageRank jako Markovův řetězec

Každá stránka je *stav*, hypertextový odkaz je *přechod* s určitou pravděpodobností; surfování webu je tak modelováno jako stochastický proces. Předpokládáme-li, že surfař kliká na odkazy rovnoměrně náhodně (nepoužívá tlačítko „zpět“ ani nezadáva URL), je každá přechodová pravděpodobnost $1/O_i$.

Pro počáteční rozdělení $\mathbf{p}_0$ a přechodovou matici A platí $\sum_i p_0(i) = 1$ a $\sum_j A_{ij} = 1$; pak je A *stochastickou maticí* Markovova řetězce a

$$p_1(j) = \sum_{i=1}^n A_{ij} p_0(i), \quad \mathbf{p}_1 = A^T \mathbf{p}_0, \quad \mathbf{p}_k = A^T \mathbf{p}_{k-1}.$$

Věta (stacionární rozdělení). Konečný Markovův řetězec daný stochastickou maticí A má *jednoznačné* stacionární rozdělení, je-li A **ireducibilní** a **aperiodická**. Pak $\lim_{k \rightarrow \infty} \mathbf{p}_k = \boldsymbol{\pi}$ nezávisle na volbě $\mathbf{p}_0$, a ve stacionárním stavu $\boldsymbol{\pi} = A^T \boldsymbol{\pi} - \boldsymbol{\pi}$ je hlavním vlastním vektorem A^T s vlastním číslem 1. V PageRanku se $\boldsymbol{\pi}$ používá jako vektor $\mathbf{P}$, což je intuitivní: odráží dlouhodobé pravděpodobnosti, že náhodný surfař stránku navštíví.

Ireducibilita znamená, že web graf G je *silně souvislý*: pro každou dvojici uzlů u, v existuje cesta z u do v . Obecný web graf ireducibilní není.

Aperiodicita: stav i je periodický s periodou $k > 1$, je-li k nejmenší číslo takové, že všechny cesty ze stavu i zpět do i mají délku, která je násobkem k . Není-li stav periodický ($k = 1$), je aperiodický; řetězec je aperiodický, jsou-li aperiodické všechny stavy. Příklad periodického řetězce s $k = 3$: matice A s jediným cyklem $1 - 2 - 3 - 1$, kde každý návrat do stavu 1 trvá $3h$ přechodů.

Řešení obou problémů jedinou strategií: přidej hranu z každé stránky na každou stránku a dej každé takové hraně malou přechodovou pravděpodobnost řízenou parametrem d . Rozšířená přechodová matice je pak zjevně ireducibilní i aperiodická.

Finální algoritmus PageRank

Po rozšíření má náhodný surfař na stránce dvě možnosti: s pravděpodobností d si náhodně vybere odkaz, který následuje; s pravděpodobností $1 - d$ skočí na náhodnou stránku. Vylepšený model:

$$\mathbf{P} = \left((1 - d) \frac{E}{n} + d A^T \right) \mathbf{P},$$

kde $E = \mathbf{e} \mathbf{e}^T$ ($\mathbf{e}$ je sloupcový vektor samých jedniček), tedy E je matice $n \times n$ samých jedniček. Matice $(1 - d) \frac{E}{n} + d A^T$ je stochastická (transponovaná), ireducibilní i aperiodická.

Škálujeme-li tak, že $\mathbf{e}^T \mathbf{P} = n$, dostaneme

$$\mathbf{P} = (1 - d) \mathbf{e} + d A^T \mathbf{P}, \quad \text{po složkách} \quad P(i) = (1 - d) + d \sum_{j=1}^n A_{ji} P(j) = (1 - d) + d \sum_{(j,i) \in E} \frac{P(j)}{O_j}.$$

Parametr d je *tlumicí faktor* z $(0, 1)$; v původním článku $d = 0,85$.

Normalizovaná varianta (součet PageRanků = 1, běžná v praxi): $P(i) = \frac{1-d}{n} + d \sum_{(j,i) \in E} \frac{P(j)}{O_j}$.

Výpočet mocninou iterací:

- 1: **PAGERANK-ITERATE**(G): $\mathbf{P}_0 \leftarrow \mathbf{e}/n$; $k \leftarrow 1$
- 2: **repeat**
- 3: $\mathbf{P}_{k+1} \leftarrow (1 - d) \mathbf{e} + d A^T \mathbf{P}_k$
- 4: $k \leftarrow k + 1$
- 5: **until** $\|\mathbf{P}_{k+1} - \mathbf{P}_k\|_1 < \varepsilon$
- 6: **return** $\mathbf{P}_{k+1}$

Příklad — výpočet PageRanku

Orientovaný graf: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow C$, $C \rightarrow A$. $n = 3$, $d = 0,85$; použijeme normalizovanou variantu, kde $\frac{1-d}{n} = 0,05$:

$$PR(A) = 0,05 + 0,85 PR(C), \quad PR(B) = 0,05 + 0,85 \frac{PR(A)}{2},$$

$$PR(C) = 0,05 + 0,85 \left(\frac{PR(A)}{2} + PR(B) \right).$$

Start $PR = 1/3$ pro všechny vrcholy:

Iterace	$PR(A)$	$PR(B)$	$PR(C)$
0	0,3333	0,3333	0,3333
1	0,3333	0,1917	0,4750
2	0,4538	0,1917	0,3546
3	0,3514	0,2429	0,4058
limita	0,3878	0,2148	0,3974

Přesné řešení dostaneme dosazením: $PR(C) = 0,0925 + 0,78625 PR(A)$ a $PR(A) = 0,05 + 0,85 PR(C)$, odkud $PR(A) = 0,1286/0,3317 = 0,3878$. Součet je vždy 1.

Interpretace: nejvyšší PageRank má C — dostává celý příspěvek od B a polovinu od A . Poučné je ale srovnání C a A : C má *dvě* vstupní hrany, A jedinou, a přesto je jejich skóre téměř shodné (0,3974 vs. 0,3878). Důvod je v tom, že C svůj PageRank *nedělí* — má jedinou výstupní hranu, takže celé své (nejvyšší) skóre předá A . **Nerozhoduje tedy počet vstupních hran, ale prestiž jejich zdrojů a jejich výstupní stupeň.** Vrchol B je nejslabší: přichází k němu jen polovina skóre A .

Výhody PageRanku. *Boj proti spamu:* stránka je důležitá, jsou-li důležité stránky na ni ukazující; pro vlastníka stránky není snadné získat vstupní vazby z jiných důležitých stránek, takže PageRank lze těžko ovlivnit. *Globálnost a nezávislost na dotazu:* hodnoty všech stránek se počítají a ukládají *offline*, nikoli v době dotazu. *Kritika:* právě nezávislost na dotazu — PageRank nerozliší stránky autoritativní obecně od stránek autoritativních k tématu dotazu.

	HITS	PageRank
Závislost na dotazu	ano (root set $\rightarrow$ base set)	ne (globální, offline)
Skóre	dvojí (hub, autorita)	jediné
Normalizace	nutná (hodnoty rostou)	není nutná (tok se zachovává)
Malice	$L^T L$ (spolucitovanost), LL^T (bibl. vazba)	A^T s tlumením d
Odolnost vůči spamu	nízká (snadné přidat výstupní vazby)	vyšší
Rychlost v době dotazu	pomalé	rychlé

Oba algoritmy byly publikovány v roce 1998 a jsou si nápadně blízké; PageRank se prosadil jako dominantní model kvůli nezávislosti na dotazu, odolnosti vůči spamu a obchodnímu úspěchu Google. *Důležitost:* hodnocení podle odkazů není jediný algoritmus používaný ve vyhledávačích — kombinuje se s mnoha obsahovými faktory.

10.14 Tematické komunity a jejich objevování

Tematická komunita

Je dána konečná množina entit $S = \{s_1, \dots, s_n\}$ téhož typu. *Tematická komunita* (*themed community*) je dvojice $C = (T, G)$, kde T je *téma* komunity a $G \subseteq S$ množina všech entit v S sdílejících téma T . Je-li $s_i \in G$, je s_i *členem* komunity C .

Vlastnosti: téma definuje komunitu (dvě komunity jsou si rovny, mají-li stejné téma); téma lze definovat libovolně (událost, koncept); entita může být v libovolném počtu komunit — komunity se tedy mohou *překrývat*; entity v S jsou téhož typu (lidé a organizace nemohou být v téže komunitě).

Hierarchie: komunita (T, G) může obsahovat podkomunity $\{(T_1, G_1), \dots, (T_m, G_m)\}$, kde T_i je podtématem T a $G_i \subseteq G$; (T, G) je pak *nadkomunitou*. Rekurzivně tak vzniká hierarchie.

Projev komunity v datech. Členové jsou nějak propojeni a přidružené texty obsahují slova indikující téma. *Webové stránky:* hypertextové odkazy (členové jsou spíše vzájemně propojeni) + obsahová slova. *E-maily:* výměna e-mailů mezi entitami + obsahová slova. *Textové dokumenty:* společný výskyt entit v téže větě či dokumentu + obsahová slova ve větách. Souvisejícím pojmem je *cílené procházení* (*focused crawling*) — kombinace obsahu a odkazů pro procházení stránek určitého tématu: sleduj odkazy a pomocí učení/klasifikace urči, zda je stránka k tématu.

Bipartitní jádra a jejich hledání

(i, j) -*bipartitní jádro* (*bipartite core*) je úplný bipartitní podgraf s alespoň i *fanoušky* (*fans*) a alespoň j *centry* (*centers*), tj. obsahuje všechny možné hrany mezi i fanoušky a j centry. Fanoušci reprezentují specializované huby, centra odpovídají autoritám.

Algoritmus:

- *Krok 1 — prořezávání.* Prořezání podle vstupního stupně: smaž všechny stránky, na které se hodně odkazuje, tj. jejich počet vstupních vazeb je větší než nějaké k (např. $k = 50$). Iterativní prořezávání fanoušků a center (pro nalezení (i, j) -jader): smaž všechny fanoušky s výstupním stupněm menším než j ; smaž všechna centra se vstupním stupněm menším než i ; odstraň z grafu hrany incidentní se smazanými uzly.
- *Krok 2 — generování všech (i, j) -jader.* Najdi všechna $(1, j)$ -jádra, tj. množinu všech vrcholů s výstupním stupněm alespoň j . Sestroj všechna $(2, j)$ -jádra kontrolou každého fanouška, který také ukazuje na nějaké centrum v $(1, j)$ -jádre. Pokračuj stejně až do i .

Omezení: algoritmus najde pouze *jádrové* stránky komunit, nikoli všechny členské stránky, a nenajde ani témata komunit ani jejich hierarchickou strukturu.

Překrývající se komunity pojmenovaných entit

Cíl: najít komunity entit z textového korpusu. **Hlavní myšlenka:** dvě entity jsou propojeny, vyskytnou-li se v téže větě (jsou-li dva lidé zmíněni v téže větě, obvykle mezi nimi existuje vztah); jedna entita se může objevit ve více komunitách; nejčastější klíčová slova reprezentují téma komunity.

Algoritmus:

1. *Postav graf vazeb.* Zpracuj každý dokument a pro každou větu identifikuj obsažené pojmenované entity; má-li věta více než jednu entitu, propoj tyto entity po dvojicích a klíčová slova věty připoj k propojeným dvojicím jako textový obsah; ostatní věty zahod.
2. *Najdi všechny trojúhelníky.* Trojúhelník tvoří tři entity svázané dohromady a představuje základní stavební blok komunit; pozorování: komunita se rozrůstá trojúhelníky sdílejícími společnou hranu.
3. *Najdi jádra komunit.* Jádro je skupina těsně svázaných trojúhelníků, tedy uvolněný úplný podgraf (klika).
4. *Shlukuj kolem jader.* Trojúhelníky a dvojice entit, které nejsou v žádném jádru, se přiřadí k jádrům podle podobnosti jejich textového obsahu.

Metoda dala slibné výsledky při hledání komunit politiků a celebrit z webových dokumentů.

10.15 Detekce komunit

Komunita

Komunitu tvoří jednotlivci takoví, že uvnitř skupiny spolu interagují častěji než s těmi mimo skupinu (v různých kontextech též *skupina*, *klastr*, *kohezní podskupina*, *modul*). *Detekce komunit* je objevování skupin v síti, kde příslušnost jednotlivců ke skupinám není explicitně dána.

Dva typy skupin v sociálních médiích: *explicitní* (vznikají přihlášením uživatelů) a *implicitní* (vznikají sociálními interakcemi). Proč extrahovat skupiny z topologie, když některé weby umožňují se ke skupinám přihlásit? Ne všechny weby komunitní platformu poskytují; ne všichni lidé se chtějí namáhat s přihlašovaním; skupiny se mohou dynamicky měnit. Síťová interakce navíc poskytuje informaci o vztazích mezi uživateli, doplňuje jiné druhy informací a pomáhá vizualizaci i navigaci.

Definice komunity může být subjektivní — tatáž síť může být chápána jako jedna hustě propojená komunita, nebo jako několik komponent.

Dvě hlavní obtíže: (1) počet komunit obvykle není znám a musí být určen; (2) komunity bývají heterogenní co do velikosti a hustoty.

10.15.1 Minimální řez, poměrový a normalizovaný řez

Minimální řez

Řez (cut) je rozdělení vrcholů grafu do dvou disjunktních množin. *Problém minimálního řezu:* najdi rozdělení grafu tak, aby byl minimalizován počet hran mezi oběma množinami (*velikost řezu*, u vážených sítí součet vah). Pro $k > 2$ se používá strategie iterativního půlení.

Nevýhoda: produkuje skupiny nevyvážených velikostí (typicky jednu skupinu tvoří jediný uzel).

Složitost: jednoduché řešení pomocí algoritmu s-t řezu založeného na maximálním toku má $O(n^5)$; implementace Kargerovým algoritmem má $O(n^4)$, ale nezaručuje nalezení minimálního řezu; pro $w_{ij} = 1$ a bez omezení na vyváženost je dvoucestný řez polynomiálně řešitelný (Kargerův randomizovaný minimální řez) v $O(n^2 \log n)$; libovolné váhy hran a omezení na vyváženost činí úlohu NP-těžkou.

Poměrový a normalizovaný řez

Aby se předešlo nevyváženému rozdělení, změníme účelovou funkci tak, aby zohlednila velikost komunity. Nechť C_i je komunita, $|C_i|$ počet jejích uzlů a $\text{vol}(C_i)$ součet stupňů v C_i :

$$\text{RatioCut}(\pi) = \frac{1}{k} \sum_{i=1}^k \frac{\text{cut}(C_i, \bar{C}_i)}{|C_i|}, \quad \text{NCut}(\pi) = \frac{1}{k} \sum_{i=1}^k \frac{\text{cut}(C_i, \bar{C}_i)}{\text{vol}(C_i)}.$$

Obě míry preferují **vyvážené** rozdělení.

10.15.2 Girvan–Newman

Algoritmus Girvan–Newman (2002)

Divizivní hierarchický algoritmus shora dolů: rozkládá počáteční úplný graf na postupně menší souvislé kusy, dokud nezbudou hrany k odstranění a každý uzel netvoří vlastní komunitu.

Myšlenka: identifikuj hrany spojující vrcholy patřící do různých komunit (*mosty*) a iterativně je odstraňuj. **Kritérium:** *mezilehlost hrany* (*edge betweenness*) — podíl nejkratších cest, které procházejí danou hranou — protože takové hrany leží na velkém počtu nejkratších cest mezi vrcholy.

- 1: **repeat**
- 2: spočti mezilehlost *všech* hran v síti
- 3: odstraň hranu s nejvyšší mezilehlostí
- 4: přepočti komponenty souvislosti (aktuální rozklad na komunity) a jeho modularitu Q
- 5: **until** v grafu nezbude žádná hrana
- 6: **return** rozklad s *maximální* modularitou Q

Krok 1 je nutné opakovat po *každém* odstranění — mezilehlosti se odstraněním hrany mění.

Výstup: hierarchická struktura reprezentovatelná dendrogramem. **Volba počtu komunit:** algoritmus vrací množinu možných řešení, ale neříká, které je nejlepší — pro každý rozklad spočti *modularitu* a vyber rozklad s nejvyšší.

Nevýhoda: vhodný jen pro sítě střední velikosti kvůli vysokým výpočetním nákladům — $O(m^2n)$ pro graf o m hranách a n vrcholech, resp. $O(n^3)$ pro řídké sítě.

Modularita

Modularita Q kvantifikuje kvalitu daného rozdělení grafu do komunit. **Základní myšlenka:** síť má smysluplnou komunitní strukturu, je-li počet hran *mezi* komunitami menší, než by se očekávalo při náhodné volbě.

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j) = \sum_c \left[\frac{l_c}{m} - \left(\frac{d_c}{2m} \right)^2 \right],$$

kde m je počet hran, A_{ij} prvek matice sousednosti (počet hran mezi i a j), k_i stupeň vrcholu i , c_i skupina vrcholu i , δ Kroneckerovo delta, l_c počet hran uvnitř komunity c a d_c součet stupňů v c . Člen $\frac{k_i k_j}{2m}$ je *očekávaný* počet hran mezi i a j (nulový model se zachovaným rozdělením stupňů). *Příklad očekávaného počtu hran:* pro uzly se stupni 3 a 2 v síti se 14 hranami je očekávaný počet hran mezi nimi $\frac{3 \cdot 2}{2 \cdot 14} = \frac{3}{14}$.

Hodnoty: $Q \in \langle -1, 1 \rangle$. Je-li kladná, existuje možnost nalezení komunitní struktury; jsou-li hodnoty kladné a velké, může rozklad odrážet skutečnou komunitní strukturu. Pro smysluplné komunity by mělo být $Q \geq 0,3$ (Clauset et al., 2004).

Omezení rozlišení (*resolution limit*): maximalizace modularity nedokáže najít komunity menší než $\sqrt{2m}$ hran — malé komunity se slučují. Řešení: parametr rozlišení γ ve členu $\gamma \frac{k_i k_j}{2m}$ nebo kritérium *map equation* (Infomap).

Příklad — krok algoritmu Girvan–Newman

Použijeme týž graf dvou trojúhelníků: hrany $AB, AC, BC, CD, DE, DF, EF$; $m = 7$.

Krok 1 — mezilehlosti hran: $C_B(CD) = 9$, $C_B(AC) = C_B(BC) = C_B(DE) = C_B(DF) = 4$, $C_B(AB) = C_B(EF) = 1$.

Krok 2 — odstraněni hranu s nejvyšší mezilehlostí: odstraníme most CD .

Krok 3 — komponenty: graf se rozpadl na $\{A, B, C\}$ a $\{D, E, F\}$.

Krok 4 — modularita. Komunita 1: vnitřních hran $l_1 = 3$ (AB, AC, BC), součet stupňů (v původním grafu) $d_1 = 2 + 2 + 3 = 7$; $2m = 14$:

$$Q_1 = \frac{3}{7} - \left(\frac{7}{14}\right)^2 = 0,4286 - 0,25 = 0,1786.$$

Komunita 2 je symetrická, $Q_2 = 0,1786$. Celkem $Q = 0,3571$.

Další odstraňování hran (např. AC) rozdělí trojúhelníky a modularita klesne — rozklad na dvě komunity je proto výsledkem algoritmu.

10.15.3 Louvain

Algoritmus Louvain (Blondel et al., 2008)

Hladová optimalizační metoda pro *aglomerativní hierarchickou* maximalizaci modularity. Dvě opakující se fáze:

Fáze 1 (lokální optimalizace). Považuj každý uzel za samostatnou komunitu.

1.1 Spočti modularitu Q .

1.2 Přesuň izolovaný uzel z jeho komunity do sousední komunity.

1.3 Spočti zisk/ztrátu modularity plynoucí ze změněného přiřazení.

1.4 Zvýší-li se modularita (je-li zisk), ponech uzel v „nové“ komunitě.

1.5 Opakuj, dokud nejsou možná další zlepšení.

Výstup: rozklad k -té úrovně (k je číslo iterace). Zisk modularity při přesunu uzlu i do komunity:

$$\Delta Q = \left[\frac{\Sigma_{in} + 2k_{i,in}}{2m} - \left(\frac{\Sigma_{tot} + k_i}{2m} \right)^2 \right] - \left[\frac{\Sigma_{in}}{2m} - \left(\frac{\Sigma_{tot}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right].$$

Fáze 2 (agregace). Vytvoř novou síť (*supergraf*) odvozenou z původní, kde každý nový uzel (*supervrchol*) je agregací uzlů přiřazených dané komunitě nalezené ve fázi 1; dva supervrcholy jsou spojeny hranou, existuje-li alespoň jedna hrana spojující dva vrcholy uvnitř odpovídajících komunit.

Iterace: opakuj fáze 1 a 2, dokud není dosaženo maxima modularity.

Výhody: dobrý výkon i na velkých grafech při nízkých výpočetních nákladech — $O(n \log n)$.

Nevýhoda: výsledky jsou citlivé na pořadí zpracování uzlů. Novější algoritmus **Leiden** (Traag et al., 2019) navíc garantuje dobře propojené komunity.

10.15.4 Taxonomie metod detekce komunit

Metody lze zhruba rozdělit do čtyř kategorií:

Uzlově orientované metody (node-centric)

Každý uzel ve skupině musí splňovat určité vlastnosti.

- **Úplná vzájemnost — kliky.** *Klika* je maximální úplný podgraf o třech a více uzlech, v němž jsou všechny uzly navzájem sousední. Nalezení maximální kliky je NP-těžké. *Myšlenka rekursivního prořezávání:* pro nalezení kliky velikosti k odstraň uzly se stupněm menším než $k - 1$. Postup: navzorkuj podsít, najdi v ní kliku (např. hladově) velikosti k ; abys našel větší kliku, odstraň všechny uzly se stupněm $\leq k - 1$; opakuj, dokud není síť dost malá. V sítích s mocným rozdělením stupňů se prořeže mnoho uzlů. Kliky se obvykle používají jako *jádro* či *semínko* pro průzkum větších komunit.
- **Dosažitelnost.** *k-klika:* maximální podgraf, kde největší geodetická vzdálenost mezi libovolnými uzly je $\leq k$ (cesta může vést i mimo podgraf, takže *k-klika* může mít *uvnitř* podgrafu průměr $\geq k$). *k-klub:* podstruktura s průměrem $\leq k$. *k-klan:* *k-klika* s průměrem $\leq k$. *Příklad:* kliky $\{1, 2, 3\}$; 2-kliky $\{1, 2, 3, 4, 5\}$, $\{2, 3, 4, 5, 6\}$; 2-kluby $\{1, 2, 3, 4\}$, $\{1, 2, 3, 5\}$, $\{2, 3, 4, 5, 6\}$.
- **Stupně uzlů.** *k-jádro* (*k-core*): podstruktura, kde je každý uzel spojen s alespoň k dalšími členy skupiny. *k-plex:* pro skupinu o n_s uzlech musí být každý uzel sousední s nejméně $n_s - k$ uzly ve skupině. Definice jsou komplementární (*k-jádro* je $(n_s - k)$ -plex), ale množina všech *k-jader* pro dané k není identická s množinou všech $(n_s - k)$ -plexů, protože velikost skupiny n_s se může lišit.
- **Relativní četnost vazeb uvnitř a vně.** *LS množiny:* každá vlastní podmnožina LS množiny má více vazeb k ostatním uzlům ve skupině než mimo ni (původně pro návrh obvodů na čipech). Příliš striktní; uvolněnou definicí je *k-komponenta* vyžadující výpočet hranové souvislosti mezi libovolnou dvojicí uzlů algoritmem minimálního řezu / maximálního toku.

Omezení: definice jsou příliš striktní (lze je použít jako jádro komunity), neškálují (používají se pro malé sítě) a někdy nejsou konzistentní s vlastnostmi velkých sítí (např. stupně uzlů v bezškálových sítích).

Metoda perkolace klik (CPM)

Klika je velmi striktní a nestabilní definice, proto se používá jako jádro pro nalezení větších komunit. *Clique Percolation Method* je příkladem metody hledající **překrývající se** komunity.

- *Vstup:* parametr k a síť.
- *Postup:* najdi všechny kliky velikosti k ; sestroj *graf klik*, kde jsou dvě kliky sousední, sdílejí-li $k - 1$ uzlů; každá komponenta souvislosti grafu klik tvoří jednu komunitu.

Příklad ($k = 3$): kliky velikosti 3 jsou $\{1, 2, 3\}$, $\{1, 3, 4\}$, $\{4, 5, 6\}$, $\{5, 6, 7\}$, $\{5, 6, 8\}$, $\{5, 7, 8\}$, $\{6, 7, 8\}$; výsledné komunity jsou $\{1, 2, 3, 4\}$ a $\{4, 5, 6, 7, 8\}$ — uzel 4 patří do *obou*.

Skupinově orientované metody (group-centric)

Kritéria vyžadují, aby určitou podmínku splňovala *celá skupina* bez zaostření na úroveň uzlu — např. dostatečně velkou hustotu skupiny. Podgraf $G_S = (V_S, E_S)$ je γ -*hustá kvazi-klika*, platí-li

$$|E_S| \geq \gamma \cdot \frac{|V_S|(|V_S| - 1)}{2}.$$

Použije se strategie podobná klikám: *lokální prohledávání* (navzorkuj podgraf a najdi maximální γ -hustou kvazi-kliku velikosti k) a *heuristické prořezávání* (odstraň uzly se stupněm menším než průměrný stupeň, tj. $< (k - 1)\gamma$).

Uvažují propojení v síti *globálně*; cílem je rozdělit uzly do disjunktních množin. Přístupy:

- **Shlukování podle podobnosti vrcholů.** Podobnost je definována podobností jejich okolí. *Strukturní ekvivalence*: dva uzly jsou strukturně ekvivalentní, právě když se připojují ke stejné množině aktérů — pro praktické použití příliš restriktivní. Používají se proto

$$\text{Jaccard}(v_i, v_j) = \frac{|N_i \cap N_j|}{|N_i \cup N_j|}, \quad \cos(v_i, v_j) = \frac{\sum_k A_{ik} A_{jk}}{\sqrt{\sum_s A_{is}^2} \cdot \sqrt{\sum_t A_{jt}^2}},$$

načež se nad vrcholy spustí klasický algoritmus k -means (vazby se chápou jako příznaky).

- **Modely latentního prostoru.** Zobrazí uzly do nízkorozměrného prostoru tak, aby byla zachována blízkost daná propojením, a pak aplikují k -means. *Vícerozměrné škálování* (MDS): sestroj matici blízkosti P reprezentující párové vzdálenosti mezi uzly (např. geodetické) a najdi souřadnice S v nízkorozměrném prostoru minimalizující zkreslení; řešení je dáno prvními l vlastními vektory a diagonální maticí největších vlastních čísel.
- **Blokové modely.** $A \approx S\Sigma S^T$, kde Σ je *míchací matice* a S *indikátorová matice komunit*; uvolníme-li S na numerické hodnoty, odpovídá optimální řešení hlavním vlastním vektorům A .
- **Spektrální shlukování.** Poměrový i normalizovaný řez lze přeformulovat jako minimalizaci $\text{tr}(S^T X S)$, kde $X = L = D - A$ je grafový Laplacián (pro poměrový řez), resp. $X = D^{-1/2} L D^{-1/2}$ normalizovaný Laplacián (D je diagonální matice stupňů). *Spektrální relaxace*: optimálním řešením jsou vlastní vektory s nejmenšími vlastními čísly. První vlastní vektor říká, že všechny uzly patří do téhož klastru, a je tedy nepoužitelný; *Fiedlerův vektor* (druhý nejmenší) dává dobré dvojrozdělení podle znamének složek a příslušné vlastní číslo (*algebraická souvislost*) měří, jak silně je graf propojen. Násobnost vlastního čísla 0 je rovna počtu komponent souvislosti.
- **Maximalizace modularity.** *Modularitní matice* $B_{ij} = A_{ij} - \frac{d_i d_j}{2m}$; optimálním řešením relaxované úlohy jsou hlavní vlastní vektory B , načež se jako post-processing aplikuje k -means.

Jednotný pohled. Modely latentního prostoru, blokové modely, spektrální shlukování i maximalizace modularity lze zapsat jako úlohu *maticové faktorizace* $\min \|X - SS^T\|_F^2$ (resp. optimalizaci stopy), kde se liší jen volbou X :

X	Metoda
$-\frac{1}{2}(P^2 - \dots)$, tj. dvojitě centrovaná matice vzdáleností	modely latentního prostoru
sociomatrice A	aproximace blokovým modelem
grafový Laplacián L	minimalizace řezu (spektrální shlukování)
modularitní matice B	maximalizace modularity

Omezení: všechny vyžadují, aby uživatel předem zadal počet komunit.

Hierarchicky orientované metody (hierarchy-centric)

Cílem je vybudovat hierarchickou strukturu komunit na základě topologie sítě, což umožňuje analýzu sítě v různých rozlišeních.

- **Divizivní hierarchické shlukování** — rozděl uzly do několika množin a každou dále dělej na menší; lze použít síťově orientované rozdělení. Konkrétní příklad: rekurzivně odstraňuj „nejslabší“ vazbu — najdi hranu s nejmenší silou, odstraň ji a aktualizuj sílu ostatních hran; opakuj, dokud se síť nerozloží na požadovaný počet komponent. Sílu vazby měříme *mezilehlostí hrany*. Reprezentant: Girvan–Newman, $O(m^2n)$.
- **Aglomerativní hierarchické shlukování** — inicializuj každý uzel jako komunitu a postupně slučuj komunity do větších podle určitého kritéria (např. podle nárůstu modularity). Reprezentant: Louvain, $O(n \log n)$.

Příklad mezilehlosti hrany: mezilehlost hrany $e(1, 2)$ je 4, protože všechny nejkratší cesty z uzlu 2 do $\{4, 5, 6, 7, 8, 9\}$ musí projít buď $e(1, 2)$, nebo $e(2, 3)$, a $e(1, 2)$ je zároveň nejkratší cestou mezi 1 a 2.

Kategorie	Reprezentanti	Složitost	Poznámka
Uzlově orientované	klika, k -klika, k -klub, k -klan, k -jádro, k -plex, LS	vysoká (NP-těžké)	příliš striktní, malé sítě
Skupinově orientované	γ -husté kvazi-kliky	vysoká	uvolnění kliky
Síťově orientované	podobnost vrcholů, MDS, blokové modely, spektrální, maximalizace Q	$O(n^3)$ i více	nutné zadat počet komunit
Hierarchicky orientované	Girvan–Newman (divizivní)	$O(m^2n)$	interpretovatelný dendrogram
Překrývající se	Louvain (aglomerativní) perkolace klik (CPM)	$O(n \log n)$ vysoká	standard pro velké sítě uzel může být ve více komunitách
Ostatní	Walktrap (krátké náhodné procházky), simulované žihání, blokové modelování	—	

10.16 Predikce vazeb

Úloha predikce vazeb

Cíl: předpovědět budoucí vazby mezi dvojicemi uzlů v síti. V komerčních sociálních sítích (Facebook, LinkedIn) jsou uživatelé často doporučováni jako potenciální přátelé. Použít lze jak podobnost obsahu, tak strukturu:

- **Obsahové míry** — využívají princip *homofilie*: uzly s podobným obsahem se spíše propojí. Zlepšují predikci, ale jsou citlivé na konkrétní síť (např. krátké a zašuměné tweety s nestandardními zkratkami). *Obsahová homofilie však nemusí implikovat strukturní propojenost!*
- **Strukturní míry** — využívají princip *triadického uzávěru*: dva uzly sdílející okolí se spíše propojí. Triadický uzávěr je velmi častý napříč doménami, míry jsou účinné v různých typech sítí a přímo použitelné. Patří sem míry založené na sousedství, Katzova míra a míry založené na náhodných procházkách. *Strukturní propojenost obvykle implikuje i obsahovou homofilii.*

Míry založené na sousedství

Nechť S_i je množina sousedů uzlu i .

Míra společných sousedů:

$$\text{CommonNeighbors}(i, j) = |S_i \cap S_j|.$$

Slabina: nezohledňuje *relativní* počet společných sousedů vůči počtu ostatních vazeb — byli-li by oba uzly velmi populární veřejné osobnosti spojené s velkým počtem aktérů, mohli by mít mnoho společných sousedů *jen náhodou*.

Jaccardova míra — měří podobnost okolí, normalizováno vůči stupňům, které se kvůli mocninnému rozdělení silně liší:

$$\text{JaccardPredict}(i, j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}.$$

Slabina: nemusí se dobře přizpůsobit *prostředním* sousedům — jsou-li společní sousedé velmi populární (mají vysoký stupeň), objevují se jako společní sousedé i u mnoha jiných dvojic, a jsou tedy pro predikci méně důležití.

Adamicova–Adarova míra — vážená verze míry společných sousedů; váha společného souseda je klesající funkcí jeho stupně, např. $1/\log |S_k|$:

$$\text{AdamicAdar}(i, j) = \sum_{k \in S_i \cap S_j} \frac{1}{\log |S_k|}.$$

Příklad: mají-li Alice a Bob čtyři společné sousedy se stupni 4, 2, 2 a 4, je

$$\frac{1}{\log 4} + \frac{1}{\log 2} + \frac{1}{\log 2} + \frac{1}{\log 4} = \frac{3}{\log 2},$$

zatímco Jaccardova míra je v téže situaci $4/9$.

Preferenční připojování: $|S_i| \cdot |S_j|$.

Katzova míra

Míry založené na sousedství nejsou účinné, je-li počet společných sousedů dvojice malý. *Příklad:* Alice a Bob mají jednoho společného souseda, Alice a Jim rovněž jednoho — míry založené na sousedství tyto případy nerozliší, ačkoli existuje nepřímá propojenost delšími cestami. V takových případech jsou vhodnější míry založené na *procházkách*.

Nechť n_{ij}^t je počet procházek délky t mezi uzly i a j a $\beta < 1$ předem zvolený parametr. *Katzova míra* je

$$\text{Katz}(i, j) = \sum_{t=1}^{\infty} \beta^t \cdot n_{ij}^t,$$

kde β je *diskontní faktor* potlačující delší procházky; pro malá β nekonečná suma konverguje. Je-li A symetrická matice sousednosti neorientované sítě, lze celou matici Katzových koeficientů K spočítat jako

$$K = \sum_{i=1}^{\infty} \beta^i A^i = (I - \beta A)^{-1} - I.$$

Řada konverguje právě tehdy, když $\beta < 1/\lambda_{\max}(A)$: vlastní čísla matice A^i jsou totiž i -té mocniny vlastních čísel A , takže členy $\beta^i A^i$ klesají geometricky jen při $\beta\lambda_{\max} < 1$. Vysoce centrální uzly mají sklon tvořit vazby s mnoha uzly — součet Katzových koeficientů uzlu i vůči ostatním uzlům se nazývá jeho *Katzova centralita*.

Příklad. Nechť Alice (A) má sousedy x, y, p ; Bob (B) má sousedy x, q ; Jim (J) má jediného souseda y ; a nechť je navíc hrana $p-q$. Pak

$$\text{CommonNeighbors}(A, B) = |\{x\}| = 1 = |\{y\}| = \text{CommonNeighbors}(A, J),$$

takže míry založené na sousedství obě dvojice *nerozliší*. Procházky ale ano: mezi A a B vedou dvě — AxB (délka 2) a $ApqB$ (délka 3) — zatímco mezi A a J jen jedna (AyJ). Pro $\beta = 0,1$ tedy (až na členy řádu β^4)

$$\text{Katz}(A, B) \approx \beta^2 \cdot 1 + \beta^3 \cdot 1 = 0,011, \quad \text{Katz}(A, J) \approx \beta^2 \cdot 1 = 0,010,$$

a vazba mezi Alicí a Bobem je předpovězena jako pravděpodobnější.

Predikce vazeb jako klasifikační úloha

Přítomnost či nepřítomnost vazby mezi dvojicí uzlů se chápe jako binární indikátor třídy. Pro každou dvojici uzlů sestrojíme vícerozměrný datový záznam:

- příznaky zahrnují všechny podobnosti mezi uzly založené na sousedství, Katzově míře či procházkách;
- navíc se používají příznaky preferenčního připojování, například stupně obou uzlů dvojice;
- výsledkem je *pozitivně-neoznačovaná* (*positive-unlabeled*) klasifikační úloha: dvojice s hranou jsou pozitivní příklady, zbývající dvojice jsou neoznačované;
- neoznačované příklady lze při trénování přibližně považovat za negativní;
- protože je ve velkých řídkých sítích negativních dvojic příliš mnoho, používá se jen jejich vzorek.

Trénovací fáze: vygeneruj data s jedním záznamem pro každou dvojici uzlů s hranou a se vzorkem záznamů z dvojic bez hrany; příznaky odpovídají extrahovaným podobnostním a strukturálním rysům; třída je přítomnost/nepřítomnost hrany; postav a natrénuj klasifikátor.

Testovací fáze: převed' každou testovanou dvojici na vícerozměrný záznam a predikuj natrénovaným klasifikátorem. Běžnou volbou bazového klasifikátoru je *logistická regrese*; kvůli nevyváženosti dat se často používají *cenově citlivé* verze klasifikátorů (viz kap. 6).

Výhody supervizovaného přístupu: snadno lze extrahovat a použít obsahové příznaky; klasifikátor se automaticky naučí relevanci příznaků; u orientovaných sítí lze příznaky extrahovat *asymetricky* (vstupní a výstupní stupně místo prostých stupňů, personalizovaný PageRank); vyšší flexibilita díky schopnosti učit se vztahy mezi vazbami a příznaky různých typů.

Shrnutí predikce vazeb. Různé míry se liší účinností na různých datech: míry založené na sousedství lze počítat efektivně i pro velmi velká data a fungují téměř stejně dobře jako ostatní nesupervizované míry; míry založené na náhodných procházkách a Katzově míře jsou užitečné pro velmi řídké sítě, kde nelze robustně měřit počet společných sousedů; supervize poskytuje lepší přesnost a adaptabilitu napříč doménami, ale je výpočetně nákladná; v posledních letech se k vylepšení používá i obsah, strukturální míry jsou však mnohem silnější (kvůli triadickým vlastnostem reálných sítí). Moderní přístupy: maticová faktorizace, *vnoření grafu* (*graph embedding* — node2vec, DeepWalk) a grafové neuronové sítě (GCN, GraphSAGE). **Vyhodnocení:** skryj část existujících hran, na zbytku uč a měř AUC nebo precision@*k* na skrytých hranách; pozor na extrémní nevyváženost ($O(n^2)$ možných vs. $O(n)$ existujících hran) — accuracy je zde nepoužitelná.

10.17 Shrnutí k zkoušce

- **SNA** studuje *vztahy*, ne entity; průsečík společenských věd, síťové analýzy a teorie grafů. Reprezentace: seznam hran, matice sousednosti (u orientovaných nesymetrická), seznam sousedů; *ego sítě* a problém vymezení hranic.
- **Síla vazeb:** Granovetterova síla slabých vazeb; homofilie / tranzitivita / mosty; *most* (odstranění rozpojí), *zkratkový most* (vzdálenost vzroste nad 2), *překryv sousedství* $\frac{|N_i \cap N_j|}{|N_i \cup N_j| - 2}$.
- **Centrality:** stupňová, blízkostní $\sum_u \frac{n-1}{d(v,u)}$, mezilehlostní $\sum \frac{\sigma_{st}(v)}{\sigma_{st}}$, vlastní ($\lambda \mathbf{x} = A\mathbf{x}$, jen *největší* vlastní číslo dá nezáporný vektor), Katzova, PageRank. Umět interpretovat (kolik lidí oslovím přímo / kdo je na cestě / jak rychle dosáhnu všech / kdo je spojen s dobře spojenými) a spočítat na malém grafu.
- **Koheze:** reciprocita (jen orientované), hustota ($\binom{n}{2}$ vs. $n(n-1)$), shlukovací koeficient $C(i) = \frac{2|e(j,k)|}{k(i)(k(i)-1)}$, průměr, excentricita, *malý svět* = vysoké C + krátké ℓ .
- **Šest vlastností reálných sítí:** malý svět, tranzitivita/shlukování, mocninné rozdělení stupňů, odolnost, vzorce míšení (asortativita), komunitní struktura. **Preferenční připojování** $\Pi(k_i) = k_i / \sum_j k_j$; důvody: popularita / kvalita / smíšený model.
- **Modelování vlivu:** *LTM* (práh θ_v z $U(0,1)$, aktivace při $\sum b_{w,v} \geq \theta_v$, orientace na příjemce) vs. *ICM* (jedna šance, pravděpodobnost $p_{w,v}$ na hraně, orientace na odesílatele). **Maximalizace vlivu** $\max_{|B| \leq k} \sigma(B)$ je NP-těžká, hladový algoritmus dá 63 % optima.

- **Vliv vs. korelace:** tři procesy — homofilie (atributy $\rightarrow$ vazby), confounding (prostředí), vliv (vazby $\rightarrow$ atributy). *Test korelace* porovná podíl heterogenních hran s $2p(1-p)$. *Logistický model* $\ln \frac{p(a)}{1-p(a)} = \alpha \ln(a+1) + \beta$, α je koeficient sociální korelace. *Shuffle test*: zpřeházej časová razítka; liší-li se α' od α , existuje vliv.
- **Test homofilie:** heterogenních hran významně méně než $2pq \Rightarrow$ homofilie; významně více $\Rightarrow$ inverzní homofilie. Umět příklad (5 z 18 vs. očekávaných 8).
- **Výběr vs. sociální vliv;** měnitelné a neměnitelné charakteristiky; longitudinální studie (obezita — Christakis a Fowler).
- **Afiliční sítě:** bipartitní, lidé + ohniska; tři uzávěry — *triadický* (tranzitivita), *ohniskový* (výběr), *členský* (sociální vliv); $T(k)$ a nulový model $1 - (1-p)^k$; empiricky velký skok z 1 na 2 a klesající výnosy.
- **Schellingův model:** lokální homofilie (práh t) $\rightarrow$ globální segregace; neměnitelné charakteristiky se stanou korelovanými s měnitelnými.
- **Strukturní balance:** v každém trojúhelníku jsou tři + nebo právě jedna +. *Věta o bilanci*: vyvážený úplný graf $\Rightarrow$ buď samí přátelé, nebo dvě skupiny X, Y . *Slabá balance*: zakázán jen typ $+, +, - \Rightarrow$ rozklad na N skupin. Aplikace: Evropa 1872–1907, Bangladéš 1972.
- **HITS:** root set $\rightarrow$ base set; $\mathbf{a} = L^T \mathbf{h}$, $\mathbf{h} = L \mathbf{a}$; iterace $\mathbf{a}_k = L^T L \mathbf{a}_{k-1}$, $\mathbf{h}_k = L L^T \mathbf{h}_{k-1}$ + normalizace; $\mathbf{h}^*$ je vlastní vektor MM^T ; $L^T L$ = spolucitovanost, LL^T = bibliografická vazba; slabiny: spamovatelnost, tematický drift, pomalost.
- **PageRank:** prestiž podle hodnoty $\mathbf{P} = A^T \mathbf{P}$; Markovův řetězec vyžaduje stochastickou, *irreducibilní* a *aperiodickou* matici — web ji nesplňuje (dangling nodes, není silně souvislý); řešení: tlumicí faktor $d = 0,85$ a hrany na všechny uzly; $P(i) = (1-d) + d \sum_{(j,i) \in E} \frac{P(j)}{O_j}$; mocninná iterace. Umět projít iteraci.
- **Tematické komunity:** (T, G) , hierarchie, překryv; (i, j) -*bipartitní jádra* (fanoušci = huby, centra = authority) + prořezávání podle stupňů; komunity pojmenovaných entit přes trojúhelníky.
- **Detekce komunit:** minimální řez (nevyvážený) $\rightarrow$ *ratio cut* a *normalized cut*; **modularita** $Q = \frac{1}{2m} \sum_{i,j} (A_{ij} - \frac{k_i k_j}{2m}) \delta(c_i, c_j)$, smysluplné $Q \geq 0,3$, omezení rozlišení; **Girvan–Newman** $O(m^2 n)$ (mezilehlost hran, vyber max. Q); **Louvain** $O(n \log n)$ (lokální ΔQ + agregace, citlivý na pořadí); taxonomie *uzlově* / *skupinově* / *síťově* / *hierarchicky* orientovaných metod; **CPM** pro překrývající se komunity.
- **Predikce vazeb:** obsahové (homofilie) vs. strukturní (triadický uzávěr) míry; společní sousedé, Jaccard, Adamic–Adar ($1/\log |S_k|$), preferenční připojování, **Katzova míra** $\sum_t \beta^t n_{ij}^t$, maticově $(I - \beta A)^{-1} - I$; jako klasifikace — positive-unlabeled, vzorkování negativ, logistická regrese, cenově citlivé varianty.

11 Praktické využití technik

11.1 Typické aplikační oblasti

Přednáška uvádí následující úlohy dobývání znalostí a jejich aplikace:

- segmentace a klasifikace klientů banky (např. detekce problémových nebo bonitních klientů);
- analýza příčin poruch v telekomunikačních sítích;
- analýza příčin změny poskytovatele služeb (internet, mobilní telefony);
- predikce spotřeby elektrické energie;
- predikce vývoje cen akcií;
- segmentace a klasifikace zákazníků pojišťovny;
- určení příčin závad automobilů;
- analýza databáze pacientů v nemocnici;
- analýza nákupního košíku.

Historické příklady statistického dobývání znalostí z dat: Florence Nightingaleová analyzovala zdravotní stav britské armády (krymská válka, Indie) a Ignaz Semmelweis analyzoval úmrtnost na horečku omladnic ve Vídni, přičemž odhalil kontaminaci z pitev jako příčinu.

11.2 Analýza nákupního košíku

Úloha: které zboží se kupuje společně. **Metody:** asociační pravidla (Apriori, FP-Growth), sekvenční vzory, korelační analýza. **Data:** charakteristiky zákazníků a znalost o jejich nákupech (co koupili, kdy); data se předzpracují do relační tabulky. **Otázky:** Existují produkty kupované současně? Co charakterizuje naše zákazníky (nízký příjem, ...)?

Reálné nasazení: Walmart (2,5 milionu zaměstnanců), Target Corporation; v ČR Delvita, Meinl, drogerie.

Akce odvozené z výsledků: plánování a uspořádání prodejny (komplementární zboží daleko od sebe, aby zákazník prošel celou prodejnou; nebo naopak blízko pro pohodlí), cílené kupony a časově omezené slevy, balíčkování produktů, doporučení v e-shopu („zákazníci, kteří koupili X, koupili také Y“), řízení skladu.

Úskalí: obrovské množství generovaných pravidel (nutná taxonomie, virtuální položky, prořezávání, míry zajímavosti); triviální pravidla („kdo koupí rohlík, koupí i housku“); zdánlivá pravidla plynoucí ze společné příčiny (obojí je v akci).

11.3 Kreditní skórování

Úloha: odhad pravděpodobnosti, že žadatel splatí úvěr. **Metody:** logistická regrese (dominuje kvůli interpretovatelnosti a regulatorním požadavkům), rozhodovací stromy, gradient boosting, scorecard modely (WOE/IV transformace).

Cílová proměnná: „default“ definovaný obvykle jako 90 dní po splatnosti během 12 měsíců. **Atributy:** příjem, závazky, historie splácení (úvěrový registr), délka zaměstnání, věk, produktová historie.

Vyhodnocení: Giniho koeficient = $2 \cdot \text{AUC} - 1$ (bankovní standard), Kolmogorovova–Smirnovova statistika, lift, stabilita v čase (*Population Stability Index*).

Etické a právní aspekty: zákaz diskriminace podle chráněných atributů (pohlaví, etnicita) — pozor na *zástupné proměnné (proxy)*, např. PSČ jako proxy etnicity; povinnost vysvětlit zamítnutí (GDPR, čl. 22). Ilustrativní je příklad z přednášky: banka Barclays plánovala směřovat zákazníky s nízkou bonitou do indických call center a movitější do britských — systém byl kritizován jako „apartheid“ a ukázal, že technicky správná segmentace může být společensky nepřijatelná.

11.4 Detekce podvodů

Úloha: identifikovat podvodné transakce, pojistné události nebo zdravotní výkony. **Charakter dat:** extrémně nevyvážené třídy (0,1–1 % pozitivních), měnící se chování protivníka (*adversarial*), vysoká cena FN i FP.

Metody: klasifikace s cenově citlivým učením a převzorkováním (SMOTE); detekce odlehlých hodnot bez učitele (izolační les, LOF, jednotřídní SVM, autoenkodéry) — podvodné vzorce se totiž mění a označovaná data zastarávají; analýza sítí (kap. 10) pro odhalení organizovaných skupin (společné adresy, telefony, IP — komunity v grafu entit); pravidlové systémy pro rychlé blokování.

Vyhodnocení: nikoli accuracy, ale $\text{precision}@k$ (kolik z k nejpodezřelejších případů je skutečně podvod), AUPRC, úspora nákladů. Model typicky nerozhoduje sám, ale řadí případy k ruční kontrole — viz úvodní příklad se zdravotní pojišťovnou: stroj vybral z 20 milionů výkonů 14 tisíc nejpodezřelejších, z nichž revizní lékaři 8 tisíc ihned potvrdili jako podvodné.

11.5 Segmentace zákazníků a churn

Segmentace: shlukování zákazníků do skupin s podobným chováním (kap. 7). Klasickým vstupem je **RFM analýza** — *recency* (jak dávno naposledy nakoupil), *frequency* (jak často), *monetary* (za kolik). Nalezené skupiny lze interpretovat jako segmenty v daném tržním prostoru a nabídnout jim speciální služby. Segmenty se popíše charakteristickými pravidly (kap. 8) a zvizualizují (kap. 9).

Predikce odchodu (*churn prediction*): klasifikace, zda zákazník do X měsíců odejde. Použití v telekomunikacích, bankovníctví, předplatných službách. Klíčové je, že model musí predikovat *dostatečně dopředu*, aby zbyl čas na retenční akci, a že se hodnotí ekonomicky: náklad na retenční nabídku vs. celoživotní hodnota zákazníka (viz lift analýza, kap. 6). Souvisí i analýza příčin změny poskytovatele služeb.

11.6 Doporučovací systémy

Přístup	Princip a vlastnosti
Obsahové (<i>content-based</i>)	doporučuje položky podobné těm, které uživatel hodnotil kladně; vyžaduje popis položek; trpí <i>přeurčeností</i> (doporučuje stále totéž)
Kolaborativní filtrování — uživatelské	najdi uživatele s podobným profilem hodnocení a doporuč, co se líbilo jim (k -NN nad maticí hodnocení)
Kolaborativní filtrování — položkové	najdi položky, které se hodnotí podobně jako ty, jež uživatel má rád; stabilnější a lépe škáluje (Amazon)
Maticová faktorizace	rozlož řídkou maticí hodnocení $R \approx UV^T$ na latentní faktory (SVD, ALS); vítěz soutěže Netflix Prize
Asociační pravidla	„kdo koupil X, koupil i Y“ — přímá aplikace MBA
Hybridní	kombinace výše uvedených; standard v praxi

Typické problémy: *studený start* (*cold start*) — nový uživatel či položka bez historie; *řídkost* matice hodnocení; *popularity bias*; *bublina* (*filter bubble*) a zpětnovazební smyčka (model ovlivňuje data, na nichž se učí další verze); vyhodnocení offline (RMSE, $\text{precision}@k$, NDCG) vs. online (A/B test, konverze) — offline metriky s online výsledky často nekorelují.

11.7 Webové a textové dolování

- Dolování obsahu webu** (*web content mining*) — klasifikace a shlukování stránek, extrakce informací, analýza sentimentu. Text se reprezentuje vektorovým modelem (TF-IDF) nebo vnořeními; nejlepším klasifikátorem klasicky bývá SVM (viz kap. 5), dnes transformerové modely.
- Dolování struktury webu** (*web structure mining*) — odkazová struktura jako graf: PageRank, HITS (*huby* a *authority*), detekce spamu (*link farms*) — přímá aplikace kap. 10.

- **Dolování používání webu** (*web usage mining*) — analýza logů: sekvenční vzory navigace uživatelů (kap. 4), segmentace návštěvníků, optimalizace struktury webu, personalizace.

11.8 Praktické využití analýzy sociálních sítí

Aplikace SNA (kap. 10) sledují společný vzorec: praktická otázka se přeloží na síťovou úlohu a tu řeší konkrétní míra či algoritmus.

Úloha	Míra / algoritmus	Co se z ní čte
Virální marketing, seeding kampaně	maximalizace vlivu, LTM/ICM, hladový algoritmus	koho oslovit, aby kaskáda zasáhla co nejvíce uživatelů
Doporučení přátel („People you may know“)	predikce vazeb: společní sousedé, Adamic-Adar, Katz	dvojice s vysokým skóre = pravděpodobné budoucí vazby
Detekce podvodných a dezinformačních skupin	detekce komunit + centrality, husté podgrafy	organizované skupiny sdílející zařízení, adresy či obsah
Vyhledávání na webu	PageRank, HITS	prestíž stránky; huby a autority
Odolnost infrastruktury	cílené vs. náhodné odebírání uzlů, mezilehlost	bezškálové sítě: robustní k náhodným výpadkům, zranitelné cíleným útokem na huby
Epidemiologie, očkovací strategie	stupňová a blízkostní centralita, huby	imunizace hubů zpomalí šíření; trasování kontaktů po hranách
Organizační analýza	mezilehlost, brokerské pozice, jádro-periferie	kdo drží tok informací, kde hrozí „single point of failure“
Kriminalistika (Krebs 9/11, syfilis v Georgii)	ego sítě, centrality, komunity	klíčoví aktéři a struktura skryté sítě

Dvě praktická úskalí: *vymezení hranice sítě* (*boundary specification*, kap. 10) — výsledek analýzy silně závisí na tom, koho do sítě zahrneme; a *záměna vlivu s homofilií* — bez shuffle testu či kontroly výběru nelze z korelace chování sousedů usuzovat, že jde o šíření vlivu (kap. 10.7).

11.9 Další oblasti

Prediktivní údržba: predikce poruch strojů ze senzorových dat (časové řady → okna → klasifikace/regrese doby do poruchy). Určení příčin závad automobilů.

Zdravotnictví: analýza databáze pacientů, predikce rizika onemocnění, detekce podvodů v účtovacích výkonech, farmakovigilance (asociace mezi léky a nežádoucími účinky). Kritické je vyhodnocení citlivosti a specifity a spolupráce s doménovým expertem.

Energetika: predikce spotřeby elektrické energie (časové řady, počasí jako externí data).

Finanční trhy: predikce vývoje cen akcií. Přednáška uvádí varovný příklad: cena akcií GameStop vzrostla z \$18,84 (31. 12. 2020) na \$347,51 (27. 1. 2021) kvůli koordinované akci internetové komunity na Twitteru a Redditu (Wallstreetbets), což způsobilo obrovské ztráty hedgeovým fondům (Melvin Capital ztratil \$2,75 miliardy). Modely naučené na historických datech selhávají, změní-li se generující proces — *concept drift*.

Netradiční aplikace: rekonstrukce roztrhaných dokumentů Stasi — archiv začal na počátku 90. let ručně skládat dokumenty roztrhané pracovníky Ministerstva pro státní bezpečnost během revoluce 1989/90; materiál byl nacpán v 16 000 pytlích. Ručně bylo rekonstruováno 1,5 milionu stran z 500 pytlů; v roce 2007 začal výzkum počítačem podporované rekonstrukce a od konce roku 2013 bylo automaticky zrekonstruováno zhruba 60 000 stran z 18 pytlů (stav srpen 2016).

11.10 Provozní a etické aspekty nasazení

Od modelu k produkci

- **Nasazení** (kap. 1, CRISP-DM fáze 6): plán nasazení, integrace do procesů, dokumentace.
- **Monitoring**: sledování *driftu dat* (rozdělení vstupů se mění) a *driftu konceptu* (mění se vztah vstupů a cíle); metriky kvality v čase; alerty a plán přeučení.
- **Zpětná vazba**: model ovlivňuje realitu, kterou měří (schválené úvěry generují data jen o schválených klientech — *reject inference*).
- **Reprodukovatelnost**: verzování dat, kódu i modelu; auditní stopa.

Etika a právo:

- *Soukromí*: GDPR — právní základ zpracování, minimalizace dat, účelové omezení; anonymizace je obtížná (deanonymizace Netflix Prize datasetu, AOL logy); *diferenciální soukromí* jako formální záruka.
- *Spravedlnost (fairness)*: model naučený na historických datech reprodukuje historickou diskriminaci; metriky *demographic parity*, *equalized odds*, *equal opportunity* — nelze je obecně splnit současně.
- *Transparentnost*: právo na vysvětlení automatizovaného rozhodnutí; AI Act pro vysoce rizikové systémy.
- *Korelace není kauzalita*: dobývání znalostí nalézá asociace; rozhodnutí o zásahu vyžaduje kauzální úvahu (randomizovaný experiment, kauzální inference).

11.11 Shrnutí k zkoušce

- Umět ke každé aplikaci přiřadit **typ úlohy** a **vhodné metody**: nákupní košík → asociační pravidla; kreditní skórování → logistická regrese / stromy (interpretovatelnost!); detekce podvodů → nevyvážené třídy, cenově citlivé učení, detekce odlehlých hodnot, analýza sítí; segmentace → shlukování + charakteristická pravidla; churn → klasifikace + lift analýza; doporučování → kolaborativní filtrování / maticová faktorizace; web → PageRank, sekvenční vzory, klasifikace textu.
- **SNA v praxi**: virální marketing → maximalizace vlivu (LTM/ICM); doporučení přátel → predikce vazeb (Adamic–Adar); podvodné a dezinformační skupiny → detekce komunit + centrality; odolnost sítí → cílené odebírání hubů; epidemie → imunizace hubů. Pozor na vymezení hranice sítě a na záměnu vlivu s homofilií.
- **Vyhodnocení musí odpovídat aplikaci**: u nevyvážených úloh nikoli accuracy, ale precision/recall, AUPRC, precision@k, úspora nákladů; u marketingu lift křivka a ekonomický výpočet.
- **Nejčastější příčiny selhání v praxi**: špatně formulovaná úloha, data leakage, concept drift, nezohledněná cena chyb, model bez plánu nasazení a monitoringu, ignorování doménového experta.
- **Etika**: proxy diskriminace, právo na vysvětlení, soukromí, korelace vs. kauzalita; příklad Barclays ukazuje, že technicky správný model může být společensky nepřijatelný.

Část II

Internet a klasifikační metody

Tato část pokrývá okruh předmětu *Internet a klasifikační metody* (M. Holeňa, MFF UK). Předmět se dívá na klasifikaci „shora dolů“ od tří internetových aplikací — filtrace spamu, doporučovacích systémů a systémů pro odhalení hrozeb v síti — a u každé skupiny metod ptá, *k čemu je v těchto aplikacích dobrá*. Tam, kde se látka překrývá s první částí textu, uvádíme jen jádro a odkazujeme na příslušnou kapitolu; důraz je zde na aplikačním pohledu a na tématech, která první část neobsahuje.

12 Tři důležité internetové aplikace klasifikačních metod

12.1 Filtrace spamu

Spam a filtrace spamu

Spam (též *UBE* — *unsolicited bulk email*) je nevyžádaná hromadně rozesílaná zpráva. *Filtrace spamu* je proces oddělení nevyžádaných zpráv od legitimních (*ham*).

Filtrace spamu je z hlediska strojového učení mimořádně náročná, protože jde o **adversariální** úlohu: protivník (spammer) aktivně pozoruje filtr a přizpůsobuje mu své zprávy, takže rozdělení dat se v čase mění (*concept drift*).

12.1.1 Klasifikační úlohy vyskytující se při filtraci spamu

Úloha	Popis
Základní binární klasifikace	zpráva $\rightarrow$ {spam, ham}; nejčastější formulace
Klasifikace do 3 tříd	zpráva $\rightarrow$ {spam, ham, nejisté} — viz níže
Klasifikace odesílatele / IP	reputace zdroje, detekce botnetů rozesílajících spam
Klasifikace typu spamu	phishing, malware, podvod (<i>419 scam</i>), reklama, sexuálně explicitní obsah — různé typy vyžadují různou reakci
Detekce phishingu	podmnožina s mnohem vyšší cenou chyby (krádež přihlašovacích údajů)
Klasifikace obrázků	<i>image spam</i> — text je vykreslen do obrázku, aby unikl textovým filtrům; nutná OCR nebo obrazová klasifikace
Klasifikace URL v těle zprávy	odkaz na známou škodlivou doménu
Klasifikace v jiných médiích	komentářový spam na webu, spam v sociálních sítích, SMS spam, spam v hodnoceních (<i>review spam</i>)

12.1.2 Klasifikace na základě obsahu zpráv a na základě metainformací

Dva zdroje příznaků

Klasifikace na základě obsahu (*content-based*) — příznaky se odvozují z textu zprávy:

- *tokenizace* na slova, případně na n -gramy znaků či slov (odolnější vůči záměrným překlepům typu V!AGRA);
- vektorový model: binární indikátory výskytu, četnosti, TF-IDF váhy;
- výběr atributů (kap. 3): informační zisk, χ^2 , vzájemná informace — slovník se redukuje z desítek tisíc na stovky až tisíce nejinformativnějších tokenů;
- příznaky z HTML struktury (skrýté texty, podíl obrázků, počet odkazů), z formátování a z obrázků (OCR).

Klasifikace na základě metainformací (*meta-information*) — příznaky mimo tělo zprávy:

- *hlavičky*: pole From, Reply-To, Received (trasa zprávy), nesoulad mezi obálkou a hlavičkou, chybějící či podivné Message-ID, časová razítka;
- *síťové a reputační údaje*: IP adresa odesílajícího serveru a jeho záznam v seznamech (DNSBL — *DNS blacklist*, *whitelist*), autonomní systém, geolokace, stáří domény;
- *autentizační mechanismy*: SPF (*Sender Policy Framework*), DKIM (podpis zprávy), DMARC (politika nad SPF a DKIM);
- *kolaborativní údaje*: kontrolní součty zprávy sdílené mezi uživateli (Razor, Pyzor, DCC) — hromadnost rozesílky je sama o sobě silným příznakem;
- *chování uživatele*: kolikrát tento příjemce podobné zprávy označil jako spam.

Metainformace jsou pro spammera obtížněji falšovatelné než obsah, ale samy o sobě nestačí (legitimní hromadné rozesílky, nově registrované legitimní domény).

12.1.3 Začlenění klasifikace do celkového procesu filtrace

Klasifikátor je v reálném filtru jen jednou — byť klíčovou — komponentou víceetapového procesu:

1. **Připojovací fáze** (*connection-level*): odmítnutí spojení podle IP reputace (DNSBL), *greylisting* (dočasné odmítnutí s výzvou k opakovanému doručení — primitivní botnety se nepokusí znovu), omezení rychlosti. Nejlevnější krok, odfiltruje většinu objemu.
2. **Protokolová fáze**: kontrola SPF, DKIM, DMARC, syntaktická správnost hlaviček.
3. **Obsahová fáze**: pravidlový skórovací systém + statistický klasifikátor (naivní Bayes, SVM, logistická regrese, dnes neuronové sítě) + kolaborativní kontrolní součty + antivirová kontrola příloh.
4. **Rozhodovací fáze**: kombinace skóre do konečného rozhodnutí s *prahy* zohledňujícími různou cenu chyb (viz kap. 13); tři možné akce — doručit, do karantény, odmítnout/smazat.
5. **Zpětná vazba a přeučení**: uživatelské označení „toto je spam“ / „toto není spam“, honeypoty (adresy sloužící jen jako past), periodické přeučení modelu. Právě tato smyčka je vstupní branou pro *bayesian poisoning* (kap. 14).

Klasifikace do 3 tříd ve spamových filtrech

Binární rozhodnutí je pro spam nevhodné, protože **cena falešné positivity je řádově vyšší** než cena falešné negativity: nedoručený legitimní e-mail (např. nabídka práce, faktura) může způsobit velkou škodu, zatímco jeden spam ve schránce je jen obtěžování.

Řešení: zavedeme *třetí třídu* — „nejisté“. Filtr pracuje se dvěma prahy $t_1 < t_2$ nad skóre s zprávy:

$$\begin{aligned} s < t_1 &\Rightarrow \text{ham (doručit),} & t_1 \leq s < t_2 &\Rightarrow \text{nejisté (karanténa),} \\ s \geq t_2 &\Rightarrow \text{spam (odmítnout).} \end{aligned}$$

Zprávy v karanténě uživatel může (ale nemusí) prohlédnout. Tím se náklad na falešnou pozitivitu sníží z „ztracená zpráva“ na „zpožděná zpráva“ za cenu určité práce uživatele. Jde o aplikaci obecného principu *klasifikace s možností odmítnutí* (*classification with reject option*): odmítnutá oblast pokrývá vstupy, kde je aposteriorní pravděpodobnost tříd blízká, a rozhodnutí se deleguje na člověka.

SpamAssassin tento princip realizuje bodovým skóre s konfigurovatelnou prahovou hodnotou (výchozí 5,0) a značkováním předmětu; systémy s karanténou (Microsoft Exchange Online Protection, Proofpoint) používají explicitně tři úrovně.

12.1.4 Příklady existujících spamových filtrů

Filtr	Charakteristika
SpamAssassin	Apache; hybridní — stovky ručně psaných pravidel s body, jejichž <i>váhy</i> byly optimalizovány (původně genetickým algoritmem, později perceptronem), plus bayesovský plugin, DNSBL, SPF/DKIM a kolaborativní kontrolní součty (Razor, Pyzor, DCC). Rozhodnutí podle součtu bodů proti prahu.
bogofilter	rychlý bayesovský filtr v C, používá Robinsonovu geometrickou kombinaci a Fisherovu metodu skládání pravděpodobností místo prostého naivního Bayese
SpamBayes	implementace tří tříd (<i>spam</i> / <i>ham</i> / <i>unsure</i>) podle Robinsonova–Fisherova chi-kvadrát skóre
CRM114	„Controllable Regex Mutilator“; nepoužívá jednotlivá slova, ale <i>řídce binární polynomiální hashování</i> (SBPH) a <i>ortogonální řídce bigramy</i> (OSB) — tedy krátké kombinace slov, což výrazně komplikuje bayesian poisoning; učení metodou <i>Train Until No Errors</i>
DSPAM, Rspamd	serverové filtry kombinující statistický klasifikátor s pravidly a reputací
Gmail / Microsoft	rozsáhlé produkční systémy s neuronovými sítěmi, průběžným přeučováním na miliardách zpráv a globální reputační infrastrukturou; deklarovaná úspěšnost přes 99,9 % při velmi nízké falešné pozitivitě

Historicky zásadní byl esej Paula Grahama *A Plan for Spam* (2002), který popularizoval naivní bayesovský filtr počítající skóre z 15 nejinformativnějších tokenů zprávy; jeho úspěch zahájil éru statistické filtrace spamu.

12.2 Doporučovací systémy

Doporučovací systém

Doporučovací systém (*recommender system*) odhaduje, jak by uživatel u ohodnotil (nebo zda by si vybral) položku i , kterou dosud nezná, a předkládá mu položky s nejvyšším odhadem. Vstupem je matice hodnocení R (uživatelé $\times$ položky), obvykle extrémně *řídka* (vyplněno bývá pod 1 % buněk).

12.2.1 Klasifikační úlohy vyskytující se v doporučovacích systémech

Úloha	Popis
Bude se položka líbit?	binární klasifikace {doporučit, nedoporučit} — nejprímější klasifikační formulace
Predikce hodnocení	regrese (odhad hvězdiček 1–5) nebo klasifikace do uspořádaných tříd (<i>ordinální klasifikace</i>)
Predikce interakce	klikne / koupí / dokouká (<i>click-through rate prediction</i>) — binární klasifikace s obrovskou nevyvážeností
Zařazení položky	klasifikace položek do žánrů či kategorií podle obsahu, aby je bylo možné doporučovat
Segmentace uživatelů	klasifikace do předem daných typů, resp. shlukování (viz níže)
Detekce útoků	<i>shilling attacks</i> — falešné profily uměle zvyšující či snižující hodnocení položky; klasifikace profilu na pravý/falešný

12.2.2 Klasifikace při obsahovém a při kolaborativním filtrování

Obsahové filtrování

Obsahové filtrování (*content-based filtering*) doporučuje položky podobné těm, které uživatel dříve hodnotil kladně. Každá položka je popsána *vektorem příznaků* (žánr, autor, klíčová slova z popisu, TF-IDF vektor textu, u hudby akustické atributy).

Klasifikace zde vystupuje takto: pro *každého uživatele zvlášť* se natrénuje binární klasifikátor, jehož trénovací množinou jsou položky, které uživatel hodnotil (kladně = pozitivní třída, záporně = negativní), a příznaky jsou atributy položek. Nová položka se klasifikuje tímto „osobním“ modelem. Používají se naivní Bayes (u textových položek velmi úspěšně), rozhodovací stromy, SVM, k -NN nad vektory položek.

Výhody: funguje i pro položku, kterou nikdo jiný nehodnotil (*nový předmět*); doporučení lze vysvětlit („protože se vám líbily jiné detektivky“). *Nevýhody:* nutnost kvalitního popisu položek; *přeúčtenost* (*overspecialization*) — doporučuje stále totéž, žádné překvapení; nefunguje pro nového uživatele bez historie.

Kolaborativní filtrování

Kolaborativní filtrování (collaborative filtering) nepoužívá obsah položek vůbec — pracuje jen se vzorcem hodnocení. Dvě základní varianty:

- **uživatelské** (*user-based*): najdi uživatele s podobným profilem hodnocení a doporuč, co se líbilo jim;
- **položkové** (*item-based*): najdi položky hodnocené podobně jako ty, které uživatel má rád. Stablnější (podobnost položek se mění pomaleji než podobnost uživatelů) a lépe škáluje — proto ji používá Amazon.

Klasifikace zde vystupuje takto: jde přímo o *k*-NN *klasifikaci* (kap. 14) v prostoru, kde jsou příznaky jednotlivá hodnocení. Predikce hodnocení uživatelsky orientovaným *k*-NN se počítá jako vážený průměr odchylek od průměru:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_k(u)} \text{sim}(u, v) (r_{vi} - \bar{r}_v)}{\sum_{v \in N_k(u)} |\text{sim}(u, v)|}.$$

Alternativně lze úlohu formulovat jako klasifikaci do tříd {líbí, nelíbí} a použít libovolný klasifikátor nad příznaky odvozenými z matice hodnocení.

Výhody: nepotřebuje popis položek; objevuje nečekaná doporučení (*serendipity*). *Nevýhody:* *studený start* (nový uživatel i nová položka), řídkost matice, popularity bias, zranitelnost vůči falešným profilům.

Hybridní systémy kombinují obojí (vážením, přepínáním, kaskádou nebo společným modelem) a jsou v praxi standardem. **Maticová faktorizace** $R \approx UV^T$ (latentní faktory, SVD, ALS) je dnes dominantní technikou kolaborativního filtrování — proslavil ji Netflix Prize.

12.2.3 Příklady existujících doporučovacích systémů

Systém	Typ	Poznámka
Pandora	obsahové	<i>Music Genome Project</i> : každou skladbu ručně ohodnotili hudebníci zhruba 450 atributy; doporučuje podle podobnosti těchto vektorů
Google News (raná verze)	obsahové + kolab.	shlukování článků podle obsahu, personalizace podle historie čtení
IMDb, Rotten Tomatoes	obsahové	„podobné filmy“ podle žánru, tvůrců a klíčových slov
GroupLens / MovieLens	kolaborativní	akademický systém (University of Minnesota, 1994); <i>MovieLens</i> je dodnes standardní veřejná datová sada pro výzkum doporučovacích systémů
Amazon	kolaborativní	<i>item-to-item collaborative filtering</i> (Linden et al., 2003) — předpočítaná matice podobnosti položek, škáluje na desítky milionů uživatelů
Netflix	kolaborativní	Netflix Prize (2006–2009, \$1M) — vítězné řešení byl ensemble s maticovou faktorizací; cílem bylo o 10 % zlepšit RMSE oproti systému Cinematch
Last.fm	kolaborativní	„scrobbling“ poslechů, podobnost umělců
YouTube, Spotify	hybridní	hluboké neuronové sítě nad chováním uživatelů, obsahové vnoření pro studený start

12.3 Systémy pro odhalení hrozeb v síti

IDS a IPS

Systém pro odhalení průniku (Intrusion Detection System, IDS) monitoruje síťový provoz nebo činnost hostitele a hlásí podezřelou aktivitu. *IPS (Intrusion Prevention System)* navíc dokáže provoz automaticky blokovat.

Dělení podle umístění: **NIDS** (síťový — analyzuje pakety a toky) a **HIDS** (hostitelský — analyzuje logy, integritu souborů, volání systémových služeb).

Dělení podle principu detekce:

- **Detekce zneužití** (*misuse / signature-based*) — porovnává provoz se *signaturami* známých útoků. Vysoká přesnost a nízká falešná pozitivita, ale *neodhalí nové útoky (zero-day)*. Odpovídá klasifikaci s učitelem nad známými třídami útoků.
- **Detekce anomálií** (*anomaly-based*) — postaví model *normálního* chování a hlásí odchylky. Odhalí i neznámé útoky, ale trpí vysokou falešnou pozitivitou. Odpovídá učení bez učitele (detekce odlehklých hodnot) nebo jednotřídní klasifikaci.

12.3.1 Klasifikační úlohy vyskytující se v systémech pro odhalení hrozeb

Dvě úrovně klasifikace anomálního chování

1. Klasifikace přítomnosti a nepřítomnosti anomálního chování — binární úloha {normální, anomální}. Toto je klíčová komponenta systému: rozhoduje, zda vůbec spustit poplach. Charakter dat: extrémní nevyváženost (útoky tvoří zlomek procenta provozu), silný *concept drift* (legitimní provoz se vyvíjí), adversariální protivník.

2. Klasifikace do jednotlivých druhů anomálního chování — vícetřídní úloha, která rozhodne, *o jaký útok jde*, a tedy jak reagovat. Klasické kategorie podle datové sady KDD Cup 99:

- **DoS** (*Denial of Service*) — zahlcení služby (neptune, smurf, teardrop);
- **Probe** — skenování a průzkum sítě (portsweep, nmap, satan);
- **R2L** (*Remote to Local*) — neoprávněný přístup ze vzdáleného stroje (hádání hesel, phishing);
- **U2R** (*User to Root*) — neoprávněné získání práv superuživatelé (buffer overflow, rootkit).

Třídy R2L a U2R jsou v datech řádově vzácnější než DoS a Probe, takže vícetřídní úloha je silně nevyvážená a přesnost na těchto třídách bývá nízká — typický příklad, kde je *accuracy* zcela zavádějící metrikou (kap. 6).

Problém základní míry (*base-rate fallacy*)

Axelsson (2000) ukázal, proč je detekce průniků tak obtížná. Nechť je apriorní pravděpodobnost útoku $P(I) = 10^{-5}$ (jedna z 100 000 událostí) a nechť má IDS vynikající citlivost $P(A | I) = 1$ a falešnou pozitivitu $P(A | \neg I) = 10^{-3}$. Pak

$$P(I | A) = \frac{P(A | I)P(I)}{P(A | I)P(I) + P(A | \neg I)P(\neg I)} = \frac{10^{-5}}{10^{-5} + 10^{-3} \cdot (1 - 10^{-5})} \approx 0,0099,$$

tedy **méně než 1 % poplachů je skutečný útok**. Aby byl systém prakticky použitelný, musí být falešná pozitivita o několik řádů nižší. Důsledek: u IDS je klíčovou metrikou *precision* při dané úrovni recall, nikoli *accuracy*, a operátoři trpí *únavou z poplachů (alert fatigue)*.

12.3.2 Příznaky a příklady existujících systémů

Typické příznaky: základní vlastnosti spojení (doba trvání, protokol, služba, počet přenesených bajtů oběma směry, příznaky TCP); obsahové příznaky (počet neúspěšných přihlášení, přístup k souborům, spuštění shellu); statistiky nad časovým oknem (počet spojení na tentýž hostitel či službu za poslední 2 sekundy, podíl chybových odpovědí); u šifrovaného provozu příznaky z metadat toků (NetFlow/IPFIX), velikosti a časování paketů, JA3 otisk TLS.

Systém	Charakteristika
Snort	nejrozšířenější open-source NIDS/IPS; signaturový, pravidla ve vlastním jazyce (hlavička <code>alert tcp any any -> \$HOME_NET 80</code> a tělo s podmínkami <code>content</code> , <code>pcrc</code> , <code>flow</code> a identifikátorem <code>sid</code>); rozsáhlé komunitní i komerční sady pravidel
Suricata	moderní vícevláknová alternativa ke Snortu, kompatibilní s jeho pravidly; podporuje extrakci souborů, TLS/HTTP logging, skriptování v Lua
Zeek (dříve Bro)	„network security monitor“ spíše než klasický IDS; generuje bohaté strukturované logy o spojeních a aplikačních protokolech, nad nimiž lze psát detekční skripty a stavět modely strojového učení — ideální zdroj příznaků
OSSEC, Wazuh	hostitelské IDS: analýza logů, kontrola integrity souborů, detekce rootkitů
Security Onion	distribuce integrující Suricatu, Zeek a nástroje pro analýzu
Komerční NDR/UEBA	Darktrace, Vectra, Cisco Stealthwatch — detekce anomálií strojovým učním nad metadaty toků

Datové sady: *KDD Cup 99* (odvozená z DARPA 1998; historicky nejpoužívanější, ale silně kritizovaná — obsahuje mnoho duplicit a nerealistické rozdělení); *NSL-KDD* (odstraněné duplicity, vyváženější); *CICIDS2017*, *UNSW-NB15*, *CTU-13* (botnety) — modernější a realističtější.

12.4 Shrnutí k zkoušce

- **Spam:** klasifikační úlohy (binární, 3 třídy, typ spamu, phishing, image spam, odesílatel); příznaky z *obsahu* (tokeny, n -gramy, TF-IDF, HTML) vs. z *metainformací* (hlavičky, IP a DNSBL, SPF/DKIM/DMARC, kolaborativní kontrolní součty); celkový proces má 5 stupňů (spojení → protokol → obsah → rozhodnutí → zpětná vazba); **3 třídy** {spam, ham, nejisté} s karanténou kvůli vysoké ceně falešné pozitivity. Příklady: SpamAssassin, bogofilter, SpamBayes, CRM114, DSPAM/Rspamd, Gmail.
- **Doporučovací systémy:** úlohy (bude se líbit, predikce hodnocení, CTR, zařazení položky, detekce falešných profilů); *obsahové filtrování* = osobní klasifikátor nad atributy položek (řeší nový předmět, trpí přeúčteností); *kolaborativní filtrování* = k -NN nad maticí hodnocení, uživatelské vs. položkové (řeší serendipitu, trpí studeným startem a řídkostí). Příklady: Pandora (obsahové), Amazon a Netflix/MovieLens (kolaborativní), YouTube/Spotify (hybridní).
- **IDS:** NIDS vs. HIDS; *detekce zneužití* (signatury, přesná, neodhalí zero-day) vs. *detekce anomálií* (odhalí nové, vysoká falešná pozitivita); dvě klasifikační úlohy — **přítomnost/nepřítomnost** anomálie a **druh** anomálie (DoS, Probe, R2L, U2R); *problém základní míry* — při $P(I) = 10^{-5}$ a $FPR = 10^{-3}$ je jen 1 % poplachů skutečný útok. Příklady: Snort, Suricata, Zeek, OSSEC; data KDD Cup 99 / NSL-KDD.
- Společný rys všech tří aplikací: **silně nevyvážené třídy a různá cena chyby** ⇒ accuracy je nepoužitelná, nutné cenově citlivé hodnocení.

13 Základní koncepty týkající se klasifikace

13.1 Klasifikace, klasifikátory, počet tříd

Klasifikace a klasifikátor

Klasifikace je přiřazení objektu $\vec{x} \in \mathcal{X}$ do jedné z konečně mnoha *tříd* $C = \{C_1, \dots, C_m\}$. *Klasifikátor* je zobrazení $h : \mathcal{X} \rightarrow C$ (případně $h : \mathcal{X} \rightarrow C \cup \{\text{odmítnutí}\}$).

Binární klasifikace ($m = 2$) vs. **klasifikace do více tříd** ($m > 2$). Vícetřídní úlohu lze převést na binární:

- *jeden proti zbytku* (*one-against-rest*) — m klasifikátorů, každý odděluje jednu třídu od všech ostatních; rozhoduje nejvyšší skóre;
- *jeden proti jednomu* (*one-against-one*) — $\binom{m}{2}$ klasifikátorů, hlasování;
- *kódy opravující chyby* (*error-correcting output codes*) — každé třídě se přiřadí binární kód, trénuje se klasifikátor pro každý bit a výsledek se dekóduje na nejbližší kódové slovo.

Některé metody (rozhodovací stromy, naivní Bayes, k -NN, neuronové sítě se softmaxem) zvládají více tříd nativně; SVM nikoli (kap. 15).

Použití klasifikace do 3 tříd ve spamových filtrech je popsáno v kap. 12: třetí třída „nejisté“ realizuje *odmítnutí rozhodnutí* a snižuje očekávanou cenu chyb.

13.2 Koncepty specifické pro binární klasifikaci

Pozitivní a negativní třída, falešná pozitivita a negativita

U binární klasifikace se jedna třída označí za **pozitivní** (obvykle ta „zajímavá“, minoritní — spam, útok, položka, která se bude líbit) a druhá za **negativní**.

Falešná pozitivita (*false positive*, FP, chyba I. druhu) — negativní objekt klasifikován jako pozitivní. **Falešná negativita** (*false negative*, FN, chyba II. druhu) — pozitivní objekt klasifikován jako negativní.

Matici záměn a odvozené míry viz kap. 6.

Chybovost a zahrnutí různé ceny chyb

Chybovost (*error rate*) = $\frac{FP+FN}{TP+TN+FP+FN} = 1 - \text{accuracy}$ implicitně předpokládá, že *obě chyby stojí stejně*. To v internetových aplikacích téměř nikdy neplatí.

Cenově citlivá klasifikace (*cost-sensitive classification*): zavedeme matici nákladů $c(i, j)$ — cena zařazení objektu skutečné třídy i do třídy j (obvykle $c(i, i) = 0$) — a minimalizujeme *očekávanou cenu* místo chybovosti:

$$\text{zvol třídu } j \text{ minimalizující } \sum_i P(C_i | \vec{x}) c(i, j).$$

Pro dvě třídy z toho plyne *optimální práh* na aposteriorní pravděpodobnost:

$$\text{klasifikuj jako pozitivní} \iff P(\text{poz} | \vec{x}) \geq \frac{c(\text{neg}, \text{poz})}{c(\text{neg}, \text{poz}) + c(\text{poz}, \text{neg})} = \frac{c_{FP}}{c_{FP} + c_{FN}}.$$

Prakticky lze cenovou citlivost realizovat: posunem prahu, *vážením* trénovacích příkladů, převzorkováním, nebo přímo v účelové funkci učícího algoritmu.

Různá cena falešné positivity a negativity při filtraci spamu

Označme $\lambda = c_{FP}/c_{FN}$ poměr cen, tj. kolikrát je horší zahodit legitimní zprávu než propustit spam. Androutsopoulos et al. zavedli standardní scénáře:

λ	Scénář	Interpretace
1	symetrický	obě chyby stejně drahé (nerealistické)
9	označkování	spam se pouze označí v předmětu; uživatel jej snadno najde
999	zahození	spam se automaticky maže — ztráta legitimní zprávy je katastrofická

Odpovídající míry: *vážená správnost* $WAcc = \frac{\lambda \cdot TN + TP}{\lambda \cdot N + P}$ (legitimní zpráva se počítá λ -krát; P je počet spamů, N počet legitimních zpráv) a *poměr celkové ceny* (*total cost ratio*, TCR)

$$TCR = \frac{\text{cena bez filtru}}{\text{cena s filtrem}} = \frac{P}{\lambda \cdot FP + FN};$$

v čitateli je cena stavu „žádný filtr“ (projde všech P spamů, každý za jednotku), ve jmenovateli cena chyb filtru. $TCR > 1$ znamená, že se filtr vyplatí. Pro $\lambda = 999$ musí mít filtr falešnou pozitivitu prakticky nulovou — odtud tříčlenné rozhodování s karanténou.

Příklad — výpočet WAcc a TCR pro tři scénáře

Testovací množina: $P = 200$ spamů a $N = 800$ legitimních zpráv. Filtr dal $TP = 190$, $FN = 10$, $FP = 8$, $TN = 792$ (tedy citlivost 95 %, ale $FPR = 8/800 = 1$ %).

λ	WAcc filtru	WAcc bez filtru	$TCR = \frac{200}{8\lambda + 10}$
1	$\frac{792+190}{800+200} = 0,982$	$\frac{800}{1000} = 0,800$	$\frac{200}{18} = 11,1$
9	$\frac{7128+190}{7200+200} = 0,989$	$\frac{7200}{7400} = 0,973$	$\frac{200}{82} = 2,44$
999	$\frac{791208+190}{799200+200} = 0,990$	$\frac{799200}{799400} = \mathbf{0,9998}$	$\frac{200}{8002} = \mathbf{0,025}$

Čtení výsledku. Při $\lambda = 1$ i $\lambda = 9$ je filtr jasně výhodný. Při $\lambda = 999$ je ale WAcc *nefiltrování* (0,9998) **vyšší** než WAcc filtru (0,990) a TCR klesne na 0,025 — filtr napáchá čtyřicetkrát větší škodu, než kolik jí zabránil. Příčinou je jediné číslo: osm falešně pozitivních zpráv. Aby byl při $\lambda = 999$ filtr vůbec užitečný ($TCR > 1$), muselo by platit $999 \cdot FP + 10 < 200$, tedy $FP = 0$ — na 800 legitimních zprávách si nesmí dovolit *ani jednu* chybu (pak $TCR = 200/10 = 20$). To je kvantitativní zdůvodnění karantény a třetí třídy „nejisté“.

13.3 Charakteristiky kvality binární klasifikace

Definice a odvození jsou v kap. 6; zde jen souhrn v terminologii předmětu:

Míra	Vzorec	Význam ve spamu
Správnost (<i>accuracy</i>)	$\frac{TP+TN}{TP+TN+FP+FN}$	podíl správně zařazených zpráv; <i>zavádějící</i> při nevyvážených datech
Přesnost (<i>precision</i>)	$\frac{TP}{TP+FP}$	kolik ze zpráv označených za spam spammem skutečně je — <i>přímo měří falešnou pozitivitu</i>
Citlivost (<i>recall, sensitivity, TPR</i>)	$\frac{TP}{TP+FN}$	kolik spamu filtr zachytí
Specifita (<i>specificity, TNR</i>)	$\frac{TN}{TN+FP}$	kolik legitimních zpráv projde; $FPR = 1 - \text{specifita}$
F_1 -míra	$F_1 = \frac{2pr}{p+r}$, obecně F_β	harmonický průměr; $\beta < 1$ zvýhodní přesnost (vhodné pro spam)
ROC křivka	TPR proti FPR	celé spektrum provozních bodů podle prahu
AUC	plocha pod ROC	1 dokonalý, 0,5 náhodný; nezávislá na prahu i na poměru tříd

Charakterizace kvality při filtraci spamu. Kvůli $\lambda \gg 1$ se používá spíše: přesnost při vysokém recallu; ROC v logaritmickém měřítku zaostřená na oblast velmi nízké FPR (např. $FPR < 10^{-3}$); vážená správnost a TCR; a v produkci přímo počet stížností uživatelů na ztracené zprávy. Samotná AUC je nedostatečná, protože průměruje i přes provozní body, které nikdy nepoužijeme.

13.4 Tvar hranice mezi třídami

Rozhodovací hranice a lineární separabilita

Rozhodovací hranice (decision boundary) je množina bodů $\{\vec{x} : h \text{ mění na } \vec{x} \text{ predikci}\}$; u pravděpodobnostních klasifikátorů je to množina, kde se aposteriorní pravděpodobnosti tříd rovnají. Dvě třídy jsou **lineárně separabilní**, existuje-li nadrovina $\langle \vec{w} \cdot \vec{x} \rangle + b = 0$ taková, že všechny objekty jedné třídy leží v jednom poloprostoru a všechny objekty druhé třídy v druhém.

Příklady tvarů hranice: lineární (perceptron, LDA, logistická regrese, lineární SVM); kvadratická (QDA, naivní Bayes se spojitými atributy); osově rovnoběžná po částech konstantní (rozhodovací stromy); libovolná po částech lineární (vícevrstvý perceptron, k -NN).

Přechod k lineární separabilitě pomocí jádrových funkcí

Nejsou-li třídy ve vstupním prostoru $\mathcal{X}$ lineárně separabilní, zobrazíme data nelineárním zobrazením $\phi : \mathcal{X} \rightarrow \mathcal{F}$ do *prostoru příznaků* $\mathcal{F}$ (obvykle mnohem vyšší dimenze), kde už lineárně separabilní být mohou. Coverova věta říká, že pravděpodobnost lineární separability roste s dimenzí prostoru.

Explicitní výpočet $\phi(\vec{x})$ je však obvykle neproveditelný (prokletí dimenzionality). *Jádrový trik:* metody, které používají data *pouze prostřednictvím skalárních součinů*, lze přepsat pomocí **jádrové funkce**

$$K(\vec{x}, \vec{z}) = \langle \phi(\vec{x}) \cdot \phi(\vec{z}) \rangle,$$

aniž bychom ϕ vůbec znali. Podle Mercerovy věty je K jádrem právě tehdy, když je symetrická a její Gramova matice je pro libovolnou konečnou množinu bodů pozitivně semidefinitní. Běžná jádra (lineární, polynomiální, gaussové RBF, sigmoidální) a podrobnosti viz kap. 5; proč je trik výhodný právě pro SVM, viz kap. 15.

13.5 Konstrukce klasifikátorů z dat: učení

Učení klasifikátorů

Učení (*training*) klasifikátoru je konstrukce zobrazení h z trénovací množiny $\{(\vec{x}_1, y_1), \dots, (\vec{x}_p, y_p)\}$ minimalizací (regularizovaného) empirického rizika

$$\hat{R}(h) = \frac{1}{p} \sum_{i=1}^p L(y_i, h(\vec{x}_i)) + \text{regularizace.}$$

Podle režimu rozlišujeme učení *dávkové* (*batch* — model se postaví najednou) a *inkrementální* (*online* — model se aktualizuje po každém příkladu; nezbytné pro spamové filtry a IDS, kde data přicházejí proudem).

Učení spamových filtrů

Specifika:

- **Zdroje označovaných dat:** uživatelská tlačítka „nahlásit spam“ / „není spam“; *honeypoty* (adresy, které nikdy nikomu nedaly souhlas, takže vše, co přijde, je spam); ručně anotované korpusy (Ling-Spam, PU1, SpamAssassin corpus, Enron-Spam, TREC Spam Track).
- **Personalizace vs. globální model:** globální model má mnohem více dat, ale pojem „spam“ je subjektivní (obchodní newsletter). V praxi hybrid: globální model + osobní korekce.
- **Inkrementální přeučování** je nutné kvůli concept driftu; typicky se uchovává jen klouzavé okno posledních zpráv.
- **Úsporné strategie tréninku:** *Train on Error* (TOE — uč pouze na chybně klasifikovaných zprávách), *Train Until No Errors* (TUNE), *Train on or near Error* (TONE — uč i na správně klasifikovaných zprávách s nejistým skóre). Šetří kapacitu modelu a zvyšují odolnost.
- **Nebezpečí:** automatické přeučení na neověřených datech otevírá cestu k otravě modelu (*bayesian poisoning*, kap. 14).

Přeučení klasifikátoru

Přeučení (*overfitting*) nastává, když klasifikátor zachytí i šum a specifika trénovací množiny: trénovací chyba klesá, ale chyba na nezávislých datech roste. Rozklad chyby na zkreslení, rozptyl a neredukovatelný šum, prostředky obrany (regularizace, prořezávání, předčasné zastavení, více dat, ensemble) a metody odhadu chyby (holdout, křížová validace, bootstrap) jsou v kap. 6.

Specificky u spamu: přeučení se projeví tím, že si filtr zapamatuje konkrétní kampaň (např. jméno konkrétního produktu) místo obecného vzorce, a při změně kampaně selže. Proto se preferují jednodušší, silně regularizované modely s průběžným přeučováním.

13.6 Klasifikace, regrese a shlukování

Souvislost klasifikace a regrese

Regrese odhaduje *spojitou* cílovou veličinu, klasifikace *diskrétní*. Vazby mezi nimi:

- Většina klasifikátorů vnitřně počítá *spojitou* „diskriminační“ či skórovací funkci $f(\vec{x})$ a teprve prahováním z ní dělá třídu: $h(\vec{x}) = \text{sign}(f(\vec{x}))$. Klasifikátor je tedy prahovaná regresní funkce.
- *Logistická regrese* je regresní model, jehož výstupem je pravděpodobnost — používá se přímo jako klasifikátor (viz logitová metoda, kap. 14).
- Naopak *regresi lze převést na klasifikaci* diskretizací cílové veličiny (ztrácíme informaci o uspořádání, řeší *ordinální klasifikace*).
- Tytéž modely bývají v obou variantách: regresní vs. klasifikační stromy (CART), regresní vs. klasifikační náhodné lesy, SVR vs. SVM.
- Liší se ztrátové funkce: kvadratická či absolutní u regrese; 0–1, logistická, hinge u klasifikace.

Role regrese v doporučovacíích systémech. Predikce hodnocení („kolik hvězdiček dá uživatel

u filmu i) je *regresní* úloha — proto se kvalita v Netflix Prize měřila RMSE. Maticová faktorizace $\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^T \mathbf{q}_i$ je regresní model s latentními faktory. Regrese se uplatní i při *learning to rank* a při odhadu pravděpodobnosti kliknutí. Naopak, má-li systém rozhodnout jen „doporučit / nedoporučit“, jde o klasifikaci — a je známo, že optimalizace RMSE nemusí vést k nejlepšímu *pořadí* doporučení (metriky precision@ k , NDCG).

Odlišnost klasifikace a shlukování

	Klasifikace	Shlukování
Typ učení	s učitelem	bez učitele
Vstup	objekty <i>s označením třídy</i>	objekty <i>bez označení</i>
Cíl	předpovědět třídu nového objektu	najít strukturu v datech
Třídy	dané předem, mají význam	vznikají z dat, význam je nutné dodat
Vyhodnocení	objektivní (chyba na testovací množině)	nepřímé (koheze, separace, externí shoda)

Metody shlukování jsou podrobně v kap. 7.

Použití shlukování v doporučovacích systémech:

- *Škálovatelnost*: místo hledání k nejbližších sousedů mezi miliony uživatelů se uživatelé nejprve rozdělí do shluků a sousedé se hledají jen uvnitř shluku — výrazné zrychlení za cenu mírné ztráty přesnosti.
- *Segmentace zákazníků*: shluky uživatelů (např. podle RFM nebo podle profilu hodnocení) definují cílové skupiny pro doporučení a marketing.
- *Shlukování položek*: položky se seskupí do témat či žánrů, což zmenší řídkost matice (hodnocení se agregují na úroveň shluku) a umožní doporučovat na úrovni kategorií.
- *Studený start*: nového uživatele lze po několika hodnoceních přiřadit k nejbližšímu shluku a doporučovat mu jeho typické položky.
- *Spoluklastrování (co-clustering)*: současné shlukování uživatelů i položek na bloky matice hodnocení.
- *Diverzita*: doporučí-li se po jedné položce z různých shluků, výsledná nabídka je pestřejší a méně trpí přeúčteností.

13.7 Shrnutí k zkoušce

- **Klasifikátor** $h : \mathcal{X} \rightarrow C$; binární vs. vícetřídní; převody one-against-rest, one-against-one, ECOC; **3 třídy u spamu** = odmítnutí rozhodnutí (karanténa).
- **FP vs. FN**; **cenově citlivá klasifikace**: minimalizuj očekávanou cenu $\sum_i P(C_i | \vec{x}) c(i, j)$; optimální práh je $c_{FP}/(c_{FP} + c_{FN})$; u spamu se uvažuje $\lambda \in \{1, 9, 999\}$ a míry WAcc a TCR (umět spočítat, kdy TCR klesne pod 1).
- **Míry kvality**: správnost, přesnost, citlivost, specifita, F -míra, ROC, AUC (kap. 6); u spamu zaostřit na oblast velmi nízké FPR.
- **Tvar hranice**: lineární separabilita; **jádrové funkce** převedou lineárně neseparabilní úlohu na separabilní ve vysokorozměrném prostoru příznaků; Coverova věta, Mercerova věta, jádrový trik.
- **Učení**: minimalizace empirického rizika; dávkové vs. inkrementální; u spamu honeypoty, uživatelská zpětná vazba, TOE/TUNE/TONE, concept drift. **Přeučení** a obrana proti němu.
- **Klasifikace vs. regrese**: klasifikátor je prahovaná regresní funkce; regrese v doporučovacích systémech (predikce hodnocení, RMSE, maticová faktorizace). **Klasifikace vs. shlukování**: s učitelem vs. bez učitele; shlukování v doporučovacích systémech (škálovatelnost, segmentace, řídkost, studený start, diverzita).

14 Hlavní typy klasifikačních metod

14.1 Rozdělení metod podle toho, zda hledají hranice mezi třídami

Dvě rodiny klasifikačních metod

Metody hledající hranice mezi třídami (*discriminative*) — přímo modelují rozhodovací hranici, resp. diskriminační funkci $f(\vec{x})$, aniž by modelovaly rozdělení dat. Patří sem perceptron a vícevrstvé perceptrony, SVM, logistická regrese v diskriminativním pojetí, rozhodovací stromy (po částech konstantní hranice).

Metody nehledající hranice — rozhodují bez explicitní hranice. Dva hlavní přístupy:

- **podobnost** — objekt se zařadí podle toho, kterým už zařazeným objektům je nejpodobnější (k -NN a další metody založené na instancích);
- **odhadování pravděpodobnosti příslušnosti k jednotlivým třídám** — spočítá se (odhad) $P(C_j | \vec{x})$ a zvolí se nejpravděpodobnější třída. Sem patří bayesovské klasifikátory, logitová metoda a diskriminační analýza prokládající vícerozměrné normální rozdělení.

Poznámka: hranice *existuje* i u metod druhé rodiny (je to množina, kde se pravděpodobnosti či podobnosti vyrovnají), metoda ji však nehledá explicitně — vzniká jako vedlejší produkt.

14.2 Klasifikátory založené na podobnosti: k -NN

k -NN klasifikátor

k *nejbližších sousedů* (*k-nearest neighbours*) klasifikuje objekt $\vec{z}$ takto: spočítá podobnost (resp. vzdálenost) $\vec{z}$ ke všem trénovacím objektům se známou příslušností do tříd, vyber k nejpodobnějších a přiřaď třídu, která mezi nimi převažuje (případně váženě podle podobnosti).

Jde o *líné učení* (*lazy learning*): trénink spočívá v uložení dat, veškerá práce se odkládá na dobu klasifikace. Podrobnosti viz kap. 5.

Volba počtu nejblížešších sousedů

- **Malé k** (v extrému $k = 1$): velmi flexibilní hranice, nízké zkreslení, ale *vysoký rozptyl* — klasifikátor kopíruje šum a jednotlivé chybně označované body.
- **Velké k** : hladká hranice, nízký rozptyl, ale *vysoké zkreslení* — v extrému $k = p$ vrací vždy majoritní třídu.
- **Jak volit:** křížovou validací (kap. 6) přes rozsah hodnot; heuristika $k \approx \sqrt{p}$; *liché k* u dvou tříd kvůli remízám.
- **Vážená varianta** (*distance-weighted k-NN*) — soused přispívá vahou $1/d^2$ nebo $\sin$; činí volbu k méně citlivou.
- U *nevyvážených* tříd je nutné hlasy vážit apriorními pravděpodobnostmi či cenami chyb, jinak minoritní třída zanikne.

Míry podobnosti užívané v k -NN klasifikátorech

Míra	Vzorec	Kdy
Euklidovská vzdálenost	$\sqrt{\sum_j (x_j - z_j)^2}$	numerické atributy podobné škály
Manhattanská	$\sum_j x_j - z_j $	robustnější k odlehlým hodnotám
Minkowského $L^{(q)}$	$(\sum_j x_j - z_j ^q)^{1/q}$	zobecnění obou
Mahalanobisova	$\sqrt{(\vec{x} - \vec{z})^T S^{-1} (\vec{x} - \vec{z})}$	korelované atributy s různým rozptylem
Kosinová podobnost	$\frac{\langle \vec{x}, \vec{z} \rangle}{\ \vec{x}\ \ \vec{z}\ }$	vysokorozměrná řídká data (text, hodnocení)
Pearsonova korelace	$\frac{S_{xz}}{\sqrt{S_x^2 S_z^2}}$	data s různým „posunem“ (přísní vs. mírní hodnotitelé)
Jaccardův koeficient	$\frac{ X \cap Z }{ X \cup Z }$	binární / množinová data (n -gramy, API volání)
Hammingova	počet neshodných pozic	kategoriální a binární atributy
Editační (Levenshteinova)	minimální počet úprav	řetězce, URL, názvy souborů

Kritické je škálování: bez normalizace atributů (kap. 2) dominují atributy s velkým rozsahem. k -NN je také citlivé na irrelevantní atributy a na prokletí dimenzionality — ve vysokých dimenzích se vzdálenosti vyrovnávají a pojem „nejbližší“ ztrácí smysl; pomáhá výběr atributů (kap. 3) nebo naučená metrika (*metric learning*).

Použití k -NN při kolaborativním filtrování a měření podobnosti

Kolaborativní filtrování je *přímočará aplikace k -NN*: „objekty“ jsou uživatelé (resp. položky) a „atributy“ jejich hodnocení. Zvláštností je, že vektory jsou z drtivé většiny *neúplné*, takže podobnost se počítá jen přes *společně hodnocené* položky $I_{uv} = I_u \cap I_v$.

Pearsonova korelace (standard pro uživatelsky orientované CF) — odečtení průměru uživatele kompenzuje, že někdo dává systematicky vyšší hodnocení:

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{vi} - \bar{r}_v)^2}}.$$

Kosinová podobnost — vhodná pro implicitní zpětnou vazbu (0/1), kde průměr nemá smysl.

Upravená kosinová podobnost (*adjusted cosine*) — standard pro *položkově* orientované CF; odečítá průměr *uživatele* (nikoli položky), čímž odstraní individuální posun:

$$\text{sim}(i, j) = \frac{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_u)(r_{uj} - \bar{r}_u)}{\sqrt{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_u)^2} \sqrt{\sum_{u \in U_{ij}} (r_{uj} - \bar{r}_u)^2}}.$$

Spearmanova korelace (Pearson nad pořadími) — když záleží jen na pořadí preferencí.

Praktické korekce: *významnostní vážení* (*significance weighting*) — podobnost spočtená ze 2 společných položek je nespolehlivá, proto se násobí faktorem $\min(|I_{uv}|, \gamma)/\gamma$ (typicky $\gamma = 50$); *smršťování* (*shrinkage*) $\frac{|I_{uv}|}{|I_{uv}| + \beta} \text{sim}$; *inverzní frekvence položky* (oblíba u všech nenese informaci, obdoba IDF); *výchozí hodnocení* (*default voting*) pro nehodnocené položky.

Predikce: viz vzorec pro $\hat{r}_{ui}$ v kap. 12.

Použití k -NN při detekci malware

Malware se reprezentuje vektorem příznaků získaných *statickou* nebo *dynamickou* analýzou:

- *statické*: bajtové n -gramy souboru, n -gramy instrukcí (*opcode*), řetězce, importované funkce (*import table*), struktura hlavičky PE, entropie sekcí (indikuje zabalení či šifrování);
- *dynamické*: posloupnost volání systémových služeb (API/syscall trace) při běhu v izolovaném prostředí (*sandbox*), síťová aktivita, zápisy do registru a souborového systému.

Podobnost se pak měří *Jaccardovým koeficientem* nad množinami n -gramů či volání, *kosinovou podobností* nad TF-IDF vektory n -gramů, editační vzdáleností nad posloupnostmi volání, nebo *normalizovanou kompresní vzdáleností* (NCD) přímo nad binárkami.

Proč právě k -NN: (a) malware se šíří v *rodinách* — nový vzorek je obvykle mutací něčeho známého, takže nejbližší soused bývá z téže rodiny; (b) klasifikace je zároveň *vysvětlením* („podobá se známému vzorku X z rodiny Y“) — což je pro analytika cenné; (c) nový vzorek stačí přidat do databáze, není nutné přeučovat model, což se hodí při desítkách tisíc nových vzorků denně. *Nevýhoda*: nákladná klasifikace nad obrovskou databází (řeší se LSH, k -d stromy, shlukováním) a zranitelnost vůči záměrné obfuskaci, která vzdálenost uměle zvětší.

14.3 Klasifikátory založené na bodových odhadech pravděpodobnosti

14.3.1 Bayesovské klasifikátory

Bayesovský a naivní bayesovský klasifikátor

Vychází z Bayesovy věty $P(H | E) = \frac{P(E|H)P(H)}{P(E)}$ a volí hypotézu s největší aposteriorní pravděpodobností (H_{MAP}). *Naivní* bayesovský klasifikátor předpokládá podmíněnou nezávislost atributů při dané třídě:

$$H_{MAP} = \arg \max_t P(H_t) \prod_{k=1}^K P(E_k | H_t).$$

Odvození, Laplaceova korekce, spojité atributy a příklad výpočtu viz kap. 5.

Bodové odhady pravděpodobnosti logitovou metodou

Logitová metoda modeluje pravděpodobnost příslušnosti ke třídě *přímo* (diskriminativně), bez modelování rozdělení atributů. Protože pravděpodobnost je omezena na $\langle 0, 1 \rangle$, modeluje se lineárně její *logit* (logaritmus šance):

$$\text{logit } P = \ln \frac{P}{1-P} = \alpha + \beta_1 x_1 + \dots + \beta_m x_m \quad \Longleftrightarrow \quad P(y = 1 \mid \vec{x}) = \frac{1}{1 + e^{-\alpha - \sum_j \beta_j x_j}}.$$

Parametry se odhadují *metodou maximální věrohodnosti*, ekvivalentně minimalizací křížové entropie; odvození adaptačních pravidel viz kap. 5. Pro více tříd se používá *multinomiální logitový model* (softmax): $P(y = j \mid \vec{x}) = \frac{e^{\beta_j^T \vec{x}}}{\sum_l e^{\beta_l^T \vec{x}}}$.

Srovnání s naivním Bayesem. Oba dávají odhad $P(C \mid \vec{x})$ a oba mají (pro binární případ s příslušnými předpoklady) *lineární* rozhodovací hranici, ale:

	Naivní Bayes	Logitová metoda
Typ modelu	generativní ($P(\vec{x} \mid C)P(C)$)	diskriminativní ($P(C \mid \vec{x})$)
Předpoklad	podmíněná nezávislost atributů	žádný o rozdělení atributů
Kvalita odhadu P	často <i>přepálená</i> k 0/1	dobře <i>kalibrovaná</i>
Korelované atributy	zkresluje (dvojí započtení)	zvládá
Malá data	lepší (silnější předpoklad)	horší
Učení	spočtení četností, triviální	iterativní optimalizace

Právě dobrá *kalibrace* pravděpodobností je důvod, proč se logitová metoda používá tam, kde se odhad P dále vkládá do rozhodování s cenami chyb (spamový práh, skórování rizika, predikce vazeb v kap. 10).

Použití bayesovských klasifikátorů ve spamových filtrech

Naivní Bayes je klasickou metodou filtrace spamu. Zpráva M se reprezentuje množinou tokenů $\{w_1, \dots, w_n\}$ a počítá se

$$P(\text{spam} \mid M) \propto P(\text{spam}) \prod_{i=1}^n P(w_i \mid \text{spam}).$$

Praktické varianty:

- *multinomiální* model (počítá četnosti výskytů) vs. *Bernoulliho* model (jen přítomnost/nepřítomnost tokenů); pro spam se osvědčil binární multinomiální.
- *Grahamova varianta*: pro každý token se spočte „spamovost“ $p(w) = \frac{b(w)/B}{b(w)/B + g(w)/G}$ (kde b, g jsou četnosti ve spamu a hamu a B, G velikosti korpusů), vezme se **15 tokenů nejvzdálenějších od 0,5** (nejinformativnějších) a jejich pravděpodobnosti se zkombinují. Použití jen několika nejinformativnějších tokenů výrazně zvyšuje odolnost proti otravě.
- *Robinsonova–Fisherova varianta* (bogofilter, SpamBayes): pravděpodobnosti tokenů se kombinují Fisherovou metodou skládání p -hodnot, což dává dvě samostatná skóre (spamovost a hamovost) a přirozeně vede k třetí třídě „nejisté“.
- *Vyhlazování*: Laplaceova korekce a „stažení“ odhadů vzácných tokenů ke střední hodnotě (Robinsonovo s).

Proč naivní Bayes na spamu funguje, ačkoli slova zjevně nezávislá nejsou? Pro *rozhodnutí* stačí správné pořadí aposteriorních pravděpodobností, nikoli jejich správné hodnoty; korelace slov sice zkreslí absolutní hodnoty, ale zpravidla ne pořadí. Navíc je model extrémně rychlý, inkrementálně učitelný a snadno personalizovatelný.

Učení bayesovských spamových filtrů

Model je určen tabulkou četností tokenů ve spamu a hamu, takže *učení je pouhá aktualizace čítačů* — ideální pro inkrementální provoz.

1. *Inicializace* na anotovaném korpusu, nebo od nuly z uživatelské pošty.
2. *Průběžné učení* z uživatelské zpětné vazby a z honeypotů.
3. *Strategie*: učit na všem, nebo jen na chybách (TOE), nebo i na nejistých případech (TONE) — viz kap. 13.
4. *Stárnutí*: tokeny, které se dlouho neobjevily, se z databáze odstraňují; jinak databáze roste a zastarává.
5. *Personalizace*: osobní databáze tokenů zvyšuje přesnost (spam se pozná i podle toho, o čem uživatel běžně komunikuje) a zároveň ztěžuje spammerovi cílený útok, protože nezná obsah cizí databáze.

Bayesian poisoning — narušitelnost učení spamery

Bayesian poisoning je útok, při němž spammer záměrně manipuluje statistiky filtru. Dva režimy: **Pasivní (evazivní) otrava — útok na klasifikaci**. Spammer do zprávy přidá slova typická pro legitimní poštu („word salad“, náhodné úryvky z knih či zpravodajství, skrytý text bílou barvou na bílém pozadí, text v HTML komentářích, slova rozbitá neviditelnými značkami). Cílem je posunout skóre zprávy směrem k hamu. Sem patří i *záměrné překlady* (VIAGRA, Vi_agra) rozbíjející tokenizaci a *obrázkový spam*, kde je text mimo dosah textového klasifikátoru.

Aktivní otrava — útok na učení. Spammer využije zpětnovazební smyčku: rozešle zprávy navržené tak, aby je uživatelé nahlásili (nebo aby se dostaly do honeypotů) a filtr se na nich přeučil chybně. Podvariantou je *cílený útok*, kdy spammer nasytí filtr tokeny typickými pro legitimní komunikaci konkrétní oběti, aby jí následně mohl doručit phishing; a *útok na dostupnost*, kdy je cílem naopak zvýšit falešnou pozitivitu tak, aby uživatel filtr vypnul.

Obrany:

- používat jen *nejinformativnější* tokeny (Grahamových 15) — přidaná neutrální slova pak skóre téměř neovlivní;
- *vícemístné příznaky* místo jednotlivých slov: CRM114 používá SBPH a ortogonální řídké bigramy, takže spammer by musel napodobit celé fráze, nikoli jednotlivá slova;
- neučit automaticky na neověřených datech; ověřovat zpětnou vazbu (reputace nahlašujícího uživatele), používat karanténu a ruční kontrolu;
- robustní a regularizované modely, ansámblů různých metod (kap. 17), doplnění o metainformace, které spammer nemůže falšovat (IP reputace, DKIM);
- omezit rychlost přeučování a udržovat „zlatý“ referenční korpus, na němž se po každém přeučení ověří, že kvalita neklesla.

Obecně jde o instanci *adversariálního strojového učení*: útoky na trénovací data se nazývají *poisoning*, útoky na klasifikaci již natrénovaného modelu *evasion*.

14.3.2 Fisherova diskriminační analýza

Fisherova diskriminační analýza

Klasifikace založená na odhadech pravděpodobnosti příslušnosti k třídám *prokládáním vícerozměrného normálního rozdělení*: předpokládáme, že objekty třídy c_t jsou realizacemi rozdělení $\mathcal{N}(\vec{\mu}_t, S_t)$, a parametry odhadneme z dat (výběrové průměry a kovarianční matice). Aposteriorní pravděpodobnost pak podle Bayesovy věty je

$$f_t(\vec{x}) = P(c_t | \vec{x}) = \frac{P(\vec{x} | c_t)P(c_t)}{\sum_k P(\vec{x} | c_k)P(c_k)},$$

a objekt zařadíme do třídy s největší hodnotou.

Fisherovo kritérium (dvě třídy): hledáme směr $\vec{w}$, který maximalizuje poměr rozptylu *mezi* třídami k rozptylu *uvnitř* tříd po projekci:

$$J(\vec{w}) = \frac{(\vec{w}^T(\vec{\mu}_1 - \vec{\mu}_2))^2}{\vec{w}^T S_W \vec{w}} \rightarrow \max, \quad \text{řešení } \vec{w} \propto S_W^{-1}(\vec{\mu}_1 - \vec{\mu}_2),$$

kde S_W je součet vnitrotřídních rozptylových matic. Fisher původně odvodil tento směr *bez* předpokladu normality — shoduje se však s bayesovsky optimálním řešením pro normální rozdělení se shodnými kovariančními maticemi.

Lineární a kvadratická diskriminační analýza

QDA (*kvadratická*) — obecný případ s *různými* kovariančními maticemi $S_1 \neq S_2$; diskriminační funkce obsahuje kvadratický člen $\frac{1}{2}\vec{x}^T(S_1^{-1} - S_2^{-1})\vec{x}$, takže hranice je kvadratická plocha (elipsoid, hyperboloid).

LDA (*lineární*) — předpoklad *shodných* kovariančních matic $S_1 = S_2 = S$; kvadratický člen se vyruší a hranice je *nadrovina*:

$$f(\vec{x}) = (\vec{\mu}_1^T - \vec{\mu}_2^T)S^{-1}\vec{x} - \frac{1}{2}(\vec{\mu}_1 - \vec{\mu}_2)^T S^{-1}(\vec{\mu}_1 - \vec{\mu}_2) - \ln \frac{P(c_1)}{P(c_2)}.$$

Úplné vzorce viz kap. 5.

	LDA	QDA
Předpoklad	$S_1 = S_2$	$S_1 \neq S_2$
Tvar hranice	nadrovina	kvadratická plocha
Počet parametrů	$O(m^2)$ jedna matice	$O(m^2)$ na každou třídu
Riziko	vyšší zkreslení	vyšší rozptyl, potřebuje více dat
Kompromis	regularizovaná DA (RDA): $S_t(\alpha) = \alpha S_t + (1 - \alpha)S$	

LDA slouží zároveň jako metoda *redukce dimenze s učitelem*: promítá data do nejvýše $m - 1$ dimenzí maximalizujících oddělení tříd (na rozdíl od PCA, která cílovou proměnnou nepoužívá).

Diskriminační analýza při klasifikaci obrázků a videí

Obrázek o rozměru 100×100 pixelů je vektor v $\mathbb{R}^{10000}$, zatímco trénovacích vzorků bývá stovky — kovarianční matice je singulární a QDA i LDA přímo nepoužitelné. Standardní řešení:

- **Eigenfaces** (Turk, Pentland, 1991) — nejprve PCA (bez učitele) do několika desítek dimenzí, kde se zachytí hlavní variabilita.
- **Fisherfaces** (Belhumeur et al., 1997) — *PCA následovaná LDA*: PCA sníží dimenzi tak, aby S_W byla regulární, a LDA pak najde směry nejlépe oddělující *osoby*. Ukázalo se, že Fisherfaces jsou výrazně odolnější vůči změnám osvětlení a výrazu než Eigenfaces, protože LDA se učí potlačit variabilitu *uvnitř* třídy (tj. mezi snímky téže osoby).
- **Video** — klasifikace akcí a gest: příznaky se extrahují z časoprostorových objemů (optický tok, histogramy gradientů) a LDA se používá jak ke klasifikaci, tak k redukci dimenze před dalším klasifikátorem. Varianty: 2D-LDA a tenzorová LDA pracující přímo s maticovou/tenzorovou strukturou snímků.

Diskriminační analýza je tak v počítačovém vidění především *nástrojem naučené projekce*; v moderních systémech ji nahradily hluboké konvoluční sítě, princip „maximalizuj rozptyl mezi třídami, minimalizuj uvnitř tříd“ však přežívá ve ztrátových funkcích typu *center loss* či *triplet loss*.

14.4 Klasifikátory hledající hranice pomocí umělých neuronových sítí

Perceptron: lineární hranice

Perceptron (Rosenblatt, 1958) počítá $y = \text{sign}(\sum_j w_j x_j + b)$ a jeho rozhodovací hranicí je **nadrovina**. Učí se jednoduchým pravidlem: je-li vzor $\vec{x}_i$ klasifikován chybně, uprav $\vec{w} \leftarrow \vec{w} + \eta y_i \vec{x}_i$.

Věta o konvergenci perceptronu: jsou-li třídy lineárně separabilní, algoritmus skončí po konečném počtu kroků. Nejsou-li separabilní, *necykluje ke konci* — proto se používá „kapesní“ varianta (*pocket algorithm*) pamatující si nejlepší dosažené váhy.

Omezení: perceptron realizuje pouze lineárně separabilní funkce — nedokáže se naučit XOR (Minsky, Papert, 1969). To vedlo k první „zimě“ neuronových sítí.

Vícevrstvý perceptron: nelineární hranice

Vícevrstvý perceptron (*multilayer perceptron*, MLP) přidá jednu či více *skrytých* vrstev s nelineární aktivační funkcí. Skrytá vrstva vytváří novou reprezentaci vstupu; výstupní vrstva nad ní realizuje lineární oddělení. Výsledná hranice v původním prostoru je *libovolně složitá po částech nelineární plocha* — MLP s jednou skrytou vrstvou a nelineární aktivací je *univerzální aproximátor*.

Trénuje se *zpětným šířením chyby* (*backpropagation*); podrobnosti, problémy (lokální minima, mizející gradient, přeučení) a varianta ELM viz kap. 5.

Vztah k jádrovým metodám: skrytá vrstva plní tutéž roli jako zobrazení ϕ u SVM — rozdíl je v tom, že MLP se reprezentaci *učí* z dat, zatímco jádro ji volí předem.

Filtrace spamu. Historicky MLP nad vybranými tokeny (srovnatelné s naivním Bayesem, ale dražší na trénink). Dnes: rekurentní sítě a zejména *transformerové* jazykové modely nad celým textem zprávy, konvoluční sítě nad obrázky u image spamu, sítě nad znakovými n -gramy odolné vůči záměrným překlepům, a vnoření URL a domén. Výhodou je, že se sítě učí *reprezentaci* a nevyžadují ruční návrh příznaků; nevýhodou je nízká interpretovatelnost a nutnost velkých dat — proto je používají velcí provozovatelé (Gmail, Microsoft) a nikoli malé filtry.

Doporučovací systémy. *Neuronové kolaborativní filtrování* nahrazuje skalární součin latentních faktorů naučenou nelineární funkcí; *autoenkodéry* rekonstruuji řídký vektor hodnocení; *sítě s vnořeními* (*embeddings*) zpracují kategoriální rysy (uživatel, položka, kontext); *sekvenční modely* (RNN, transformery) modelují pořadí interakcí v relaci; architektura *wide & deep* kombinuje zapamatování konkrétních kombinací s generalizací. Produkčním příkladem je dvoustupňový systém YouTube (kandidátní síť + hodnotící síť).

Odhalování hrozeb v síti. MLP nad příznaky spojení bylo klasickým přístupem na KDD Cup 99. Dnes: *autoenkodéry* pro detekci anomálií (velká rekonstrukční chyba = anomálie) — výhodou je učení jen z normálního provozu; *LSTM* nad posloupnostmi paketů, systémových volání či logů (predikce dalšího prvku, odchylka = anomálie); *konvoluční sítě* nad surovými bajty paketů; *grafové neuronové sítě* nad grafem komunikace mezi hosty (kap. 10) pro odhalení laterálního pohybu a botnetů. Sítě zvládají i vícetřídní rozlišení druhů útoků, včetně vzácných tříd R2L a U2R, jsou-li doplněny převzorkováním a cenově citlivou ztrátou.

14.5 Shrnutí k zkoušce

- **Dělení metod:** hledající hranice (perceptron, MLP, SVM, stromy) vs. nehledající hranice — ty stojí buď na *podobnosti* (k -NN), nebo na *odhadu pravděpodobnosti* (bayesovské klasifikátory, logitová metoda, diskriminační analýza).
- k -NN: volba k (malé $\rightarrow$ rozptyl, velké $\rightarrow$ zkreslení; křížová validace, liché k , vážené hlasování); míry podobnosti (euklidovská, Manhattan, Mahalanobis, kosinová, Pearson, Jaccard, Hamming, editační); nutnost normalizace a citlivost na irelevantní atributy.
- k -NN v kolaborativním filtrování: uživatelské vs. položkové; *Pearson* (odečte průměr uživatele), *kosinová* (implicitní zpětná vazba), *upravená kosinová* (položkové CF); významnostní vážení a smršťování; predikční vzorec $\hat{r}_{ui}$.
- k -NN v detekci malware: statické (n -gramy bajtů a opcodů, importy, entropie) a dynamické (trasy API volání) příznaky; Jaccard, kosinová, NCD; výhodou je rodinová struktura malwaru a vysvětlitelnost.
- **Naivní Bayes:** $H_{MAP} = \arg \max P(H_t) \prod P(E_k | H_t)$; ve spamu Grahamova varianta s 15 nejinformativnějšími tokeny, Robinsonova–Fisherova kombinace; učení = aktualizace čítačů, TOE/TONE, stárnutí, personalizace.
- **Logitová metoda:** $\ln \frac{P}{1-P}$ lineární v attributech, maximální věrohodnost; diskriminativní, dobře kalibrovaná, zvládá korelované atributy — na rozdíl od naivního Bayese.
- **Bayesian poisoning:** pasivní (word salad, skrytý text, překlapy, image spam — útok na *klasifikaci*) vs. aktivní (manipulace zpětné vazby — útok na *učení*); obrany: nejinformativnější tokeny, vícemístné příznaky (SBPH/OSB v CRM114), neučit na neověřených datech, metainformace, referenční korpus.
- **Diskriminační analýza:** prokládání vícerozměrného normálního rozdělení; Fisherovo kritérium $J(\vec{w})$ a $\vec{w} \propto S_W^{-1}(\vec{\mu}_1 - \vec{\mu}_2)$; **LDA** (shodné S , lineární hranice) vs. **QDA** (různé S , kvadratická hranice); v obrazech Eigenfaces (PCA) vs. **Fisherfaces** (PCA + LDA, odolné vůči osvětlení).
- **Neuronové sítě:** perceptron = lineární hranice, věta o konvergenci, nedokáže XOR; MLP = nelineární hranice, univerzální aproximátor, backpropagation; aplikace ve spamu (transformery, image spam), doporučování (neuronové CF, autoenkodéry, sekvenční modely) a IDS (autoenkodéry, LSTM, grafové sítě).

15 Kdy dělá klasifikátor nejméně chyb na nových vstupech?

15.1 Generalizační schopnost

Generalizační schopnost

Generalizační schopnost (generalization ability) klasifikátoru je jeho přesnost na *nových* vstupech, tj. na datech, která nebyla použita při učení. Formálně jde o *skutečné riziko*

$$R(h) = \mathbb{E}_{(\vec{x}, y) \sim P} [L(y, h(\vec{x}))],$$

zatímco učení minimalizuje pouze *empirické riziko* $\hat{R}(h)$ na trénovací množině. Rozdíl $R(h) - \hat{R}(h)$ je *generalizační mezera*.

Statistická teorie učení (Vapnik, Chervonenkis) dává horní meze typu

$$R(h) \leq \hat{R}(h) + \Phi\left(\frac{d_{VC}}{p}\right),$$

kde p je počet trénovacích vzorů a d_{VC} VC dimenze třídy hypotéz (kapacita modelu). Odtud *princip strukturální minimalizace rizika (structural risk minimization)*: nestačí minimalizovat trénovací chybu — je nutné současně omezovat kapacitu modelu. To je teoretický základ SVM.

Předpoklad o šířce pásu mezi třídami

Předpoklad: u binárního klasifikátoru pro *lineárně separabilní* třídy se generalizační schopnost **zvýší**, zvětšíme-li *šířku pásu (margin)* mezi třídami.

Intuice: oddělujících nadrovin je nekonečně mnoho a všechny mají nulovou trénovací chybu, takže trénovací chyba mezi nimi nerozliší. Nadrovina těsně přiléhající k datům je však křehká — malý posun nového bodu (šum v měření, mírně jiná realizace téhož rozdělení) jej přenesse na druhou stranu. Nadrovina s co nejširším pásem naopak ponechá kolem sebe „bezpečnostní zónu“.

Teoretické opodstatnění: pro nadroviny s okrajem γ nad daty v kouli o poloměru R je efektivní VC dimenze omezena hodnotou $\min\{\lceil R^2/\gamma^2 \rceil, m\} + 1$ — **nezávisle na dimenzi m prostoru**. Široký okraj tedy snižuje kapacitu, a tím podle strukturální minimalizace rizika i mez chyby. To je klíč k tomu, proč SVM nezkolabují ve vysokorozměrných prostorech příznaků.

15.2 Hledání klasifikátoru s nejširším pásem jako optimalizační úloha

Optimalizační formulace

Trénovací množina $\{(\vec{x}_1, y_1), \dots, (\vec{x}_r, y_r)\}$, $y_i \in \{1, -1\}$. Hledáme nadrovinu $\langle \vec{w} \cdot \vec{x} \rangle + b = 0$ s maximálním okrajem. Po normalizaci škály tak, že nejbližší body splňují $|\langle \vec{w} \cdot \vec{x} \rangle + b| = 1$, jsou *opěrné nadroviny tříd* $H_+ : \langle \vec{w} \cdot \vec{x} \rangle + b = 1$ a $H_- : \langle \vec{w} \cdot \vec{x} \rangle + b = -1$ a šířka pásu je

$$\text{margin} = d_+ + d_- = \frac{2}{\|\vec{w}\|}.$$

Maximalizace okraje je tedy ekvivalentní minimalizaci $\|\vec{w}\|$:

$$\text{minimalizuj } \frac{\langle \vec{w} \cdot \vec{w} \rangle}{2} \quad \text{za podmínky} \quad y_i(\langle \vec{w} \cdot \vec{x}_i \rangle + b) \geq 1, \quad i = 1, \dots, r.$$

Jde o úlohu *konvexního kvadratického programování* s lineárními omezeními — má tedy **jediné globální optimum** (na rozdíl od neuronových sítí, kde hrozí lokální minima).

Role vektorů z opěrných nadrovin: support vektory

Lagrangeova funkce $L_P = \frac{1}{2} \langle \vec{w} \cdot \vec{w} \rangle - \sum_i \alpha_i [y_i (\langle \vec{w} \cdot \vec{x}_i \rangle + b) - 1]$, $\alpha_i \geq 0$. Kuhnovy–Tuckerovy podmínky jsou pro konvexní účelovou funkci s lineárními omezeními *nutné i postačující*. Podmínka komplementarity

$$\alpha_i [y_i (\langle \vec{w} \cdot \vec{x}_i \rangle + b) - 1] = 0$$

říká, že $\alpha_i > 0$ mohou mít *pouze* body ležící přesně na opěrných nadrovinách H_+ a H_- — to jsou **support vektory** (*opěrné vektory*); pro všechny ostatní body je $\alpha_i = 0$.

Důsledky:

- Řešení $\vec{w} = \sum_{i \in sv} \alpha_i y_i \vec{x}_i$ závisí *jen* na support vektorech — ostatní data lze zahodit, aniž se nadrovina změní. Model je *řídový*.
- Support vektorů bývá málo (jednotky procent dat), takže je klasifikace rychlá.
- Podíl support vektorů dává odhad chyby: leave-one-out chyba $\leq \frac{\#sv}{r}$.
- Naopak: přidání či odebrání bodu daleko od hranice model nezmění — SVM je robustní vůči „snadným“ datům, ale citlivá vůči šumu *blízko* hranice.

Duální úloha (Wolfeho duál) obsahuje data pouze prostřednictvím skalárních součinů:

$$L_D = \sum_{i=1}^r \alpha_i - \frac{1}{2} \sum_{i,j=1}^r y_i y_j \alpha_i \alpha_j \langle \vec{x}_i \cdot \vec{x}_j \rangle \rightarrow \max, \quad \sum_i \alpha_i y_i = 0, \quad \alpha_i \geq 0,$$

a klasifikace testovacího objektu $\vec{z}$ je $\text{sign}(\sum_{i \in sv} \alpha_i y_i \langle \vec{x}_i \cdot \vec{z} \rangle + b)$. Odvození viz kap. 5.

15.3 SVM pro lineárně neseparabilní třídy

Rozšíření o toleranci vůči šumu: měkký okraj

Reálná data obsahují šum a překryv tříd, takže tvrdá podmínka $y_i (\langle \vec{w} \cdot \vec{x}_i \rangle + b) \geq 1$ nemusí být splnitelná — a i kdyby byla, její vynucení by vedlo k přeučení. Zavedeme *pomocné proměnné* (*slack variables*) $\xi_i \geq 0$ měřící porušení podmínky:

$$\text{minimalizuj } \frac{\langle \vec{w} \cdot \vec{w} \rangle}{2} + C \sum_{i=1}^r \xi_i \quad \text{za podmínek} \quad y_i (\langle \vec{w} \cdot \vec{x}_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Interpretace: $\xi_i = 0$ — bod je správně a mimo pás; $0 < \xi_i \leq 1$ — bod je správně, ale uvnitř pásu; $\xi_i > 1$ — bod je klasifikován chybně. Suma $\sum \xi_i$ je horní mezí počtu chyb.

Parametr $C > 0$ řídí kompromis mezi šířkou pásu a počtem chyb: velké C silně penalizuje chyby (úzký pás, riziko přeučení), malé C preferuje široký pás (více chyb, lepší zobecnění). Volí se křížovou validací. V duální úloze se změní jediná věc: $0 \leq \alpha_i \leq C$ (*box constraint*).

Varianta ν -SVM nahrazuje C parametrem $\nu \in (0, 1]$, který přímo shora omezuje podíl chyb a zdola podíl support vektorů — je intuitivnější na nastavení.

Proč je pro SVM výhodný přechod pomocí jádrových funkcí?

Toto je klíčová otázka tématu. Odpověď má **čtyři části**:

1. Duální formulace pracuje s daty jen přes skalární součiny. V L_D i v rozhodovací funkci vystupují data *výhradně* jako $\langle \vec{x}_i \cdot \vec{x}_j \rangle$, resp. $\langle \vec{x}_i \cdot \vec{z} \rangle$. Po zobrazení ϕ by tam bylo $\langle \phi(\vec{x}_i) \cdot \phi(\vec{x}_j) \rangle$, což je přesně $K(\vec{x}_i, \vec{x}_j)$. Stačí tedy *dosadit jádro* — zobrazení ϕ nemusíme znát ani nikdy vyčíslit. U metod, které potřebují souřadnice jednotlivých bodů (např. rozhodovací stromy), tento trik použít nelze.

2. Složitost nezávisí na dimenzi prostoru příznaků. Duální úloha má tolik proměnných α_i , kolik je *trénovacích vzorů* — nikoli kolik je dimenzí. Přechod do prostoru o milionech (u RBF jádra dokonce *nekonečně mnoha*) dimenzí tedy **nestojí nic navíc**. Explicitní transformace by naopak byla výpočetně neproveditelná.

3. Kapacita je řízena okrajem, ne dimenzí. Právě zde se uplatní mez z předchozí sekce: efektivní VC dimenze závisí na R^2/γ^2 , nikoli na m . SVM proto ve vysokorozměrném prostoru *netrpí prokletím dimenzionality* — na rozdíl od většiny jiných metod, které by v něm okamžitě přeučily. Bez principu maximálního okraje by byl jádrový trik k ničemu, protože v obrovské dimenzi lze data separovat vždy, a to bezcenně.

4. Řešení zůstává řídké. I v nekonečně rozměrném prostoru je rozhodovací funkce $\text{sign}(\sum_{i \in sv} \alpha_i y_i K(\vec{x}_i, \vec{z}) + b)$ konečnou sumou přes support vektory, takže je vyčíslitelná.

Navíc: konvexitá úlohy se dosazením jádra *zachová* (jádrová matice je pozitivně semidefinitní podle Mercerovy věty), takže i jádrová SVM má jediné globální optimum.

Používání SVM při klasifikaci do více tříd

SVM je ze své podstaty **dvoutřídní** — hledá jednu oddělující nadrovinu. Pro $m > 2$ tříd:

- *jeden proti zbytku* — m SVM, každá odděluje jednu třídu od ostatních; rozhodne největší hodnota $f_j(\vec{z})$. Levné (m modelů), ale trénovací množiny jsou nevyvážené a skóre různých SVM nejsou přímo srovnatelná (řeší se kalibrací, např. Plattovým škálováním na pravděpodobnosti);
- *jeden proti jednomu* — $\binom{m}{2}$ SVM, každá nad dvojicí tříd, hlasování. Více modelů, ale každý se učí na menších datech, takže celkově bývá rychlejší; standard v knihovně LIBSVM;
- *DAG-SVM* — tytéž modely jako one-against-one, ale vyhodnocují se v orientovaném acyklickém grafu, což vyžaduje jen $m - 1$ vyhodnocení;
- *kódy opravující chyby* (ECOC) — každé třídě binární kód, klasifikátor pro každý bit, dekódování na nejbližší kódové slovo; robustní vůči chybám jednotlivých klasifikátorů;
- *přímé vícetřídní formulace* (Weston–Watkins, Crammer–Singer) — jediná optimalizační úloha se všemi třídami; elegantní, ale výpočetně náročnější a v praxi se používají méně.

15.4 Použití SVM v internetových aplikacích

SVM pro filtraci spamu

SVM patří k *nejlepším klasifikátorům pro klasifikaci textu* a spam je textová úloha. Důvody vhodnosti:

- textová data jsou *vysokorozměrná* (desítky tisíc tokenů) a *řídká* — přesně režim, kde SVM díky maximálnímu okraji nepřeučí a kde často stačí *lineární* jádro;
- většina tokenů je relevantní, takže agresivní výběr atributů není nutný;
- řídkost modelu (jen support vektory) drží klasifikaci rychlou i při velkých korpusech.

Praxe: lineární SVM (LIBLINEAR, SVM^{light}) nad binárními či TF-IDF příznaky; *online* varianty (Pegasos, stochastický gradientní sestup s hinge ztrátou) umožňují inkrementální učení potřebné kvůli concept driftu; asymetrická cena chyb se realizuje *různými* C_+ a C_- pro pozitivní a negativní třídu, což přímo implementuje požadavek $\lambda \gg 1$ z kap. 13. *Nevýhoda:* model není interpretovatelný (u lineárního jádra lze alespoň číst váhy tokenů) a přeučení celého modelu je dražší než aktualizace bayesovských čítačů.

SVM v doporučovacích systémech

- *Obsahové filtrování*: pro každého uživatele se natrénuje osobní SVM nad atributy položek — typicky málo trénovacích vzorů ve vysoké dimenzi, tedy opět příznivý režim pro maximální okraj.
- *Klasifikace „líbí/nelíbí“* místo predikce hodnocení; ordinální varianta pro hvězdičky.
- *Ranking SVM (learning to rank)*: místo absolutního hodnocení se učí *pořadí* — pro každou dvojici položek, kde uživatel preferuje i před j , se požaduje $f(\vec{x}_i) > f(\vec{x}_j) + 1 - \xi_{ij}$. To odpovídá tomu, co doporučovací systém skutečně dělá (řadí), lépe než RMSE.
- *Maximum-margin maticová faktorizace (MMMF)* — přenáší princip maximálního okraje na doplňování matice hodnocení.
- *Studený start*: SVM nad demografickými a kontextovými atributy uživatele.

SVM při detekci malware

Nad příznaky popsanými v kap. 14 (n -gramy bajtů a opcodů, importy, trasy API volání) je SVM standardní volbou:

- vysoká dimenze a řídkost příznaků opět svědčí lineárnímu jádru;
- *řetězcová a stromová jádra* umožňují pracovat přímo se strukturovanými reprezentacemi (posloupnosti volání, grafy toku řízení) bez ručního převodu na vektory — to je další případ, kde je jádrový trik nenahraditelný;
- *jednotřídní SVM (one-class SVM)* se učí jen z „čistých“ vzorků a hlásí odchylky — vhodné pro detekci dosud neznámého malware a stejně tak pro detekci anomálií v IDS;
- cenově citlivé nastavení $C_+ \neq C_-$ odráží, že nezachycený malware je dražší než falešný poplach (na rozdíl od spamu, kde je poměr obrácený).

15.5 Aktivní učení

Aktivní učení

Aktivní učení (active learning) je režim, v němž si učicí algoritmus sám **vybírá**, které nezařazené objekty nechá označkovat „učitelem“ (*oracle* — člověk, drahý experiment). Motivace: neoznačkových dat bývá mnoho a jsou levná, zatímco označkování je drahé (analytik musí prozkoumat vzorek malware, uživatel musí ohodnotit film).

Režimy: *pool-based* (k dispozici je zásoba neoznačkových dat, vybírá se z ní), *stream-based* (data přicházejí proudem a pro každé se rozhodne, zda se zeptat), *membership query synthesis* (algoritmus si dotaz vytvoří sám).

Strategie výběru dotazů:

- *vzorkování podle nejistoty (uncertainty sampling)* — vyber objekt, u něhož je model nejméně jistý (nejnižší $\max_j P(C_j | \vec{x})$, nejmenší rozdíl dvou nejvyšších pravděpodobností, největší entropie);
- *dotaz na komisi (query by committee)* — vyber objekt, na němž se nejvíce neshodne soubor modelů;
- *očekávaná změna modelu či očekávané snížení chyby* — výpočetně nákladné, ale nejlépe podložené;
- *hustotně vážené varianty* — preferuj objekty, které jsou nejisté a zároveň reprezentativní pro hustou oblast dat (jinak se algoritmus soustředí na odlehlé hodnoty).

Relevance aktivního učení pro SVM klasifikátory

Aktivní učení a SVM se mimořádně dobře doplňují, a to ze **strukturálního** důvodu: *Model SVM je určen výhradně support vektory* — tedy body na okraji pásu. Všechny ostatní trénovací body jsou pro řešení irelevantní. Označkovat bod daleko od hranice je proto plýtvání; označkovat bod blízko hranice s vysokou pravděpodobností přidá nový support vektor a model změní. *Vzorkování podle nejistoty* se zde redukuje na jednoduché a velmi levné pravidlo:

$$\text{vyber } \vec{x} \text{ minimalizující } |f(\vec{x})| = |\langle \vec{w} \cdot \vec{x} \rangle + b|,$$

tedy **bod nejbližší aktuální oddělovací nadrovině**.

Teoretické zdůvodnění (Tong, Koller, 2001): SVM lze chápat jako aproximaci středu *prostoru verzí* (*version space*) — množiny všech hypotéz konzistentních s dosud označovanými daty. Každý nový označovaný bod tento prostor půlí nadrovinou; dotaz na bod nejbližší aktuální hranici půlí prostor verzí co nejrovnoměrněji, čímž zaručuje nejrychlejší (přibližně exponenciální) redukci nejistoty. Autoři popsali strategie *Simple margin* (nejbližší bod), *MaxMin margin* a *Ratio margin*.

Praktický dopad: v klasifikaci textu a detekci malware dosahuje aktivní SVM téže přesnosti s řádově menším počtem označovaných vzorků než náhodný výběr. *Úskalí*: počáteční model musí být alespoň hrubě rozumný („studentý start“ aktivního učení); u silně nevyvážených dat hrozí, že se algoritmus nikdy nedostane k minoritní třídě, což se řeší kombinací s hustotním vážením.

Využití aktivního učení v doporučovacích systémech

Aktivní učení řeší **studentý start**: nový uživatel nemá historii, systém se ho tedy musí na několik položek *zeptat* — a otázka zní, *na které*. Kritéria pro volbu položek k ohodnocení:

- *popularita* — uživatel položku pravděpodobně zná a dokáže ji ohodnotit (jinak dotaz vyjde naprázdno);
- *entropie hodnocení* — položka, kterou uživatelé hodnotí velmi různě, nese hodně informace o vkusu; naopak film, který se líbí všem, nerozliší nic;
- *kombinace popularita $\times$ entropie* — v praxi nejlepší jednoduchá heuristika, protože samotná entropie vede na neznámé okrajové tituly;
- *personalizovaná adaptivní strategie* — otázky se volí postupně podle předchozích odpovědí, typicky rozhodovacím stromem, jehož vnitřní uzly jsou otázky na položky (metody IGCN, funkční maticní faktorizace);
- *očekávané snížení chyby predikce* nad zbytkem katalogu.

Totéž se používá i *průběžně* u existujících uživatelů („ohodnoťte tyto tři filmy a zlepšíme vaše doporučení“) a při zavedení *nové položky* do katalogu — systém ji cíleně nabídne uživatelům, jejichž hodnocení o ní řekne nejvíce. Praktickým omezením je, že dotazy jsou pro uživatele zátěží: je nutné vyvážit informační přínos a ochotu odpovídat.

15.6 Shrnutí k zkoušce

- **Generalizační schopnost** = přesnost na nových vstupech; $R(h) \leq \hat{R}(h) + \Phi(d_{VC}/p)$; *strukturální minimalizace rizika* — minimalizovat trénovací chybu a zároveň omezovat kapacitu.
- **Předpoklad o šířce pásu**: širší pás $\rightarrow$ lepší generalizace. Efektivní VC dimenze je omezena hodnotou $\min\{R^2/\gamma^2, m\} + 1$, tedy *nezávisle na dimenzi* prostoru.
- **Optimalizační úloha**: $\min \frac{1}{2} \langle \vec{w} \cdot \vec{w} \rangle$ za $y_i(\langle \vec{w} \cdot \vec{x}_i \rangle + b) \geq 1$; $\text{margin} = 2/\|\vec{w}\|$; konvexní QP $\Rightarrow$ jediné globální optimum.
- **Support vektory** = body na opěrných nadrovinách s $\alpha_i > 0$ (podmínka komplementarity); model je řídký, závisí jen na nich; LOO chyba $\leq \#sv/r$.
- **Měkký okraj**: $\xi_i \geq 0$, účelová funkce $+C \sum \xi_i$, duálně $0 \leq \alpha_i \leq C$; C řídí kompromis; ν -SVM.
- **Proč jádra právě u SVM**: (1) duál obsahuje jen skalární součiny; (2) složitost závisí na počtu vzorů, ne na dimenzi; (3) kapacitu řídí okraj, ne dimenze — proto žádné prokletí dimenzionality;

(4) řešení zůstává řídké; navíc se zachová konvexita.

- **Více tříd:** one-against-rest, one-against-one (LIBSVM), DAG-SVM, ECOC, přímé formulace.
- **Aplikace:** spam (lineární SVM nad TF-IDF, online varianty, $C_+ \neq C_-$), doporučování (osobní SVM, Ranking SVM, MMMF), malware (řetězcová a stromová jádra, jednotřídní SVM).
- **Aktivní učení:** pool/stream/synthesis; strategie nejistota, komise, očekávaná změna. **Pro SVM** je přirozené, protože model určují jen support vektory $\Rightarrow$ ptej se na bod *nejbližší nadrovině* ($\min |f(\vec{x})|$); teoreticky půlení prostoru verzí (Tong, Koller). **V doporučovacích systémech** řeší studený start — volba položek podle popularity $\times$ entropie, adaptivní stromy.

16 Kdy je klasifikace srozumitelná uživateli?

16.1 Vyjádření klasifikace jazykem formální logiky

Snaha o srozumitelné vyjádření klasifikace vede přirozeně k *jazyku formální logiky*: klasifikátor se vyjádří jako množina výroků, kterým člověk rozumí, může je ověřit doménovou znalostí a případně ručně upravit. To je přesný opak „černých skříněk“ jako SVM s jádrem či hlubokých sítí. Motivace je nejen pohodlí — v regulovaných doménách jde o právní požadavek (kap. 9), u spamu a IDS o možnost ověřit, proč byla zpráva zablokována, a u doporučování o důvěryhodnost („doporučujeme proto, že...“).

Klasifikační pravidla: implikace a ekvivalence booleovské logiky

Klasifikační pravidlo má tvar

$$\underbrace{A_1(\vec{x}) \wedge A_2(\vec{x}) \wedge \cdots \wedge A_k(\vec{x})}_{\text{antecedent } \varphi} \implies \underbrace{\text{class}(\vec{x}) = C}_{\text{sukcedent } \psi},$$

kde A_i jsou elementární podmínky („příjem = vysoký“, „počet_odkazů > 10“).

Implikace $\varphi \Rightarrow \psi$ říká pouze: *splňuje-li objekt antecedent, patří do třídy C*. Neříká nic o objektech, které antecedent nesplňují — ty může pokrývat jiné pravidlo. Sada pravidel proto může být *neúplná* (nutné implicitní pravidlo) a *nekonzistentní* (dvě pravidla pro různé třídy pokrývají tentýž objekt — řeší se uspořádáním pravidel nebo hlasováním; viz kap. 5).

Ekvivalence $\varphi \Leftrightarrow \psi$ je silnější: říká, že objekt patří do C *právě tehdy*, splňuje-li antecedent — tedy navíc, že objekt mimo antecedent do C *nepatří*. Ekvivalenční pravidlo je tedy zároveň *nutnou i postačující* podmínkou příslušnosti do třídy. Je informativnější, ale v datech se prosadí mnohem hůř (musí platit „oběma směry“). Srovnej s charakteristickými (nutná podmínka, $\Rightarrow$ ze třídy) a diskriminačními (postačující podmínka, $\Leftarrow$ do třídy) pravidly v kap. 8.

Klasifikační pravidla ve fuzzy logice

Booleovská pravidla mají *ostré* hranice: podmínka „věk > 30“ rozdělí uživatele ve věku 30 a 31 let, ačkoli jsou si prakticky totožní. *Fuzzy logika* nahradí dvouhodnotovou pravdivost *stupněm pravdivosti* z intervalu $\langle 0, 1 \rangle$ a elementární podmínky *lingvistickými proměnnými* s fuzzy množinami („mladý“, „střední“, „starý“) danými funkcemi příslušnosti $\mu : \mathcal{X} \rightarrow \langle 0, 1 \rangle$.

Konjunkce se realizuje *t-normou* T : minimum $T(a, b) = \min(a, b)$, součin $T(a, b) = ab$, Łukasiewiczova $T(a, b) = \max(0, a + b - 1)$. Stupeň splnění antecedentu je $\mu_\varphi(\vec{x}) = T(\mu_{A_1}(x_1), \dots, \mu_{A_k}(x_k))$.

Fuzzy implikace $I : \langle 0, 1 \rangle^2 \rightarrow \langle 0, 1 \rangle$ zobecňuje $a \Rightarrow b$; nejužívanější:

Název	Předpis $I(a, b)$
Kleeneho–Dienesova (S-implikace)	$\max(1 - a, b)$
Łukasiewiczova	$\min(1, 1 - a + b)$
Reichenbachova	$1 - a + ab$
Gödelova (R-implikace k minimu)	1 pro $a \leq b$, jinak b
Goguenova (R-implikace k součinu)	1 pro $a \leq b$, jinak b/a

Všechny splňují okrajové podmínky klasické implikace ($I(1, 1) = I(0, 0) = I(0, 1) = 1$, $I(1, 0) = 0$), liší se však uvnitř. *S-implikace* vznikají z $\neg a \vee b$, *R-implikace* jako residuum t-normy: $I(a, b) = \sup\{c : T(a, c) \leq b\}$.

Fuzzy ekvivalence (biresiduum) $E(a, b) = \min(I(a, b), I(b, a))$; pro Łukasiewiczovu implikaci vychází elegantně $E(a, b) = 1 - |a - b|$.

Fuzzy klasifikační pravidlo má tvar

IF x_1 je A_1 AND ... AND x_k je A_k THEN třída je C s vahou w .

Objekt se klasifikuje tak, že se pro každé pravidlo spočte stupeň splnění antecedentu, vynásobí se vahou pravidla a třídy se sečtou (*vážené hlasování*) nebo vyhraje pravidlo s nejvyšším stupněm (*winner takes all*).

Výhody: plynulé hranice (žádný skok u hraničních hodnot), přirozená lingvistická interpretace, schopnost pracovat s neurčitostí a s překrývajícími se třídami. **Nevýhody:** nutnost zvolit funkce příslušnosti a operátory; větší počet parametrů; při mnoha attributech kombinatorická exploze pravidel. Souvislost s fuzzy diskretizací a „fuzzy objekty“ s vahou < 1 viz kap. 2.

16.2 Získávání klasifikačních pravidel evolučními algoritmy

Proč evoluční algoritmy

Prostor možných sad pravidel je obrovský, diskrétní a nespojitý; účelová funkce (přesnost, srozumitelnost, pokrytí) není diferencovatelná a bývá *vícekritériální* (přesnost *versus* počet a délka pravidel). To jsou přesně podmínky, kde jsou vhodné *genetické a další evoluční algoritmy*: pracují s populací kandidátních řešení, kombinují je křížením a mutací a vybírají podle fitness — bez nároků na hladkost. Navíc snadno zvládnou vícekritériální optimalizaci (Paretova fronta přesnost $\times$ srozumitelnost).

Alternativou zůstávají hladové metody z kap. 5 (sekvenční pokrývání, RIPPER), které jsou mnohem rychlejší, ale uvíznou v lokálním optimu.

Michiganský a pittsburgský přístup

Zásadní otázkou je, *co je jedinec* v populaci.

Michiganský přístup (*Michigan approach*):

- **jedinec = jedno pravidlo**; *celá populace* tvoří výslednou sadu pravidel;
- chromozom je krátký a pevné délky $\Rightarrow$ evoluce je rychlá a lze ji provozovat *online* (průběžná adaptace);
- **problém přiřazení zásluh** (*credit assignment*): fitness jednotlivého pravidla musí odrážet jeho přínos pro *celek*, ale měří se izolovaně. Pravidla si zároveň konkurují (soutěží o místo v populaci) a musí spolupracovat (pokrýt různé oblasti). Řeší se mechanismy jako *bucket brigade*, sdílení fitness (*fitness sharing*) vynucující pokrytí různých oblastí, nebo posílené učení;
- typičtí zástupci: *klasifikátorové systémy* (*learning classifier systems*, LCS) — Hollandův ZCS, Wilsonův XCS (fitness založená na *přesnosti predikce*, nikoli na síle pravidla, což řeší problém přeživších nepřesných pravidel), UCS pro učení s učitelem.

Pittsburgský přístup (*Pittsburgh approach*):

- **jedinec = celá sada pravidel**; populace obsahuje konkurenční sady;
- fitness přímo měří kvalitu *celé* sady (přesnost na trénovacích datech, případně s penalizací za velikost) $\Rightarrow$ **problém přiřazení zásluh odpadá** a lze přirozeně optimalizovat i srozumitelnost;
- chromozom má *proměnnou délku* $\Rightarrow$ nutné speciální operátory křížení a mutace (přidání, odebrání, zobecnění, zúžení pravidla); prostor prohledávání je mnohem větší;
- výpočetně *výrazně dražší* (každé vyhodnocení fitness znamená klasifikovat celou trénovací množinu celou sadou);
- typičtí zástupci: **GABIL**, GIL, SAMUEL, GAssist.

	Michiganský	Pittsburgský
Jedinec	jedno pravidlo	celá sada pravidel
Řešení	celá populace	nejlepší jedinec
Délka chromozomu	pevná, krátká	proměnná, dlouhá
Fitness	lokální (pravidlo)	globální (sada)
Přiřazení zásluh	problém	odpadá
Interakce pravidel	implicitní	explicitně optimalizovaná
Výpočetní náklady	nízké	vysoké
Vhodné pro	online, adaptivní prostředí	dávkové učení, důraz na srozumitelnost

Hybridní: iterativní učení pravidel (*iterative rule learning*, IRL, též genetické kooperativně-kompetitivní učení). Evoluční algoritmus hledá vždy *jedno* nejlepší pravidlo (jako Michigan, tedy levně), příklady pokryté tímto pravidlem se z trénovací množiny odstraní a postup se opakuje (jako sekvenční pokrývání). Fitness je jednoznačná a náklady zůstávají nízké; cenou je, že interakce mezi pravidly se optimalizují jen nepřímo. Zástupci: SIA, HIDER, SLAVE (pro fuzzy pravidla).

Použití klasifikačních pravidel v internetových aplikacích

Filtrace spamu. Pravidlové systémy jsou v praxi zcela zásadní — *SpamAssassin* je v jádru sada stovek ručně psaných pravidel (regulární výrazy nad tělem a hlavičkami, testy struktury zprávy), z nichž každé přispívá do celkového skóre svou vahou. Právě *váhy* pravidel byly historicky optimalizovány **genetickým algoritmem** (později perceptronem) na anotovaném korpusu — učebnicový příklad evolučního učení klasifikačních pravidel v produkčním nasazení. Výhody: administrátor pravidlům rozumí, může je přidat proti konkrétní kampani během minut a vysvětlit uživateli, proč byla zpráva zablokována (*SpamAssassin* vypisuje seznam bodovaných pravidel do hlavičky). Nevýhoda: ruční údržba a snadná evazivnost.

Doporučovací systémy. Pravidla ve tvaru „koupil-li X a Y, doporuč Z“ jsou přímo *asociační pravidla* (kap. 4), resp. třídní asociační pravidla (CAR) a klasifikátor CBA. Výhodou je vysvětlitelnost doporučení a možnost obchodníka doplnit ruční pravidla („k tiskárně nabídní toner“). Fuzzy pravidla se hodí pro plynulé preference („je-li uživatel *spíše* milovník sci-fi a film je *poměrně* nový, pak doporuč silně“). Evoluční algoritmy se používají k hledání pravidel s vysokou zajímavostí a zároveň krátkým antecedentem.

Detekce malware. Signatury antivirů jsou v podstatě klasifikační pravidla; moderním standardem je jazyk **YARA** (pravidlo = podmínky nad řetězci, bajtovými vzory a metadaty souboru). Pravidla lze generovat automaticky z množiny vzorků téže rodiny a optimalizovat evolučně tak, aby pokryla co nejvíce vzorků rodiny při nulové falešné pozitivitě na čistých souborech. Výhodou je auditovatelnost (analytik pravidlo přečte a ověří) a rychlost vyhodnocení; nevýhodou křehkost vůči obfuskaci.

16.3 Získávání pravidel pomocí observačního kalkulu

Observační kalkul a metoda GUHA

Observační kalkul (*observational calculus*) je formální logický kalkul, jehož formule se vyhodnocují na *datové matici* (na konečném modelu), nikoli na abstraktní struktuře. Jeho jádrem jsou *zobecněné kvantifikátory*, které váží dvě booleovské formule (antecedent φ , sukcedent ψ) a jejichž pravdivost závisí pouze na *čtyřpolní tabulce* spočtené na datech.

Metoda **GUHA** (*General Unary Hypotheses Automaton*; Hájek, Havel, Chytil, 1966; monografie Hájek, Havránek: *Mechanizing Hypothesis Formation*, 1978) je česká metoda automatického generování hypotéz: systematicky vygeneruje všechny formule daného tvaru, vyhodnotí je na datech a vypíše ty, které platí. Jde o historicky první systematickou metodu dobývání asociací — předchází Apriori o čtvrt století. Dnešní implementací je systém *LISp-Miner* s procedurou *4ft-Miner*.

Čtyřpolní tabulka formulí φ a ψ na datové matici o n objektech:

	ψ	$\neg\psi$	Σ
φ	a	b	$a + b$
$\neg\varphi$	c	d	$c + d$
Σ	$a + c$	$b + d$	n

Asociace se zapisuje $\varphi \approx \psi$, kde $\approx$ je zobecněný kvantifikátor, tj. funkce $\{0,1\}$ ze čtveřice (a, b, c, d) .

Konstrukce observačních pravidel pomocí odhadů pravděpodobností

První rodina kvantifikátorů vychází z *bodových odhadů* podmíněných pravděpodobností spočtených přímo z četností. Parametry: práh $p \in (0, 1]$ a minimální podpora Base.

Kvantifikátor	Podmínka	Interpretace
Fundovaná implikace $\Rightarrow_{p, \text{Base}}$	$\frac{a}{a+b} \geq p$	alespoň 100p% objektů s φ má ψ — <i>přesně confidence a support</i> z kap. 4
Dvojitě fundovaná implikace $\Leftrightarrow_{p, \text{Base}}$	$\frac{a}{a+b+c} \geq p$	Jaccardova míra: mezi objekty s φ <i>nebo</i> ψ jich alespoň 100p% splňuje obojí
Fundovaná ekvivalence $\equiv_{p, \text{Base}}$	$\frac{a+d}{n} \geq p$	alespoň 100p% objektů se v φ a ψ <i>shoduje</i> (obojí platí, nebo obojí neplatí)
Nadprůměrnost $\sim_{p, \text{Base}}^+$	$\frac{a}{a+b} \geq (1+p) \frac{a+c}{n}$	relativní četnost ψ mezi objekty s φ je alespoň o 100p% vyšší než v celých datech — <i>odpovídá liftu</i>

Ke každé podmínce ve druhém sloupci navíc vždy patří konjunkce s $a \geq \text{Base}$, která zajišťuje, že se pravidlo neopírá o pár objektů. Tyto kvantifikátory jsou *deskriptivní*: tvrdí něco o pozorovaných datech, nikoli o populaci, z níž data pocházejí.

Konstrukce observačních pravidel pomocí testování hypotéz

Druhá rodina kvantifikátorů je *statistická*: pravidlo platí, právě když příslušný *statistický test* na hladině významnosti α zamítne nulovou hypotézu. Tvrzení pak neplatí jen o datech, ale (s danou spolehlivostí) i o populaci.

Dolní kritická implikace $\Rightarrow_{p, \alpha, \text{Base}}^!$ — jednostranný *binomický test* hypotézy $H_0 : P(\psi | \varphi) \leq p$ proti $H_1 : P(\psi | \varphi) > p$:

$$\sum_{i=a}^{a+b} \binom{a+b}{i} p^i (1-p)^{a+b-i} \leq \alpha \quad \wedge \quad a \geq \text{Base}.$$

Levá strana je p -hodnota: pravděpodobnost, že bychom při skutečné podmíněné pravděpodobnosti p pozorovali alespoň a úspěchů z $a+b$ pokusů. Je-li malá, zamítáme H_0 a tvrdíme, že skutečná podmíněná pravděpodobnost je *opravdu* vyšší než p .

Fisherův kvantifikátor $\sim_{\alpha, \text{Base}}^F$ — *Fisherův exaktní test* nezávislosti φ a ψ (viz kap. 3), doplněný požadavkem na *kladný* směr asociace:

$$ad > bc \quad \wedge \quad \sum_{i=a}^{\min(a+b, a+c)} \frac{\binom{a+b}{i} \binom{c+d}{a+c-i}}{\binom{n}{a+c}} \leq \alpha \quad \wedge \quad a \geq \text{Base}.$$

Kvantifikátor $\chi^2 \sim_{\alpha, \text{Base}}^{\chi^2}$ — táž myšlenka s testem χ^2 (asymptotický, vyžaduje dostatečné četnosti); **horní kritická implikace** $\Rightarrow_{p, \alpha, \text{Base}}^?$ testuje opačnou nerovnost.

Příklad — vyčíslení GUHA kvantifikátorů na čtyřpolní tabulce

Nechť $\varphi \equiv$ „vysoký příjem“, $\psi \equiv$ „úvěr schválen“, $n = 200$:

	ψ	$\neg\psi$	Σ
φ	$a = 60$	$b = 20$	80
$\neg\varphi$	$c = 40$	$d = 80$	120
Σ	100	100	200

- *Fundovaná implikace*: $\frac{a}{a+b} = \frac{60}{80} = 0,75$, tedy $\varphi \Rightarrow_{p, \text{Base}} \psi$ platí pro každé $p \leq 0,75$ (a $\text{Base} \leq 60$). Support je $60/200 = 30\%$.
- *Dvojitě fundovaná implikace*: $\frac{a}{a+b+c} = \frac{60}{120} = 0,50$ — přísnější, protože penalizuje i objekty s ψ bez φ .
- *Fundovaná ekvivalence*: $\frac{a+d}{n} = \frac{140}{200} = 0,70$.
- *Nadprůměrnost*: $\frac{a}{a+b} = 0,75$ oproti $\frac{a+c}{n} = 0,50$; podmínka $0,75 \geq (1+p) \cdot 0,50$ dá $p \leq 0,5$ — profil zvyšuje šanci na ψ o 50 %, což je přesně $\text{lift} = 0,75/0,50 = 1,5$.
- *Dolní kritická implikace* pro $p = 0,6$, $\alpha = 0,05$: $\sum_{i=60}^{80} \binom{80}{i} 0,6^i 0,4^{80-i} = 0,0036 \leq 0,05$ — zamítáme H_0 , takže tvrdíme $P(\psi | \varphi) > 0,6$ i pro populaci.
- *Fisherův kvantifikátor*: $ad = 4800 > bc = 800$ (kladný směr) a jednostranná p -hodnota je $5,3 \cdot 10^{-9} \leq 0,05$ — asociace je vysoce významná.
- *Kvantifikátor χ^2* : očekávané četnosti jsou 40, 40, 60, 60, odchylky ± 20 , tedy $\chi^2 = \frac{400}{40} + \frac{400}{40} + \frac{400}{60} + \frac{400}{60} = 33,33 > 3,84$ — rovněž zamítáme nezávislost.

Poučení: deskriptivní kvantifikátory dají „0,75“, statistické k tomu dodají, že to není náhoda. Kdyby byla táž čtyřpolní tabulka desetkrát menší ($a = 6$, $b = 2$, $c = 4$, $d = 8$), zůstala by fundovaná implikace stejná (0,75), ale binomický test by dal p -hodnotu 0,315, Fisherův test 0,085 a χ^2 pouhých $3,33 < 3,84$ — *ani jeden* statistický kvantifikátor by tedy neplatil. Právě v tom je rozdíl obou rodin.

Srovnání obou rodin kvantifikátorů

Proč dvě rodiny? Odhady pravděpodobností jsou levné a přímo interpretovatelné, ale při malých četnostech mohou být iluzorní (pravidlo s $a = 3$, $b = 0$ má confidence 100 %, aniž by to cokoli znamenalo). Statistické kvantifikátory tuto past ošetří přímo v definici pravidla. Cenou je vyšší výpočetní náročnost a nutnost korekce na *mnohonásobné testování* — GUHA generuje statisticky obrovské množství hypotéz, takže část „významných“ nálezů je nutně náhodná (řeší se Bonferroniho korekcí či kontrolou FDR).

Vztah k asociačním pravidlům. Fundovaná implikace je klasické asociační pravidlo se support a confidence; GUHA je tedy obecnějším rámcem, v němž jsou Apriori a spol. speciálním případem. GUHA navíc umožňuje v antecedentu i sukcedentu *libovolné* booleovské formule (včetně disjunkcí a negací), nikoli jen konjunkce, a nabízí i podmíněné (*conditional*) varianty a kvantifikátory nad více kontingenčními tabulkami (SD4ft-Miner pro srovnání dvou skupin).

16.4 Klasifikační stromy

Klasifikační stromy a pravidla z nich

Klasifikační strom (*classification tree*, rozhodovací strom) je model, kde každý vnitřní uzel testuje jeden atribut, každá hrana odpovídá výsledku testu a každý list nese třídu. Definice, algoritmy (TDIDT, ID3, C4.5, CART, CHAID), kritéria dělení (informační zisk, poměrný informační zisk, Gini, χ^2), učení, prořezávání (*reduced-error pruning*, *subtree raising*, *cost-complexity pruning*, *rule post-pruning*) a propočítaný příklad jsou v kap. 5.

Získávání pravidel ze stromu: každá cesta od kořene k listu odpovídá právě jednomu pravidlu — vnitřní uzly s hodnotami na příslušných hranách tvoří konjunktivní antecedent, list tvoří sukcedent. Takto vzniklá sada je zpočátku *vzájemně vylučná a vyčerpávající*; po prořezání pravidel (odstranění zbytečných podmínek) tuto vlastnost ztrácí a je nutné pravidla uspořádat podle odhadované přesnosti.

Proč jsou stromy „srozumitelné“: model lze nakreslit a přečíst; každé rozhodnutí je posloupností jednoduchých testů; strom sám ukazuje *důležitost* atributů (blíže ke kořeni = důležitější). *Meze srozumitelnosti:* strom hlouběji než 4–5 úrovní už člověk nepřečte; stromy jsou nestabilní (malá změna dat → jiný strom), takže „vysvětlení“ nemusí být spolehlivé; a hranice jsou vždy osově rovnoběžné, takže šikmý vztah strom aproximuje mnoha schody.

Použití klasifikačních stromů v doporučovacích systémech a dalších internetových aplikacích

Doporučovací systémy:

- *Adaptivní dotazování při studeném startu* — strom, jehož vnitřní uzly jsou otázky „jak hodnotíte film X?“ a listy skupiny uživatelů s předpočítanými doporučeními; systém se ptá jen podél cesty stromem, takže po několika otázkách má profil nového uživatele (funkční maticová faktorizace, IGCN). Toto je zároveň realizace aktivního učení z kap. 15.
- *Klasifikace položek* do žánrů a kategorií z metadat.
- *Vysvětlitelná doporučení* — pravidlo získané ze stromu se přímo zobrazí uživateli.
- *Predikce kliknutí* — v praxi dominují *soubory* stromů (gradient boosting), viz kap. 17.

Další internetové aplikace:

- *Filtrace spamu* — stromy nad příznaky z hlaviček a metadat (kde je atributů málo a jsou heterogenní) fungují dobře; nad tisíce textových tokeny naopak hůře než SVM či naivní Bayes, protože jediný strom využije jen několik atributů.
- *Detekce průniků* — stromy patří k nejúspěšnějším metodám na KDD Cup 99 a zvládají snadno vícetřídní rozlišení druhů útoků; navíc je pravidlo z listu okamžitě převeditelné na signaturu pro Snort.
- *Detekce malware* — klasifikace do rodin, vysvětlitelná analytikovi.
- *Web mining* — klasifikace webových stránek, predikce odchodu zákazníka, kreditní skórování (kap. 11), hodnocení kvality reklam.

16.5 Shrnutí k zkoušce

- **Klasifikační pravidlo** $\varphi \Rightarrow \psi$ (postačující podmínka) vs. $\varphi \Leftrightarrow \psi$ (nutná i postačující); sada pravidel může být neúplná a nekonzistentní.
- **Fuzzy pravidla:** stupeň pravdivosti v $\langle 0, 1 \rangle$, lingvistické proměnné, konjunkce t-normou; *fuzzy implikace* — Kleeneho–Dienesova $\max(1 - a, b)$, Łukasiewiczova $\min(1, 1 - a + b)$, Gödelova, Goguenova, Reichenbachova; S-implikace vs. R-implikace (residuum); *fuzzy ekvivalence* $E(a, b) = \min(I(a, b), I(b, a))$, u Łukasiewicze $1 - |a - b|$. Výhoda: plynulé hranice a lingvistická interpretace.
- **Evoluční hledání pravidel:** prostor je diskrétní, fitness nediferencovatelná a vícekritériální. **Michiganský přístup:** jedinec = pravidlo, populace = sada, levné a online, ale *problém přiřazení zásluh* (LCS, XCS). **Pittsburgský přístup:** jedinec = celá sada, fitness globální, žádný problém zásluh, ale proměnná délka chromozomu a vysoké náklady (GABIL, GAssist). Hybrid: *iterativní*

učení pravidel (SIA, HIDER, SLAVE).

- **Aplikace pravidel:** SpamAssassin (stovky pravidel s vahami optimalizovanými genetickým algoritmem), doporučování (asociační pravidla, CAR/CBA, fuzzy preference), malware (YARA signatury).
- **Observační kalkul a GUHA:** formule se vyhodnocují na datové matici; *zobecněný kvantifikátor* závisí jen na čtyřpolní tabulce (a, b, c, d) .
- **Kvantifikátory přes odhady pravděpodobností:** fundovaná implikace $\frac{a}{a+b} \geq p$ (= confidence), dvojitě fundovaná implikace $\frac{a}{a+b+c} \geq p$, fundovaná ekvivalence $\frac{a+d}{n} \geq p$, nadprůměrnost (= lift); vždy $+a \geq \text{Base}$.
- **Kvantifikátory přes testování hypotéz:** dolní kritická implikace (binomický test), Fisherův kvantifikátor (Fisherův exaktní test + $ad > bc$), χ^2 kvantifikátor. Výhoda: platnost i pro populaci; nevýhoda: náklady a mnohonásobné testování. Umět je vyčíslit na konkrétní čtyřpolní tabulce a ukázat, že při desetkrát menších četnostech zůstane fundovaná implikace stejná, ale statistické kvantifikátory přestanou platit.
- **Klasifikační stromy:** učení a prořezávání viz kap. 5; *cesta kořen-list = pravidlo*; srozumitelné do hloubky 4–5, nestabilní, osově rovnoběžné hranice. Aplikace: adaptivní dotazování při studeném startu, klasifikace položek, IDS (pravidlo $\rightarrow$ signatura), malware, web mining.

17 Tým zvládne více než jedinec

17.1 Spojování více klasifikátorů do týmu

Tým klasifikátorů

Tým (*multiple classifier system*) kombinuje rozhodnutí několika klasifikátorů $h_1, \dots, h_T$ do jediného výsledku. Základní kombinační pravidla:

- *většinové hlasování* (*majority voting*) — vyhraje třída, kterou předpoví nejvíce členů;
- *vážené hlasování* — členové mají různou **důvěru** α_t : $H(\vec{x}) = \arg \max_j \sum_t \alpha_t \mathbb{I}[h_t(\vec{x}) = j]$;
- *kombinace pravděpodobností* — průměr, součin, maximum či minimum odhadů $P_t(C_j | \vec{x})$;
- *učená kombinace* (*stacking*) — výstupy členů se stanou vstupy *meta-klasifikátoru*, který se naučí, komu kdy věřit.

Proč tým funguje? Aby byl tým lepší než nejlepší jednotlivec, musí členové (a) být *přiměřeně přesní* (lepší než náhoda) a (b) *dělat různé chyby* — být **různorodí** (*diverse*). Jsou-li chyby nezávislé a každý člen má chybu $\epsilon < 0,5$, klesá chyba většinového hlasování T členů s rostoucím T exponenciálně (Condorcetova věta o porotě). Různorodost je tedy stejně důležitá jako přesnost — soubor T identických klasifikátorů nepřinese nic.

Zahrnutí různé důvěry různým klasifikátorům

Váha α_t může být určena:

- *apriorně* — podle znalosti o spolehlivosti metody či modality;
- *z výkonu na validační množině* — např. $\alpha_t \propto \log \frac{1-\epsilon_t}{\epsilon_t}$ (přesně tak to dělá AdaBoost, kde $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$);
- *dynamicky, v závislosti na vstupu* — *dynamický výběr klasifikátoru*: pro daný vstup se použije člen, který je nejlepší v jeho okolí (lokální kompetence měřená např. přesností na k nejbližších validačních vzorcích). Zobecněním je *směs expertů* (*mixture of experts*), kde „hradlovací“ síť (*gating network*) sama určuje váhy expertů jako funkci vstupu;
- *učením* — meta-klasifikátor ve stackingu se váhy naučí implicitně.

Týmy různých druhů vs. soubory klasifikátorů stejného druhu

Heterogenní tým (*multiple classifier system*, „tým různých klasifikátorů“) — členové jsou klasifikátory *různých typů* (SVM + naivní Bayes + strom + k -NN) trénované obvykle na týchž datech. Různorodost pramení z odlišných *induktivních biasů* jednotlivých metod: každá metoda dělá jiné chyby, protože předpokládá jiný tvar hranice. Kombinuje se hlasováním nebo stackingem.

Homogenní soubor (*ensemble*) — členové jsou klasifikátory *téhož typu*, obvykle téhož algoritmu; různorodost se musí vytvořit *uměle*: různými podmnožinami dat (bagging), různými vahami dat (boosting), různými podmnožinami atributů (random subspace), různou inicializací či náhodností v algoritmu (náhodné lesy).

	Heterogenní tým	Homogenní soubor
Zdroj různorodosti	odlišné induktivní biasy metod	vnesená náhodnost / vážení
Počet členů	malý (jednotky)	velký (stovky)
Kombinace	hlasování, stacking	hlasování, průměr
Náročnost	implementace mnoha metod	jedna metoda, mnoho běhů
Typický zástupce	stacking, směs expertů	bagging, boosting, náhodný les

Použití týmu různých klasifikátorů při klasifikaci multimediálních dat

Multimediální dokument (video, webová stránka, příspěvek na sociální síti) má několik *modalit*: obraz, zvuk, řeč (přepis), text, metadata, u webu i odkazovou strukturu. Každá modalita má zcela odlišnou reprezentaci, takže je přirozené natrénovat pro každou *vlastní* klasifikátor a jejich výstupy zkombinovat. Tři strategie:

- **časná fúze** (*early fusion*, na úrovni příznaků) — vektory příznaků všech modalit se zřetězí a použije se jediný klasifikátor. Zachytí korelace mezi modalitami, ale vede na velmi vysokou dimenzi a selže, chybí-li některá modalita.
- **pozdní fúze** (*late fusion*, na úrovni rozhodnutí) — každá modalita má vlastní klasifikátor a kombinují se až jejich výstupy (vážené hlasování, stacking). Robustní vůči chybějící modalitě, umožňuje různou důvěru modalitám a různé metody pro různé modalities — v praxi převažuje.
- **hybridní fúze** — kombinace obojího, případně víceúrovňová.

Příklady: klasifikace videa v TRECVID (vizuální rysy + přepis řeči + zvuk); detekce nevhodného obsahu na sociálních sítích (obraz + text + chování uživatele); *image spam* (OCR text + obrazové rysy + metainformace zprávy); klasifikace webových stránek (text stránky + text odkazů na ni + struktura, kap. 10). Váhy modalit se často určují dynamicky podle jejich *kvality* u konkrétního dokumentu (rozmazané video → nižší váha vizuálního klasifikátoru).

17.2 Metody vytváření týmů klasifikátorů

Bagging a boosting

Bagging (*bootstrap aggregating*) — z trénovacích dat se losováním *s vrácením* vytvoří T bootstrapových vzorků velikosti n (každý obsahuje zhruba 63,2 % různých objektů), na každém se natrénuje klasifikátor a výsledek je většinovým hlasem. **Snižuje rozptyl**, nikoli zkreslení; členy lze trénovat *paralelně*; ideálním základem jsou hluboké (nestabilní) rozhodovací stromy.

Boosting — členy se trénují *sekvenčně*; každému trénovacímu objektu je přiřazena váha, která se po každém kole zvýší u chybně klasifikovaných objektů, takže se další klasifikátor soustředí na dosud obtížné případy. Výsledkem je vážená kombinace s vahami $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$. **Snižuje zkreslení**; nelze paralelizovat; je citlivý na šum a odlehlé hodnoty. Algoritmus AdaBoost, srovnání s náhodnými lesy a moderní varianty (gradient boosting, XGBoost, LightGBM) viz kap. 5.9.

Hierarchické týmy

Hierarchický tým organizuje klasifikátory do struktury, kde není každý člen dotázán na každý vstup. Hlavní typy:

Kaskáda (*cascade*) — klasifikátory jsou zapojeny *za sebou* od nejlevnějšího k nejdražšímu. Každý stupeň buď vstup s jistotou rozhodne, nebo jej předá dál. Cílem je *rychlost*: naprostou většinu vstupů odbaví levné první stupně a drahé modely vidí jen malý zlomek dat.

- *Viola-Jones* (detekce obličejů, 2001) — kaskáda stále složitějších boostovaných klasifikátorů; první stupeň odmítne přes 50 % okének s téměř nulovou cenou. Klasický příklad.
- *Filtrace spamu* — přesně proces z kap. 12: IP reputace → SPF/DKIM → pravidla → statistický klasifikátor → antivirus. Každý stupeň je optimalizován tak, aby měl velmi nízkou falešnou pozitivitu.
- *IDS* — rychlé signaturové porovnání na plné rychlosti linky, teprve podezřelý provoz jde do drahé behaviorální analýzy.

Hierarchie podle taxonomie tříd (*hierarchical classification*) — struktura kopíruje hierarchii tříd: první klasifikátor rozhodne hrubou kategorií, další rozliší podtřídy. Příklady: malware → typ (trojan / worm / ransomware) → rodina → varianta; IDS → normální/anomální → DoS/Probe/R2L/U2R → konkrétní útok; katalog e-shopu → kategorie → podkategorie. Výhody: každý klasifikátor řeší jednodušší úlohu s vlastními příznaky, lze zohlednit různou cenu chyb na různých úrovních. Nevýhoda: chyba na vyšší úrovni je *nenapravitelná* (šíří se dolů).

Směs expertů (*mixture of experts*) — hradlovací síť rozdělí vstupní prostor mezi experty a naučí se, kterému expertovi v které oblasti věřit. Trénuje se společně s experty. Moderní podobou jsou *sparse mixture-of-experts* vrstvy v rozsáhlých neuronových sítích.

Stacking — dvouúrovňová hierarchie: bazové modely na první úrovni, meta-model na druhé; meta-model se trénuje na predikcích získaných křížovou validací, aby nedošlo k úniku informace. Lze skládat i do více úrovní.

Delegující a arbitrážní klasifikátory — klasifikátor, který si není jist, *deleguje* rozhodnutí na jiný člen (analogie odmítnutí rozhodnutí); *arbitr* je trénován právě na těch případech, na nichž se bazové modely neshodly.

Použití souborů klasifikátorů při detekci malware

Detekce malware je oblast, kde jsou soubory klasifikátorů standardem:

- *Různorodost pohledů*: statické příznaky (*n*-gramy, importy, entropie), dynamické příznaky (trasy volání ze sandboxu) a metainformace (původ souboru, podpis, reputace) tvoří tři velmi odlišné pohledy → přirozený heterogenní tým s pozdní fúzí.
- *Odolnost vůči obfuskaci*: útočník obvykle dokáže obejít *jeden* typ detekce (např. zabalením binárky zničí statické *n*-gramy), ale obejít současně statickou, dynamickou i reputační složku je mnohem těžší. Soubor tedy zvyšuje nejen přesnost, ale i *adversariální robustnost*.
- *Náhodné lesy a gradient boosting* nad tisíci příznaků patří k nejúspěšnějším metodám (např. v soutěži Microsoft Malware Classification Challenge zvítězily boostované stromy); jsou rychlé, zvládají heterogenní příznaky bez normalizace a dávají *důležitost příznaků* pro analýzu.
- *Kaskáda v antivirovém produktu*: kontrolní součet známých souborů → signatury → heuristiky → model strojového učení → sandbox (nejdražší). Uživatel nesmí čekat, takže drahé stupně vidí zlomek souborů.
- *Vícemotorové skenování* (VirusTotal) je fakticky tým desítek nezávislých komerčních detektorů; agregace jejich verdiktů (počet detekcí) se běžně používá jako pseudo-označování při trénování nových modelů.

17.3 Náhodné lesy

Klasifikační náhodné lesy

Náhodný les (*random forest*, Breiman, 2001) je soubor rozhodovacích stromů, do jejichž budování byla explicitně vnesena **náhodnost**, aby byly stromy méně korelované. Motivace: mají-li členové rozptyl σ^2 a párovou korelaci ρ , je rozptyl průměru

$$\rho\sigma^2 + \frac{(1-\rho)\sigma^2}{T},$$

takže člen $\rho\sigma^2$ *nezávisí* na počtu stromů a omezuje zisk z pouhého baggingu $\Rightarrow$ je nutné snížit ρ .

Algoritmus: pro každý z T stromů: vytvoř bootstrapový vzorek; nad ním pěstuj strom tak, že v *každém uzlu* vybereš náhodně m atributů a nejlepší dělení hledáš jen mezi nimi; strom neprořezávej. Výstup: většinový hlas (klasifikace), resp. průměr (regrese).

Parametry: m (běžná volba $\sqrt{\text{počet atributů}}$ pro klasifikaci, $m/3$ pro regresi; les na tento parametr není příliš citlivý) a počet stromů T (přidávat, dokud chyba klesá — přeučení hrozí minimálně). Podrobnosti viz kap. 5.9.

Užitečné vedlejší produkty: *out-of-bag* odhad chyby zdarma (každý strom neviděl 36,8 % dat); *důležitost atributů* (průměrný pokles Gini nebo pokles přesnosti po permutaci atributu v OOB vzorku); *blízkost (proximity)* dvou objektů = podíl stromů, v nichž skončí v téže listu — použitelná pro shlukování, detekci odlehlých hodnot i doplnění chybějících hodnot.

Typy náhodných lesů

Liší se především *způsobem, jak se vnáší náhodnost*:

Typ	Charakteristika
Forest-RI (<i>random input selection</i>)	Breimanův standard: bootstrap + náhodná podmnožina m atributů v každém uzlu; dělení podle jednoho atributu $\Rightarrow$ osově rovnoběžné hranice
Forest-RC (<i>random combination</i>)	dělení podle náhodné <i>lineární kombinace</i> L náhodně zvolených atributů s náhodnými koeficienty; dává <i>šikmé (oblique)</i> hranice a je vhodný, když je atributů málo (a náhodný výběr m z nich tedy nedává dost různorodosti)
Metoda náhodných podprostorů (Ho, 1998)	každý strom se učí na náhodně zvoleném <i>podprostoru atributů</i> , celý pro všechny uzly stromu; bez bootstrapu. Historicky předchází náhodné lesy
Extrémně randomizované stromy (Extra-Trees; Geurts et al., 2006)	navíc se <i>prahy dělení volí náhodně</i> (nikoli optimálně) a obvykle se nepoužívá bootstrap (učí se na celých datech). Ještě nižší korelace a rozptyl, mírně vyšší zkreslení; výrazně rychlejší trénink
Rotační les (<i>rotation forest</i>)	před stavbou každého stromu se na náhodné podmnožiny atributů aplikuje PCA a strom se učí v otočených souřadnicích; kombinuje přesnost s různorodostí
Online / Mondrian lesy	inkrementální varianty schopné učit se z datového proudu — relevantní pro spam a IDS, kde data přicházejí průběžně

Ortogonalně se lesy dělí na *klasifikační* a *regresní* (v listech průměr, kritérium součet čtverců), dále na *kvantilové* (odhadují celé podmíněné rozdělení), *survival forests* a *izolační les (isolation forest)* — ten používá zcela náhodné stromy k *detekci odlehlých hodnot*, protože anomálie se v náhodném stromu izoluje mělčeji než běžný bod; je to jedna z nejpoužívanějších metod detekce anomálií v IDS.

Aktivní učení náhodných lesů

Náhodný les je *přírozená komise* (*committee*): jeho stromy jsou různorodé modely natrénované na týchž datech. *Dotaz na komisi* (*query by committee*) se tedy realizuje bez jakékoli nadstavby — stačí sledovat **neshodu mezi stromy**:

- *entropie hlasů*: $-\sum_j \frac{V_j}{T} \log \frac{V_j}{T}$, kde V_j je počet stromů hlasujících pro třídu j ; vyber objekt s maximální entropií;
- *okraj hlasování* (*vote margin*): rozdíl podílu hlasů pro nejčetnější a druhou nejčetnější třídu; vyber objekt s *nejmenším* okrajem;
- *Kullbackova–Leiblerova divergence* predikcí jednotlivých stromů od průměrné predikce.

Doplňkově lze využít *blížkost* (proximity): objekt, který je daleko od všech označovaných objektů (padá do listů, kde nic označovaného není), je *reprezentativně cenný* — kombinace nejistoty a reprezentativnosti brání tomu, aby se výběr soustředil na odlehlé hodnoty.

Praktický význam: v detekci malware a v IDS je označování drahé (analytik musí vzorek prozkoumat) a neoznačovaných dat jsou miliony, takže aktivní učení lesa výrazně snižuje anotační náklady. Oproti aktivnímu učení SVM je výhodou, že les zvládá nativně více tříd a heterogenní příznaky; nevýhodou, že neshoda stromů je hrubší mírou nejistoty než vzdálenost od nadrovin.

Použití náhodných lesů pro filtraci spamu a v doporučovacích systémech

Filtrace spamu:

- Náhodné lesy jsou silné zejména nad *heterogenními* příznaky, kde se míchají textové, strukturní a metainformační rysy (počet odkazů, poměr velkých písmen, stáří domény, výsledek SPF, skóre DNSBL, podíl obrázků) — na rozdíl od SVM nevyžadují normalizaci ani transformaci a nevadí jim nelineární interakce („mnoho odkazů a zároveň neznámá doména“).
- Poskytují *důležitost příznaků*, což správci filtru ukáže, které signály fungují.
- Dávají dobře použitelné *skóre* (podíl hlasů), z něhož lze prahováním odvodit tři třídy {spam, ham, nejisté} — viz kap. 12.
- Snadno se nastaví *cenová citlivost* (váhy tříd, vážené hlasování), což je u spamu klíčové.
- *Nevýhoda*: nad tisíci řídkými textovými tokeny bývají slabší než lineární SVM a jsou dražší na uložení i vyhodnocení; proto se v praxi používají nad *agregovanými* příznaky, nikoli nad surovým slovníkem.

Doporučovací systémy:

- *Predikce hodnocení a kliknutí* z kombinace obsahových atributů položky, atributů uživatele a kontextu (denní doba, zařízení) — stromové soubory jsou zde standardem, protože zachytí interakce mezi atributy bez ručního návrhu;
- *Řazení výsledků* (*learning to rank*) — boostované stromy (LambdaMART) jsou dlouhodobě silnou baseline;
- *Hybridizace* — náhodný les jako meta-model ve stackingu, který kombinuje výstup kolaborativního a obsahového modelu a rozhoduje, kterému v jaké situaci věřit (typicky: pro nové uživatele obsahovému, pro aktivní uživatele kolaborativnímu);
- *Studený start* — model nad demografickými atributy, kde kolaborativní filtrování selhává;
- *Vysvětlitelnost* — důležitost příznaků ukáže, co doporučení řídí, byť jednotlivé doporučení vysvětlitelné není.

17.4 Shrnutí k zkoušce

- **Tým funguje**, jsou-li členové *přesní a různorodí* (dělají různé chyby); Condorcetova věta. **Různá důvěra**: pevné váhy, váhy z validace ($\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$), dynamický výběr podle lokální kompetence, směs expertů s hradlovací sítí, stacking.
- **Heterogenní tým** (různé metody, různé induktivní biasy, jednotky členů) vs. **homogenní soubor** (jedna metoda, uměle vnesená různorodost, stovky členů).
- **Multimediální data**: každá modalita vlastní klasifikátor; *časná fúze* (zřetězení příznaků) vs. *pozdní fúze* (kombinace rozhodnutí — robustnější, převažuje) vs. hybridní.

- **Metody vytváření týmů:** *bagging* (bootstrap, paralelní, snižuje rozptyl), *boosting* (sekvenční, váhy objektů, snižuje zkreslení, citlivý na šum), *hierarchické týmy* — kaskáda (Viola–Jones, spamový pipeline, IDS), hierarchie podle taxonomie tříd (malware, druhy útoků), směs expertů, stacking, delegující a arbitrážní klasifikátory.
- **Malware:** soubory zvyšují přesnost i *adversariální robustnost* (útočník musí obejít statickou, dynamickou i reputační složku); boostované stromy a náhodné lesy; kaskáda v antiviru; VirusTotal jako tým desítek motorů.
- **Náhodné lesy:** bootstrap + m náhodných atributů v každém uzlu ($\sqrt{n}$), bez prořezávání; snižují ρ , a tím i člen $\rho\sigma^2$ nezávislý na počtu stromů; OOB odhad chyby, důležitost atributů, blízkost.
- **Typy náhodných lesů:** Forest-RI (standard), Forest-RC (náhodné lineární kombinace $\rightarrow$ šikmé hranice, vhodné při málo attributech), metoda náhodných podprostorů (Ho), Extra-Trees (náhodné prahy, bez bootstrapu, rychlé), rotační les (PCA), online/Mondrian lesy; izolační les pro detekci anomálií.
- **Aktivní učení lesů:** les je *přirozená komise* $\Rightarrow$ query by committee — entropie hlasů, okraj hlasování, KL divergence; doplnkově blízkost pro reprezentativnost. Šetří anotační náklady u malware a IDS.
- **Aplikace lesů:** spam (heterogenní příznaky, důležitost příznaků, skóre pro tři třídy, cenová citlivost; slabší nad surovým slovníkem), doporučování (predikce hodnocení a CTR, LambdaMART, stacking, studený start).

Konec učebního textu k okruhům „Dobývání znalostí“ a „Internet a klasifikační metody“.